



Centro de Investigación y de Estudios Avanzados
del Instituto Politécnico Nacional
Unidad Guadalajara

Selección de algoritmos para la toma de decisiones en el IoT basada en un modelo semántico

Tesis que presenta:
Liliana Durán Polanco

Para obtener el grado de:
Doctora en Ciencias

en la especialidad de:
Ingeniería Eléctrica

Directores de Tesis:
Dr. Mario Angel Siller González Pico

Selección de algoritmos para la toma de decisiones en el IoT basada en un modelo semántico

**Tesis de Doctorado en Ciencias
Ingeniería Eléctrica**

Por:

Liliana Durán Polanco

Maestra en Ciencias en Ingeniería Eléctrica
CINVESTAV, Guadalajara 2019-2021

Becario de SECIHTI, No. 1001334

Directores de tesis:

Dr. Mario Angel Siller González Pico

Centro de Investigación y de Estudios Avanzados del Instituto Politécnico Nacional
Unidad Guadalajara, Febrero de 2026.



Centro de Investigación y de Estudios Avanzados
del Instituto Politécnico Nacional
Unidad Guadalajara

Algorithm selection based on a semantic model for decision-making in the IoT

A thesis presented by:
Liliana Durán Polanco

to obtain the degree of:
Doctor in Science

in the subject of:
Electrical Engineering

Thesis Advisors:
Dr. Mario Angel Siller González Pico

Algorithm selection based on a semantic model for decision-making in the IoT

**Doctor of Science Thesis
In Electrical Engineering**

By:

Liliana Durán Polanco

Master of Science in Electrical Engineering
CINVESTAV, Guadalajara 2019-2021

Scholarship granted by SECIHTI, No. 1001334

Thesis Advisors:

Dr. Mario Angel Siller González Pico

Center for Research and Advanced Studies in the National Polytechnical Institute,
Guadalajara Unit, February 2026.

Resumen

La toma de decisiones adaptativa en sistemas del Internet de las Cosas (IoT) requiere mecanismos capaces de responder a la evolución de los requerimientos funcionales, originada por la dinámica del sistema y por sus interacciones con contextos físicos, sociales y tecnológicos. Para abordar este desafío, se propone un mecanismo basado en un modelo semántico para la reconfiguración adaptativa de servicios de toma de decisiones. El enfoque adopta el paradigma de la computación autónoma, con énfasis en la auto-configuración como medio para facilitar la adaptación en tiempo de ejecución.

El núcleo del enfoque propuesto es O-DMIoT, una ontología para la toma de decisiones en sistemas IoT que formaliza las relaciones entre problemas de decisión, soluciones algorítmicas, contextos de ejecución y mecanismos de evaluación. Los problemas de decisión se caracterizan mediante dimensiones estructurales y contextuales, mientras que los algoritmos se describen en términos de supuestos de diseño, estrategias algorítmicas y métricas de evaluación. Esta caracterización semántica permite una selección de algoritmos trazable y la reconfiguración de servicios a nivel de ejecución. La evaluación centrada en el ser humano se integra mediante modelado basado en agentes, lo que permite analizar efectos conductuales y relacionados con la confianza sin introducir supuestos específicos del dominio en la ontología. Este análisis se apoya en la derivación de métricas de Calidad de Experiencia (QoE), que facilitan la evaluación comparativa y la reconfiguración informada.

El marco propuesto se valida a través de tres casos de estudio en dominios epidemiológicos y agrícolas, demostrando su aplicabilidad bajo condiciones de evolución del sistema. En conjunto, este trabajo establece una base formalmente fundamentada, adaptativa y sensible al factor humano para el diseño de sistemas IoT resilientes y escalables.

Abstract

Adaptive decision-making in Internet of Things (IoT) systems requires mechanisms that respond to evolving functional requirements driven by system evolution and interactions with physical, social, and technological contexts. To address this challenge, this work proposes a semantic-driven mechanism for the adaptive reconfiguration of decision-making services. The approach adopts the autonomic computing paradigm, emphasizing self-configuration to facilitate runtime adaptation.

The core of the proposed approach is O-DMIoT, an ontology for decision-making in IoT systems that formalizes relationships among decision problems, algorithmic solutions, execution contexts, and evaluation mechanisms. Decision problems are defined by structural and contextual dimensions, while algorithms are described by design assumptions, algorithmic strategies, and evaluation metrics. This semantic characterization supports traceable algorithm selection and service-level reconfiguration. Human-centered evaluation is integrated using agent-based modeling, which enables assessment of behavioral and trust-related effects without introducing domain-specific assumptions into the ontology. Quality of Experience (QoE) metrics are derived to facilitate comparative evaluation and informed reconfiguration.

The framework is validated through three case studies in epidemiological and agricultural domains, demonstrating applicability under evolving system conditions. This work establishes a formally grounded, adaptive, and human-aware foundation for resilient and scalable IoT decision-making systems.

Dedicatoria

A mi hermana y a mi mamá

Agradecimientos

- A mi asesor, el Dr. Mario, por la paciencia y por recordarme que los procesos complejos pueden ser abordados abstrayéndolos lo suficiente.
- A los profesores del área de computación, por compartir su conocimiento, consejos y, sobre todo, por cultivar la tenacidad. Esa que no aparece en los planes de estudio, pero que resulta indispensable para llegar hasta aquí.
- A mis compañeros del grupo de redes, por demostrar que las buenas conexiones persisten aun cuando los nodos cambian de lugar. Su disposición para ayudar, su compañía y sus consejos fueron invaluable en este proceso.
- A mis compañeros del laboratorio de computación, por las aventuras, las risas y los aprendizajes compartidos que hicieron el camino más ligero. Pero, sobre todo, por el apoyo —en todas las dimensiones imaginables— y el cariño que me brindaron.
- A la comunidad del Cinvestav, por ser realmente una comunidad: una que acoge, apoya y acompaña. Durante toda mi estancia encontré ayuda en los momentos necesarios, solo portando la mochila con el logo.
- A mis amigos fuera del Cinvestav, que nunca leerán estas líneas, pero que fueron motivación, familia y un espacio seguro para descargar frustraciones cuando el sistema parecía no responder.
- A quien eligió acompañarme día a día durante este proceso, incluso cuando la tesis ocupó más espacio del debido. Gracias por la paciencia, el cariño y la estabilidad en medio del ruido.
- A mi familia, por ser el sistema más estable en un entorno lleno de incertidumbre. Gracias por acompañarme, incluso cuando el proceso exigió más paciencia de la razonable. A mi hermana, en particular, por estar siempre presente.
- Quiero aclarar que decidí no mencionar nombres, no por falta de gratitud, sino por el deseo de no olvidar a nadie. Este trabajo también es de quienes estuvieron presentes en formas pequeñas, silenciosas o difíciles de enumerar.
- A SECIHTI, por proporcionar el apoyo económico que hizo posible la realización de este trabajo.

Contents

1. Introduction	19
1.1. Context	19
1.2. Problem statement	20
1.3. Motivation	20
1.4. Hypothesis	21
1.5. Research objectives	21
1.5.1. General objective	21
1.5.2. Specific objectives	21
1.6. Structure	22
1.6.1. Contribution 1 — Semantic Foundations for Algorithm Selection in IoT	22
1.6.2. Contribution 2 — Human-Centric Evaluation Framework	23
1.6.3. Contribution 3 — Semantic-Driven Algorithm Selection and Adaptive Service Reconfiguration	23
2. Theoretical Foundations	27
2.1. Self-configuration in autonomig computing	27
2.2. Ontology	27
2.2.1. Domain ontology	27
2.2.2. Domain ontology	28
2.3. The algorithm selection problem	28
2.3.1. The extended algorithm selection framework	29
2.4. Reinforcement learning	29
2.4.1. Markov decision process	29
2.5. Agent-based models (ABM)	30
2.6. Functional requirements	30
2.7. Metrics	30
2.7.1. Algorithm characteristics	31
2.7.2. Quality of Service	31
2.7.3. Quality of Experience	32
References	32
3. Semantic-Driven Decision-Making and Adaptive Reconfiguration in IoT Systems	35
3.1. Semantic Modeling for Decision-Making in IoT	35
3.2. Ontology-Based Decision Models and Algorithm Mapping	36
3.3. Autonomic Computing and Adaptive Reconfiguration	36
3.4. Semantic Reasoning with Decision-Theoretic and Learning-Based Methods	37
3.5. Evaluation Frameworks and Human-Centered Considerations	38
3.6. Domain Independence and Generalizability	38
3.7. Synthesis and Research Gaps	39

References	39
4. A taxonomy for decision making in IoT systems (TDMIoT)	43
4.1. Introduction	43
4.2. Background and related work	44
4.2.1. Taxonomies of Decision-Making in IoT and Related Fields . .	44
4.2.2. Foundations of Decision Analysis for IoT Systems	46
4.3. Research approach	46
4.3.1. Objectives of the Taxonomy	48
4.3.2. Taxonomy Development	48
4.3.3. Taxonomy Deployment	52
4.4. Taxonomy for Decision Making in IoT systems (TDMIoT)	53
4.4.1. Preliminaries and Definitions	53
4.4.2. Structure of TDMIoT	54
4.4.3. TDMIoT as a Framework for Decision Process Design	60
4.5. Research evaluation	61
4.5.1. TDMIoT as a Classification Artifact	65
4.5.2. TDMIoT as a Design Framework	66
4.6. Implications of TDMIoT	68
4.6.1. TDMIoT as a description mechanism	68
4.6.2. TDMIoT as a design framework for practitioners	68
4.6.3. Enabling Self-Configuration in Autonomic Computing	69
4.7. Conclusion	69
References	70
5. An Ontology for Decision Making in the IoT (O-DMIoT)	78
5.1. Introduction	78
5.2. Background and related work	79
5.2.1. Ontologies in IoT	79
5.2.2. Ontologies for decision-making / algorithm characterization .	81
5.3. Ontology Scope and Requirements	82
5.3.1. Domain and Purpose	83
5.3.2. Objectives	83
5.3.3. Competency Questions	83
5.3.4. Usage Scenarios and Stakeholders	84
5.4. Ontological Foundations	85
5.4.1. Overview of UFO	85
5.4.2. UFO for Decision-Making in IoT Systems	85
5.4.3. Key UFO Constructs Used	87
5.5. Methodology and Development Approach	88
5.5.1. Identification of Purpose and Requirements	88
5.5.2. Knowledge Acquisition and Conceptualization	88
5.5.3. Ontological Design and UFO Grounding	89
5.5.4. OntoUML Conceptual Modeling	89
5.5.5. Implementation and OWL/RDF Formalization	89
5.5.6. Verification and Validation	89
5.5.7. Methodological Integration with the Thesis Framework	90
5.6. Conceptual Model of O-DMIoT	90
5.6.1. Core Concepts and Conceptual Relationships	90

5.6.2.	Requirement Layer	95
5.6.3.	Design Layer	98
5.6.4.	Execution Layer	102
5.6.5.	Evaluation and Metrics as an Integrative Cross-Cutting Role .	104
5.7.	Formalization and Implementation	108
5.7.1.	OWL Encoding of O-DMIoT	108
5.7.2.	Logical Constraints and Reasoning	108
5.7.3.	Consistency and Extensibility	110
5.8.	Validation	111
5.8.1.	Competency Question Coverage	111
5.8.2.	Foundational Ontology Compliance (UFO)	111
5.8.3.	Application-driven Validation (Case Studies)	113
5.9.	Role of O-DMIoT in Thesis Framework	114
5.10.	Conclusions	114
	References	115

6. Agent-based evaluation of information and communication technology interventions 119

6.1.	Introduction	119
6.2.	Background: Non-Pharmaceutical Interventions and ICT in Pandemic Response	120
6.3.	An agent-based model to evaluate NPI to support Post-Pandemic work activities and the well being of workers	121
6.3.1.	Model Overview	121
6.4.	Modeling contributions	126
6.4.1.	Technological NPI model	127
6.4.2.	IoT monitoring infrastructure	127
6.4.3.	Human Behavior Modeling	131
6.5.	Evaluation and Computational Experiments	138
6.5.1.	Scenario Setup and Model Assumptions	138
6.5.2.	Experimental results	140
6.5.3.	Robustness and Sensitivity Considerations	146
6.6.	Deriving Human-Centric Metrics for O-DMIoT	146
6.6.1.	QoE Application Across Simulation Scenarios	147
6.6.2.	Mapping QoE to O-DMIoT Layers	149
6.7.	Implications for O-DMIoT	149
6.7.1.	Implications for Problem Characterization	149
6.7.2.	Implications for Algorithm Design and Selection	150
6.7.3.	Implications for Metrics and Evaluation	150
6.7.4.	Implications for Adaptation and Reconfiguration Mechanisms	150
6.8.	Conclusions	150
	References	151

7. Agent-based analysis of trust and adoption dynamics accross populations 154

7.1.	Introduction	154
7.2.	Background: Epidemiology, Information Interventions, and Trust Dynamics	155

7.3.	An agent-based model for Epidemics and Vaccination Response Dynamics	156
7.3.1.	Model overview	156
7.3.2.	Conceptual Architecture	157
7.3.3.	Agents and System Components	158
7.3.4.	Information Flow and Decision Cycle	160
7.4.	Behavioral Foundations and Supporting Submodels	160
7.4.1.	Epidemiological Submodel	161
7.4.2.	Vaccine Schedule Submodel	162
7.4.3.	Intention	162
7.5.	Modelling contributions	163
7.5.1.	Information Exchange Submodel	163
7.5.2.	Trust Submodel	166
7.5.3.	Decision-making Submodel	167
7.5.4.	Modeling Contribution Summary	170
7.6.	Experimental Setup and Population Scenarios	170
7.6.1.	Scenario Design	170
7.6.2.	Observed Variables	171
7.6.3.	Experimental Results and Analysis	173
7.6.4.	Summary of findings	176
7.7.	Derived Human-Centric Metrics	176
7.7.1.	QoE Instantiation	176
7.7.2.	Metric definition	177
7.7.3.	Scenario-Based Computation	177
7.8.	Human-Centric Evaluation as QoE in O-DMIoT	178
7.8.1.	Interpreting QoE Across O-DMIoT Layers	178
7.8.2.	Linking Observed Outcomes to Design and Execution Decisions	179
7.9.	Implications for O-DMIoT	180
7.9.1.	Implications for Problem Characterization	180
7.9.2.	Implications for Algorithm Design and Selection	181
7.9.3.	Implications for Metrics and Evaluation	181
7.9.4.	Implications for Adaptation and Reconfiguration Mechanisms	181
7.10.	Conclusions	181
	References	182

8. Algorithm Selection Based on a Semantic Model for Decision-Making in the IoT 189

8.1.	Introduction	189
8.2.	Algorithm Selection: Background and Related Work	190
8.2.1.	Theoretical Foundations of Algorithm Selection	191
8.2.2.	Semantic and Knowledge-Based Characterization Approaches	191
8.2.3.	Algorithm Selection in IoT and Dynamic Environments	191
8.2.4.	Learning-Based and Decision-Theoretic Perspectives	192
8.2.5.	Identified Research Gap	192
8.3.	Semantic Model for Algorithm Selection	193
8.3.1.	Problem characterization in O-DMIoT	193
8.3.2.	Algorithm Characterization in O-DMIoT	194
8.4.	Algorithm Selection as a Planning Problem	194

8.5.	Algorithm Selection as a Markov Decision Process	195
8.5.1.	MDP definition	195
8.5.2.	Ontology-based state representation	195
8.5.3.	Action semantics	196
8.5.4.	Reward	196
8.6.	Semantic-driven Monte Carlo Tree Search with Thompson Sampling .	196
8.6.1.	Assumptions	196
8.6.2.	Tree Structure	197
8.6.3.	State and Action Space Formalization	197
8.6.4.	Selection Policy (Thompson Sampling)	198
8.6.5.	Expansion Phase	198
8.6.6.	Rollout and Semantic Evaluation	198
8.6.7.	Reward Function	199
8.6.8.	Backpropagation	199
8.6.9.	Integration of Semantic Reasoning within Monte Carlo Tree Search	200
8.7.	Proof of Concept: Algorithm Selection Scenario	201
8.7.1.	Scenario Description	202
8.7.2.	Candidate Algorithms and Metrics	202
8.7.3.	Selection Outcome	203
8.8.	Discussion and Limitations	204
	References	205

9. Adaptive Reconfiguration of IoT services: a proof of concept in the field of protected and controlled agriculture 208

9.1.	Introduction	208
9.2.	Case Study Overview	209
9.3.	Simulation Environment	210
9.3.1.	Agent-based Modeling Approach	210
9.3.2.	Simulation Platform	211
9.4.	Crop Growth Environmental Model	213
9.4.1.	TOMGRO Crop Growth Model	213
9.4.2.	TOMGRO Adaptation for Agent-Based Simulation	213
9.4.3.	Model Assumptions	214
9.4.4.	Inputs and Outputs	214
9.5.	IoT System Model	215
9.5.1.	IoT System Architecture	215
9.5.2.	Evolution Modules	216
9.5.3.	Model Assumptions	216
9.6.	Adaptive Reconfiguration Mechanism	217
9.6.1.	Reconfiguration Triggers	217
9.6.2.	Problem Characterization	217
9.6.3.	Invocation of the Synthesizer	218
9.7.	Proof of Concept Execution	218
9.7.1.	End-to-End Reconfiguration Flow	220
9.7.2.	Observations	225
9.8.	Discussion and Limitations	226
	References	227

10. Conclusions	228
10.1. Summary of contributions	228
10.2. Limitations and Future Directions	228
11. Publications Derived from This Thesis	230
A. Appendix A	231
A. Formal Specification of the O-DMIoT Ontology	231
A.1. Ontology Overview	231
A.1.1. Visual Ontology Model	231
A.2. Core Ontological Elements	233
A.2.1. Core Data Properties	233
A.2.2. Logical Constraints and Axioms	233
A.2.3. Query listing	233
A.2.4. OWL Implementation and Availability	234
B. Appendix B	236
B. Computational Artifacts and Reproducibility	236
B.1. Agent-Based Models	236
B.2. Algorithm Selection and Reconfiguration Mechanisms	236

List of Figures

1.1. Thesis structure mapped to the MAPE-K autonomic control loop . . .	25
4.1. Related work summary	45
4.2. Research and development methodology	48
4.3. Systematic literature review	52
4.4. TDMIoT structure for analysis	53
4.5. TDMIoT structure for analysis	55
4.6. TDMIoT as a Framework for Decision Process Design	61
4.7. TDMIoT classification - Part 1	63
4.8. TDMIoT classification - Part 2	64
5.1. Methodological workflow for the development of O-DMIoT	91
5.2. The core concepts of O-DMIoT	92
5.3. Conceptual layered structure of the O-DMIoT ontology	93
5.4. O-DMIoT Requirement Layer	96
5.5. O-DMIoT Design layer	99
5.6. O-DMIoT Execution layer	103
5.7. O-DMIoT Evaluation layer	106
5.8. Excerpt of the OWL formalization of O-DMIoT illustrating the en- coding of the <i>Requirement layer</i>	109
5.9. Example of a Description Logic (DL) query executed in Protégé to retrieve all algorithm designs applicable to a specific decision problem with a classification objective.	110
5.10. Example of a Description Logic (DL) query executed	112
6.1. Interaction between urban infrastructure, IoT monitoring, and the POI recommendation algorithm.	122
6.2. High-level architecture of the ABM	123
6.3. Causal loop diagram of behavioral dynamics and ICT-mediated in- tervention effects.	126
6.4. Modeling contributions class diagram.	132
6.5. Sequence diagram of the interaction between the Worker model and the Technological NPI	137
6.6. Infection dynamics by work scheme across simulation scenarios	143
6.7. Mean anxiety levels by work scheme across simulation scenarios . . .	144
6.8. Recreational activities performed by anxiety level across simulation scenarios (logarithmic scale)	145
7.1. State of the art in agent-based modeling of COVID-19	156
7.2. ABM overview	158
7.3. Conceptual architecture and feedback mechanisms underlying the proposed agent-based model	159

7.4.	Operationalization of deliberative vaccination decisions using Decision Field Theory	169
7.5.	Comparative dynamics of epidemiological outcomes and vaccination behavior across experimental scenarios	174
7.6.	Evolution of population-level intention and trust across experimental scenarios, grouped by the presence or absence of information-based interventions	175
7.7.	Distribution of vaccinated individuals by epidemiological state at the end of the simulation across all experimental scenarios	175
8.1.	Semantic-driven Monte Carlo Tree Search with Thompson Sampling .	201
9.1.	Deployment view of the proof-of-concept environment	211
9.2.	Overview of the simulation environment, the TOMGRO crop growth model, the evolution of the IoT system, and the semantic-driven re-configuration process.	212
9.3.	Semantic-driven reconfiguration decision trace	219
9.4.	Initial configuration of the simulation environment	220
9.5.	Simulation state after activation of the vision-based crop monitoring evolution module	221
9.6.	Simulation state after reconfiguration, showing the execution of the vision-based crop health classification service resulting from algorithm selection and service binding.	224
A.1.	O-DMIoT full diagram	232

List of Tables

3.1. Analytical comparison of representative state-of-the-art approaches and this thesis with respect to semantic modeling, decision representation, and adaptation mechanisms.	40
3.2. Analytical comparison of representative state-of-the-art approaches and this thesis with respect to evaluation practices, semantic traceability, and validation scope.	41
4.1. Decision analysis	47
5.1. Representative ontologies related to decision-making and algorithm characterization	82
5.2. Competency questions mapping	93
6.1. TDMIoT classification of the POI recommendation algorithm	130
6.2. Model parameters used across simulation scenarios. Parameters are grouped conceptually into epidemiological dynamics, vaccination process, and population structure models.	141
6.3. Comparative evaluation of simulation scenarios across epidemiological, behavioral, and socio-technical dimensions.	142
6.4. Application of the proposed QoE metric across the three simulation scenarios	148
7.1. Analysis by wave of infections. NPI - Information flow intervention regarding COVID-19 vaccination / PI - COVID-19 vaccine	172
7.2. Application of the proposed QoE metric across the eight simulation scenarios	178
8.1. Summary of A-Box individuals instantiated in O-DMIoT for the proof of concept	203
8.2. Problem instantiation and experimental configuration for the algorithm selection proof of concept	204
8.3. Summary of semantic-driven MCTS execution for the algorithm selection proof of concept	205
9.1. Initial system state prior to evolution	220
9.2. Semantic characterization of the vision-based crop monitoring requirement	222
9.3. System state after evolution activation	222
9.4. System state after completion of adaptive reconfiguration	224
9.5. Proof-of-concept configuration for the vision-based crop monitoring evolution module	225
A.1. Core classes of the O-DMIoT ontology	233

A.2. Representative Description Logic axioms of O-DMIoT 234

A.3. Mapping of competency questions to example Description Logic (DL)
queries executable in Protégé 235

1. Introduction

1.1. Context

The Internet of Things (IoT) has evolved into a cyber-physical ecosystem. Within it, diverse devices, software components, stakeholders, and ongoing data streams interact. This diversity makes IoT systems inherently dynamic. With that dynamism comes the challenge of ensuring functional correctness while maintaining performance and quality of service throughout execution. Recent systematic reviews emphasize that IoT applications must operate in the face of uncertainty and constant change, underscoring the importance of runtime adaptation [5, 1].

Software solutions that rely on fixed configurations and lack adaptability are unlikely to succeed in future IoT environments, especially in domains such as smart cities, healthcare, and agriculture. Robust decision-making is required to manage shifting contexts and unpredictable conditions [10]. Without adaptive algorithms, protocols, and services, performance, robustness, and scalability suffer as operational conditions change. Autonomic computing addresses this challenge, serving as a foundational paradigm for managing complexity in distributed environments. Autonomic approaches combine monitoring, analysis, and self-optimization, enabling systems to adjust to contextual changes. They use predictive models and learning mechanisms to tune parameters and optimize decisions at runtime [8, 12]. These principles increasingly shape IoT architectures, supporting adaptive communication protocols, service orchestration, and resource allocation.

A central challenge lies in the reliance of IoT systems on decision-making algorithms whose behavior is highly sensitive to contextual factors such as data quality, human activity patterns, and domain-specific uncertainties. Algorithms defined at design time may become suboptimal or even fail when conditions change, leading to misalignment between system behavior and stakeholder requirements. Although research on self-adaptive IoT architectures has advanced architectural and MAPE-loop-based solutions [2, 14, 6, 9, 4, 3], recent reviews highlight that adaptation mechanisms remain primarily focused on infrastructure layers, parameter tuning, or domain-specific optimizations [5, 7]. Less attention has been given to modular or high-level mechanisms capable of reconfiguring the services that provide algorithmic decision-making.

Semantic representations can address this gap by enabling clear descriptions of functional requirements, service capabilities, constraints, and decision logic in machine-interpretable knowledge bases. Prior work shows that semantic models support automated reasoning and adaptation in dynamic IoT environments [11, 13]. Yet, integration with autonomic self-configuration is underexplored. Combining semantic representation with runtime adaptation enables adjusting not only system parameters but also algorithmic logic.

To address these challenges and bridge the gap between infrastructure-level adaptation and high-level algorithmic flexibility, this thesis proposes an adaptation mech-

anism grounded in semantic modeling and autonomic computing. By building on the need for clear functional representations and dynamic reconfiguration, the approach enables the runtime adaptation of IoT services that provide decision-making algorithms. The primary objective is to enhance adaptability across heterogeneous application domains by ensuring ongoing alignment between deployed algorithms and evolving functional requirements. In this work, functional requirements are formalized as semantic specifications encompassing decision objectives, constraints, and contextual assumptions that directly affect the suitability of the algorithm. Rather than serving merely as documentation, the semantic model functions as an executable knowledge base, enabling reasoning-driven adaptation. Reconfiguration is triggered by detected changes in requirements or contextual conditions that undermine the suitability of the current algorithm. The proposed approach emphasizes service-level and algorithm-level reconfiguration, specifically algorithm selection and parameterization, excluding dynamic code synthesis and service orchestration.

1.2. Problem statement

IoT is a technology with broad domain applicability, where each domain presents unique characteristics, operational conditions, and sources of uncertainty. Despite these differences, many IoT application domains rely on similar classes of decision-making algorithms, typically defined at design time and fixed throughout system operation.

This work addresses the problem of adapting an IoT system to changes in functional requirements. Such changes may arise from the system's evolution or emergent phenomena resulting from its interactions with other systems (physical, social, and technological). In particular, this work focuses on adapting a decision-making algorithm within the context of an IoT system, recognizing its central role in driving system behavior and ensuring alignment with stakeholder needs as conditions evolve. This study is particularly relevant to smart cities, healthcare, and agriculture IoT systems, where dynamic adaptation is crucial due to the varying operational conditions and stakeholder requirements. By narrowing the focus to these domains, the research aims to present more targeted insights and solutions applicable to other similar IoT environments.

1.3. Motivation

The IoT is a transformative technology connecting a vast network of physical and virtual elements, integrating diverse devices, applications, and platforms. However, the increasing complexity and scale of IoT systems introduce design, deployment, and maintenance challenges not typically encountered in traditional software development. This thesis is motivated by the pressing need for adaptable mechanisms that address these challenges and ensure IoT systems remain robust, efficient, and responsive as they evolve in demanding real-world contexts.

Meeting the requirements of a wide range of stakeholders further complicates IoT system design. Inherent IoT characteristics such as dynamism, heterogeneity, scale, and interoperability heighten the need for sophisticated, adaptive architectures. IoT systems may operate at large scales or in remote, unpredictable environments, inter-

acting with other systems and producing emergent behaviors. These realities create a significant research gap: it is crucial to identify and implement runtime adaptation strategies that optimize system performance and resilience, especially those that address changes in the algorithmic logic driving system behavior. This thesis responds directly to these challenges.

1.4. Hypothesis

As IoT systems scale, the ability to adapt their behaviors at runtime becomes essential. This adaptation must respond effectively to changes in the surrounding physical environment and the underlying computational infrastructure while minimizing manual intervention. Autonomic computing, through its self-management property, offers a promising foundation for addressing this challenge. Under this approach, the following working hypotheses guide the present thesis:

- The self-configuration property of autonomic computing enables dynamic re-configuration of services within a deployed IoT system during runtime, addressing evolving requirements and contextual changes.
- A service provides specific functionality and thus delivers the appropriate granularity for exploring the self-configuration of an IoT system.
- A specification expressed through a semantic knowledge base enables the precise representation of functional requirements; likewise, available services can be semantically described and linked to these requirements.
- As IoT systems interact with dynamic environments, their behavior relies on explicit decision-making processes. These processes are implemented through decision-making algorithms provided as services. Consequently, reconfiguring these services enables the system to adapt its behavior to evolving contexts and requirements.

1.5. Research objectives

1.5.1. General objective

Develop a reconfiguration mechanism for IoT that adjusts services that provide decision-making algorithms to improve adaptability to changes in functional requirements across different application domains.

1.5.2. Specific objectives

- Analyze at a meta-level the decision-making algorithms used in the different IoT domains and their associated requirements.
- Conceptualize a semantic model that describes decision-making algorithms in the IoT.
- Build a process for selecting decision-making algorithms based on a semantic model.

- Map the elements that make up the adaptation mechanism to a reference architecture.
- Integrate agent-based modeling as an evaluation strategy.

1.6. Structure

The present thesis is organized into ten chapters. Each supports the objective of enabling adaptability to changes in the functional requirements of decision-making algorithms through semantic modeling. The chapters are grouped into three main research contributions that build the conceptual, evaluative, and operational foundations of the proposed approach.

Chapter 2 presents the theoretical framework and key concepts. Chapter 3 provides an overview of autonomic computing in IoT systems and summarizes how semantic models have been employed to support adaptive behavior. Chapters 4 to 9 develop the core components of the thesis: semantic foundations, human-centric evaluation, and runtime adaptive mechanisms. Finally, Chapter 10 synthesizes the findings derived from the three contributions, discusses the limitations of the proposed approach, and outlines promising directions for future research. Together, these chapters consolidate the contribution of this thesis and outline a path forward for semantic-driven, human-centric, and autonomic IoT systems.

1.6.1. Contribution 1 — Semantic Foundations for Algorithm Selection in IoT

Contribution 1 establishes the conceptual and formal basis for representing decision-making in IoT systems using semantic models. It begins with a theoretical classification of decision problems and algorithmic approaches and culminates in a formal ontology that enables machine-interpretable reasoning over IoT decision processes.

- **Chapter 4: A Taxonomy for Decision-Making in IoT Systems (TD-MIoT)**

This chapter proposes a high-level taxonomy that conceptualizes decision-making as an association between problem types and algorithmic solutions. It systematizes decision-making across IoT application domains, identifies common algorithmic approaches, and exposes design biases that influence solution selection.

- **Chapter 5: Ontology for Decision-Making in the IoT (O-DMIoT)**

Building on the taxonomy, this chapter formalizes a semantic model that captures the relationships among problems, algorithmic strategies, design elements, and service descriptions. O-DMIoT enables explicit reasoning about algorithm suitability and serves as the core knowledge representation throughout the thesis.

Together, these chapters form the semantic foundation required for algorithm selection and later serve as the basis for service reconfiguration.

1.6.2. Contribution 2 — Human-Centric Evaluation Framework

Contribution 2 introduces a human-centric evaluation perspective that incorporates human behavior, trust dynamics, and societal responses into the assessment of algorithm performance. Through agent-based simulations, it demonstrates how decision algorithms, classified and semantically represented using the artefacts introduced in previous chapters, affect stakeholders under realistic constraints. This evaluation provides insights into the design of fairer, more effective, and more robust algorithms in social contexts. In this thesis, agent-based modeling is used as an external evaluation mechanism to assess the impact of reconfiguration decisions on human-centered Quality of Experience (QoE) metrics.

- **Chapter 6: Agent-based evaluation of information and communication technology interventions**

This chapter proposes an agent-based model (ABM) to assess IoT-mediated decision interventions during the COVID-19 pandemic. It introduces qualitative metrics—such as anxiety reduction and behavioral responses to assess the technology impact on well-being and establish a notion of qualitative Quality of Experience (QoE) metrics.

- **Chapter 7: Agent-Based Analysis of Trust and Adoption Dynamics Across Populations**

In the context of the ABM approach, this chapter analyzes how trust, cultural factors, and social interaction patterns affect vaccine adoption. The results highlight the importance of integrating human-centric metrics—beyond purely technical performance—into the evaluation of algorithms and services.

These chapters validate and enrich the semantic framework by showing that the real-world consequences of algorithmic decisions create a feedback loop: human behavior influences how algorithms are selected and how services adapt their behavior.

1.6.3. Contribution 3 — Semantic-Driven Algorithm Selection and Adaptive Service Reconfiguration

Contribution 3 operationalizes the semantic and evaluative foundations into a complete runtime mechanism for algorithm selection and service reconfiguration in IoT environments.

- **Chapter 8: Algorithm Selection Based on a Semantic Model for Decision-Making in the IoT**

This chapter proposes a runtime algorithm-selection strategy guided by O-DMIoT. The system implements a search mechanism using Monte Carlo Tree Search (MCTS) that leverages semantic reasoning and qualitative evaluation metrics. This mechanism establishes the first operational layer of adaptive decision-making.

• Chapter 9: Adaptive Reconfiguration of IoT Services

This chapter integrates the whole pipeline—problem characterization, semantic reasoning, algorithm selection, and evaluation—into a working reconfiguration mechanism deployed in a controlled agricultural environment. The case study demonstrates how semantic models and adaptive strategies improve responsiveness and operational performance in real IoT settings.

Figure 1.1 presents an overview of the thesis structure and illustrates how the research objectives are operationalized through the proposed contributions, in alignment with the Monitor–Analyze–Plan–Execute over Knowledge (MAPE-K) autonomic control loop. The objectives of analyzing decision-making algorithms across IoT domains and conceptualizing a semantic model are addressed through the taxonomy and ontology developed in Contribution 1, which populate the knowledge and analysis phases of the loop. The objective of developing a semantic-based algorithm selection process is fulfilled in Contribution 3, where a planning mechanism based on Monte Carlo Tree Search implements the plan phase using semantic reasoning. The integration of agent-based modeling as an evaluation strategy, as described in Contribution 2, supports the monitoring and analysis of human-centric Quality of Experience metrics. Finally, the general objective of enabling adaptive reconfiguration is accomplished by executing the selected decisions at the service level, thereby closing the autonomic loop through a proof of concept in controlled agriculture. Collectively, these contributions illustrate the interconnection and progressive realization of the research objectives within a unified self-adaptive IoT framework.

Chapter References

- [1] I. Alfonso et al. “Self-adaptive architectures in IoT systems: a systematic literature review”. In: *Journal of Internet Services and Applications* 12.1 (Dec. 2021), p. 14. URL: <https://jisajournal.springeropen.com/articles/10.1186/s13174-021-00145-8> (visited on 11/28/2025).
- [2] Q. M. Ashraf et al. “Autonomic protocol and architecture for devices in Internet of Things”. In: *2014 IEEE Innovative Smart Grid Technologies - Asia (ISGT ASIA)*. 2014, pp. 737–742.
- [3] R. Ben Halima et al. “MAPE-K patterns for self-adaptation in cyber-physical systems”. en. In: *J. Supercomput.* 79.5 (Mar. 2023), pp. 4917–4943.
- [4] A. Bucchiarone et al. “A MAPE-K Approach to Autonomic Microservices”. In: *2022 IEEE 19th International Conference on Software Architecture Companion (ICSA-C)*. 2022, pp. 100–103.
- [5] L. Erazo-Garzón et al. “Self-adaptive Internet of Things Systems: A Systematic Literature Review”. In: *International Conference on Applied Technologies*. Ed. by M. Botto-Tobar et al. Vol. 2051. Series Title: Communications in Computer and Information Science. Cham: Springer Nature Switzerland, 2024, pp. 137–157. URL: https://link.springer.com/10.1007/978-3-031-58950-8_11 (visited on 11/28/2025).

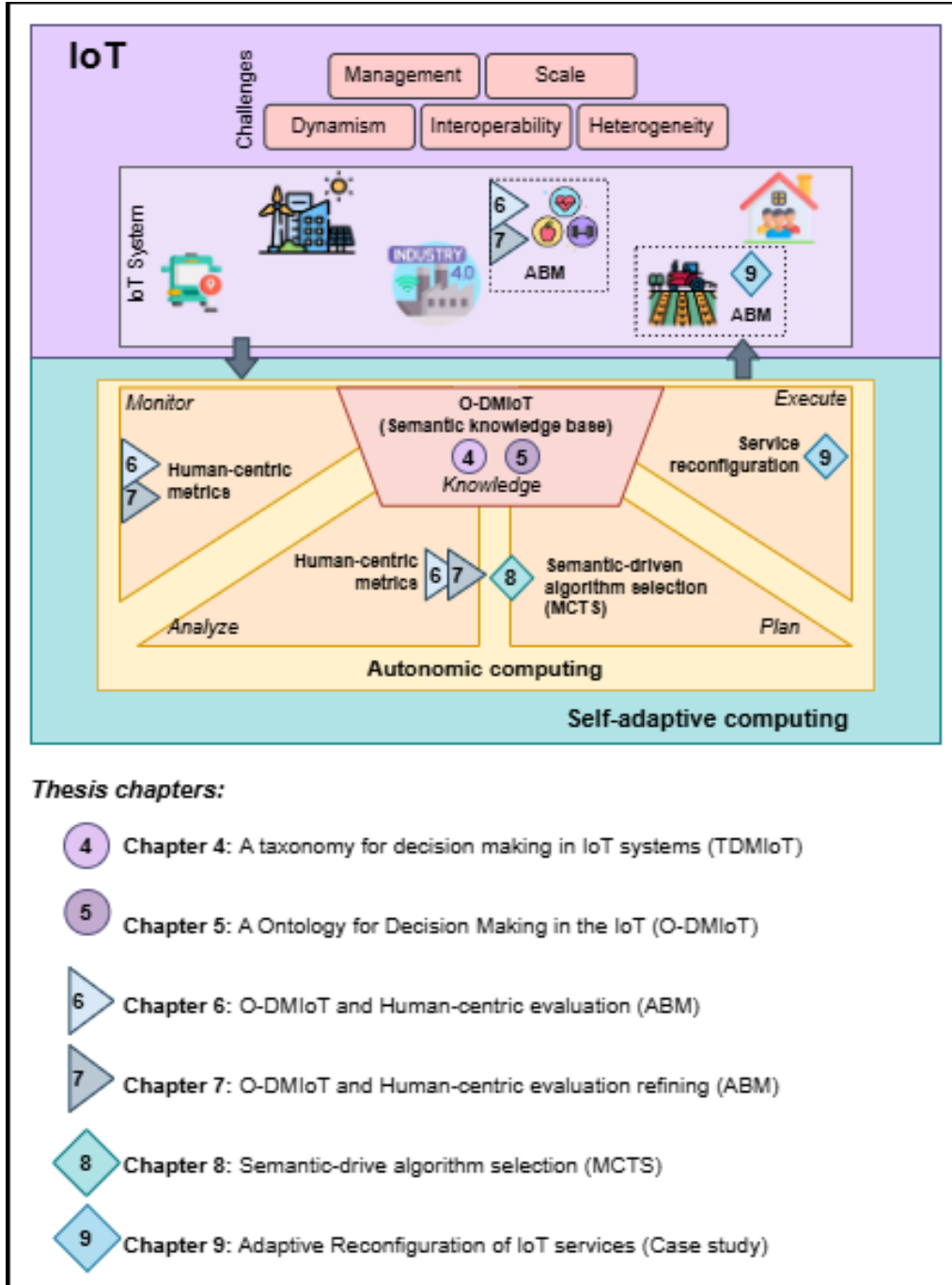


Figure 1.1.: **Thesis structure mapped to the MAPE-K autonomic control loop.** The figure illustrates how each major contribution of this thesis is positioned within the Monitor–Analyze–Plan–Execute over Knowledge (MAPE-K) model for self-adaptive computing in IoT systems. The semantic foundations, represented through the O-DMIoT ontology, populate the knowledge, analysis, and planning phases. The algorithm selection contribution operationalizes the planning phase via a Monte Carlo Tree Search (MCTS) mechanism guided by semantic reasoning. Human-centric evaluation supports monitoring and analysis through qualitative metrics, while adaptive service reconfiguration executes the selected decisions, closing the autonomic control loop across heterogeneous IoT application domains.

- [6] A. Jaleel et al. “Autonomic interoperability manager: A service-oriented architecture for full-stack interoperability in the Internet-of-Things”. In: *ICT Express* 8.4 (2022), pp. 507–512. URL: <https://www.sciencedirect.com/science/article/pii/S2405959521001491>.
- [7] E. Jawabreh and A. Taweel. “Self-adaptation in Microservice Systems: A Literature Review”. In: *Service-Oriented Computing – ICSOC 2024 Workshops*. Ed. by S. Kallel et al. Vol. 15833. Series Title: Lecture Notes in Computer Science. Singapore: Springer Nature Singapore, 2026, pp. 28–40. URL: https://link.springer.com/10.1007/978-981-96-7238-7_3 (visited on 11/28/2025).
- [8] J. O. Kephart and D. M. Chess. “The vision of autonomic computing”. In: *Computer* 36.1 (2003), pp. 41–50.
- [9] A. Mitra et al. “Self-Adaptive IoT protocol for Dynamic Smart City Environment”. In: *International Journal of Sciences and Innovation Engineering* 2.6 (June 24, 2025), pp. 857–868. URL: <https://www.ijsci.com/index.php/home/article/view/305> (visited on 11/28/2025).
- [10] B. C. Nogueira et al. “Self-Adaptation in IoT Systems for Smart Cities”. In: *2023 Symposium on Internet of Things (SIoT)*. 2023 Symposium on Internet of Things (SIoT). Oct. 2023, pp. 1–5. URL: <https://ieeexplore.ieee.org/document/10390083> (visited on 11/28/2025).
- [11] A. Paul et al. “Deep Learning-Based Semantic Interaction Network: Advancing IoT Data Modeling for Interoperability”. In: *2024 IEEE International Conference on Communications Workshops (ICC Workshops)*. IEEE, June 2024, pp. 135–140. URL: <http://dx.doi.org/10.1109/ICCWorkshops59551.2024.10615419>.
- [12] R. R. Achary, and P. M. Paul. “Intelligent Autonomic Computing: Achieving Self-Optimization Through Autonomous Learning”. In: *2025 3rd International Conference on Inventive Computing and Informatics (ICICI)*. 2025 3rd International Conference on Inventive Computing and Informatics (ICICI). June 2025, pp. 1463–1468. URL: <https://ieeexplore.ieee.org/document/11069688> (visited on 11/28/2025).
- [13] R. Ranpara. “A semantic and ontology-based framework for enhancing interoperability and automation in IoT systems”. In: *Discover Internet of Things* 5.1 (Mar. 2025). URL: <http://dx.doi.org/10.1007/s43926-025-00122-8>.
- [14] L. E. Villela Zavala, A. Ordoñez García, and M. Siller. “Architecture and Algorithm for IoT Autonomic Network Management”. In: *2019 International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*. 2019, pp. 861–867.

2. Theoretical Foundations

2.1. Self-configuration in autonomig computing

In the context of autonomic computing, self-configuration refers to the ability of a system to adapt to changes in its environment. The self-configuration process enables a system to automatically update any missing or obsolete components based on error/alert messages generated by a monitoring system [9]. Autonomous systems are configured automatically following high-level policies that delineate the desired outcome rather than the means of achieving it.

2.2. Ontology

The concept of ontology was first employed to conceptualize a portion of the real world. Historically, the first objective and use of an ontology was to enable a software system to reason about entities in the real world. A second objective was to provide a unified conceptualization of a domain that disparate agents accept. In this aspect, ontologies generate considerable interest across different fields, including information systems design, systems integration and interoperation, natural language processing, and information retrieval [18]. An ontology comprises two key elements:

1. A representative vocabulary comprising different symbols (class names, relationship names, etc.).
2. A set of definitions that specify the meaning of the vocabulary.

2.2.1. Domain ontology

A domain ontology encompasses the terminology, classification, taxonomy, relationships (including hierarchies and constraints), and domain axioms. An ontology must be machine-readable [8]. It should be noted, however, that an ontology is not a program that can be executed. It serves to represent knowledge in a declarative manner, thereby providing a foundation for the utilization of programs. An ontology should provide a robust and enduring "knowledge skeleton" to which all other knowledge should adhere.

According to [1] a domain ontology can be defined as an structure $:= (C, \leq_C, R, \sigma, \leq_R, IR)$ where:

- C is a set of concepts
- R is a set of relationships
- $\leq_C$ is a partial orden on C
- $\sigma \rightarrow C \times C$ is function called seignature

- $\leq_R$ is a partial order on R called relationship
- IR is a set on inference rules expresses in a language L

2.2.2. Domain ontology

A domain ontology captures the terminology, classifications, taxonomies, relationships (including hierarchies and constraints), and axioms relevant to a specific domain of discourse. Ontologies must be machine-readable and formally specified [8]. However, an ontology is not an executable artifact; it represents domain knowledge in a declarative form, providing a formal foundation for computational systems to reason and operate. In this context, a domain ontology establishes a stable "knowledge skeleton" that supports consistency, reuse, and interoperability across systems.

Foundational works in computer science [10, 11] formally characterize a domain ontology as a logical theory that explicitly specifies a conceptualization of a domain.

Let:

- $\mathcal{C}$ denote a conceptualization of the domain, understood as an abstract and intensional structure that identifies the relevant entities and relations independently of any particular representation;
- L be a formal language equipped with a well-defined syntax and model-theoretic semantics;
- $\text{Ax}(L)$ denote the set of all well-formed axioms expressible in L .

A domain ontology $\mathcal{O}$ is defined as a finite set of axioms such that:

$$\mathcal{O} \subseteq \text{Ax}(L), \quad (2.1)$$

The axioms in $\mathcal{O}$ provide an explicit and partial specification of the conceptualization $\mathcal{C}$. The intended models of $\mathcal{O}$ must remain consistent with the domain interpretation associated with $\mathcal{C}$ and must be independent of specific reasoning procedures or implementation choices.

In foundational ontology theory, the notion of conceptualization is intentionally left abstract and is not formalized as a mathematical structure. This is because any ontology can only provide a partial and approximate specification of domain meaning.

2.3. The algorithm selection problem

The algorithm selection problem proposed by Rice [17] consists of finding the most suitable algorithm for a specific problem. The classical algorithm selection problem is defined by a triple $\langle X, A, P \rangle$, where:

- X represents a set of problems
- A represents a set of algorithms
- $P : A \times X \rightarrow \mathbb{R}$ represents a quality function of the algorithm to solve a problem.

The objective is to determine a mapping $\alpha : X \rightarrow A$ that associates each problem with an algorithm.

2.3.1. The extended algorithm selection framework

At [12], an extension of the algorithm selection problem is proposed. In this extension, an additional feature space, designated as F , characterizes problems. Similarly, a feature space, denoted as G , extracts algorithm characteristics. Subsequently, the selection mapping is a function of both problem and algorithm features. The revised problem formulation is as follows: The objective is to identify an optimal algorithm $\alpha \in A$, selected via the mapping $S(f(x), g(a))$, that maximizes the performance mapping $p(a(x)) \in P$, given a problem $x \in X$ characterized by $f(x) \in F$ and algorithms $a \in A$ characterized by $g(a) \in G$.

2.4. Reinforcement learning

Reinforcement learning is a machine learning branch that aims to develop an understanding of how to map situations to actions in order to maximize a numerical reward signal. The reinforcement learning framework entails the continuous interaction of an agent with an environment, enabling the agent to achieve a desired outcome despite the presence of uncertainties. The objective is accomplished through the execution of actions that are not explicitly conveyed to the agent. In contrast, the agent identifies which actions result in a greater reward. In this regard, one of the challenges inherent to reinforcement learning is the trade-off between *exploration* and *exploitation*. The term *exploration* is used to describe the process of interacting with the environment in order to gain insight into the potential rewards that may be obtained. The term *exploitation* is used to describe the utilisation of the available information. It is essential that the agent leverages these tasks in order to achieve its desired outcome. A fundamental aspect of reinforcement learning is the acknowledgment that past actions exert an influence on future states, thereby encompassing the problem in a comprehensive manner. [22]

2.4.1. Markov decision process

A Markov decision process (MDP) models a sequential decision process, where a system evolves and is controlled. The dynamics follow a probabilistic transition function Φ that maps states s and actions a to new states s . With each action, a reward r is received [21]. The main characteristic of a Markov decision process is that the previous state has a direct effect on the action selection.

Formally, an MDP is a tuple $M = \langle S, A, \Phi, R \rangle$, where:

- S represents a set of states
- A refers to a set of actions
- $\Phi : S \times A \times S$ is a transition function
- $R : S \times A$ is a reward function.

MDPs are a mathematically idealized form of the reinforcement learning problem.

2.5. Agent-based models (ABM)

Agent-based modeling and simulation represents a computational approach to modeling systems composed of autonomous elements that interact with one another. Agent-based models permit the construction of a complex system and the simulation of its dynamic evolution over time [16, 5].

By modeling agents individually, one can observe the full effects of the diversity that exists among agents in their attributes and behaviors, as this diversity gives rise to the behavior of the system as a whole. Therefore, this modeling approach is especially relevant in contexts where human behavior and interactions play a central role in shaping system outcomes.

2.6. Functional requirements

Functional requirements specify the expected behavior of a system by defining the actions the system should perform in response to specific inputs or situations. In software engineering, functional requirements describe the services, functions, or tasks that a system must provide, as well as the conditions under which these functions are executed [20]. These requirements are distinguished from non-functional requirements, which constrain how the system performs its functions by specifying quality attributes such as performance, reliability, or usability, rather than introducing new system behavior [13]. Consequently, functional requirements capture the primary objectives that guide system behavior and decision-making. In dynamic and adaptive systems, functional requirements may evolve in response to changes in the operational environment, system configuration, or stakeholder needs. This evolution necessitates decision mechanisms capable of reassessing and adjusting system behavior at runtime. From a systems engineering perspective, functional requirements serve as the principal link between high-level system goals and operational decisions, enabling adaptive systems to reason about alternative actions while preserving functional correctness under changing conditions [15].

2.7. Metrics

Metrics function as formal instruments for evaluating, comparing, and analyzing the behavior and outcomes of computational systems. In adaptive and decision-making systems, metrics are crucial for determining whether system behavior aligns with specified objectives and requirements. Rather than prescribing system behavior, metrics establish a basis for observing, quantifying, and interpreting the effects of decisions and adaptations.

From a theoretical perspective, metrics are defined as mappings from observed attributes or behaviors of real-world entities to numerical values that reflect the degree to which specified properties or criteria are satisfied, in accordance with measurement theory [6]. These criteria may derive from functional requirements specifying expected system behavior, from performance considerations typically expressed as non-functional requirements, or from high-level stakeholder objectives refined into evaluable requirements during the requirements engineering process [20]. Consequently, metrics serve as a vital connection between abstract requirements and con-

create system behavior, enabling the evaluation and comparison of alternative system configurations or decision outcomes.

In adaptive systems, metrics are particularly important because system configurations and decision mechanisms may evolve over time. Metrics facilitate the continuous evaluation of these changes, support comparisons among alternative configurations, and inform adaptation decisions. Multiple types of metrics are often required to capture diverse aspects of system behavior, such as technical performance, quality of service, and achievement of functional objectives. Therefore, the selection of appropriate metrics is a foundational component in the design and evaluation of adaptive decision-making systems.

2.7.1. Algorithm characteristics

Algorithm characteristics denote intrinsic properties of algorithmic designs that are independent of specific execution contexts or application domains. These characteristics provide a foundation for comparing alternative algorithms and assessing their suitability for particular problem settings.

Temporal and spatial complexity represent fundamental algorithm characteristics. Temporal complexity quantifies how an algorithm's execution time scales with input size, while spatial complexity measures the memory resources required during execution. These properties are commonly expressed using asymptotic notation, such as Big-O notation, which abstracts away constant factors and implementation details to highlight scalability [4].

In decision-making systems, algorithm characteristics such as computational complexity are particularly relevant when algorithms must operate under resource constraints or within dynamic environments. Explicit identification of these characteristics enables algorithm selection and configuration decisions to account for both functional correctness and considerations of feasibility and efficiency.

2.7.2. Quality of Service

Quality of Service (QoS) encompasses measurable, system-oriented properties that characterize the performance and reliability of a system or service during operation. QoS metrics are typically objective and quantify parameters such as response time, throughput, availability, and resource utilization.

In software and service-oriented systems, QoS provides a framework for evaluating the effectiveness with which a system delivers its intended functionality under specified operational conditions. Standards such as ISO/IEC 25010 define software quality by the extent to which a system fulfills stated and implied needs, and further decompose quality into characteristics that can be assessed using appropriate metrics [23].

In adaptive systems, QoS metrics are essential for monitoring runtime behavior and evaluating the impact of configuration or decision changes [3, 14]. By providing quantitative indicators of system performance, QoS enables comparisons among alternative system configurations and supports evidence-based adaptation decisions.

2.7.3. Quality of Experience

Quality of Experience (QoE) extends system-level evaluation by focusing on the perceptions and experiences of users interacting with a system. In contrast to QoS, which measures objective technical properties at the network and application levels, QoE captures how users perceive and value a system's properties. Foundational studies define QoE as a perceptual layer above traditional QoS mechanisms, representing the user's overall experience with the system's service [19].

Within the QoE framework, user experience is influenced by both application-level and network-level QoS factors, as well as by subjective elements such as satisfaction, interest, and expectations. Since these subjective aspects are not fully represented by technical performance metrics, explicit QoE modeling is required [7]. Ontology-based representations have been developed to formalize QoE concepts, enabling the integration of objective QoS metrics, user feedback, and adaptation mechanisms within a unified conceptual framework.

Recent research has sought to unify definitions of QoE across domains, characterizing it as the degree of delight or annoyance experienced by a user under specific conditions [2]. Collectively, these perspectives establish QoE as a complementary evaluation dimension that captures the human-centered impact of system behavior and informs the assessment of adaptive and socio-technical systems.

References

- [1] Alexander Maedche et al. *SEAL-II - The Soft Spot between Richly Structured and Unstructured Knowledge*. URL: <https://lib.jucs.org/article/27804/> (visited on 08/16/2024).
- [2] P. L. Callet et al. *Qualinet White Paper on Definitions of Quality of Experience*. Tech. rep. Qualinet (www.qualinet.eu), Mar. 2013. URL: <https://hal.science/hal-04638470> (visited on 01/31/2026).
- [3] B. H. C. Cheng et al. "Software Engineering for Self-Adaptive Systems: A Research Roadmap". In: *Software Engineering for Self-Adaptive Systems*. Springer, 2009, pp. 1–26.
- [4] T. H. Cormen, ed. *Introduction to algorithms*. 3rd ed. OCLC: ocn311310321. Cambridge, Mass: MIT Press, 2009, p. 359.
- [5] A. T. Crooks and A. J. Heppenstall. "Introduction to Agent-Based Modelling". en. In: *Agent-Based Models of Geographical Systems*. Ed. by A. J. Heppenstall et al. Dordrecht: Springer Netherlands, 2012, pp. 85–105. URL: https://doi.org/10.1007/978-90-481-8927-4_5 (visited on 08/16/2024).
- [6] N. E. Fenton and S. L. Pfleeger. *Software Metrics: A Rigorous and Practical Approach*. 3rd. CRC Press, 2014.
- [7] E. Gallo, M. Siller, and J. Woods. "An Ontology for the Quality of Experience framework". In: *2007 IEEE International Conference on Systems, Man and Cybernetics*. ISSN: 1062-922X. Oct. 2007, pp. 1540–1544. URL: <https://ieeexplore.ieee.org/document/4414109> (visited on 01/31/2026).

- [8] D. Gašević, D. Djuric, and V. Devedžić. “Ontologies”. In: *Model Driven Engineering and Ontology Development*. Springer Berlin Heidelberg, 2009, pp. 45–80. URL: http://dx.doi.org/10.1007/978-3-642-00282-3_2.
- [9] S. S. Gill et al. “AI for next generation computing: Emerging trends and future directions”. en. In: *Internet of Things* 19 (Aug. 2022), p. 100514. URL: <https://linkinghub.elsevier.com/retrieve/pii/S254266052200018X> (visited on 06/12/2023).
- [10] T. R. Gruber. “A Translation Approach to Portable Ontology Specifications”. In: *Knowledge Acquisition* 5.2 (1993), pp. 199–220.
- [11] N. Guarino, D. Oberle, and S. Staab. “What Is an Ontology?” In: *Handbook on Ontologies*. Ed. by S. Staab and R. Studer. Springer, 2009, pp. 1–17.
- [12] M. Hilario et al. “Ontology-Based Meta-Mining of Knowledge Discovery Workflows”. en. In: *Meta-Learning in Computational Intelligence*. Ed. by N. Jankowski, W. Duch, and K. Grabczewski. Studies in Computational Intelligence. Berlin, Heidelberg: Springer, 2011, pp. 273–315. URL: https://doi.org/10.1007/978-3-642-20980-2_9 (visited on 05/02/2023).
- [13] *IEEE Recommended Practice for Software Requirements Specifications*. IEEE, 1998.
- [14] J. O. Kephart and D. M. Chess. “The vision of autonomic computing”. In: *Computer* 36.1 (2003), pp. 41–50.
- [15] A. Kossiakoff et al. *Systems Engineering: Principles and Practice*. 2nd. Hoboken, NJ: Wiley, 2015.
- [16] C. M. Macal and M. J. North. “AGENT-BASED MODELING AND SIMULATION”. en. In: ().
- [17] J. R. Rice. “The Algorithm Selection Problem”. en. In: *Advances in Computers*. Vol. 15. Elsevier, 1976, pp. 65–118. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0065245808605203> (visited on 01/09/2023).
- [18] C. Roussey et al. “An Introduction to Ontologies and Ontology Engineering”. en. In: *Ontologies in Urban Development Projects*. Ed. by G. Falquet et al. Advanced Information and Knowledge Processing. London: Springer, 2011, pp. 9–38. URL: https://doi.org/10.1007/978-0-85729-724-2_2 (visited on 09/08/2021).
- [19] M. Siller. “QoS arbitration for improving the QoE in multimedia transmission”. en. In: *International Conference on Visual Information Engineering (VIE 2003). Ideas, Applications, Experience*. Vol. 2003. Guildford, UK: IEE, 2003, pp. 238–241. URL: https://digital-library.theiet.org/content/conferences/10.1049/cp_20030531 (visited on 01/31/2026).
- [20] I. Sommerville. *Software Engineering*. 10th. Boston, MA: Pearson Education, 2016.
- [21] L. E. Sucar. “Markov Decision Processes”. In: *Probabilistic Graphical Models*. Book Authors: _:n3863 Series Title: Advances in Computer Vision and Pattern Recognition. London: Springer London, 2015, pp. 199–216. URL: http://link.springer.com/10.1007/978-1-4471-6699-3_11 (visited on 06/09/2021).

- [22] R. S. Sutton and A. G. Barto. *Reinforcement learning: an introduction*. Adaptive computation and machine learning. Cambridge, Mass: MIT Press, 1998.
- [23] *Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – System and Software Quality Models*. ISO/IEC, 2011.

3. Semantic-Driven Decision-Making and Adaptive Reconfiguration in IoT Systems

Internet of Things (IoT) systems operate in dynamic, heterogeneous, and context-sensitive environments, where static configurations are insufficient to cope with evolving requirements, changing environmental conditions, and diverse stakeholder interactions. Consequently, research has increasingly emphasized autonomous decision-making and adaptive reconfiguration mechanisms that allow IoT systems to adjust their behavior at runtime.

The state of the art spans several interrelated research directions. Semantic technologies provide machine-interpretable representations of domain knowledge, context, and constraints. Autonomic computing frameworks introduce architectural mechanisms for monitoring and adaptation through control loops. Decision-theoretic and learning-based methods address uncertainty, optimization, and long-term adaptation. Although these approaches are often examined independently, adaptive IoT systems integrate them to achieve robust adaptation.

In this regard, this chapter reviews the literature using analytical dimensions that reflect common design and evaluation practices in adaptive IoT systems. It specifically examines semantic representations of system knowledge, mechanisms for decision and algorithm selection, adaptation and reconfiguration architectures, evaluation practices, and claims of generalizability. This structure enables a systematic analysis of how existing approaches support adaptive decision-making.

3.1. Semantic Modeling for Decision-Making in IoT

Semantic modeling establishes a foundational layer in adaptive IoT systems by enabling interoperability and reasoning across heterogeneous components. This is achieved by abstracting low-level sensing and communication details. Semantic models are primarily employed to represent devices, observations, contextual information, and system relationships in a machine-interpretable format.

Early research emphasized the semantic description of sensors and observations to support data integration and interoperability across IoT platforms. Subsequent studies have expanded semantic models to improve situation awareness and decision-making by enriching contextual representations and enabling reasoning over system state [13, 10]. For example, [9] presents a context management framework that utilizes semantic models to capture contextual state and apply probabilistic reasoning for adaptation under uncertainty. Similarly, [2] introduces a multilevel semantic framework that integrates ontologies with probabilistic models to enable predictive

decision-making based on inferred contextual situations. These approaches employ semantic representations to provide structured descriptions of environmental and system states. In these works, decision support is derived indirectly through inference mechanisms operating over semantically grounded variables.

3.2. Ontology-Based Decision Models and Algorithm Mapping

Beyond context representation, several studies examine the use of semantic models to support algorithm selection or configuration in adaptive systems. In these approaches, semantic representations describe system conditions, goals, or constraints, which are subsequently associated with specific algorithms or control strategies through predefined mappings.

In [7], a semantic model is employed to select among alternative feedback control schemes based on system conditions, enabling runtime controller selection within a control-oriented application domain. Similarly, [8] applies ontology-based knowledge modeling to support self-organized reconfiguration in cyber-physical production systems, where production goals and system states are semantically represented and linked to reconfiguration strategies. Domain-specific approaches further integrate semantic models with multicriteria decision-making techniques, as illustrated by [1] in smart manufacturing environments, where ontologies structure decision criteria used to rank and select reconfiguration options. Together, these studies demonstrate that semantic knowledge can guide algorithmic decisions by establishing explicit mappings between domain-specific semantic descriptions and algorithmic or strategic alternatives within a given application context.

In these approaches, the logic for algorithm or strategy selection is embedded within control rules, planning mechanisms, or domain-specific decision workflows. Semantic representations inform the selection process by constraining or contextualizing available alternatives; however, they do not provide an explicit, reusable characterization of the decision problem itself. Consequently, decision formulations remain closely tied to specific application domains and predefined sets of alternatives, rather than being articulated through general, domain-independent semantic models of decision problems. This lack of domain-independent models is a critical limitation as it restricts the scalability and adaptability of solutions across different IoT environments.

3.3. Autonomic Computing and Adaptive Reconfiguration

Autonomic computing provides the architectural foundation for self-managing IoT systems and is commonly structured around the MAPE-K (Monitor – Analyze – Plan – Execute – Knowledge) control loop. Within this paradigm, autonomic principles are widely applied in service-based and industrial IoT systems to support runtime orchestration and reconfiguration through continuous monitoring and control actions [4, 6].

To enable these adaptive capabilities, semantic technologies are often integrated to enhance the knowledge component of the control loop. Semantic models offer machine-interpretable representations of system state, context, and adaptation policies, facilitating context interpretation during monitoring and supporting reasoning during analysis and planning phases [11, 9, 2, 6]. In this capacity, semantics serve as an intermediary between sensed system conditions and predefined adaptation mechanisms.

Within most autonomic approaches, adaptation decisions are operationalized through planning mechanisms or policy rules that associate specific system conditions with corresponding reconfiguration actions. Decision logic is therefore embedded within the structure of the control loop, with reconfiguration strategies defined as part of the planning or execution phases rather than derived from an explicit semantic representation of the decision problem [4, 6].

These autonomic mechanisms should be distinguished from control-based adaptation approaches. Control-based methods aim to maintain system performance relative to fixed control objectives using predefined system models, typically through parameter tuning or policy adjustment. In contrast, the autonomic frameworks discussed in this section function at a higher level of abstraction, where adaptation may involve modifications to decision logic, reconfiguration strategies, or algorithmic choices as system requirements and operating contexts change.

3.4. Semantic Reasoning with Decision-Theoretic and Learning-Based Methods

To address uncertainty and optimization in adaptive IoT systems, semantic models are increasingly combined with probabilistic and learning-based decision mechanisms. In these approaches, semantic representations structure contextual variables, system states, or constraints, while decision-theoretic or learning-based models operate on these semantically grounded elements to determine adaptation actions.

Bayesian networks are frequently combined with ontologies to enable uncertainty-aware reasoning over contextual information and system state. In ontology-based context management frameworks, semantic models define the structure and meaning of contextual variables, while probabilistic inference mechanisms estimate system conditions and support adaptation decisions based on inferred situations [9, 2]. In this context, semantics serve as a mediation layer that constrains and informs probabilistic reasoning, rather than providing an explicit representation of decision requirements.

Decision-theoretic approaches formalize adaptation under uncertainty by embedding decision-making within optimization models. In [3], Partially Observable Markov Decision Processes (POMDPs) are employed to represent self-adaptation under partial observability, demonstrating improved decision quality for non-functional requirement satisfaction. In these cases, decision objectives, uncertainty models, and adaptation actions are defined within the POMDP structure, while semantic representations primarily support state definition or constraint specification.

Learning-based approaches, particularly reinforcement learning, enable adaptive policies to improve through experience. For instance, SMASH and Q-SMASH integrate semantic interoperability platforms with reinforcement learning and multi-

agent architectures to support human-centered IoT adaptation [12]. In [5], it is demonstrated how ontologies can structure reinforcement learning state spaces.

In these approaches, semantic models define the representation of states, inputs, or constraints, whereas adaptation logic and decision criteria are embedded within the learning or optimization mechanism. As a result, decision problem formulations remain closely associated with the selected decision-theoretic or learning model.

3.5. Evaluation Frameworks and Human-Centered Considerations

Evaluation of adaptive IoT systems is primarily conducted through case studies and prototype implementations in domains such as smart homes, industrial IoT, and smart cities [10, 4, 6]. Evaluation frameworks are generally based on system-level performance indicators, with common metrics including reconfiguration latency, decision accuracy, resource efficiency, and system stability [11, 4, 6, 3]. These metrics are used to assess the effectiveness and robustness of adaptation mechanisms under varying operational conditions.

Certain approaches integrate user preferences or behavioral aspects into adaptation logic, either as part of the system context or as parameters influencing adaptation decisions. Trust-aware adaptation mechanisms [12] and behavior-aware energy management strategies [14] illustrate this integration. In these works, human-related factors shape contextual inputs or decision parameters; however, evaluation remains primarily focused on system-level performance outcomes. Consequently, human impact is not directly considered, and formal evaluation frameworks that explicitly relate human-centered outcomes to system reconfiguration using specific metrics are scarce.

3.6. Domain Independence and Generalizability

Several semantic-driven approaches seek to achieve domain independence by introducing modular ontologies and generic autonomic architectures that separate domain-specific knowledge from adaptation logic. In these approaches, semantic models primarily structure domain concepts, system configurations, and contextual information, while adaptation mechanisms are designed to function over these representations independently of any specific application domain.

In practice, empirical validation of these approaches is typically performed within a single application domain, such as smart environments, industrial IoT, or energy management, where decision objectives, constraints, and evaluation criteria are defined by domain-specific requirements [12, 8, 14]. Thus, generality is usually demonstrated through the reuse of architectural components and adaptation mechanisms, while specific decision formulations and evaluation setups are instantiated separately for each application scenario. However, this reliance on single-domain validation limits the robustness of claims regarding generalizability. To strengthen validation efforts, future research should aim to conduct empirical tests across multiple domains, focusing on diverse application scenarios. This would provide a more comprehensive understanding of how generalizable the proposed approaches truly are and under what conditions they can be effectively applied.

3.7. Synthesis and Research Gaps

The literature shows significant progress in integrating semantic modeling, autonomous control, and decision-making methods to support adaptive behavior in IoT systems. However, a synthesis of the reviewed approaches reveals several recurring limitations.

- **Decision problem representation and algorithm selection.**

Decision problems are seldom modeled as first-class semantic entities. Instead, algorithm selection and configuration are typically embedded within application-specific control logic, planning mechanisms, or decision models. Consequently, the relationship between decision requirements and algorithmic solutions remains tightly coupled to specific application contexts, limiting reuse and traceability at higher levels of abstraction.

- **Evaluation scope and human-centered considerations.**

Evaluation practices primarily focus on technical performance metrics, such as latency, accuracy, resource efficiency, or stability. Although some approaches include human-related factors as contextual inputs or adaptation parameters, explicit evaluation of human-centered outcomes, such as behavioral impact, trust, or user experience, remains limited despite the inherently socio-technical nature of IoT systems.

- **Reconfiguration granularity and decision traceability.**

Reconfiguration mechanisms are generally defined at the architectural or service orchestration level, with adaptation actions triggered through policies, planning logic, or control loops. However, the connection between system evolution, requirements, and algorithm selection is not made explicit through semantic models, which limits traceability and systematic reasoning about the rationale and process of reconfiguration.

Tables 3.1 and 3.2 synthesize the main characteristics of representative approaches reviewed in this chapter and contrast them with the focus of this thesis. The comparison is not intended to rank approaches or evaluate their effectiveness within a shared application domain. Rather, it highlights how representative part of the literature address various aspects of adaptive IoT systems at different levels of abstraction. The approaches under comparison address diverse problem settings and domains. The analytical dimensions in the tables illustrate differences in modeling scope, decision representation, and evaluation focus.

References

- [1] M. B. Canciglieri et al. “Smart Reconfigurable Manufacturing System Based on Semantic Models and Multicriteria Decision Making”. In: *IEEE Access* 13 (2025), pp. 193489–193507. URL: <https://ieeexplore.ieee.org/document/11240171/> (visited on 01/28/2026).

Analytical Dimension	Semantic Context Models [9, 2]	Ontology-Guided Algorithm Mapping [7, 8]	Autonomic & Learning-based Adaptation [4, 3, 12]	This Thesis
Primary focus	Context representation and situation awareness	Mapping domain knowledge to predefined algorithms or strategies	Runtime adaptation via control loops, optimization, or learning	Semantic characterization of decision problems and adaptive reconfiguration
Role of semantics	Representation of devices, observations, and contextual state	Representation of domain concepts, goals, and decision criteria	Structuring system state, policies, or learning states	Representation of decision problems, algorithm designs, and evaluation criteria
Decision problem representation	Implicit, derived from inferred context	Embedded in domain-specific logic or workflows	Encoded within control, optimization, or learning models	Explicit, first-class semantic entities
Algorithm selection	Not addressed (fixed reasoning mechanism)	Explicit but domain-scoped and predefined	Fixed decision mechanism (e.g., policies, POMDPs, reinforcement learning)	Traceable selection and reconfiguration based on semantic problem characterization
Adaptation mechanism	Context inference and rule-based triggering	Strategy or controller selection within a domain	MAPE-K control loops, optimization, or learning	Semantic-driven service-level reconfiguration

Table 3.1.: Analytical comparison of representative state-of-the-art approaches and this thesis with respect to semantic modeling, decision representation, and adaptation mechanisms.

- [2] M. Casillo et al. “A multilevel graph approach for IoT-based complex scenario management through situation awareness and semantic approaches”. en. In: *Journal of Reliable Intelligent Environments* 10.4 (Dec. 2024), pp. 395–411. URL: <https://link.springer.com/10.1007/s40860-024-00224-0> (visited on 01/28/2026).
- [3] L. Garcia, H. Samin, and N. Bencomo. “Decision Making for Self-Adaptation Based on Partially Observable Satisfaction of Non-Functional Requirements”. In: *ACM Trans. Auton. Adapt. Syst.* 19.2 (Apr. 2024), 11:1–11:44. URL: <https://dl.acm.org/doi/10.1145/3643889> (visited on 01/28/2026).
- [4] G. Garzone, N. Guermouche, and T. Monteil. “Autonomic Management Approach for Dynamic Service Based IoT Systems”. In: *2018 International Symposium on Networks, Computers and Communications (ISNCC)*. Rome: IEEE, June 2018, pp. 1–8. URL: <https://ieeexplore.ieee.org/document/8530975/> (visited on 01/28/2026).
- [5] S. Ghanadbashi et al. “Handling uncertainty in self-adaptive systems: an ontology-based reinforcement learning model”. en. In: *Journal of Reliable Intelligent Environments* 10.1 (Mar. 2024), pp. 19–44. URL: <https://link.springer.com/10.1007/s40860-022-00198-x> (visited on 01/28/2026).
- [6] A. N. Lam, O. Haugen, and J. Delsing. “Dynamical Orchestration and Configuration Services in Industrial IoT Systems: An Autonomic Approach”. In: *IEEE Open Journal of the Industrial Electronics Society* 3 (2022), pp. 128–145. URL: <https://ieeexplore.ieee.org/document/9706288/> (visited on 01/28/2026).

Analytical Dimension	Semantic Context Models [9, 2]	Ontology-Guided Algorithm Mapping [7, 8]	Autonomic & Learning-based Adaptation [4, 3, 12]	This Thesis
Human-centered considerations	Indirect	Domain-specific	Incorporated as contextual inputs or adaptation parameters	Integrated as explicit evaluation via QoE and agent-based modeling
Evaluation focus	Inference or prediction quality	Domain-specific performance indicators	Latency, stability, accuracy, and resource efficiency	Combined technical performance and human-centered outcomes
Semantic traceability	Context to inferred situation	Domain knowledge to strategy selection	State to policy or action	Decision problem to algorithm to execution to evaluation
Validation scope	Validated within a specific application domain	Validated within a specific application domain	Validated within a specific application domain	Validated across multiple domains using a unified semantic model

Table 3.2.: Analytical comparison of representative state-of-the-art approaches and this thesis with respect to evaluation practices, semantic traceability, and validation scope.

- [7] G. M. Milis, C. G. Panayiotou, and M. M. Polycarpou. “Semantically Enhanced Online Configuration of Feedback Control Schemes”. In: *IEEE Transactions on Cybernetics* 48.3 (Mar. 2018), pp. 1081–1094. URL: <http://ieeexplore.ieee.org/document/7891022/> (visited on 01/28/2026).
- [8] T. Müller et al. “Architecture and knowledge modelling for self-organized reconfiguration management of cyber-physical production systems”. en. In: *International Journal of Computer Integrated Manufacturing* 36.12 (Dec. 2023), pp. 1842–1863. URL: <https://www.tandfonline.com/doi/full/10.1080/0951192X.2022.2121425> (visited on 01/28/2026).
- [9] T. Nguyen-ANH and Q. Le-TRUNG. “An IoT Reconfiguration Framework Applied Ontology-based Modeling and Bayesian-based Reasoning for Context Management”. In: *2019 6th NAFOSTED Conference on Information and Computer Science (NICS)*. Hanoi, Vietnam: IEEE, Dec. 2019, pp. 540–545. URL: <https://ieeexplore.ieee.org/document/9023885/> (visited on 01/28/2026).
- [10] C. Perera et al. “Context Aware Computing for The Internet of Things: A Survey”. In: *IEEE Communications Surveys & Tutorials* 16.1 (2014), pp. 414–454. URL: <https://ieeexplore.ieee.org/document/6512846> (visited on 01/28/2026).
- [11] F. Poggi et al. “Semantic Run-Time Models for Self-Adaptive Systems: A Case Study”. In: *2016 IEEE 25th International Conference on Enabling Technologies: Infrastructure for Collaborative Enterprises (WETICE)*. Paris, France: IEEE, June 2016, pp. 50–55. URL: <http://ieeexplore.ieee.org/document/7536429/> (visited on 01/28/2026).
- [12] H. Rahimi et al. “SMASH: A Semantic-Enabled Multi-agent Approach for Self-adaptation of Human-Centered IoT”. en. In: *Advances in Practical Appli-*

- cations of Agents, Multi-Agent Systems, and Social Good. The PAAMS Collection*. Ed. by F. Dignum, J. M. Corchado, and F. De La Prieta. Vol. 12946. Series Title: Lecture Notes in Computer Science. Cham: Springer International Publishing, 2021, pp. 201–213. URL: https://link.springer.com/10.1007/978-3-030-85739-4_17 (visited on 01/28/2026).
- [13] I. B. Rodriguez et al. “Ontology-based support for reconfigurable adaptive group communication architecture”. en. In: ().
- [14] B. Teixeira et al. “Semantic rule-based approach for automated energy resource management in buildings”. en. In: *Applied Energy* 401 (Dec. 2025), p. 126675. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0306261925014059>.

4. A taxonomy for decision making in IoT systems (TDMIoT)

Abstract. Decision-making in IoT systems is supported by algorithms that must adapt to diverse application domains and changing requirements. However, existing semantic representations mainly describe IoT components and interactions without addressing the logical evolution of decision processes. This chapter introduces the Taxonomy for Decision-Making in IoT Systems (TDMIoT), developed using a conceptual-empirical methodology. The taxonomy conceptualizes decision-making as a problem-solution association and classifies algorithmic approaches across IoT domains. A systematic literature review is conducted to validate TDMIoT as a classification scheme, revealing prevalent design biases favoring prediction-oriented, data-driven algorithms. The taxonomy is further evaluated through its application in the COVID-19 Crowd Management project, demonstrating its potential as a design framework for selecting appropriate decision models. This chapter lays the semantic foundation for the ontology and runtime algorithm selection mechanisms developed later in this thesis.

4.1. Introduction

Internet of Things (IoT) based systems are inherently dynamic: diverse stakeholders demand different objectives [25], components constantly entering and leaving the network, and a continuously changing operational environment [39], raise uncertainties from both the physical and digital layers, driving changes in functional requirements.

Developing IoT systems is challenging as designers must anticipate and respond to unexpected behaviors emerging from the interactions between the diverse components. Currently, the unexpected behaviors are modeled and addressed with specific decision-making processes using decision theory ([87, 31]), machine learning models ([29, 3]), control theory methods ([71, 79]) or even an ensemble of techniques [14, 28]). Nevertheless, in a dynamic infrastructure as the IoT, these models become obsolete.

Addressing the challenges imposed by the IoT requires automated approaches centered on interoperability. However, existing service design and adaptation methods frequently lack the semantic expressiveness and structural mechanisms needed to

support runtime reconfiguration at the level of logical functionality [15, 56, 61, 9, 54, 40, 23, 78, 13, 5]. To overcome these limitations, it is necessary to focus on the core of IoT adaptability: the decision-making processes.

Although IoT applications span multiple domains, the decision-making algorithms they rely on share common characteristics. Characterizing these algorithms and their associated elements will provide a descriptive framework for mapping relationships, developing or upgrading decision processes, and establishing a shared vocabulary. To this end, this chapter presents the Taxonomy for Decision-Making in IoT Systems (TDMIoT). The taxonomy serves as a conceptual framework that categorizes the essential elements of decision-making processes in IoT, abstracting them as a *problem-solution* association providing the foundation for a semantic knowledge base that supports the evolution of logical functionality.

4.2. Background and related work

Recent IoT taxonomies have primarily addressed cybersecurity [68, 4, 48], IoT elements [15, 56, 5, 61, 9, 54, 40, 24], data [1, 84], services [19, 37, 75, 41], AI algorithms [38], and domain-specific specifications [51, 6, 64, 8, 72, 52, 20, 46, 85, 58]. As noted by the systematic review in [67], research efforts have consistently concentrated on these established topics. In contrast, taxonomies dedicated to decision-making remain underexplored.

To ground the proposed taxonomy, this section reviews the state of the art in approaches relevant to decision-making in IoT. The aim is to identify existing efforts that address either the structural representation of decision processes or the analytical study of decision problems. Accordingly, the review is organized into two parts. The first subsection examines taxonomical frameworks that describe decision-making processes (Decision-Making Block in Fig. 4.1), focusing on how these efforts classify and structure decision elements. The second subsection explores works that provide a broader analysis of processes, models, and problems involved in decision-making (Analysis Block in Fig. 4.1). Together, these perspectives highlight both the progress made and the gaps that motivate the development of the Taxonomy for Decision-Making in IoT Systems (TDMIoT).

4.2.1. Taxonomies of Decision-Making in IoT and Related Fields

Taxonomies that explicitly address decision-making in the Internet of Things (IoT) remain scarce. However, other areas of computer science, like self-adaptive systems and multi-agent systems, have structured and analyzed decision processes. These fields face challenges similar to those of IoT, such as operating under uncertainty, managing dynamic environments, and coordinating heterogeneous entities. For this reason, their contributions provide valuable reference points, even if they cannot be directly applied to IoT.

In the field of self-adaptive systems, existing taxonomies have primarily focused on characterizing uncertainty. Works such as [65, 47, 55] propose classifications of uncertainty types to which adaptive systems are exposed, thereby supporting the design of decision mechanisms that anticipate environmental variability. While these

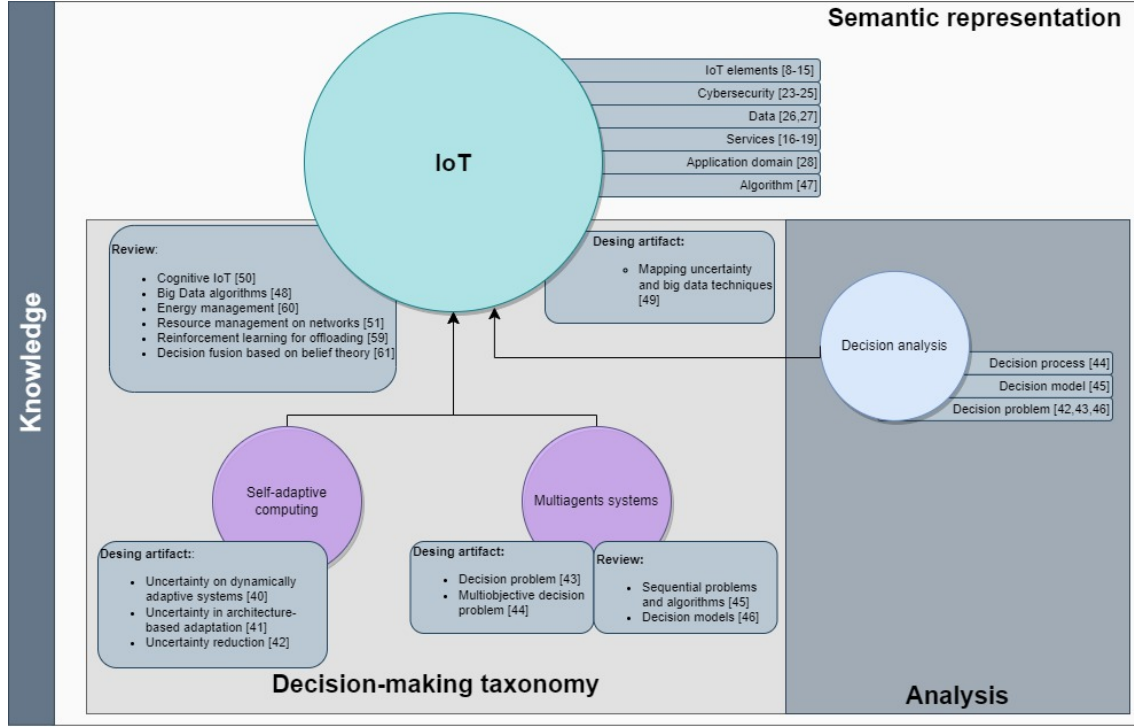


Figure 4.1.: Related work summary

taxonomies offer a systematic way to reason about adaptation, their direct applicability to IoT is limited. The IoT introduces specific challenges (heterogeneity of devices, strong interconnectivity, highly dynamic configurations, and large-scale deployments [59]) which go beyond the assumptions underlying self-adaptive systems.

Similarly, research in multi-agent systems has produced taxonomies to classify decision problems and models. For instance, the taxonomy proposed by [32] characterizes decision problems from the perspective of agent requirements, providing guidance for algorithm selection. However, it does not explicitly address scenarios involving heterogeneous agents. Conversely, [63] introduces a taxonomy focused on multi-objective decision problems, which incorporates heterogeneity but remains limited to a specific subset of decision scenarios. Other works [74, 69] focus on characterizing decision models, either sequential or general, and provide comparative insights and examples of application. Nevertheless, these efforts emphasize the disciplinary origin of models rather than the decision-making process itself. Consequently, although these taxonomies enrich our understanding of decision-making mechanisms, they offer only a partial perspective, insufficient for the needs of IoT.

Within IoT research itself, most taxonomies have concentrated on describing “things,” security, or the system as a whole [7]. Efforts directly related to decision-making typically classify algorithms rather than characterizing the decision problems they are intended to solve. For example, [26, 33] present taxonomies of big data techniques, which are relevant to specific decision processes but are disconnected from the problems that motivate their use. Similarly, [88] classifies decision models for cognitive IoT within disciplinary boundaries, while [49] provides a categorization of decision-making methods under uncertainty, focusing specifically on network resource management. Although these studies provide useful catalogues of techniques

and models, they do not analyse the relationship between decision problems and their corresponding solution approaches.

In summary, while self-adaptive and multi-agent systems provide valuable insights into decision-making taxonomies, and IoT research has begun to classify decision-support algorithms, none of the existing taxonomies address the decision-making process comprehensively. In particular, no prior work has explicitly examined the relationship between decision problems and their associated algorithms in the context of IoT. This gap emphasizes the need for a taxonomy such as the one proposed in this thesis.

4.2.2. Foundations of Decision Analysis for IoT Systems

Decision analysis provides the theoretical and methodological basis for addressing decision problems in a systematic manner [10]. Within computer science, research in this area has emphasized the development of methodologies [43], the design of frameworks to support decision problems [83], and the modeling of processes that aim to replicate human behavior [30].

A central aspect of decision analysis is the decomposition of a decision problem into smaller elements and the subsequent structuring of these elements, a process that can be understood as problem characterization. One of the earliest and most influential proposals is that of Simon [77], who divided the decision process into three stages: *intelligence*, *design*, and *choice*. This foundational characterization has informed numerous subsequent approaches, though the specific stages and emphases vary across studies.

To situate this thesis within the broader body of work, Table 4.1 summarizes relevant characterizations of decision analysis found in the literature. These characterizations differ in scope and detail, but all converge on the importance of representing knowledge to support decision-making. This perspective is particularly relevant for IoT systems, where decision processes must be systematically described in order to enable algorithmic support and runtime adaptability.

4.3. Research approach

The primary objective of this research is to develop a taxonomy that characterizes decision processes within IoT application domains. This taxonomy is intended to establish explicit associations between decision problems and algorithmic approaches. In order to achieve this objective, a research and development (R&D) methodology was adopted, which was structured in two phases: taxonomy development and taxonomy deployment.

The first phase addresses taxonomy development, following the conceptual–empirical approach proposed by [57]. This phase is organized into three stages: (i) **Conceptualization**, where the design guidelines and metacharacteristics are established; (ii) **Analysis**, which involves the collection and management of metadata to support iterative refinement; and (iii) **Refinement**, aimed at consolidating and improving the structure of the taxonomy.

The second phase focuses on the application and evaluation of the developed taxonomy as a knowledge representation to support decision-making in IoT. This phase consists of a single stage, **Research Evaluation**, in which the taxonomy is

Reference	Goal	Characterize	Analysis Perspective	Conceptualization	Characterization
2009 [32]	Identify classes of decision making problems where provable solutions exists	Decision problem	Requirements of an agent to solve a decision problem	A decision consists of selecting the best action based on the evaluation of the consequences.	• Requirements of an agent to solve a decision problem • Individual Input vs Group input • Learning vs Non-learning • Collaboration vs Competition
2010 [43]	Create an ontology for decision making in the field of Information Systems Engineering	Decision problem	Expected result	Decision is an act initiated to satisfy a purpose and allows judgement about the potential actions.	• Type of decision problem: choice, classification, and ranking • Alternatives • Criterion to evaluate alternatives
2019 [42]	Generalization of heuristic-based decision making	Decision process	Heuristic decision making	Decision consist on choosing an alternative a^* from a set of alternatives.	• Set of alternatives A. • Cues to evaluate the alternatives $c_i \in C$. • Cues are functions that map alternatives to real values or a ranking function.
2020 [34]	Analyze the change patterns in a decision model to evolve the model	Decision model	Decision model and notation standard	Decision is a tuple $\{I_d, O_d, L\}$ where I represents a subset of input symbols, O represents a set of outputs or results and L stands for a logic rule that define the relation between I and O	• Inputs • Outputs • Logic rules
2020 [63]	Analyze multi-objctive multi-agent decision problems to structure the field	Decision problem	Multi-objective multi-agent decision problems are modeled as multi-objective stochastic games (MOSG).	MOSG are formally defined by a tuple $M = (S, A, T, R)$ with $n \geq 2$ agents and $d \geq 2$ objectives, where S stands for the system state space, A represents the set of joint actions, T is a probability transition function and R refers to the environment rewards that have a vectorial form to represent the d objectives.	• Utility (Team, Social choice, Individual) • Reward (Team, Individual)

Table 4.1.: Decision analysis

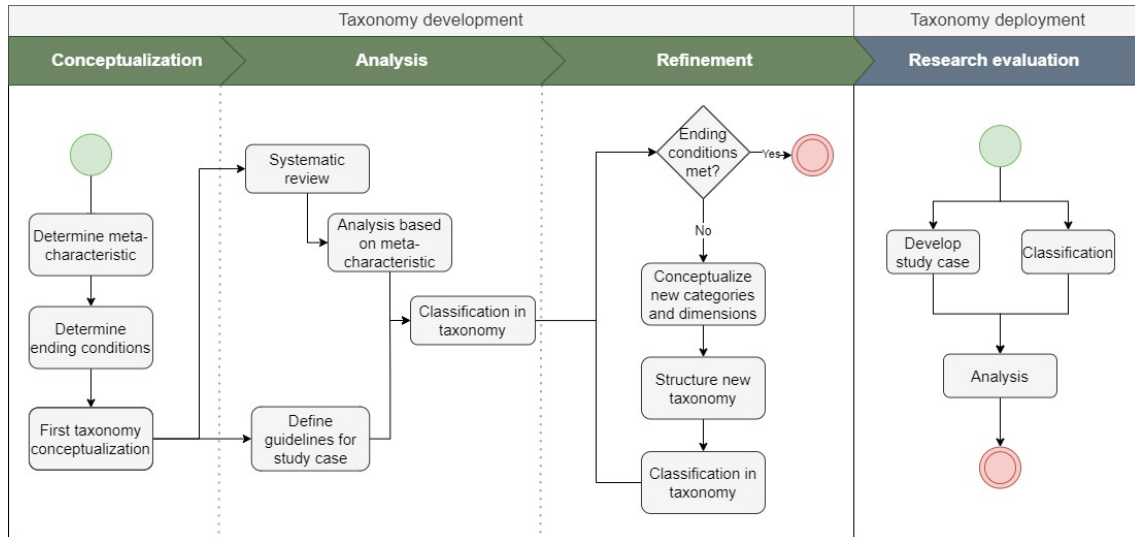


Figure 4.2.: Research and development methodology

assessed as a design asset in exploratory designs and new deployments. Here, the metacharacteristics and classification framework guide the selection of algorithms as part of the design engineering process. Figure 4.2 illustrates the overall methodology and its two phases.

4.3.1. Objectives of the Taxonomy

The TDMIoT taxonomy is designed to fulfill the following objectives:

1. To analyze the state of the art in decision-making processes within IoT, with particular attention to the conceptualization of decision processes as *problem-solution* associations.
2. To enhance the understanding of the relationships among the constituent elements of decision-making in IoT systems.
3. To provide a semantic tool that facilitates the design and selection of algorithmic approaches for decision-making problems in IoT.

4.3.2. Taxonomy Development

The development of the Taxonomy for Decision-Making in IoT (TDMIoT) followed a structured methodology designed to ensure conceptual clarity, analytical rigor, and practical relevance. The process was organized into three stages: **Conceptualization**, focused on defining the meta-characteristic, dimensions, and characteristics; **Analysis**, aimed at validating the preliminary taxonomy through literature review and gap identification; and **Refinement**, dedicated to iterative improvement and consolidation against predefined ending conditions. Together, these stages provided a systematic pathway for constructing a taxonomy capable of characterizing decision-making in IoT across diverse domains.

Conceptualization

The first stage of the methodology focuses on conceptualizing the taxonomy. In this stage, the study guidelines were defined, and an initial version of the taxonomy was constructed. The main activities involve (i) defining the meta-characteristic, (ii) establishing the ending conditions, and (iii) producing the first conceptualization of the taxonomy. These activities are described below.

- **Defining the meta-characteristic.** The central purpose of TDMIoT is to characterize the decision-making process in IoT systems as a *problem–solution* association. To this end, the decision processes reported in the literature were examined by formulating four guiding questions:
 1. Who makes the decision?
 2. What is the goal of the decision?
 3. What elements are involved in the decision?
 4. What algorithms are selected?
- **Definition of ending conditions.** Taxonomy development is iterative; therefore, it requires clearly established criteria to determine when the process can be considered complete. In this work, the ending conditions proposed by [57] were adopted. These conditions are divided into:
 - *Subjective ending conditions*, assess whether the taxonomy is practically useful and satisfies the qualities expected of a robust classification tool. Evaluation is conducted against five characteristics, each addressed through guiding questions:
 1. *Conciseness*:
 - * Does the number of dimensions allow the taxonomy to be meaningful without being difficult to manage or overwhelming?
 2. *Robustness*:
 - * Do the categories and characteristics sufficiently differentiate the objects to be meaningful?
 - * Given the properties of the sample objects, what can be inferred about the objects from their classification?
 3. *Exhaustiveness*:
 - * Can all objects—or a representative random sample—within the domain of interest be classified?
 - * Are all relevant dimensions of the objects identified?
 4. *Expandability*:
 - * Can a new characteristic be easily added to an existing dimension?
 5. *Descriptiveness*:
 - * What do the dimensions and characteristics describe about a given object?

- *Objective ending conditions*, evaluate whether the taxonomy satisfies its formal definition. These criteria ensure that the taxonomy structure adequately represents all study objects and cases. The objective ending conditions applied in TDMIoT are as follows:
 1. All study objects and cases have been examined.
 2. All objects have been classified.
 3. No object is classified under more than one characteristic within the same dimension.
 4. At least one object is classified under each characteristic of every dimension.
 5. No new dimensions or characteristics were added or modified in the last iteration without restructuring.
 6. Each dimension is unique.
 7. Each characteristic is unique within its dimension.
- ***First conceptualization of the taxonomy.*** Based on empirical knowledge of decision processes in IoT systems, a first draft of the taxonomy was created. This initial version consisted of four dimensions—Processing, Objective, Interaction, and Uncertainty—and served as the starting point for the subsequent iterative refinement process.

Together, these conditions provide both rigor and transparency to the iterative development of the taxonomy.

Analysis

The objective of the Analysis stage is to generate the empirical evidence and tools necessary to refine the taxonomy. This stage involves a systematic review of the literature on decision-making processes in IoT application domains and the definition of guidelines for a study case. Both tools are selected in alignment with the objectives of TDMIoT and serve as the basis for classifying decision processes in the taxonomy.

- ***Literature review for validation.*** To develop TDMIoT as a classification artifact, a systematic review of research studies addressing decision-making processes in IoT was conducted, following the methodology proposed by [80] (see Figure 4.3). The review follows the standard steps of preparation, retrieval, appraisal, and synthesis. The Scopus database was selected as the primary source, based on its extensive coverage of computational science research [12]. In the preparation step, the research question was defined as:

“From an algorithmic perspective, how are decision-making processes addressed in case studies across various IoT application domains?”

General inclusion criteria were set for articles from IoT-related subfields in Engineering, Decision Theory, and Computing. Keywords such as decision-making and decision-making under uncertainty were combined with domain-specific IoT terms. Additional inclusion criteria required that studies implement an algorithm or decision-making model, propose automated decision-making, and provide information to address the meta-characteristic questions.

Exclusion criteria filtered out algorithm evaluation studies, ontology- or rule-based systems, and business process modeling approaches. The initial retrieval returned 1,604 articles. After removing duplicates, 1,173 articles remained. Titles were syntactically filtered for irrelevant keywords, reducing the set to 958. Semantic filtering of titles, abstracts, and full texts subsequently reduced the selection to 56 relevant articles. Among these, fifty addressed decision problems in IoT application domains, and six, while not strictly IoT, presented relevant decision-making problems.

- **Identification of gaps and inconsistencies.** Each of the 56 selected articles was analyzed to extract data aligned with the meta-characteristic defined during the Conceptualization stage. The answers to the meta-characteristic questions were compiled into a structured summary for each article, hereafter referred to as a meta-extract. These meta-extracts provide the empirical foundation for classifying decision processes in TDMIoT.
- **Definition of Study Case Guidelines.** A taxonomy functions as a practical tool to support decision-making processes [81]. In this context, defining clear study case guidelines ensures that decision processes can be evaluated and classified effectively. The following requirements were established for a decision process within an IoT application domain:
 1. The decision process must be defined using a systematic problem-solving approach [53].
 2. The process must be classifiable within TDMIoT.
 3. The process must not be rule-based.
 4. The effectiveness of the decision process must be measurable in terms of achieving its objective.
- **Classification in the Taxonomy.** Using the 56 meta-extracts, the decision processes were mapped onto the initial draft of TDMIoT. Out of 56, only 22 processes could be classified, indicating that the first draft required refinement. Consequently, additional dimensions and categories were identified, forming the basis for the next stage of taxonomy development.

Refinement

The refinement stage corresponds to the iterative process of improving the taxonomy until it fulfills the evaluation criteria defined during conceptualization. This process ensures that the taxonomy evolves in a structured and systematic manner, balancing conceptual soundness with empirical validity. Each iteration follows a sequence of activities designed to evaluate progress, identify necessary adjustments, and consolidate improvements. The workflow in each iteration comprises the following activities:

- **Evaluation of the ending conditions.** The conditions established in the conceptualization stage, following [57], are used as reference points. After classifying the 56 meta-extracts, the taxonomy is reviewed against these conditions to assess whether the iteration can be concluded.

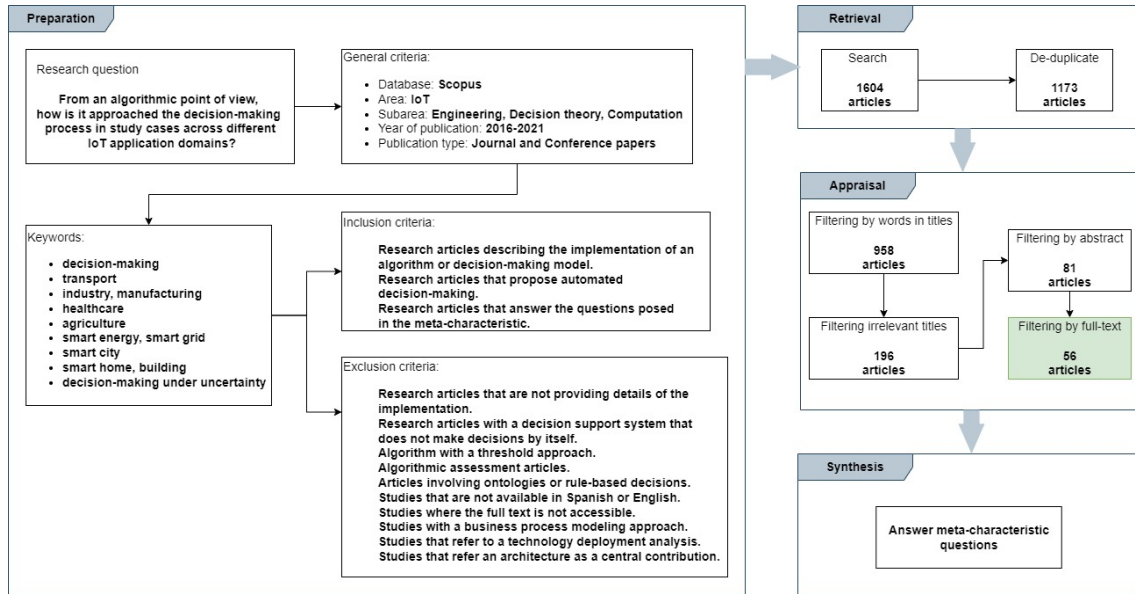


Figure 4.3.: Systematic literature review

- *Conceptualization of new categories and characteristics.* If the ending conditions are not yet satisfied, a further analysis of decision-making processes is carried out to identify missing aspects. Based on this analysis, additional dimensions or characteristics are conceptualized.
- *Structuring of a new taxonomy version.* The newly defined categories and characteristics are integrated into the taxonomy, producing an updated version.
- *Reclassification of meta-extracts.* The revised taxonomy is then applied to reclassify the 56 meta-extracts, ensuring consistency and testing the adequacy of the modifications.

In the case of TDMIoT, five iterations were required before the taxonomy satisfied the defined ending conditions.

4.3.3. Taxonomy Deployment

Research evaluation

Once an iteration of the taxonomy fulfills the termination conditions, its utility must be evaluated. According to Usman et al. [81], the usefulness of a taxonomy as a classification tool can be assessed by examining its ability to (i) classify existing literature and (ii) serve as a general design framework within a case study. As a classification artifact, its effectiveness is determined by the proportion of works that can be successfully categorized within its scope. The use cases employed to validate the TDMIoT taxonomy are presented in Section 4.5.2.

The evaluation stage of the proposed methodology comprises three main steps:

- *Case study development.* This step involves designing and developing a case study within an IoT application domain. The goal is to characterize

doi	ye	Domain	Objective	Processing	Processing layer	Interaction	Uncertainty	Knowledge representation	Algorithmic approach	Algorithmic technique	Algoritmo
10.1007/s00170-015-7350-5	2016	Industry	Optimization	Centralized	Application	System	Parameter	Hybrid	Criterion-driven	Heuristic	(ACOS) Ant Colony
10.1109/NCA.2015.30	2016	Home	Classification	Centralized	Perception	System	Model	Data	Data-driven	Supervised learni	Red Neuronal (Mu
10.1109/BIGCOMP.2016.7425965	2016	Home	Classification	Centralized	Application	Human	Model	Data	Data-driven	Random forest	Random Forest (St
10.1109/GLOCOM.2016.7841487	2016	City	Optimization	Centralized	Application	System	Parameter	Hybrid	Criterion-driven	Heuristic	Algoritmo genetic
10.1109/ACCESS.2017.2748561	2017	City	Classification	Centralized	Application	Human	Parameter	Data	Data-driven	Supervised learni	Support Vector Mi
10.3390/s17051182	2017	Home	Classification	Centralized	Application	System	Parameter	Data	Probability-driven	Bayesian	Expectation Maxim
10.1016/j.eswa.2017.03.062	2017	Home	Classification	Centralized	Application	Human	Measure	Data	Data-driven	Supervised learni	Statistical analysi
10.1109/ACCESS.2017.2702715	2017	Industry	Optimization	Centralized	Application	System	Parameter	Hybrid	Criterion-driven	Heuristic	Particle Swarm Op
10.1002/cpe.3843	2017	Home	Classification	Centralized	Application	System	Model	Hybrid	Utility-driven	Game theory	Stochastic Game N
10.1109/ICCCNT.2017.8204069	2017	Health	Estimation	Centralized	Application	Human	Parameter	Symbolic	Probability-driven	Stochastic	Hidden Markov Mi
10.3390/en10111818	2017	Home	Optimization	Centralized	Application	System	Parameter	Hybrid	Criterion-driven	Heuristic	PSO y GA (Método
10.3390/s18061711	2018	Home	Classification	Centralized	Application	System	Model	Data	Data-driven	Supervised learni	Red neuronal (Mé
10.1109/ICCCPS.2018.00014	2018	City	Conflict resolution	Centralized	Application	System	Parameter	Mathematical	Criterion-driven	Constrained	Integer Linear Pro
10.1109/SCCIC.2018.8584552	2018	City	Estimation	Centralized	Application	System	Measure	Mathematical	Probability-driven	Evidence theory	Método basado er
10.1109/ICCUBEA.2018.8697871	2018	Health	Classification	Centralized	Application	System	Parameter	Data	Data-driven	Supervised-learn	Decision Tree C4.5
10.1109/JIOT.2018.2803201	2018	Home	Estimation	Centralized	Application	Human	Parameter	Data	Probability-driven	Bayesian	Bayesian belief ne
10.1109/MCOM.2017.1700547	2019	City	Forecast	Centralized	Application	System	Measure	Mathematical	Probability-driven	Stochastic	Holt's linear trend

Figure 4.4.: TDMIoT structure for analysis

the decision process as a *problem-solution* association. An adequate problem characterization reduces the search space for suitable decision-making algorithms. Once the problem is defined, a solution is proposed and implemented for evaluation.

- **Classification.** In this step, the meta-extracts are categorized using the final version of the taxonomy. The classification results are then organized and structured. For TDMIoT, the classification structure is illustrated in Figure 4.4, encompassing the 56 meta-extracts obtained during the taxonomy development.
- **Analysis.** Analysis. The final step assesses the performance of the taxonomy in terms of its utility. The analysis is aligned with the taxonomy objectives: as a design framework, it must be evaluated in terms of usability and its capacity to support the design of new solutions; as a classification artifact, it should provide semantic knowledge that facilitates problem characterization. For TDMIoT, these results are presented in detail in Section 4.5.1.

4.4. Taxonomy for Decision Making in IoT systems (TDMIoT)

Before presenting the proposed Taxonomy for Decision Making in IoT systems (TDMIoT), it is necessary to establish the conceptual foundations and terminology upon which it is built. This ensures that the scope and interpretation of the taxonomy are clear and consistent with prior research in decision sciences and IoT systems. Two key concepts are introduced here: the notion of a taxonomy and the definition of a decision problem.

4.4.1. Preliminaries and Definitions

Taxonomy

In this work, the term taxonomy refers to a structured classification system that organizes a domain of knowledge into dimensions and categories. Following the formalization proposed by [62], a hierarchical taxonomy T is defined as a set of

n categories or dimensions $D_i (i = 1, \dots, n)$, each consisting of m (where $m \geq 2$) mutually exclusive and collectively exhaustive characteristics $C_{ij} (j = 1, \dots, m)$ such that each object evaluated by the taxonomy has one and only one characteristic C_{ij} for each dimension D_i :

$$T = \{D_1, D_2, \dots, D_n | D_i = \{C_{i1}, \dots, C_{im}\}\} \quad (4.1)$$

In other words, the taxonomy provides a systematic framework to classify and compare entities according to a fixed set of criteria.

Decision problem

Decision-making lies at the core of IoT systems, where autonomous or semi-autonomous entities must select actions under varying conditions. In this context, a decision problem can be defined as the selection of an action from a set of alternatives in response to an event that produces a consequence [50]. More formally, a decision problem is characterized by three components:

1. A set of actions

$$A = \{a_1, a_2, \dots, a_n\} \quad (4.2)$$

2. A set of events $E_i = \{e_{i1}, e_{i2}, \dots, e_{im}\}$ associated with every action a_i

$$\forall a_i \exists E_i \quad (4.3)$$

3. A set of consequences $C_i = \{c_{i1}, c_{i2}, \dots, c_{im}\}$ defined by the relation:

$$R : (e_{ij}, a_i) \longrightarrow c_{ij} \quad (4.4)$$

where c_{ij} refers to a consequence.

$$\forall a_i \exists C_i \quad (4.5)$$

Within this framework, the best action corresponds to the one that yields the most desirable consequence according to the evaluation criteria. A decision process, therefore, is the method or procedure by which this best action is selected.

4.4.2. Structure of TDMIoT

The Taxonomy for Decision Making in IoT systems (TDMIoT) (Figure 4.5) was developed using a conceptual–empirical approach [57]. Its primary purpose is to characterize decision processes in the IoT application domains. To achieve this, TDMIoT adopts an Input–Process–Output (IPO) perspective [27, 17], which allows the decision process to be described as a structured association between problems and solutions.

The choice of the Input–Process–Output (IPO) model as the structural foundation of the taxonomy is motivated by both its conceptual clarity and its suitability for representing decision-making in IoT domains. The IPO model provides a unifying abstraction that aligns naturally with the way IoT systems mediate decision-making processes. Decision-making in IoT can be systematically decomposed into:

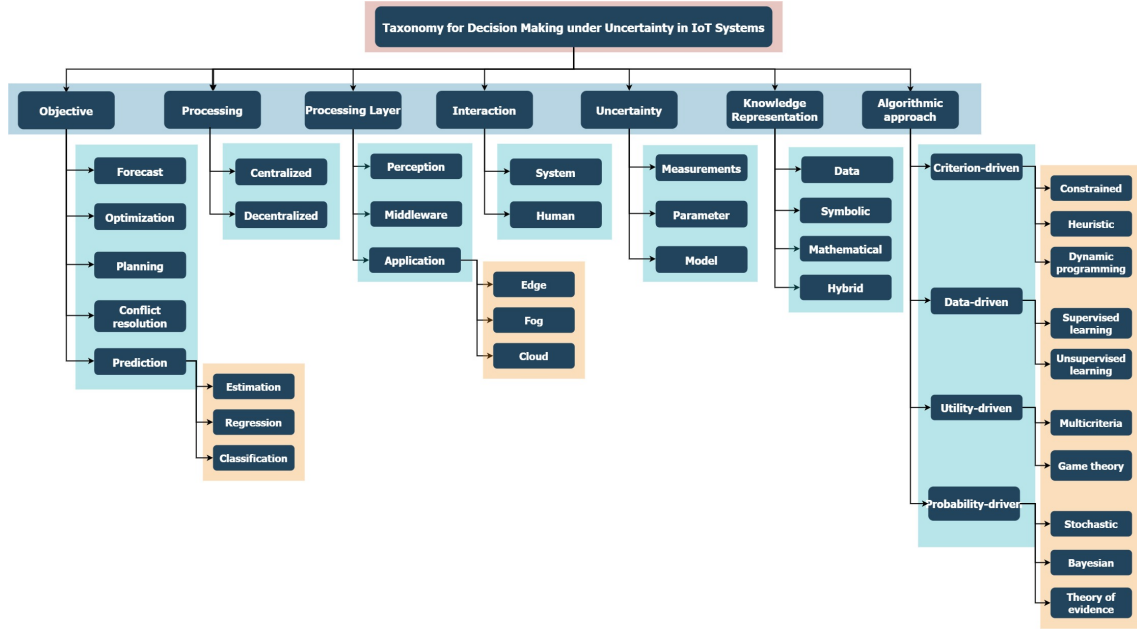


Figure 4.5.: TDMIoT structure for analysis

- Inputs, which capture contextual factors, uncertainties, objectives, and requirements gathered from the environment or stakeholders.
- Processes, which involve algorithmic reasoning, knowledge representation, and the selection of strategies for decision resolution.
- Outputs, which correspond to actions, recommendations, or system configurations that realize the intended decision in practice.

This tripartite structure allows the taxonomy to remain application-agnostic while still being expressive enough to capture the diversity of decision problems across IoT domains. From a methodological standpoint, adopting the IPO model also supports the dual role of the taxonomy: as a classification tool, it provides clear dimensions for organizing literature and case studies; and as a design artifact, it offers a structured lens to guide the characterization of problems and the specification of algorithmic solutions.

Building on this foundation, the taxonomy is composed of seven dimensions derived from the defined meta-characteristic established during the methodology (see Section 4.3). Five of these dimensions—*objective*, *interaction*, *processing*, *level*, and *uncertainty*—describe a decision problem. The remaining two dimensions—*knowledge representation* and *algorithmic approach*—abstract the solution space. Collectively, these dimensions enable the classification of decision processes as structured *problem–solution* associations.

Dimensions

Objective

The Objective dimension classifies decision problems according to their intended goal. Defining the objective is essential, as it determines the scope of the deci-

sion process and influences the choice of algorithms that can be applied. Within TDMIoT, five principal objectives are identified:

- **Forecasting** involves estimating the value of future data through time series analysis. Forecasting involves estimating the value of future data through time series analysis.
- **Optimization**, which seeks to select the best alternative from a set of possible options.
- **Planning**, where the focus is on constructing a sequence of actions that lead to a desired goal.
- **Conflict resolution** aims to achieve consensus among multiple entities with potentially competing interests and preferences.
- **Prediction** estimates unknown values from observed data. Prediction can be further refined into:
 1. *Estimation*, when predicted values are probabilities bounded within $[0,1]$
 2. *Regression*, when predicted values are continuous and unbounded.
 3. *Classification*, when predicted values are categorical.

The **Objective** dimension establishes the underlying rationale that guides the decision-making process.

Processing

The *Processing* dimension characterizes a decision process according to how the decision algorithm execution is managed. This dimension reflects the distribution of computational responsibilities. Two categories are defined within this dimension:

- **Centralized** processing occurs when a single entity has access to all the necessary resources—such as data, models, or contextual information—and executes the decision process in its entirety.
- **Decentralized** processing involves multiple entities that exchange information or share computational tasks to execute the decision process collectively.

By distinguishing between centralized and decentralized arrangements, this dimension emphasizes how the structural organization of computation shapes the nature of decision-making in IoT systems.

Processing Layer

The *Processing Layer* dimension classifies a decision process according to the architectural layer in which it is executed. This dimension highlights the physical or logical location of computation within an IoT system. Three categories are defined within this dimension:

- **Perception Layer** refers to local executions of the decision process within IoT devices, typically constrained by limited computational resources.

- **Middleware Layer** corresponds to an intermediate layer where decision processes are executed to manage communication and coordination between applications, devices, or systems.
- **Application Layer** encompasses executions in fog, edge, or cloud environments, where greater computational power is required to support decision processes that cannot be performed locally.

Interaction

The *Interaction* dimension characterizes a decision process according to the origin of its inputs. In IoT systems, inputs can arise from heterogeneous sources, including devices, humans, or external systems (natural or artificial). This dimension captures the nature of those sources, recognizing that IoT decision-making is inherently situated within environments where physical and virtual entities continuously exchange information.

Two categories are defined within this dimension: *system* and *human*:

- **System** interaction refers to decision problems that rely on outputs from non-human sources, such as data streams, objects, electronic signals, or computational modules. These decision problems are typically embedded in linear sequences of operations where system-to-system communication predominates.
- **Human** interaction involves decision problems where human presence, behavior, or intervention plays a central role in the decision process. Human interaction can be direct, such as through deliberate actions (e.g., placing an object), or indirect, as in cases where user-related information is collected passively or inferred without explicit mediation.

By distinguishing between system and human interactions, this dimension highlights the dual socio-technical nature of IoT decision-making, acknowledging that both autonomous systems and human agents can serve as critical input providers to the decision process.

Uncertainty

The concept of uncertainty pertains to the sources of restrained knowledge that impact decision-making processes. Uncertainty hinders the accurate description of the current state of the system or its future evolution [18]. Identifying the source of uncertainty is essential, since each algorithmic approach integrates strategies to address it. In the IoT context, uncertainty emerges from the cyber-physical nature of systems and is classified into three types [21]:

- **Measurements.** Refers to uncertainty associated with data acquisition. It arises from factors such as scaling, interpolation, noise, data fusion, or pre-processing. In this case, the actual data y and the measured data y' differ by

$$\Delta y = y' - y, \quad (4.6)$$

although this difference cannot be determined with precision.

- **Parameter.** Refers to uncertainty related to variables that cannot be controlled, inferred, or exactly known. It is common in optimization problems, where assumptions or bounds must be established. For a parameter P , uncertainty is represented as

$$P_{low} \leq P < P_{max}. \quad (4.7)$$

- **Model.** Refers to uncertainty derived from the absence of a precise model to support decision-making. It occurs when: (i) only datasets with relevant variables are available to infer a model; (ii) the deployment environment is partially known or produces noisy observations; or (iii) observable events depend on unobservable factors. Quantification requires large amounts of data. If X_{actual} is the true value and X_{pred} the predicted one, model uncertainty can be expressed as

$$B = \frac{X_{actual}}{X_{pred}}. \quad (4.8)$$

Knowledge representation

The Knowledge Representation dimension classifies decision processes according to how information relevant to the problem is modeled. Knowledge representation plays a central role in decision-making, as it determines not only what information is available but also how it can be structured and exploited during the process [86]. Four main forms of representation are considered:

- **Data Representation** consists of sets of records describing observations or characteristics, without explicit specification of relationships among them.
- **Symbolic Representation** encodes variables and their interrelations through explicit rules or logical schemes.
- **Mathematical Representation** formalizes variables and their relationships using functions, equations, or probabilistic structures.
- **Hybrid Representation** combines two or more of these approaches, enabling richer descriptions. For instance, a Markov decision process integrates symbolic elements (state-action schema) with mathematical elements (transition and reward functions).

Algorithmic approach

The Algorithmic Approach dimension classifies decision processes according to the type of algorithms employed to generate solutions. In contrast to the previous dimensions, which describe the decision problem, this dimension addresses how the problem is solved through algorithmic means. The classification is structured in two levels (Figure 4.5). The first level identifies the *approach* that drives the decision logic within the algorithm. The second level specifies the *techniques* employed under each approach to implement the decision-making process.

- **Criterion-driven (Level 1).** This approach includes algorithms guided by explicit mathematical criteria or structured search strategies. Three main techniques are identified:
 1. **Constrained (Level 2).** Technique that covers algorithms that employ mathematical programming under well-defined constraints, including linear, non-linear, and geometric programming [70].
 2. **Heuristic (Level 2).** Technique that use heuristics or metaheuristics to explore the solution space efficiently, offering approximate solutions through strategies such as genetic algorithms, particle swarm optimization (PSO), or simulated annealing [60].
 3. **Dynamic programming (Level 2).** Technique that decomposes decision problems into subproblems and reuses partial solutions, with reinforcement learning techniques such as Policy-Gradient, SARSA, and Q-Learning serving as representative examples [16].
- **Data-driven (Level 1).** This approach comprises algorithms that derive models directly from data, adapting to observed patterns without assuming an underlying theory. Following standard machine learning taxonomy [36], two categories are distinguished:
 1. **Supervised learning (Level 2).** Methods that use this technique train on labeled data to predict outcomes. This include algorithms such as linear regression, neural networks, and support vector machines.
 2. **Unsupervised learning (Level 2)** Methods that use this technique analyze unlabeled data to uncover hidden structures. Methods using this technique include clustering, principal component analysis, and latent variable models.
- **Utility-driven (Level 1).** Algorithms in this approach apply principles of utility theory to evaluate and rank alternatives. Unlike criterion-driven approaches, which emphasize objective functions and constraints, utility-driven approaches prioritize preference modeling. Two representative techniques are considered:
 1. **Multicriteria (Level 2).** This technique address problems involving multiple conflicting criteria, employing techniques such as Analytical Hierarchy Process (AHP), Multiple Attribute Utility Theory (MAUT), and Case-Based Reasoning (CBR) [82].
 2. **Game-theory (Level 2)** technique analyze strategic interactions among rational agents, providing a framework for competitive or cooperative decision-making, with algorithms such as minimax serving as canonical examples [44].
- **Probability-driven (Level 1).** This approach encompasses algorithms that rely on probabilistic models to make a decision. Three techniques are identified:
 1. **Stochastic (Level 2).** Technique encompassing algorithms based on stochastic models. A stochastic process is a set of random variables X_t

indexed by the set T (where $t \in T$). This type of process can be interpreted as a phenomenon that evolves over time with random dynamics and is exemplified by algorithms such as Kalman filters, ARMA models, and Markov Decision Processes [35].

2. **Bayesian (Level 2)** comprises algorithms based on Bayesian models. In a Bayesian model, a problem is formulated as a set of random variables $X_1, \dots, X_n$ with conditional relationships. The goal of a Bayesian model is to determine the posterior distribution $P(X_1|X_2)$ that represents an update of knowledge. The posterior distribution can be calculated with the available knowledge known as the prior distribution $P(X_1)$ and the observed evidence is referred to as the likelihood distribution, $P(X_2|X_1)$ [73]. Bayesian networks and classifiers are representative algorithms within this technique.
3. **Theory of evidence (Level 2)**. Technique that includes decision-making algorithms built on Evidence Theory or Dempster-Shafer Theory (DST). The DST framework combines evidence from multiple sources and computes belief measures. In this framework, there is a set of mutually exclusive elements in a domain (called the discernment framework Θ), a hypothesis (a subset of the discernment framework $H \subseteq \Theta$), and a belief function that determines the degree of belief in a hypothesis ($\beta(H)$). The discernment framework represents the evidence. Based on this evidence, a hypothesis is formulated and a set of calculations is applied to measure its strength through a belief function, and then a decision is finally made [76, 66]. This strategy is often used in data fusion applications

4.4.3. TDMIoT as a Framework for Decision Process Design

One of the objectives of this thesis is to provide a semantic knowledge representation that supports the design and reconfiguration of decision processes in IoT systems. In this sense, TDMIoT can be understood not only as a classification tool but also as a design framework. Its role is to guide engineers in structuring and reducing the search space of potential algorithmic solutions for a given problem within the diverse IoT domains.

The rationale for this positioning is that the dimensions captured in TDMIoT can be aligned with the activities of the generic *problem–solution* cycle, as proposed by [53]. Through this alignment, TDMIoT provides systematic guidance on how to transition from problem characterization to selecting an algorithmic solution (See Fig. 4.6). More specifically:

1. **Problem definition** corresponds to establishing the dimensions of *objective*, *processing layer*, and *interaction*.
2. **Problem analysis** is supported by the dimensions *processing*, *uncertainty*, *knowledge representation* and the first level of *algorithmic approach*.
3. **Evaluation of the solution** involves selecting a technique that is consistent with the chosen *algorithmic approach*.

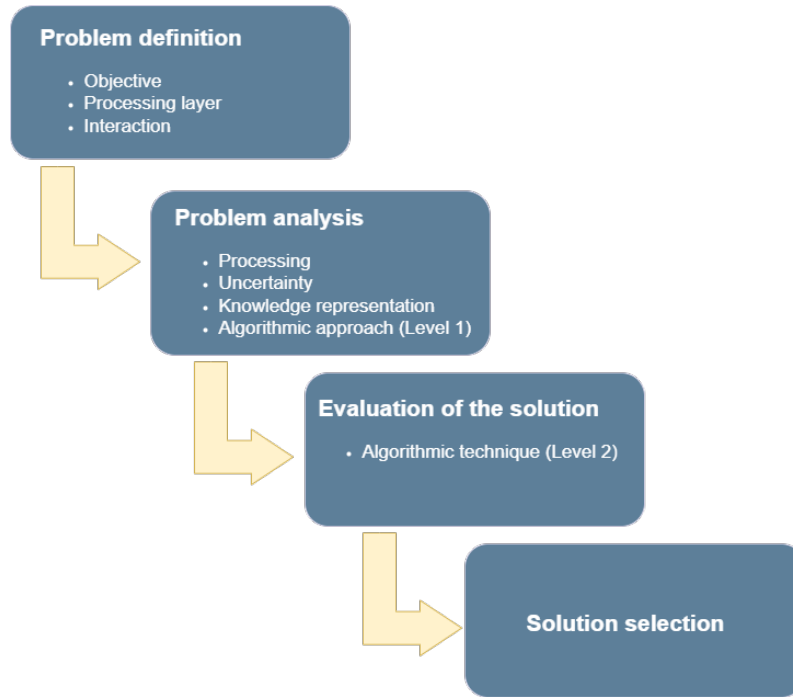


Figure 4.6.: TDMIoT as a Framework for Decision Process Design

4. **Solution selection** is achieved by identifying the specific algorithm within the relevant technique category and by classifying the required knowledge representation.

By mapping its dimensions to these design activities, TDMIoT becomes an actionable framework for decision process design. This ensures that the taxonomy is not only descriptive but also prescriptive in guiding how IoT decision-making problems can be approached and solved in a structured and systematic manner.

4.5. Research evaluation

The Taxonomy for Decision-Making in IoT (TDMIoT) was conceived as both a classification artifact and a design framework. As a classification artifact, it is intended to systematize knowledge about decision processes across diverse IoT domains. As a design framework, it aims to support the construction or reconfiguration of decision-making models by providing a semantic structure that reduces the search space of possible solutions. To evaluate the adequacy of TDMIoT in relation to these dual purposes, a systematic literature review was conducted. A total of 56 articles were selected and classified according to the dimensions of the taxonomy. The resulting meta-extracts (structured summaries of each article) were mapped onto the taxonomy, yielding 37 distinct combinations of TDMIoT categories. Figures 4.7 and 4.8 illustrate the mapping of articles, their algorithmic approaches, and their application domains. The analysis of these classifications was then structured according to the three objectives of this chapter (Section 4.3.1), namely:

1. To analyze the state of the art in decision-making processes within IoT, with

particular attention to the conceptualization of decision processes as *problem-solution* associations.

2. To enhance the understanding of the relationships among the constituent elements of decision-making in IoT systems.
3. To provide a semantic tool that facilitates the design and selection of algorithmic approaches for decision-making problems in IoT.

The following subsections present the evaluation results aligned with these objectives.

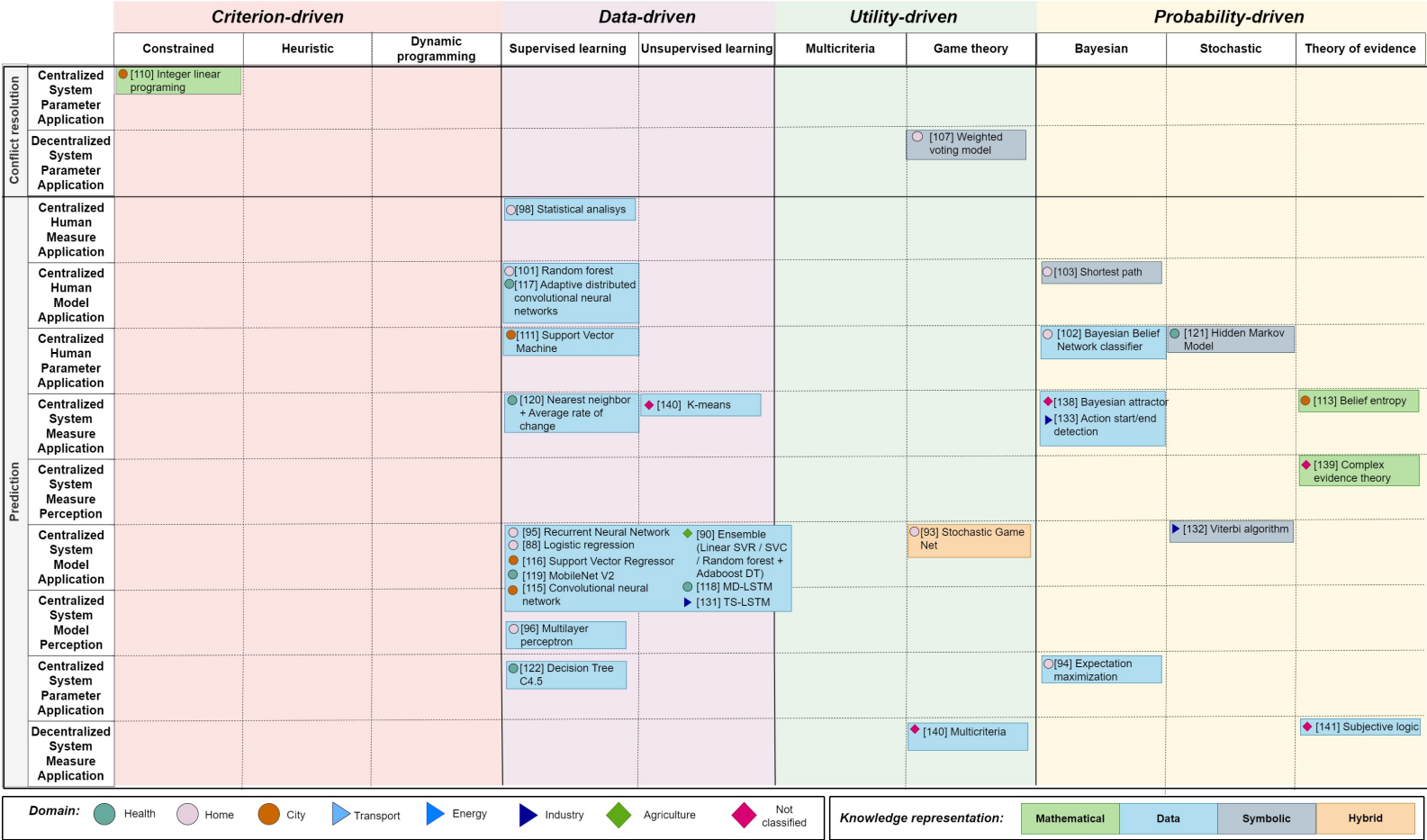


Figure 4.7.: TDMIoT classification - Part 1

		Criterion-driven			Data-driven		Utility-driven		Probability-driven		
		Constrained	Heuristic	Dynamic programming	Supervised learning	Unsupervised learning	Multicriteria	Game theory	Bayesian	Stochastic	Theory of evidence
Forecast	Centralized Human Measure Application									◆ [92] Goidanich model	
	Centralized Human Model Application				▶ [134] LSTM						
	Centralized System Measure Application								○ [100] Dynamic Bayesian Model	● [112] Holt Method ◆ [91] ARIMA ● [123] Weight adaptive Kalman filter	
	Centralized System Model Application				◆ [89] LSTM ▶ [135] Factorized Conditional Restricted Boltzmann Machine ▶ [136] XGB+LSTM ▶ [97] GRU						
	Centralized System Model Middleware				○ [114] LSTM						
Optimization	Centralized Human Parameter Application							▶ [125] Nash Equilibrium			
	Centralized System Parameter Application		▶ [126] Ant colony optimization with selection (ACO) ● [108] Genetic algorithm ▶ [128] PSO (Particle swarm optimization) ○ [104] GA / PSO ▶ [129] ACO-DR / ACO-DRC ● [109] GA / PSO ▶ [127] ACO				▶ [130] Utility model				
	Decentralized System Parameter Application							◆ [142] Nash Equilibrium		○ [106] A3C-Apex	
Planning	Centralized Human Model Application			○ [105] Q-Learning							
	Centralized Human Parameter Application									○ [99] Value Iteration	
	Centralized System Model Perception									◆ [143] Value Iteration	
	Centralized System Parameter Perception				▶ [137] Deep Q-Network						
	Decentralized System Parameter Application		▶ [124] Non-dominated sorting genetic algorithm II								
Domain:		Health	Home	City	Transport	Energy	Industry	Agriculture	Not classified	Knowledge representation:	
										Mathematical	Data
										Symbolic	Hybrid

Figure 4.8.: TDMIoT classification - Part 2

4.5.1. TDMIoT as a Classification Artifact

Objective 1: Analyzing the state of the art in IoT, considering the decision process as a problem-solution association

The classification of 56 articles allowed a comprehensive characterization of the decision-making landscape in IoT research. Several trends emerged:

- The majority of IoT decision problems are formulated as *prediction* tasks, primarily addressed through *supervised learning* techniques.
- The dominant algorithmic approach is *data-driven*, highlighting the significant impact of machine learning methods in the field.
- The most common knowledge representation is *data* in its raw form, with limited reliance on *symbolic*, *mathematical*, or *hybrid* representations.
- Decision processes are most often located at the *application* level. In contrast, relatively few works address decision-making at the *perception* or *middleware* levels, despite their importance for communication and resource allocation.
- The majority of decision-making processes remain *centralized*, despite recent advances in device capabilities and the emergence of 5G technology, indicating a potential for decentralized and edge intelligence.
- Most works assume a *system-level* perspective in the interaction dimension, with limited emphasis on human-machine interaction.

Objective 2: Provide a better understanding of the relationships between the elements that make up an IoT systems decision-making process

Beyond characterizing trends, the classification also enabled the identification of systematic relationships between TDMIoT dimensions:

- *Utility-driven* approaches are frequently associated with *decentralize* processing.
- *Measure* uncertainty is commonly linked to *probability-driven* algorithms.
- *Model* uncertainty tends to be handled by *data-driven* methods.
- Problems with *Planning* objective are often addressed with *criterion-driven* algorithms.
- Problems with *Forecasting* objective are typically linked with *centralize* processing.
- *Data* knowledge representations are not exclusive to *data-driven* techniques, but are also used in *dynamic programming*, *Bayesian*, *game theory*, and the *theory of evidence*.
- *Heuristic* algorithmic approaches are consistently executed at the *application* layer, underlining their computational demands.

- *Symbolic* knowledge representations often appear in conjunction with *probability-driven* algorithmic approaches.
- Problems with *Planning* objective are generally supported by *hybrid* knowledge representations.

These findings imply that the type of decision problem strongly influences the choice of algorithm and representation. They also demonstrate that TDMIoT can reveal structural patterns across the literature, patterns which can subsequently inform more flexible and adaptive decision-making models.

Objective 3: Serve as a semantic tool for algorithmic design in decision-making processes IoT systems

The associations identified above also provide evidence of TDMIoT utility as a semantic tool for design. For instance:

- Problems classified under *Forecasting* objective were associated both with $\{measure\ uncertainty + data-driven\}$ and with $\{measure\ uncertainty + probability-driven\}$ approaches. While traditional literature treats forecasting as a probabilistic modeling task, recent works have increasingly employed recurrent neural networks a *supervised learning* technique.
- Decision problems with *Optimization* as objective were generally associated with $\{parameter\ uncertainty + criterion-driven\}$ approaches, confirming the reliance on models with parameter-bound constraints and objective functions.
- *Planning* problems tended to combine *hybrid* knowledge representations with *probability-driven* or *criterion-driven* algorithm approaches, illustrating the need for symbolic–mathematical integration when modeling dynamic environments.

These examples illustrate how the taxonomy supports the establishment of *problem–solution* associations that might otherwise be implicit or scattered in the literature. While an expert may recognize such links intuitively, a systematic relational model such as TDMIoT makes them explicit, structured, and machine-readable. This confirms the potential of the taxonomy to act as a semantic knowledge representation for algorithmic design in IoT decision-making.

4.5.2. TDMIoT as a Design Framework

TDMIoT was conceived to guide the construction of IoT decision-making models by structuring them in a *problem–solution* association. To evaluate this function, a case study was employed, following the methodology outlined in Figure 4.6. For illustration, the case study of COVID-19 Crowd Management is presented, described initially in [22]. This case was selected because it addresses a real-world and time-sensitive IoT decision problem: managing crowds during the COVID-19 pandemic by recommending Points of Interest (POIs) that minimize congestion. The system aims to suggest the closest POI to a user’s location while also considering occupancy levels, thus balancing accessibility and public health concerns. The problem was

systematically analyzed using the TDMIoT framework. This analysis is organized into four distinct stages: (1) defining the problem, (2) analyzing the problem, (3) assessing potential solutions, and (4) selecting the best solution (according to Fig. 4.6). The structured mapping of this process is presented for clarity.

Stage 1: Problem Definition

The initial step consisted of defining the decision problem within TDMIoT dimensions. The *objective* was framed as *estimation*, with the decision occurring at the *application* layer and involving primarily *system interaction*. The task was to calculate POI congestion without requiring continuous monitoring of the user. This framing established the taxonomic anchor points for subsequent analysis.

Stage 2: Problem Analysis

The analysis phase examined the suitability of various *algorithmic approaches*. The options were systematically discarded:

- *Data-driven* is discarded by the lack of sufficient historical data to train predictive models.
- *Probability-driven* is discarded as no probabilistic model applied in the context of the COVID-19 pandemic to model the flows of people during the day in a POI exists.
- *Criterion-driven* approaches are more suitable for *optimization* or *planning* than for the estimation task at hand.

The problem was best aligned with a *utility-driven* approach and characterized by *model uncertainty*. Furthermore, the solution architecture was determined to require *centralized processing*, where a recommendation system receives requests and provides responses.

Stage 3: Solution Assessment

Based on the taxonomic classification, two candidate solution families were considered:

- *Multicriteria* technique as evaluate preferences among alternatives. However, potentially applicable, these techniques do not adequately capture how the choices of other users affect congestion levels.
- *Game theory* technique, particularly the field of Behavioral Game Theory, explicitly models human decision-making behaviors based on experimentation. This approach was found to be better aligned with the problem requirements, as it allows modeling of how users decisions regarding POIs are interdependent.

Stage 4: Solution Selection

Within the behavioral game theory family, the logit level-k model was identified as the most suitable algorithmic technique. This model assumes that human agents make mistakes when the cost of doing so is relatively low, and it has been successfully applied to problems involving limited-capacity choices (e.g., diners selecting between restaurants). In this case study, the logit level-k model was adapted to estimate POI occupancy and, consequently, predict user choices. The selected *knowledge representation* was *mathematical*, which is consistent with the formalism required to implement behavioral game theory.

4.6. Implications of TDMIoT

4.6.1. TDMIoT as a description mechanism

TDMIoT serves not only as a taxonomy but also as a descriptive mechanism that supports both research and practical development of IoT decision-making systems. As a classification scheme, TDMIoT enables the direct specification of decision problems and the algorithmic approaches used to solve them. This facilitates comparative analysis across studies, enabling researchers to clearly define the novelty of their contributions. Moreover, the systematic classification of 56 studies revealed significant associations and gaps among the dimensions, highlighting the potential of TDMIoT to uncover emerging opportunities for exploration in IoT domains.

The evaluation of TDMIoT confirms its capacity to meet the three objectives of this thesis. First, it provides a structured overview of the state of the art in IoT decision-making. Second, it exposes the structural relationships among dimensions, demonstrating how objectives, processing, uncertainty, and algorithmic approaches interrelate. Third, it validates TDMIoT as a semantic tool for guiding algorithmic design. At the same time, the evaluation highlights underexplored areas, such as decentralized decision-making, middleware-level processes, and hybrid knowledge representations, which represent promising directions for future research where TDMIoT can serve as both a guide and a model.

Finally, IoT-based systems are typically developed within a service-oriented architecture. In this context, TDMIoT provides a shared vocabulary for describing the logical functionality of services, thereby enabling the construction of databases that map decision problems to algorithmic approaches. Such resources could be further leveraged to implement adaptive and learning mechanisms, reinforcing the role of TDMIoT as a foundation for dynamic, knowledge-driven IoT reconfiguration.

4.6.2. TDMIoT as a design framework for practitioners

The case study presented in Section 4.5.2 demonstrates the effectiveness of TDMIoT as a design framework. By systematically guiding the process from problem definition through to solution selection, TDMIoT enabled the identification of a non-trivial solution that may not have been immediately apparent. Specifically, the framework facilitated the exclusion of unsuitable approaches, classified the decision problem under model uncertainty, and ultimately selected a behavioral game-theoretic technique as the most appropriate solution.

The structured mapping confirms that TDMIoT not only organizes existing research but also provides practical guidance for designing new decision-making models in IoT contexts. In this sense, the third objective of the thesis—validating TDMIoT as a semantic tool for algorithmic design—is fulfilled.

From a practical perspective, TDMIoT supports developers and engineers by offering guidelines to analyze and define decision-making problems within IoT application domains. This contribution is particularly relevant given the multidisciplinary nature of IoT development, where practitioners may lack expertise in either software engineering or domain-specific concepts. By narrowing the search space for algorithmic solutions and guiding the selection of techniques, TDMIoT simplifies the design process.

Nonetheless, the framework is highly dependent on the user’s understanding of the problem and the classification process. As such, it remains prone to interpretation errors. This limitation highlights the importance of developing supporting tools, such as semantic knowledge bases or decision-support systems, that can minimize ambiguity and enhance the robustness of the framework in real-world design scenarios.

4.6.3. Enabling Self-Configuration in Autonomic Computing

While TDMIoT proves effective as a design framework, its contribution extends beyond supporting the classification of decision-making problems and narrowing the search space for algorithmic solutions. In the broader context of autonomic computing, TDMIoT can serve as the foundation for enabling the self-configuration property, a central capability of adaptive IoT systems.

The framework currently provides a semantic representation of decision-making problems and algorithmic strategies, but its utility is limited to guiding human designers. To move towards autonomic behavior, TDMIoT must be extended with fine-grained features and explicit relationships between problem types and algorithmic techniques. This extension is crucial to address the automatic algorithm selection problem, where the system itself identifies and deploys the most suitable algorithm at runtime.

From this perspective, TDMIoT connects directly with the thesis objective of developing a reconfiguration mechanism for IoT systems. By serving as the semantic knowledge base that supports algorithm selection, it establishes the conditions for runtime adaptation and system updates without human intervention. In this way, TDMIoT moves from being a design-time aid for developers to becoming a cornerstone in enabling self-configuring IoT services.

4.7. Conclusion

In light of the research objectives, the Taxonomy for Decision-Making in IoT (TDMIoT) has provided a structured synthesis of the state of the art in decision-making processes within IoT application domains. The analysis revealed that the majority of studies have centered on decision processes with *prediction* as the primary objective, and that the most prevalent algorithmic approach has been *data-driven*. Moreover, by systematically organizing dimensions and subcategories, TDMIoT enabled the identification of meaningful relationships across categories. For instance, it was

observed that *decentralized* processing is frequently associated with *utility-driven* approaches, while measure uncertainty tends to align with *probability-driven* techniques. The discovery of such associations demonstrates the potential of TDMIoT as a semantic tool capable of linking decision problems with suitable algorithmic solutions.

The utility of TDMIoT as a design framework was further validated in its application to the COVID-19 Crowd Management project, where it guided the selection of algorithmic solutions for *utility-driven* models. This case study exemplifies the practical value of the taxonomy in supporting the systematic design of decision processes in real-world IoT scenarios.

Beyond its immediate role as a descriptive framework, TDMIoT also addresses a critical gap in the field: the absence of a structured vocabulary to capture the interplay between objectives, processing layers, interaction types, and uncertainties in IoT-based decision-making. By situating algorithms in relation to these aspects, TDMIoT provides the foundation for a shared semantic representation of IoT decision processes.

Finally, the structuring of the decision-making problem space achieved through TDMIoT establishes the conceptual basis upon which the Ontology for Decision-Making in IoT (O-DMIoT) is developed in the following chapter. While TDMIoT organizes and classifies decision processes, O-DMIoT extends this framework into a formal semantic representation, enabling reasoning, interoperability, and ultimately supporting the autonomic self-configuration of IoT systems.

References

- [1] H. B. Abdalla. “A brief survey on big data: technologies, terminologies and data-intensive applications”. In: *Journal of Big Data* 9.1 (Nov. 2022). URL: <https://doi.org/10.1186/s40537-022-00659-3>.
- [2] D. H. Abdulazeez and S. K. Askar. “Offloading Mechanisms Based on Reinforcement Learning and Deep Learning Algorithms in the Fog Computing Environment”. In: *IEEE Access* 11 (2023), pp. 12555–12586. URL: <https://doi.org/10.1109/access.2023.3241881>.
- [3] S. Ahmad et al. “Optimal Policy-Making for Municipal Waste Management Based on Predictive Model Optimization”. In: *IEEE Access* 8 (2020), pp. 218458–218469. URL: <https://ieeexplore.ieee.org/document/9284435/>.
- [4] M. A. Akbar et al. “A Multicriteria Decision Making Taxonomy of IOT Security Challenging Factors”. In: *IEEE Access* 9 (2021), pp. 128841–128861. URL: <https://doi.org/10.1109/access.2021.3104527>.
- [5] M. B. Alaya et al. “Toward semantic interoperability in oneM2M architecture”. In: *IEEE Communications Magazine* 53.12 (Dec. 2015). Conference Name: IEEE Communications Magazine, pp. 35–41.
- [6] N. Alex, C. C. Sobin, and J. Ali. “A Comprehensive Study on Smart Agriculture Applications in India”. In: *Wireless Personal Communications* 129.4 (Mar. 2023), pp. 2345–2385. URL: <https://doi.org/10.1007/s11277-023-10234-5>.

- [7] F. Alkhabbas, R. Spalazzese, and P. Davidsson. “Characterizing Internet of Things Systems through Taxonomies: A Systematic Mapping Study”. en. In: *Internet of Things 7* (Sept. 2019), p. 100084. URL: <https://www.sciencedirect.com/science/article/pii/S2542660519300307> (visited on 03/16/2023).
- [8] S. Alshathri et al. “Digital Twin-Based Automated Fault Diagnosis in Industrial IoT Applications”. In: *Computers, Materials & Continua* 75.1 (2023), pp. 183–196. URL: <https://doi.org/10.32604/cmc.2023.034048>.
- [9] M. Bermudez-Edo et al. “IoT-Lite: A Lightweight Semantic Model for the Internet of Things”. In: *2016 Intl IEEE Conferences on Ubiquitous Intelligence & Computing, Advanced and Trusted Computing, Scalable Computing and Communications, Cloud and Big Data Computing, Internet of People, and Smart World Congress (UIC/ATC/ScalCom/CBDCCom/IoP/SmartWorld)*. July 2016, pp. 90–97.
- [10] M. Bohanec. “Decision Making: A Computer-Science and Information-Technology Viewpoint”. In: *Interdisciplinary Description of Complex Systems - scientific journal* 7 (Dec. 2009), pp. 22–37.
- [11] S. Boulkaboul and D. Djenouri. “DFIOT: Data Fusion for Internet of Things”. In: *Journal of Network and Systems Management* 28.4 (Mar. 2020), pp. 1136–1160. URL: <https://doi.org/10.1007/s10922-020-09519-y>.
- [12] A. Cavacini. “What is the best database for computer science journal articles?” en. In: *Scientometrics* 102.3 (Mar. 2015), pp. 2059–2071.
- [13] C. Cetina et al. “Autonomic Computing through Reuse of Variability Models at Runtime: The Case of Smart Homes”. In: *Computer* 42.10 (Oct. 2009). Conference Name: Computer, pp. 37–43.
- [14] Y.-A. Chen et al. “An Ensemble Learning Model for Agricultural Irrigation Prediction”. In: *2021 International Conference on Information Networking (ICOIN)*. Jeju Island, Korea (South): IEEE, Jan. 2021, pp. 311–316.
- [15] M. Compton et al. “The SSN ontology of the W3C semantic sensor network incubator group”. en. In: *Journal of Web Semantics* 17 (Dec. 2012), pp. 25–32. URL: <https://www.sciencedirect.com/science/article/pii/S1570826812000571> (visited on 02/07/2023).
- [16] T. H. Cormen, ed. *Introduction to algorithms*. 3rd ed. OCLC: ocn311310321. Cambridge, Mass: MIT Press, 2009, p. 359.
- [17] A. Curry. *Managing information and systems : the business perspective*. London New York: Routledge, 2006.
- [18] F. Cuzzolin. “Uncertainty measures: The big picture”. en. In: *arXiv:2104.06839 [cs, math, stat]* (Apr. 2021). arXiv: 2104.06839. URL: <http://arxiv.org/abs/2104.06839>.
- [19] S. De et al. “Service modelling for the Internet of Things”. In: *2011 Federated Conference on Computer Science and Information Systems (FedCSIS)*. Sept. 2011, pp. 949–955.

- [20] M. Deng, C. C. Menassa, and V. R. Kamat. “From BIM to digital twins: a systematic review of the evolution of intelligent building representations in the AEC-FM industry”. In: *Journal of Information Technology in Construction* 26 (Feb. 2021), pp. 58–83. URL: <https://doi.org/10.36680/j.itcon.2021.005>.
- [21] M. Drog. “Basics on Uncertainties”. en. In: *Dealing with Uncertainties: A Guide to Error Analysis*. Ed. by M. Drog. Berlin, Heidelberg: Springer, 2009, pp. 17–48.
- [22] L. Durán-Polanco and M. Siller. “Crowd management COVID-19”. In: *Annual Reviews in Control* (2021).
- [23] O. Eris, J. L. Drury, and D. Ercolini. “A Collaboration-centric Taxonomy of the Internet of Things: Implications for Awareness Support”. en. In: *Internet of Things* 15 (Sept. 2021), p. 100403. URL: <https://www.sciencedirect.com/science/article/pii/S2542660521000470> (visited on 03/16/2023).
- [24] O. Eris, J. L. Drury, and D. Ercolini. “A Collaboration-centric Taxonomy of the Internet of Things: Implications for Awareness Support”. en. In: *Internet of Things* 15 (Sept. 2021), p. 100403. URL: <https://www.sciencedirect.com/science/article/pii/S2542660521000470> (visited on 03/16/2023).
- [25] M. Fahmideh et al. “Software Engineering for Internet of Things: The Practitioners’ Perspective”. en. In: (), p. 26.
- [26] M. Ge, H. Bangui, and B. Buhnova. “Big Data for Internet of Things: A Survey”. en. In: *Future Generation Computer Systems* 87 (Oct. 2018), pp. 601–614.
- [27] A. Goel. *Computer Fundamentals*. Pearson Education, 2010.
- [28] P. Gope et al. “A Secure IoT-Based Modern Healthcare System With Fault-Tolerant Decision Making Process”. In: *IEEE Journal of Biomedical and Health Informatics* 25.3 (Mar. 2021), pp. 862–873. URL: <https://ieeexplore.ieee.org/document/9134756/>.
- [29] M. A. Guillén-Navarro et al. “A deep learning model to predict lower temperatures in agriculture”. In: *Journal of Ambient Intelligence and Smart Environments* 12.1 (Jan. 2020), pp. 21–34. URL: <https://www.medra.org/servlet/aliasResolver?alias=iospress&doi=10.3233/AIS-200546>.
- [30] R. Guizzardi et al. “A core ontology on decision making”. English. In: *CEUR workshop proceedings*. Vol. 2728. ISSN: 1613-0073. Rheinisch Westfälische Technische Hochschule, Nov. 2020, pp. 9–21. URL: <https://research.utwente.nl/en/publications/a-core-ontology-on-decision-making> (visited on 07/29/2022).
- [31] N. Gulati and P. D. Kaur. “A game theoretic approach for conflict resolution in argumentation enabled social IoT networks”. en. In: *Ad Hoc Networks* 107 (Oct. 2020), p. 102222.
- [32] C. Guttman. “Towards a Taxonomy of Decision Making Problems in Multi-Agent Systems”. en. In: *Multiagent System Technologies*. Ed. by L. Braubach et al. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2009, pp. 195–201.

- [33] R. H. Hariri, E. M. Fredericks, and K. M. Bowers. “Uncertainty in big data analytics: survey, opportunities, and challenges”. en. In: *Journal of Big Data* 6.1 (Dec. 2019), p. 44. URL: <https://journalofbigdata.springeropen.com/articles/10.1186/s40537-019-0206-3>.
- [34] F. Hasić et al. “Decision model change patterns for dynamic system evolution”. en. In: *Knowledge and Information Systems* 62.9 (Sept. 2020), pp. 3665–3696. URL: <https://doi.org/10.1007/s10115-020-01469-w> (visited on 07/29/2022).
- [35] U. Hassler. “Basic Concepts from Probability Theory”. In: *Stochastic Processes and Calculus*. Series Title: Springer Texts in Business and Economics. Cham: Springer International Publishing, 2016, pp. 13–43.
- [36] T. Hastie. *The elements of statistical learning : data mining, inference, and prediction*. New York: Springer, 2009.
- [37] K. Hur, X. Jin, and K.-H. Lee. “Automated deployment of IoT services based on semantic description”. In: *2015 IEEE 2nd World Forum on Internet of Things (WF-IoT)*. Dec. 2015, pp. 40–45.
- [38] L. Ismail and R. Buyya. “Artificial Intelligence Applications and Self-Learning 6G Networks for Smart Cities Digital Ecosystems: Taxonomy, Challenges, and Future Directions”. In: *Sensors* 22.15 (Aug. 2022), p. 5750. URL: <https://doi.org/10.3390/s22155750>.
- [39] *ITU-T Y.4000/Y.2060 (06/2012)*. Tech. rep. ITU-T Y.4000/Y.2060 (06/2012). June 2012. URL: <http://handle.itu.int/11.1002/1000/11559>.
- [40] K. Janowicz et al. “SOSA: A lightweight ontology for sensors, observations, samples, and actuators”. en. In: *Journal of Web Semantics* 56 (May 2019), pp. 1–10. URL: <https://linkinghub.elsevier.com/retrieve/pii/S1570826818300295> (visited on 02/07/2023).
- [41] X. Jin et al. “A fast and scalable approach for IoT service selection based on a physical service model”. en. In: *Information Systems Frontiers* 19.6 (Dec. 2017), pp. 1357–1372. URL: <https://doi.org/10.1007/s10796-016-9650-1> (visited on 05/03/2023).
- [42] A. Kirsch. “A unifying computational model of decision making”. en. In: *Cognitive Processing* 20.2 (May 2019), pp. 243–259. URL: <https://doi.org/10.1007/s10339-019-00904-3> (visited on 07/04/2022).
- [43] E. Kornysheva and R. Deneckère. “Decision-Making Ontology for Information System Engineering”. en. In: *Conceptual Modeling – ER 2010*. Ed. by J. Parsons et al. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2010, pp. 104–117.
- [44] R. Laraki, J. Renault, and S. Sorin. “Introduction”. en. In: *Mathematical Foundations of Game Theory*. Ed. by R. Laraki, J. Renault, and S. Sorin. Universitext. Cham: Springer International Publishing, 2019, pp. 1–12.
- [45] J. Li et al. “Towards decision support systems for energy management in the smart industry and Internet of Things”. In: *Computers & Industrial Engineering* 161 (Nov. 2021), p. 107671. URL: <https://doi.org/10.1016/j.cie.2021.107671>.

- [46] M. Mahbub, M. M. Hossain, and M. S. A. Gazi. “IoT-Cognizant cloud-assisted energy efficient embedded system for indoor intelligent lighting, air quality monitoring, and ventilation”. In: *Internet of Things* 11 (Sept. 2020), p. 100266. URL: <https://doi.org/10.1016/j.iot.2020.100266>.
- [47] S. Mahdavi-Hezavehi, P. Avgeriou, and D. Weyns. “Chapter 3 - A Classification Framework of Uncertainty in Architecture-Based Self-Adaptive Systems With Multiple Quality Requirements”. en. In: *Managing Trade-Offs in Adaptable Software Architectures*. Ed. by I. Mistrik et al. Boston: Morgan Kaufmann, Jan. 2017, pp. 45–77.
- [48] Y. Meidan et al. “D-Score: An expert-based method for assessing the detectability of IoT-related cyber-attacks”. In: *Computers & Security* 126 (Mar. 2023), p. 103073. URL: <https://doi.org/10.1016/j.cose.2022.103073>.
- [49] N. A. Mejía, G. S. Boada, and J. R. de Almeida Amazonas. “Decision Making Under Uncertainty for the Deployment of Future Networks in IoT Scenarios”. en. In: *Broadband Communications, Networks, and Systems*. Ed. by H. Gao et al. Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering. Cham: Springer International Publishing, 2021, pp. 174–184.
- [50] M. Mendoza and E. Gutiérrez-Peña. “Decision Theory”. en. In: *International Encyclopedia of Education (Third Edition)*. Ed. by P. Peterson, E. Baker, and B. McGaw. Oxford: Elsevier, Jan. 2010, pp. 119–124.
- [51] K. Messaoudi et al. “A survey of UAV-based data collection: Challenges, solutions and future perspectives”. In: *Journal of Network and Computer Applications* 216 (July 2023), p. 103670. URL: <https://doi.org/10.1016/j.jnca.2023.103670>.
- [52] A. Mijuskovic. “Towards integration of logistics processes from a cloud/fog-edge computing perspective”. In: *2021 IEEE 25th International Enterprise Distributed Object Computing Workshop (EDOCW)*. IEEE, Oct. 2021. URL: <https://doi.org/10.1109/edocw52865.2021.00069>.
- [53] M. Mohaghegh and A. Furlan. “Systematic problem-solving and its antecedents: a synthesis of the literature”. en. In: *Management Research Review* 43.9 (Feb. 2020), pp. 1033–1062. URL: <https://www.emerald.com/insight/content/doi/10.1108/MRR-06-2019-0284/full/html>.
- [54] M. Mohsin et al. “IoTChecker: A data-driven framework for security analytics of Internet of Things configurations”. en. In: *Computers & Security* 70 (Sept. 2017), pp. 199–223. URL: <https://www.sciencedirect.com/science/article/pii/S0167404817301190> (visited on 02/13/2023).
- [55] G. A. Moreno et al. “Uncertainty reduction in self-adaptive systems”. en. In: *Proceedings of the 13th International Conference on Software Engineering for Adaptive and Self-Managing Systems*. Gothenburg Sweden: ACM, May 2018, pp. 51–57.
- [56] B. A. Mozzaquatro, R. Jardim-Goncalves, and C. Agostinho. “Towards a reference ontology for security in the Internet of Things”. In: *2015 IEEE International Workshop on Measurements & Networking (M&N)*. Oct. 2015, pp. 1–6.

- [57] R. C. Nickerson, U. Varshney, and J. Muntermann. “A method for taxonomy development and its application in information systems”. en. In: *European Journal of Information Systems* 22.3 (May 2013), pp. 336–359. URL: <https://www.tandfonline.com/doi/full/10.1057/ejis.2012.26>.
- [58] E. Oztemel and S. Gursev. “Literature review of Industry 4.0 and related technologies”. In: *Journal of Intelligent Manufacturing* 31.1 (July 2018), pp. 127–182. URL: <https://doi.org/10.1007/s10845-018-1433-8>.
- [59] K. K. Patel, S. M. Patel, and P. Scholar. “Internet of Things-IOT: Definition, Characteristics, Architecture, Enabling Technologies, Application & Future Challenges”. en. In: (2016), p. 10.
- [60] J. Pearl. “Heuristics: Intelligent search strategies for computer problem solving”. In: (Jan. 1984). URL: <https://www.osti.gov/biblio/5127296>.
- [61] L. Petnga and M. Austin. “An ontological framework for knowledge modeling and decision support in cyber-physical systems”. en. In: *Advanced Engineering Informatics* 30.1 (Jan. 2016), pp. 77–94. URL: <https://linkinghub.elsevier.com/retrieve/pii/S1474034616300039> (visited on 03/01/2023).
- [62] N. Prat, I. Comyn-Wattiau, and J. Akoka. “A Taxonomy of Evaluation Methods for Information Systems Artifacts”. en. In: *Journal of Management Information Systems* 32.3 (July 2015), pp. 229–267. URL: <http://www.tandfonline.com/doi/full/10.1080/07421222.2015.1099390>.
- [63] R. Rădulescu et al. “Multi-objective multi-agent decision making: a utility-based analysis and survey”. en. In: *Autonomous Agents and Multi-Agent Systems* 34.1 (Apr. 2020), p. 10.
- [64] W. Rafique, A. S. Hafid, and J. Qadir. “Developing smart city services using intent-aware recommendation systems: A survey”. In: *Transactions on Emerging Telecommunications Technologies* 34.4 (Jan. 2023). URL: <https://doi.org/10.1002/ett.4728>.
- [65] A. J. Ramirez, A. C. Jensen, and B. H. C. Cheng. “A taxonomy of uncertainty for dynamically adaptive systems”. In: *2012 7th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*. Zurich, Switzerland: IEEE, June 2012, pp. 99–108.
- [66] T. Reineking. “Belief Functions: Theory and Algorithms”. en. In: (), p. 165.
- [67] A. Rhayem, M. B. A. Mhiri, and F. Gargouri. “Semantic Web Technologies for the Internet of Things: Systematic Literature Review”. In: *Internet of Things* 11 (Sept. 2020), p. 100206. URL: <https://doi.org/10.1016/j.iot.2020.100206>.
- [68] E. Rios et al. “Continuous Quantitative Risk Management in Smart Grids Using Attack Defense Trees”. In: *Sensors* 20.16 (Aug. 2020), p. 4404. URL: <https://doi.org/10.3390/s20164404>.
- [69] Y. Rizk, M. Awad, and E. W. Tunstel. “Decision Making in Multiagent Systems: A Survey”. In: *IEEE Transactions on Cognitive and Developmental Systems* 10.3 (Sept. 2018), pp. 514–529. URL: <https://ieeexplore.ieee.org/document/8365805/>.

- [70] J. J. Salazar González. *Programación matemática*. Spanish. OCLC: 48639998. Madrid: Díaz de Santos, 2001.
- [71] U. J. L. dos Santos et al. “AgriPrediction: A proactive internet of things model to anticipate problems and improve production in agricultural crops”. en. In: *Computers and Electronics in Agriculture* 161 (June 2019), pp. 202–213.
- [72] D. Saraswat et al. “Explainable AI for Healthcare 5.0: Opportunities and Challenges”. In: *IEEE Access* 10 (2022), pp. 84486–84517. URL: <https://doi.org/10.1109/access.2022.3197671>.
- [73] R. van de Schoot et al. “Bayesian statistics and modelling”. en. In: *Nature Reviews Methods Primers* 1.1 (Jan. 2021). Bandiera_abtest: a Cg_type: Nature Research Journals Number: 1 Primary_atype: Reviews Publisher: Nature Publishing Group Subject_term: Scientific community;Statistics Subject_term_id: scientific-community;statistics, pp. 1–26. URL: <https://www.nature.com/articles/s43586-020-00001-2>.
- [74] S. Seuken and S. Zilberstein. “Formal models and algorithms for decentralized decision making under uncertainty”. en. In: *Autonomous Agents and Multi-Agent Systems* 17.2 (Oct. 2008), pp. 190–250.
- [75] N. Seydoux et al. “IoT-O, a Core-Domain IoT Ontology to Represent Connected Devices Networks”. en. In: *Knowledge Engineering and Knowledge Management*. Ed. by E. Blomqvist et al. Lecture Notes in Computer Science. Cham: Springer International Publishing, 2016, pp. 561–576.
- [76] G. Shafer. “Dempster-shafer theory”. In: *Encyclopedia of artificial intelligence* 1 (1992), pp. 330–331.
- [77] H. A. Simon. *The new science of management decision*. en. New York: Harper & Brothers, 1960. URL: <http://content.apa.org/books/13978-000> (visited on 10/17/2022).
- [78] D. Sinreich. “An architectural blueprint for autonomic computing”. In: 2006. URL: <https://www.semanticscholar.org/paper/An-architectural-blueprint-for-autonomic-computing-Sinreich/47c37d43f43e2be57f6f2bc668979f784911e953> (visited on 06/23/2023).
- [79] A. Tolba, O. Said, and Z. Al-Makhadmeh. “MDS: Multi-level decision system for patient behavior analysis based on wearable device information”. en. In: *Computer Communications* 147 (Nov. 2019), pp. 180–187.
- [80] G. Tsafnat et al. “Systematic review automation technologies”. In: *Systematic Reviews* 3.1 (July 2014), p. 74. URL: <https://doi.org/10.1186/2046-4053-3-74>.
- [81] M. Usman et al. “Taxonomies in software engineering: A Systematic mapping study and a revised taxonomy development method”. en. In: *Information and Software Technology* 85 (May 2017), pp. 43–59.
- [82] M. Velasquez and P. Hester. “An analysis of multi-criteria decision making methods”. In: *International Journal of Operations Research* 10 (May 2013), pp. 56–66.

- [83] J. Wątróbski. “An Ontology Supporting Multiple-Criteria Decision Analysis Method Selection”. en. In: *Intelligent Decision Technologies 2016*. Ed. by I. Czarnowski et al. Smart Innovation, Systems and Technologies. Cham: Springer International Publishing, 2016, pp. 89–99.
- [84] M. Yalaoui and S. Boukhedouma. “A survey on data quality: principles, taxonomies and comparison of approaches”. In: *2021 International Conference on Information Systems and Advanced Technologies (ICISAT)*. IEEE, Dec. 2021. URL: <https://doi.org/10.1109/icisat54145.2021.9678209>.
- [85] J. Yan and B. Li. “Research hotspots and tendency of intelligent manufacturing”. In: *Chinese Science Bulletin* 65.8 (Feb. 2020), pp. 684–694. URL: <https://doi.org/10.1360/n972019-00125>.
- [86] V. Y. Yoon and M. Adya. “Expert Systems Construction”. en. In: *Encyclopedia of Information Systems*. Elsevier, 2003, pp. 291–300. URL: <https://linkinghub.elsevier.com/retrieve/pii/B012227240400068X> (visited on 10/28/2022).
- [87] Yuchae Jung and Yong Ik Yoon. “Monitoring senior wellness status using multimodal biosensors”. In: *2016 International Conference on Big Data and Smart Computing (BigComp)*. Hong Kong: IEEE, Jan. 2016, pp. 435–438. URL: <https://ieeexplore.ieee.org/document/7425965/>.
- [88] K. Zaheer et al. “A Survey of Decision-Theoretic Models for Cognitive Internet of Things (CIoT)”. In: *IEEE Access* 6 (2018). Conference Name: IEEE Access, pp. 22489–22512.

5. An Ontology for Decision Making in the IoT (O-DMIoT)

Abstract. The highly dynamic nature of IoT systems necessitates explicit representations of evolving functional requirements and their impact on decision-making and system behavior. Although existing IoT ontologies primarily focus on interoperability and data representation, they offer limited support for reasoning about the effects of changing requirements on algorithm selection, execution, and evaluation. This chapter presents the Ontology for Decision-Making in IoT (O-DMIoT), a semantic model developed to address this limitation. O-DMIoT establishes a unified framework that connects decision problem characterization, algorithm design, service execution, and performance-based evaluation within IoT systems. The ontology is based on the Unified Foundational Ontology (UFO) to ensure conceptual rigor and is developed using the SABiO methodology, which includes requirement identification, competency questions, conceptual modeling, and formalization. O-DMIoT employs a layered structure that differentiates requirement, design, and execution aspects, and models evaluation as a cross-cutting concern using Quality of Service and Quality of Experience metrics. Formalized in the OWL, the ontology supports semantic reasoning, traceability, and extensibility, and provides the semantic foundation for the adaptive algorithm selection and reconfiguration mechanisms discussed in subsequent chapters.

5.1. Introduction

The growing complexity of Internet of Things (IoT) systems has heightened the demand for systematic mechanisms that facilitate decision-making across heterogeneous environments. IoT applications incorporate a wide range of devices, services, data sources, and stakeholders, frequently functioning under conditions of uncertainty and changing functional requirements. Although ontologies are widely used to address interoperability and semantic integration in IoT systems [10, 35, 8, 34], most existing models primarily represent sensors, data, and services, offering limited support for reasoning about decision-making processes, algorithm selection, and evaluation.

Concurrently, research on decision-making and algorithm-selection ontologies has demonstrated the utility of semantic models for guiding method selection, workflow configuration, and performance assessment in domains such as data mining, multi-criteria decision analysis, and service selection [23, 39, 27]. Nevertheless, these

approaches are often domain-specific, address only isolated phases of the decision lifecycle, or do not explicitly integrate with IoT execution and service reconfiguration.

This chapter addresses this gap by introducing the Ontology for Decision-Making in IoT (O-DMIoT), a semantic model developed to characterize decision-making problems, algorithmic solutions, execution contexts, and evaluation mechanisms within IoT systems. O-DMIoT extends the Taxonomy for Decision-Making in IoT (TDMIoT) presented in previous chapters and operationalizes it as a reference ontology that is machine-processable, formally grounded, and suitable for reasoning and reconfiguration.

The ontology is developed using the Systematic Approach for Building Ontologies (SABiO) methodology [17], which ensures a systematic and purpose-driven ontology engineering process, and is grounded in the Unified Foundational Ontology (UFO) [21] to provide ontological rigor and conceptual clarity. By distinguishing among requirement, design, and execution layers, and by modeling evaluation as a cross-cutting concern based on performance and metrics, O-DMIoT enables semantic traceability throughout the lifecycle of decision-making algorithms.

The remainder of this chapter is organized as follows. Section 5.3 defines the scope, objectives, and competency questions guiding the ontology. Section 5.4 presents the ontological foundations and UFO grounding. Section 5.5 details the development methodology. Sections 5.6 and 5.7 describe the conceptual model and its Web Ontology Language (OWL) implementation, respectively. Section 5.8 discusses validation. Finally, the chapter concludes by positioning O-DMIoT within the overall thesis framework.

5.2. Background and related work

5.2.1. Ontologies in IoT

Ontologies play a central role in addressing the heterogeneity inherent to IoT systems by providing shared, machine-interpretable vocabularies that enable semantic interoperability across devices, platforms, and applications [10, 35, 8, 34]. As IoT environments integrate diverse sensing technologies, communication protocols, and application domains, ontologies have become a foundational mechanism for data integration, reasoning, and cross-system understanding.

Early IoT ontologies concentrated on representing sensors, observations, and basic device metadata. The Semantic Sensor Network ontology (SSN) and its lightweight core, SOSA, provide a widely adopted conceptualization of sensors, observations, actuators, and deployments, thereby establishing a foundation for interoperability across heterogeneous sensing infrastructures [13, 16]. These ontologies adopt a *data* and *observation* centric perspective and have been frequently reused or extended in subsequent models.

Subsequent ontologies extended the scope to include architectural and service-level modeling. IoT-O offers an ontology for describing IoT entities, services, and observations within service-oriented architectures [3], thereby supporting integration across platforms and middleware [40]. DogOnt addresses intelligent environments by modeling devices, functionalities, and interactions in domotic systems, which enables semantic interoperability at the level of device capabilities and control [11]. These

developments illustrate the progression from sensing-focused models to system-level semantics.

To address scalability and performance challenges in dynamic, resource-constrained environments, lightweight ontologies such as IoT-Lite and IoT-Stream have been developed [10, 16, 34]. These models intentionally limit expressiveness to essential concepts, thereby supporting efficient querying, high-rate data streams, and edge deployment. Although effective for data annotation and discovery, they generally lack richer semantic expressiveness and advanced reasoning capabilities.

Parallel research explored ontologies that prioritize coordination, interaction, and intelligence within IoT systems. The Web of Objects (WoO) initiative conceptualizes IoT entities as interactive objects capable of participating in complex application workflows [12], which enables semantic orchestration of heterogeneous components using in later ontological approaches [14]. Ontology-driven multi-agent approaches, such as GAIA-MAS, integrate IoT environments with agent-based organizational models, emphasizing roles, interactions, and coordination mechanisms [5, 4]. These studies underscore the importance of interaction and decision-making semantics beyond basic sensing and data exchange.

Specialized ontologies such as IoTSec and IoTChecker address security and trust by modeling security properties, configurations, vulnerabilities, and dependencies among IoT components [30, 29]. These ontologies illustrate the application of semantic reasoning to detect misconfigurations and assess security compliance. Nevertheless, their focus remains primarily on technical security issues, without addressing broader decision-making or human-centered considerations.

More recent efforts aim to reduce fragmentation by developing unified or cross-domain ontologies. FIESTA-IoT integrates multiple existing ontologies, including SSN and IoT-Lite, to enable semantic interoperability across heterogeneous testbeds and platforms [1]. IoTO++ and related extensions aim to provide more comprehensive coverage of IoT systems by incorporating concerns such as management, governance, and extensibility [2]. Similarly, efforts like SemIoE and IoT fabric models with SDDC integration address large-scale, cross-domain interoperability and industrial IoT scenarios [14, 2, 6].

Despite significant progress, systematic reviews indicate that no single IoT ontology has achieved universal adoption [35, 8, 34]. Interoperability continues to be the primary design objective, whereas issues such as privacy, ethics, socio-technical interaction, and human-centered evaluation are inconsistently addressed and often considered secondary [25, 14, 34]. Additionally, evaluation practices for IoT ontologies remain heterogeneous, with limited consensus on validation criteria, benchmarking methods, or comparative assessment [35, 1].

Empirical studies nonetheless demonstrate tangible benefits of ontology-based approaches in IoT systems, such as enhanced data reuse, increased automation, and measurable reductions in integration overhead and communication latency when semantic processing is implemented at the edge [36, 40, 38]. These findings confirm the practical value of ontologies as enabling infrastructure rather than merely descriptive artifacts.

The evolution of IoT ontologies demonstrates a gradual transition from isolated, domain-specific models to lighter, reusable, and more unified semantic frameworks [14, 1, 2]. However, existing ontologies do not explicitly address decision-making problems, algorithm characterization, performance-based evaluation, and reconfig-

uration within a single coherent semantic model. These limitations motivate the development of O-DMIoT, which extends existing IoT ontologies to explicitly support decision-making, algorithm selection, and evaluation across heterogeneous and socio-technical IoT contexts.

5.2.2. Ontologies for decision-making / algorithm characterization

Beyond general-purpose IoT ontologies, a separate body of research addresses the explicit modeling of decision-making processes, algorithm characteristics, and selection mechanisms. These ontologies are designed to support informed selection among alternative methods, services, or algorithms by providing semantic representations of problems, alternatives, criteria, and performance indicators.

One line of research focuses on ontologies for algorithm recommendation and meta-learning. For example, [32] introduces an ontology-based advisory function for recommending feature selection algorithms based on dataset characteristics and quality measures. Similarly, OMA-ML [24] utilizes an ontology to guide the selection of machine learning pipelines in AutoML contexts, taking into account task type, dataset properties, and execution constraints. While these approaches demonstrate the utility of ontologies for algorithm selection, they are generally limited to supervised learning tasks and machine learning pipelines, with restricted applicability outside data mining domains.

A related group of ontologies addresses meta-mining and workflow selection. The Data Mining Optimization Ontology (DMOP) [23] formalizes tasks, algorithms, models, workflows, and experiments to support meta-learning and meta-mining throughout the knowledge discovery process. DMOP enables reasoning about the suitability of algorithms for specific problem classes by correlating algorithmic characteristics with dataset properties. Although highly expressive, DMOP is tailored to data mining workflows and does not explicitly address IoT-specific issues such as service deployment, interaction modalities, or runtime reconfiguration.

Other works focus on *decision method selection*, particularly in the context of multi-criteria decision analysis (MCDA). Wątróbski [39] presents an ontology that characterizes MCDA methods based on their methodological capabilities, thereby supporting the selection of suitable decision methods for specific problems. OntoDTA [9] similarly offers ontology-guided assistance for selecting decision tree algorithms, primarily targeting novice users. While these ontologies effectively demonstrate ontology-driven method selection, they are limited to specific classes of decision techniques.

A more abstract approach is taken by ontologies that seek to formalize the decision-making process itself. Kornyshova and Deneckère [28] propose a decision-making ontology for information system engineering, modeling decision problems, alternatives, criteria, and evaluation mechanisms. Similarly, [22] introduces a foundational ontology of decision making based on economic preference theory, with a primary focus on human decision processes. Although these ontologies provide valuable conceptual clarity, they operate at a high level of abstraction and are not directly aligned with algorithmic execution or service-oriented decision-making in IoT systems.

In the IoT domain, decision-related ontologies frequently arise in the context of service selection. For instance, the Physical Service Model (PSM) ontology [26] mod-

els devices, resources, and services, as well as quality of service (QoS) attributes such as availability, spatial coverage, reliability, and cost, thereby enabling preference-aware service selection. While these approaches explicitly incorporate runtime and physical constraints, they often intertwine domain-specific assumptions with decision logic, which limits their reuse across different application domains.

Several common limitations are evident across these works (See Table 5.1). Most decision-making and algorithm-selection ontologies are restricted to specific domains, such as data mining, MCDA, or physical services, and often focus on isolated phases of the decision lifecycle or conflate problem characterization, algorithm description, and evaluation criteria. Furthermore, few proposals explicitly separate problem definition, design-time algorithm characterization, execution as services, and cross-cutting evaluation mechanisms.

These limitations highlight the need for a domain-independent ontology that supports decision-making about algorithms in IoT systems, rather than decision-making within a single algorithmic paradigm. O-DMIoT addresses this gap by offering a layered semantic model that distinguishes problem characterization, algorithm design, execution as services, and performance-based evaluation. This approach enables algorithm selection and reconfiguration across heterogeneous IoT contexts without embedding domain-specific decision logic.

Focus	Representative ontology	Scope	Limitation
Algorithm recommendation	FSDCQ [32]	Feature selection	ML-specific
Meta-mining	DMOP [23]	Data mining workflows	Non-IoT
MCDA	Wątróbski [39]	Decision methods	Technique-specific
Decision process	Kornysheva & De-neckère [27]	IS engineering	High abstraction
IoT service selection	PSM [26]	Physical services	Domain-bound

Table 5.1.: Representative ontologies related to decision-making and algorithm characterization

5.3. Ontology Scope and Requirements

The ontology addresses decision-making in IoT systems by providing a conceptual framework that defines decision problems, characterizes possible solutions, and guides algorithm selection. Its primary purpose is to support the reconfiguration of IoT systems in response to evolving functional requirements, thereby enabling adaptive and autonomic behaviors.

5.3.1. Domain and Purpose

Domain

Decision-making processes in IoT systems at the application level.

Scope

O-DMIoT focuses on decision-making problems at the application layer with centralized processing, aligning with practical algorithm selection scenarios.

Assumptions

- A decision-making process in an IoT application domain is a *problem-solution* association.
- A *problem* is the characterization of a decision-making process requirements in the IoT.
- A *solution* is the characterization of a decision-making algorithm.
- This work considers IoT systems under a service-oriented architecture.

5.3.2. Objectives

The main objectives of O-DMIoT are:

- Define decision problems in the IoT domain through a structured and semantically grounded representation.
- Characterize solutions and the relationships between problems, algorithms, and execution environments.
- Guide algorithm selection by aligning problem characteristics with algorithmic design strategies.
- Facilitate reconfiguration of IoT systems by enabling semantic interoperability across services.

5.3.3. Competency Questions

The Ontology for Decision Making in the IoT (O-DMIoT) aims to answer the following questions:

Problem-Oriented Competency Questions

- **CQ1.** What constitutes a decision problem in the IoT?
- **CQ2.** What are the characteristics of a problem?
- **CQ3.** How are problem characteristics related to each other?

Solution-Oriented Competency Questions

- **CQ4.** What constitutes a solution to a problem?
- **CQ5.** What are the characteristics of a problem solution?
- **CQ6.** What are the elements of a requirement specification?
- **CQ7.** What are the elements of a design specification?
- **CQ8.** How does execution operationalize a problem solution within an IoT system?

Evaluation-Oriented Competency Questions

- **CQ9.** What constitutes the evaluation of a solution?
- **CQ10.** Which metrics can be used to assess whether a solution meets requirements?

The stated competency questions define the minimal and sufficient conceptual requirements for describing decision problems, solutions, and evaluation in IoT systems. The questions were derived from (i) the TDMIoT taxonomy, the requirements of algorithm selection, and the reconfiguration objectives of autonomic IoT systems. Together, they comprehensively address problem definition, solution characterization, execution, and evaluation, which constitute the essential dimensions for effective algorithm selection.

5.3.4. Usage Scenarios and Stakeholders

The ontology is intended for use in the following scenarios:

- **Research:** Researchers can compare and describe decision-making models in IoT application domains, identifying gaps, and exploring novel approaches.
- **IoT system development:** The ontology provides a shared vocabulary that can be used by developers and engineers designing IoT systems in multidisciplinary contexts to express problems and solutions.
- **Autonomic Computing Systems:** The ontology enables the self-configuration property by providing the semantic foundation for algorithm selection in the IoT decision-making processes.

Representative use cases for O-MDIoT include:

- Selecting an appropriate decision-making algorithm at the application level in the IoT.
- Reconfiguring IoT services to adapt to new functional requirements.
- Evaluating and comparing algorithms in terms of their objectives, processing characteristics, and uncertainties handled.

5.4. Ontological Foundations

5.4.1. Overview of UFO

Foundational ontologies provide high-level categorizations that can serve across different domains. Foundational ontologies aim to provide common ground among domain ontologies to ease integration. The Unified Foundational Ontology (UFO) is a foundational ontology for conceptual modeling [21]. UFO goals are to provide foundations for domain analysis in conceptual modeling, support ontological analysis, conceptual clarification, and semantic explanation of content embedded in representational artifacts, and support humans in tasks such as domain understanding, problem-solving, and negotiating meaning. UFO is organized into three specifications:

1. **UFO-A.** An ontology of endurants. Endurants refer to individuals that exist in time with all their parts. In this context, an individual refers to a concrete instance of a class in the ontology, representing a specific entity in the modeled domain, as defined in Description Logic[7] and OWL [33].
2. **UFO-B.** An ontology of perdurant built on top of UFO-A. Perdurants refer to individuals who develop over time by accumulating temporal parts.
3. **UFO-C.** An ontology of social agents built upon UFO-A and UFO-B.

The layered construction serves to exemplify the principle of reuse.

UFO has been selected as the foundational ontology of this work due to its comprehensive nature and use in similar domains [15, 37]. Furthermore, UFO is based on formal ontology and philosophical theories, thereby ensuring high clarity and ontological rigor. Finally, UFO is used to improve the quality and consistency of conceptual models in information systems engineering.

Additionally, UFO can be represented through a UML extension called OntoUML, used for conceptual modeling and domain ontology engineering [20, 19]. OntoUML provides elements for the visual representation of the conceptual modeling developed in this work.

5.4.2. UFO for Decision-Making in IoT Systems

The adoption of the Unified Foundational Ontology (UFO) in this thesis is not limited to providing a generic conceptual background. Instead, it establishes the ontological categories and characteristics that underpin the modeling of IoT decision-making. The dimensions proposed in the Taxonomy for Decision-Making in IoT (**TDMIoT**) can be systematically grounded in UFO categories, ensuring conceptual clarity and semantic rigor. In this way, the ontology proposed in this thesis (**ODMIoT**) is not developed in isolation but emerges as a refinement of **TDMIoT**, transforming it into a semantic model capable of supporting interoperability and reconfiguration in IoT systems.

Endurants: characterization of problems

Endurants are entities that persist through time while keeping their identity. Their defining characteristic is persistence. In the context of decision-making in IoT, this

corresponds to the stable qualities of the problem, already captured in **TDMIoT** as:

- **Objective** (e.g., prediction, optimization) persists as a defining feature of the problem.
- **Uncertainty** represents an inherent quality of the problem space.
- **Knowledge representation** (data, symbolic, mathematical, hybrid) is a structural choice that does not change during execution.

Thus, in **TDMIoT** these dimensions describe the problem space, and in O-DMIoT they are formalized as enduring entities, providing the ontological foundation for characterizing IoT decision problems.

Perdurants: capturing the temporality of algorithmic execution

Perdurants are entities that unfold in time and can be divided into temporal parts. Their defining characteristic is temporality, since they represent activities, events, or processes rather than stable objects. In **TDMIoT**, this maps to the execution-related dimensions:

- **Processing** (centralized or decentralized) specifies how an algorithm is carried out as an activity over time, either within a single entity or distributed across multiple entities.
- **Processing layer** (application, middleware, perception) locates this execution within the IoT architecture, indicating where the process is instantiated and executed.

These dimensions are temporal, not because the decision problem itself changes, but because the execution of a solution is inherently a process. In other words, they capture the dynamic unfolding of how a decision is produced, distinguishing it from the stable qualities of the problem space.

Agents: intentionality and roles in services

Agents are a special kind of *endurant* characterized by intentionality and the capacity to play roles. In **TDMIoT**, the corresponding dimension is **interaction**, which distinguishes between human and system contributions to decision-making. In O-DMIoT, this expands to:

- Services that implement algorithms and can be reconfigured, aligning with autonomic computing principles.
- Users as intentional agents who provide goals or preferences that shape decision problems.

In summary, *endurants* corresponds to the stable definition of the problem space, *perdurants* captures the execution of algorithmic processes, and intentionality/roles *agents* represent the entities responsible for carrying out decisions and interacting

within IoT systems. **TDMIoT** first operationalized this structure through its dimensions, and O-DMIoT refines it by providing a formal ontological model grounded in UFO. This alignment ensures that the ontology systematically links problem dimensions, algorithmic design, and service execution, offering a coherent foundation for the specification presented in the following section.

5.4.3. Key UFO Constructs Used

O-DMIoT is developed based on a selected subset of constructs from UFO that are specifically relevant for modeling decision-making processes in IoT systems. These constructs serve as conceptual building blocks for representing decision problems, algorithmic solutions, execution contexts, and evaluation mechanisms with semantic precision. The primary constructs used in O-DMIoT are outlined below by UFO category and their representation in OntoUML.

- *Kind (Endurant)*: Represents entities that persist through time while maintaining their identity. In OntoUML, *Kinds* define the fundamental categories of a domain and provide identity criteria for their instances.
- *Subkind*: Represents a rigid specialization of a *Kind* that preserves the identity criteria of its parent while introducing additional distinguishing characteristics. In OntoUML, *Subkinds* refine a kind by partitioning it into more specific categories, enabling finer-grained classification without altering the fundamental nature of the entities involved.
- *Role (Agentive Endurant)*: A context-dependent type that an entity assumes in a particular situation. *Roles* are essential for capturing the ways in which agents, whether human or artificial, participate in and influence decision-making.
- *Phase (Endurant)*: Represents mutually exclusive states in the lifecycle of an entity. Phases are used to distinguish different conceptual states without introducing explicit temporal constructs.
- *Category*: Represents a rigid, abstract type that generalizes multiple kinds sharing common properties. Categories are used to define high-level classifications that cut across different concrete entities.
- *Quality (Mode)*: Represents intrinsic, dependent properties that characterize an entity and can vary across instances. Qualities are essential for modeling evaluative and measurable aspects of decision-making solutions.
- *Perdurant (Event or Process)*: Entities that unfold over time and can be divided into temporal parts. *Perdurants* are necessary for representing algorithmic execution, processing activities, and other temporal dynamics in IoT decision-making. *Relator*: Represents dependent entities that mediate and establish relations between two or more entities. In OntoUML, *Relators* reify relationships by capturing contextual and semantic dependencies that connect entities, enabling explicit representation of associations such as derivation, deployment, and evaluation among problems, solutions, and services.

- *Mode (Quality)*: Represents intrinsic and dependent entities that characterize other entities. In OntoUML, *Modes* capture variable aspects of entities that do not exist independently but inhere in the entities they characterize, allowing properties such as performance or quality to change while the identity of the characterized entity remains the same.

5.5. Methodology and Development Approach

The O-DMIoT was developed using the SABiO methodology [17]. SABiO offers a structured and iterative framework for ontology engineering, ensuring that ontologies are purpose-driven, conceptually robust, and formally consistent. This section details the instantiation of each SABiO activity in the development of O-DMIoT. Figure 5.1 presents the application of the SABiO methodology in developing O-DMIoT, emphasizing the progression from knowledge sources and conceptual modeling to formal implementation and validation.

5.5.1. Identification of Purpose and Requirements

According to SABiO, ontology development begins with explicit identification of purpose, scope, and requirements. The purpose of O-DMIoT is to provide a semantic model that characterizes decision-making problems and algorithmic solutions in IoT systems, specifically to support algorithm selection and service reconfiguration.

Requirements were identified through:

- The Taxonomy for Decision-Making in IoT (TDMIoT).
- Analysis of decision-making processes reported in IoT literature.
- The needs of autonomic computing systems that must adapt to evolving functional requirements.

These requirements were formalized as competency questions, presented in Section 5.3.3, which define the minimal set of queries the ontology must support. This activity aligns with the Identification of purposes and obtaining requirements phase in SABiO.

5.5.2. Knowledge Acquisition and Conceptualization

The knowledge acquisition phase in SABiO emphasizes the collection and structuring of domain knowledge. For O-DMIoT, this activity involved a meta-analysis of decision-making processes in IoT systems, with TDMIoT serving as the primary knowledge source.

TDMIoT offers a consolidated perspective on problem dimensions, algorithmic strategies, and execution characteristics. These dimensions were analyzed and reorganized to identify core concepts, relationships, and constraints relevant to application-level decision-making. This step connects the descriptive nature of the taxonomy with the prescriptive requirements of an ontology.

The outcome of this phase is a domain conceptualization that distinguishes between problem characterization, solution design, and execution aspects, establishing the foundation for the layered structure of O-DMIoT.

5.5.3. Ontological Design and UFO Grounding

In the design phase prescribed by SABiO, the conceptual model is refined and anchored in a foundational ontology. O-DMIoT is based on the Unified Foundational Ontology (UFO), which supplies well-defined ontological categories including endurants, perdurants, roles, and relators.

Each concept identified during knowledge acquisition was mapped to a corresponding UFO construct. Stable characteristics of decision problems were modeled as endurants, algorithm execution and processing aspects as perdurants, and intentional entities such as services and users as agents assigned specific roles. This grounding promotes conceptual clarity, prevents category errors, and facilitates extensibility.

The result of this phase is a reference ontology, articulated at the conceptual level, that captures the essential semantics of decision-making in IoT independently of implementation details.

5.5.4. OntoUML Conceptual Modeling

The reference ontology was modeled using OntoUML, a graphical and formally grounded notation aligned with UFO. OntoUML facilitated the representation of core concepts, their specializations, and the relationships among the problem, design, and execution layers. This conceptual model serves several purposes: it enables validation through expert review, ensures traceability between requirements and concepts, and offers a clear specification for subsequent formalization. At this stage, O-DMIoT remains independent of any particular semantic web language, consistent with SABiO's distinction between conceptual modeling and implementation.

5.5.5. Implementation and OWL/RDF Formalization

During the SABiO implementation phase, the OntoUML conceptual model was formalized using OWL and RDF (Resource Description Framework Schema), resulting in the operational ontology provided in the Appendix A.2.4. OntoUML constructs were systematically translated into OWL classes, object properties, and data properties, preserving the ontological distinctions established during the design phase.

This formalization enables machine-processable reasoning, semantic querying, and interoperability with IoT services. The ontology adopts an *open-world* assumption, allowing partial knowledge and incremental extension, which is essential for dynamic IoT environments.

5.5.6. Verification and Validation

In accordance with SABiO, verification and validation activities were conducted throughout the development process. Verification ensured that the ontology is syntactically correct and internally consistent, while validation assessed whether O-DMIoT fulfills its intended purpose.

Validation is further detailed in Section 5.8 and encompasses competency question coverage, foundational ontology compliance, and application-driven validation through case studies. This ongoing validation process confirms that O-DMIoT is

complete with respect to its requirements, minimal in conceptualization, and suitable for supporting algorithm selection and reconfiguration.

5.5.7. Methodological Integration with the Thesis Framework

Although SABiO guides the development of O-DMIoT, the ontology is embedded within a comprehensive thesis workflow encompassing algorithm selection, simulation-based evaluation, and service deployment. Within this framework, O-DMIoT serves not as the entire methodology, but as the semantic foundation that underpins and links these subsequent stages.

Specifically, the ontology enables structured problem characterization, informs algorithm design and selection, and supports the runtime evaluation of algorithmic services. Consequently, O-DMIoT acts as a unifying knowledge layer that facilitates integration among the various technical components developed in the thesis.

5.6. Conceptual Model of O-DMIoT

5.6.1. Core Concepts and Conceptual Relationships

The core concepts of O-DMIoT define the fundamental ontological commitments required to represent decision-making in IoT systems. These concepts provide a shared semantic foundation for characterizing decision-making needs, specifying algorithmic solutions, and evaluating execution. They address the competency questions introduced in Section 5.3 and summarized in Table 5.2.

Figure 5.2 presents a high-level conceptual overview of O-DMIoT, highlighting the principal concepts and their semantic relationships. This overview abstracts from implementation details and introduces the layered organization of the ontology. Detailed representations of each layer are provided in subsequent figures and sections.

At the highest level, O-DMIoT organizes decision-making algorithms knowledge into three main layers *Requirement*, *Design*, and *Execution* with *Evaluation* acting in a cross-cutting role. This structure reflects the lifecycle of decision-making algorithms in IoT systems, from identifying decision needs to operational deployment and solution assessment.

The *Requirement layer* specifies decision-making needs derived from a problem. In this layer, a *Problem* represents a decision-making situation characterized by a set of intrinsic dimensions, including objective, uncertainty, knowledge representation, interaction, processing, and processing layer. Problems generate one or more *Requirements*, which formalize solution criteria. Requirements are detailed through *Specifications* that define required inputs, outputs, and parameters, and are characterized by performance criteria for assessment. This layer establishes a clear boundary between problem characterization and solution design.

The *Design layer* defines how a solution is conceived, separate from its execution. In this layer, a *Solution* is implemented through an *Algorithm* whose structure is determined during design. Design includes the assumptions, constraints, and defining characteristics such as complexity, data representation, and algorithmic approach. By modeling design decisions, O-DMIoT supports systematic comparison of alternative algorithms against problem requirements, which is essential for algorithm

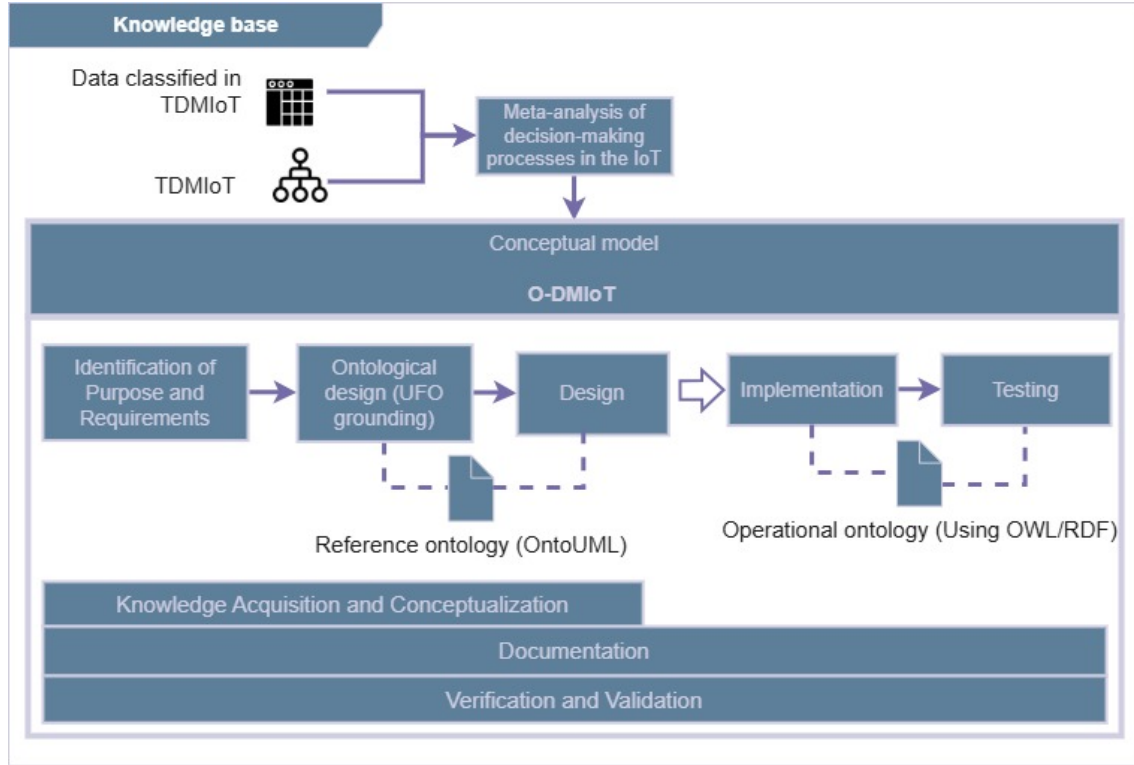


Figure 5.1.: **Methodological workflow for the development of O-DMIoT.**

The figure depicts the instantiation of the SABiO ontology engineering methodology in developing O-DMIoT. Domain knowledge from TDMIoT and a meta-analysis of decision-making processes in IoT inform the identification of purpose and requirements, followed by systematic knowledge acquisition and conceptualization. The ontology is designed and anchored in the Unified Foundational Ontology (UFO), resulting in a reference ontology modeled in OntoUML. This reference ontology is then formalized as an operational ontology using OWL/RDF. Verification, validation, and documentation are integrated as cross-cutting activities throughout the process to ensure conceptual soundness, formal consistency, and alignment with the intended use of O-DMIoT within the thesis framework.

selection.

The *Execution layer* operationalizes a designed algorithm within an IoT system. In this layer, an *Algorithm* is instantiated and executed as a *Service*. Execution is mediated by deployment and configuration relationships that link algorithm instances to specific services and data items. Services consume specific inputs and parameters, produce outputs, and enable the integration of decision-making functionality into the IoT infrastructure. Separating design from execution enables reconfiguration, allowing services to be replaced or reconfigured without changing problem definitions or design specifications.

Evaluation has a cross-cutting role spanning the *Requirement*, *Design*, and *Execution* layers. It is supported by explicit *Metrics*, which assess requirement satisfaction (performance), design properties, or quality of experience. By modeling evaluation independently, O-DMIoT ensures traceability from problem objectives to algorithm design and execution outcomes.

These core concepts and their layered organization, presented in Fig. 5.3, form the foundation of O-DMIoT. They provide a coherent semantic structure that supports the competency questions in Table 5.2, enable systematic reasoning about algorithm selection, and establish the foundation for the reinforcement learning-based reconfiguration mechanisms introduced in later chapters.

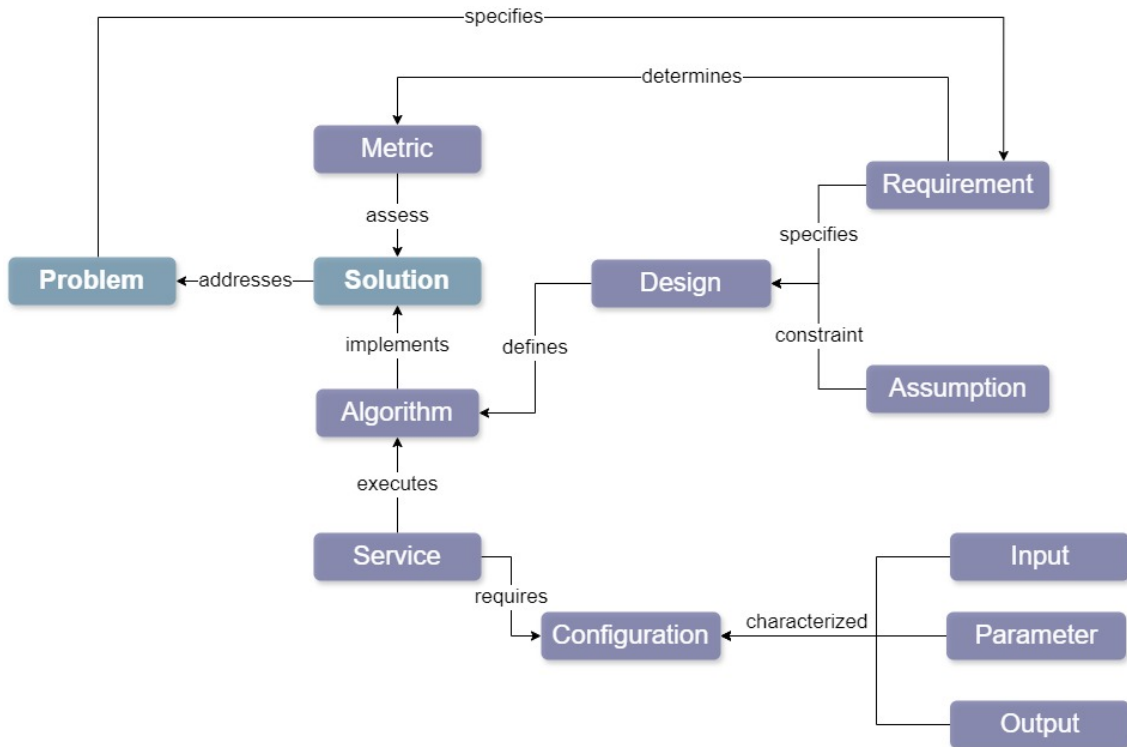


Figure 5.2.: The core concepts of O-DMIoT

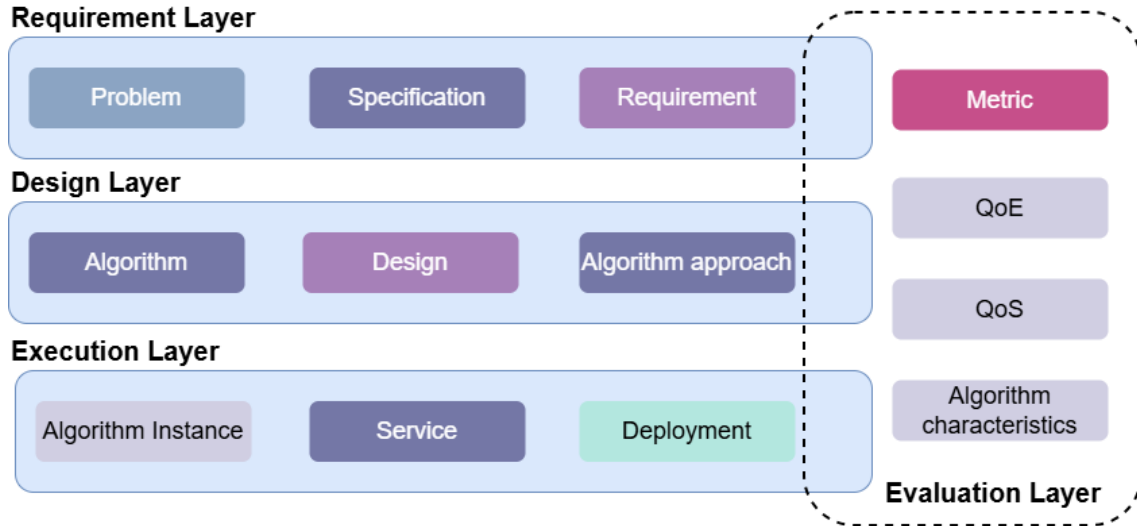


Figure 5.3.: Conceptual layered structure of the O-DMIoT ontology

Table 5.2.: **Competency questions mapping.** The table summarizes how the competency questions defined for O-DMIoT are addressed through the ontology core conceptualizations. This table serves as a conceptual validation artifact and complements the formal axioms and OWL implementation presented in Chapter 5.

<i>Competency question</i>	<i>Core conceptualization</i>	<i>Fig.</i>
CQ1. What constitutes a decision problem in the IoT?	A decision problem represents a category of decision-making situations in which an action or outcome must be selected based on available information and contextual constraints within an IoT-enabled environment.	Fig. 5.4
CQ2. What are the characteristics of a problem?	The Problem category is defined by conceptual dimensions (<i>Objective</i> , <i>Interaction</i> , <i>Knowledge_Representation</i> , <i>Processing</i> , <i>Processing_Layer</i> , and <i>Uncertainty</i>). These dimensions are instantiated in concrete problem specifications through corresponding modes, which serve as criteria for classifying decision-making problems in IoT systems.	Fig. 5.4
CQ3. How are problem characteristics related to each other?	The characteristics that distinguish a Problem are organized into complementary groups of conceptual dimensions. <i>Objective</i> , <i>Uncertainty</i> , <i>Interaction</i> , and <i>Knowledge_Representation</i> define the conceptual problem space, while <i>Processing</i> and <i>Processing_Layer</i> constrain how concrete problem specifications can be operationalized within an IoT system.	Fig. 5.4

Continued on next page

<i>Competency question</i>	<i>Core conceptualization</i>	<i>Fig.</i>
CQ4. What constitutes a solution to a problem?	A Solution is a category that classifies algorithmic approaches capable of addressing a given problem. A solution is operationalized through an Algorithm , which specifies how problem requirements are satisfied and may be instantiated and executed as services in concrete system deployments.	Fig. 5.5
CQ5. What are the characteristics of a problem solution?	A problem solution is characterized during the Design phase of the Algorithm . In this phase, the algorithm is defined by explicit assumptions, constraints, and design characteristics such as complexity, algorithmic approach, and data representation, which determine its suitability for the problem.	Fig. 5.5
CQ6. What are the elements of a requirement specification?	A Requirement Specification defines the required inputs, outputs, and parameters that a solution must support, as well as the expected behavioral properties of that solution. These expectations are captured through a <i>Performance</i> mode, evaluated using one or more <i>Metrics</i> .	Fig. 5.4
CQ7. What are the elements of a design specification?	A Design Specification is established during the <i>Design</i> phase and consists of explicit <i>Assumptions</i> and <i>Constraints</i> . It determines the algorithmic characteristics required to satisfy a given requirement specification, including the chosen <i>Algorithmic Approach</i> , <i>Data Representation</i> , and associated <i>Complexity</i> properties.	Fig. 5.5
CQ8. How does execution operationalize a problem solution within an IoT system?	Execution operationalizes a solution by instantiating an Algorithm within a Service , mediated by an algorithm deployment relationship. The service consumes specified inputs, produces outputs, and enables observation of solution behavior.	Fig. 5.6
CQ9. What constitutes the evaluation of a solution?	Evaluation involves assessing the <i>Performance</i> associated with a solution, contextualized through explicit Design Evaluation or Service Evaluation relators. These relators connect performance to either an algorithm design or a deployed service instance.	Fig. 5.7

Continued on next page

<i>Competency question</i>	<i>Core conceptualization</i>	<i>Fig.</i>
CQ10. Which metrics can be used to assess whether a solution meets requirements?	A solution is assessed using <i>Metrics</i> that measure its associated <i>Performance</i> . These metrics include <i>Quality of Service</i> , <i>Quality of Experience</i> , and <i>Algorithm Characteristic</i> metrics, enabling evaluation from system-level, human-centered, and design-oriented perspectives.	Fig. 5.7

5.6.2. Requirement Layer

The *Requirement layer* formally specifies decision-making needs based on a problem in an IoT context. It defines, independently of any solution, the conditions a decision-making solution must meet to be valid. This layer bridges problem characterization and algorithm design, ensuring traceability, completeness, and evaluability of decision requirements.

Figure 5.4 presents the *Requirement layer* of O-DMIoT. At the top of the layer, a *Problem* represents a decision-making situation characterized by a fixed set of intrinsic dimensions: *Objective*, *Uncertainty*, *Knowledge Representation*, *Interaction*, *Processing*, and *Processing Layer*. These dimensions are modeled as modes that characterize the problem and define its decision context. A problem, however, is not directly solved by an algorithm; instead, it gives rise to one or more explicit requirements.

The relator *Derives* captures the transition from problem characterization to requirement specification. It links a problem to the specifications that formalize its requirements. Explicitly modeling this dependency ensures requirements are grounded in the problem semantics, not introduced arbitrarily.

A *Requirement* marks the stage in an algorithm's lifecycle when decision-making needs are formally specified. Each requirement is linked to one or more *Specifications*, which define what a solution must provide. A *specification* includes three key elements: *Inputs*, *Outputs*, and *Parameters*. These elements are modeled as roles played by *Data Item* instances, allowing a data entity to serve different functions within a specification.

Specifications are further defined by *Performance* criteria, which specify how to assess requirement fulfillment. By associating performance at the requirement level, O-DMIoT ensures evaluation criteria are established before algorithm design and execution, and remain independent of implementation details.

The *Requirement layer* offers a complete and minimal semantic description of decision-making needs. It sets the constraints and evaluation criteria that guide design decisions, while remaining independent of algorithmic strategies and execution methods.

Formal Semantics of the Requirement Layer

Logical vocabulary

Let the domain of discourse include the following unary predicates following names in Figure 5.4:

- $Problem(x)$
- $Requirement(x)$
- $Specification(x)$
- $DataItem(x)$
- $Input(x), Output(x), Parameter(x)$

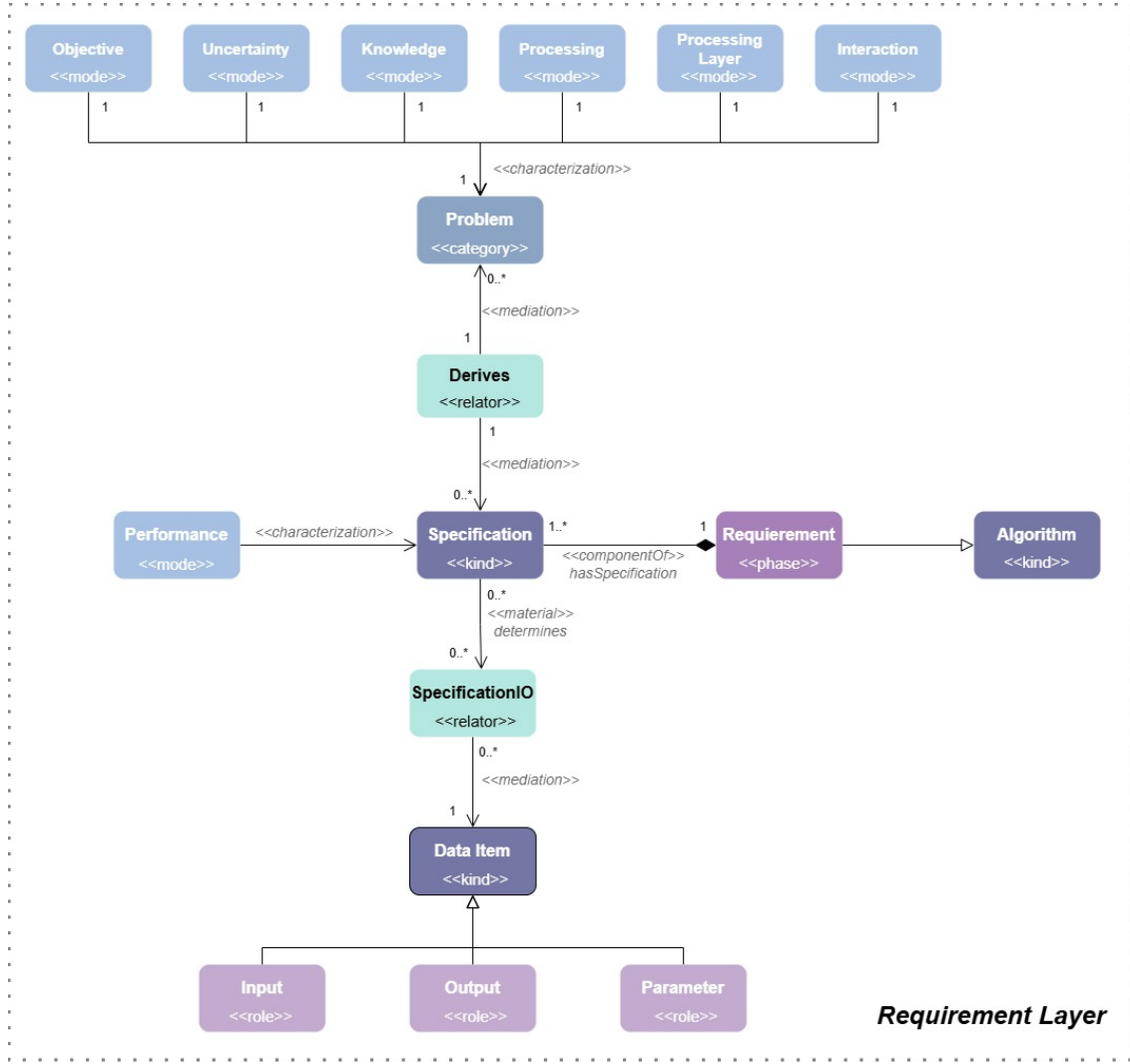


Figure 5.4.: O-DMIoT Requirement Layer

- $Performance(x)$
- $Objective(x), Uncertainty(x), Knowledge(x)$
- $Processing(x), ProcessingLayer(x), Interaction(x)$

Individual Terms:

Individual terms denote concrete instances of ontological classes. Each variable is typed according to the class it instantiates. The following individual terms are used throughout the axiomatization of the Requirement layer:

- p denotes an individual of type *Problem*.
- o denotes an individual of type *Objective*.
- u denotes an individual of type *Uncertainty*.
- k denotes an individual of type *Knowledge*.
- pr denotes an individual of type *Processing*.

- pl denotes an individual of type *ProcessingLayer*.
- i denotes an individual of type *Interaction*.
- r denotes an individual of type *Requirement*.
- s denotes an individual of type *Specification*.
- d denotes an individual of type *DataItem*.
- $perf$ denotes an individual of type *Performance*.

Binary predicates:

- $characterizedBy(p, q)$
- $derives(p, s)$
- $hasSpecification(r, s)$
- $determines(s, d)$
- $playsRole(d, r)$
- $characterize(s, perf)$

Axiom 5.6.1 (Problem Characterization). Every problem is fully characterized by exactly one instance of each problem dimension.

$$\begin{aligned} \forall p (\text{Problem}(p) \rightarrow \\ \exists ! o, u, k, pr, pl, i (\\ & \text{characterizedBy}(p, o) \wedge \text{Objective}(o) \wedge \\ & \text{characterizedBy}(p, u) \wedge \text{Uncertainty}(u) \wedge \\ & \text{characterizedBy}(p, k) \wedge \text{Knowledge}(k) \wedge \\ & \text{characterizedBy}(p, pr) \wedge \text{Processing}(pr) \wedge \\ & \text{characterizedBy}(p, pl) \wedge \text{ProcessingLayer}(pl) \wedge \\ & \text{characterizedBy}(p, i) \wedge \text{Interaction}(i) \\ &)) \end{aligned}$$

Axiom 5.6.2 (Derivation of Specifications from Problems). Every problem gives rise to one or more specifications through the *Derives* relator.

$$\forall p (\text{Problem}(p) \rightarrow \exists s (\text{Specification}(s) \wedge \text{derives}(p, s)))$$

This axiom ensures that specifications are semantically grounded in problem characterization.

Axiom 5.6.3 (Requirements Aggregate Specifications). Every requirement is associated with one or more specifications.

$$\forall r (\text{Requirement}(r) \rightarrow \exists s (\text{Specification}(s) \wedge \text{hasSpecification}(r, s)))$$

Axiom 5.6.4 (Specification Determines Data Items). Every specification determines one or more data items.

$$\forall s (\text{Specification}(s) \rightarrow \exists d (\text{DataItem}(d) \wedge \text{determines}(s, d)))$$

This axiom ensures that specifications are semantically grounded in problem characterization.

Axiom 5.6.5 (Data Items Play Exactly One Role per Specification). Each data item involved in a specification plays exactly one role: input, output, or parameter.

$$\forall d(\text{DataItem}(d) \rightarrow (\text{Input}(d) \vee \text{Output}(d) \vee \text{Parameter}(d)))$$

where $\vee$ denotes exclusive disjunction (exactly one is true).

Axiom 5.6.6 (Performance Characterizes Specifications). Every specification is characterized by at least one performance criterion.

$$\forall s(\text{Specification}(s) \rightarrow \exists p(\text{Performance}(p) \wedge \text{characterizes}(s, p)))$$

The presented axioms formalize the core semantics of the Requirement layer, ensuring that decision-making algorithm needs are (i) grounded in problem characterization, (ii) explicitly specified through inputs, outputs, and parameters, and (iii) evaluable through performance criteria. This formalization establishes a logical foundation for reasoning about algorithm selection and reconfiguration in subsequent layers.

5.6.3. Design Layer

The *Design layer* describes the conception of a decision-making solution independently of its execution context. Whereas the *Requirement layer* specifies the criteria a solution must fulfill, the *Design layer* formalizes the algorithmic strategies for addressing these requirements. This layer is central to O-DMIoT, as it provides the semantic basis for comparing, selecting, and reconfiguring algorithms in response to evolving decision-making needs.

Figure 5.5 illustrates the *Design layer* of O-DMIoT. In this layer, a *Solution* denotes an abstract resolution to a decision problem. Each solution is realized through an *Algorithm*, which encapsulates the decision logic required to satisfy the requirements derived from the problem. The structure of an algorithm is established during the *Design* phase, which aggregates the assumptions, constraints, and defining characteristics that determine the algorithm applicability.

The *Design* phase explicitly distinguishes design-time decisions from runtime concerns. It captures the conditions under which an algorithm is valid (*Assumptions*), the limitations that restrict its use (*Constraints*), and the intrinsic properties that define its computational and representational behavior. These properties include *Complexity*, which encompasses spatial and temporal computational requirements; *Data Representation*, which specifies the encoding of knowledge; and *Algorithmic Approach*, which identifies the principle guiding the decision-making process.

Algorithmic approaches are categorized into mutually exclusive subtypes that reflect established paradigms in decision-making and artificial intelligence, including criterion-driven, data-driven, utility-driven, probability-driven, and rule-driven approaches. This classification establishes a semantically grounded vocabulary for distinguishing algorithms according to their underlying decision logic, rather than implementation-specific details.

By modeling design elements as ontological entities, O-DMIoT enables systematic comparison of alternative algorithms with respect to a consistent set of requirements. This approach is essential for algorithm selection, as it allows the selection process to reason over design characteristics rather than relying on ad hoc or purely empirical criteria. The *Design layer* therefore serves as the semantic interface between requirement specification and execution, enabling principled adaptation and reconfiguration in IoT systems.

Formal Semantics of the Design Layer

To ensure semantic rigor, the *Design layer* is partially formalized using first-order logic. This formalization explicitly articulates the ontological constraints governing algorithm

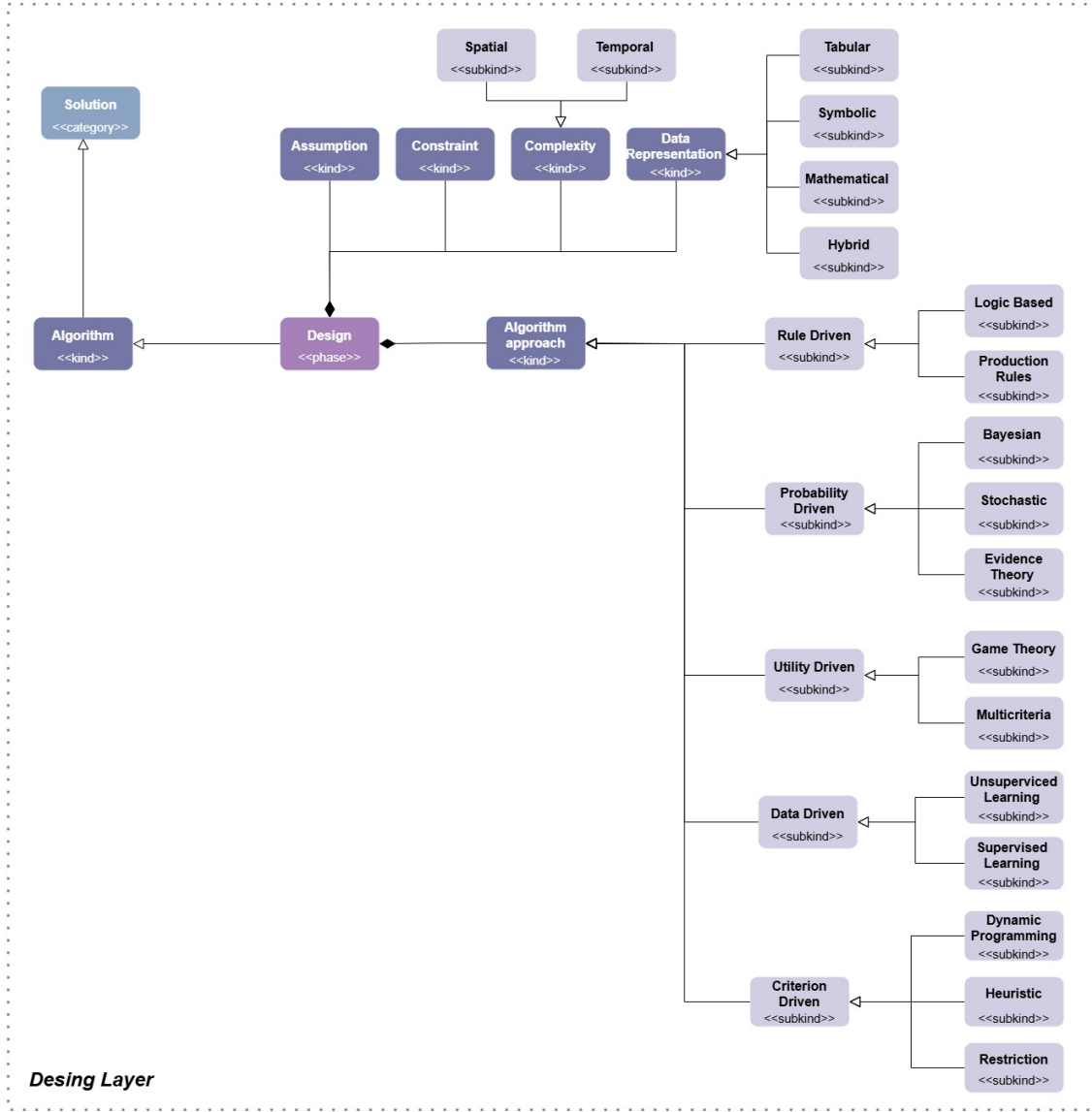


Figure 5.5.: O-DMIoT Design layer

design and supports reasoning about algorithm selection and applicability. The axioms presented here define the core semantic commitments of the Design layer.

Logical vocabulary

Let the domain of discourse include the following unary predicates resembling the terminology in Figure 5.5:

- $Solution(x)$
- $Algorithm(x)$
- $Design(x)$
- $Assumption(x)$
- $Constraint(x)$
- $Complexity(x)$

- $DataRepresentation(x)$
- $AlgorithmApproach(x)$

The following unary predicates specialize the previous concepts:

- $SpatialComplexity(x), TemporalComplexity(x)$
- $Tabular(x), Symbolic(x), Mathematical(x), Hybrid(x)$
- $CriterionDriven(x)$
- $DataDriven(x)$
- $UtilityDriven(x)$
- $ProbabilityDriven(x)$
- $RuleDriven(x)$

Individual Terms:

- s denotes an individual of type *Solution*.
- a denotes an individual of type *Algorithm*.
- d denotes an individual of type *Design*.
- as denotes an individual of type *Assumption*.
- c denotes an individual of type *Constraint*.
- cx denotes an individual of type *Complexity*.
- r denotes an individual of type *DataRepresentation*.
- ap denotes an individual of type *AlgorithmApproach*.
- sc denotes an individual of type *SpatialComplexity*.
- tc denotes an individual of type *TemporalComplexity*.

Binary Predicates:

- $implement(a, s)$
- $definedBy(a, d)$
- $assumes(d, a)$
- $constraints(d, cx)$
- $hasComplexity(d, cx)$
- $usesRepresentation(d, r)$
- $hasApproach(d, ap)$

Axiom 5.6.7 (Solutions Are Implemented by Algorithms). Every solution is realized through exactly one algorithm.

$$\forall s(Solution(s) \rightarrow \exists! a(Algorithm(a) \wedge implements(a, s)))$$

Axiom 5.6.8 (Algorithms Are Defined by a Design Phase). Every algorithm is defined by exactly one design.

$$\forall a(\text{Algorithm}(a) \rightarrow \exists! d(\text{Design}(d) \wedge \text{definedBy}(a, d)))$$

Axiom 5.6.9 (Design Includes Assumptions and Constraints). Every design includes one or more assumptions and constraints.

$$\begin{aligned} \forall d(\text{Design}(d) \rightarrow \\ (\exists a(\text{Assumption}(a) \wedge \text{assumes}(d, a)) \wedge \\ \exists c(\text{Constraint}(c) \wedge \text{constrains}(d, c)))) \end{aligned}$$

Axiom 5.6.10 (Design Specifies Algorithmic Characteristics). Every design specifies exactly by one data representation and one algorithmic approach.

$$\begin{aligned} \forall d(\text{Design}(d) \rightarrow \\ \exists! r, ap(\\ \text{DataRepresentation}(r) \wedge \text{usesRepresentation}(d, r) \wedge \\ \text{AlgorithmApproach}(ap) \wedge \text{hasApproach}(d, ap) \\)) \end{aligned}$$

Axiom 5.6.11 (Design Specifies Spatial Complexity). Every design specifies exactly one spatial complexity.

$$\forall d(\text{Design}(d) \rightarrow \exists! sc(\text{SpatialComplexity}(sc) \wedge \text{hasComplexity}(d, sc)))$$

Axiom 5.6.12 (Design Specifies Temporal Complexity). Every design specifies exactly one temporal complexity.

$$\forall d(\text{Design}(d) \rightarrow \exists! tc(\text{TemporalComplexity}(tc) \wedge \text{hasComplexity}(d, tc)))$$

Axiom 5.6.13 (Complexity Dimensions Are Disjoint). Spatial and temporal complexity are distinct dimensions.

$$\forall x(\neg(\text{SpatialComplexity}(x) \wedge \text{TemporalComplexity}(x)))$$

Axiom 5.6.14 (Complexity Generalization). Spatial and temporal complexity are specializations of complexity.

$$\forall x((\text{SpatialComplexity}(x) \vee \text{TemporalComplexity}(x)) \rightarrow \text{Complexity}(x))$$

Axiom 5.6.15 (Algorithmic Approaches Are Mutually Exclusive). Each algorithmic approach is exactly one of: criterion-driven, data-driven, utility-driven, probability-driven, or rule-driven.

$$\begin{aligned} \forall ap(\text{AlgorithmApproach}(ap) \rightarrow \\ (\text{CriterionDriven}(ap) \vee \text{DataDriven}(ap) \vee \\ \text{UtilityDriven}(ap) \vee \text{ProbabilityDriven}(ap) \vee \\ \text{RuleDriven}(ap))) \end{aligned}$$

where $\vee$ denotes exclusive disjunction (only one is true).

The axioms of the *Design layer* formalize the structure of algorithm conception in O-DMIoT and they ensure that each algorithm is associated with a single, well-defined design specifying its applicability conditions, computational characteristics, and decision-making algorithm paradigm. By constraining algorithm design in this way, O-DMIoT enables principled reasoning about algorithm selection and supports the adaptive reconfiguration mechanisms introduced in subsequent chapters.

5.6.4. Execution Layer

The *Execution layer* represents the operational realization of a decision-making solution within an IoT system. The *Requirement layer* specifies the essential requirements for a solution, and the *Design layer* defines its algorithmic conception. The *Execution layer* formalizes the instantiation, configuration, and execution of a designed algorithm within a running system. This layer links abstract algorithm design and concrete service-based deployment, which is critical for system reconfiguration and runtime adaptation.

Figure 5.6 illustrates the *Execution layer* of O-DMIoT. Within this layer, an *Algorithm* defined during the Design phase is instantiated as an *Algorithm Instance*. This instance constitutes a concrete realization of the algorithm, associated with specific configuration values and execution conditions. The distinction between algorithm and algorithm instance enables O-DMIoT to differentiate between design-time properties and runtime instantiations, thereby supporting scenarios where the same algorithm design is executed multiple times with varying configurations.

The execution of an algorithm instance within the IoT infrastructure is governed by the relator *Algorithm Deployment*. This relator explicitly defines the relationship between an algorithm instance and the *Service* that operationalizes it. A *service* functions as the runtime entity providing decision-making capabilities within the IoT system, exposing inputs and outputs, and enabling interaction with other system components.

Algorithm deployment binds each algorithm instance to a single service and associates it with the data items necessary for execution. These data items are configured in accordance with the specifications defined at the *Requirement layer*, thereby ensuring traceability from problem definition to runtime execution. By explicitly modeling deployment as a relator, O-DMIoT enables reasoning about service replacement, reconfiguration, and migration without modifying the underlying algorithm design.

The *Execution layer* establishes the semantic foundation for autonomic behavior in IoT systems. It enables decision-making services to be dynamically instantiated, replaced, or reconfigured in response to evolving requirements or environmental conditions, while preserving consistency with the problem definition and design specifications.

Formal Semantics of the Execution Layer

To ensure semantic rigor and enable reasoning about runtime behavior, the Execution layer is partially formalized using typed first-order logic. This formalization explicitly defines the constraints governing algorithm instantiation, deployment, and service execution.

Logical vocabulary

Let the domain of discourse include the following unary predicates mapping the terms in Figure 5.6:

- $Algorithm(x)$
- $AlgorithmInstance(x)$
- $Design(x)$
- $Service(x)$
- $DataItem(x)$

Individual Terms:

- a denotes an individual of type *Algorithm*.

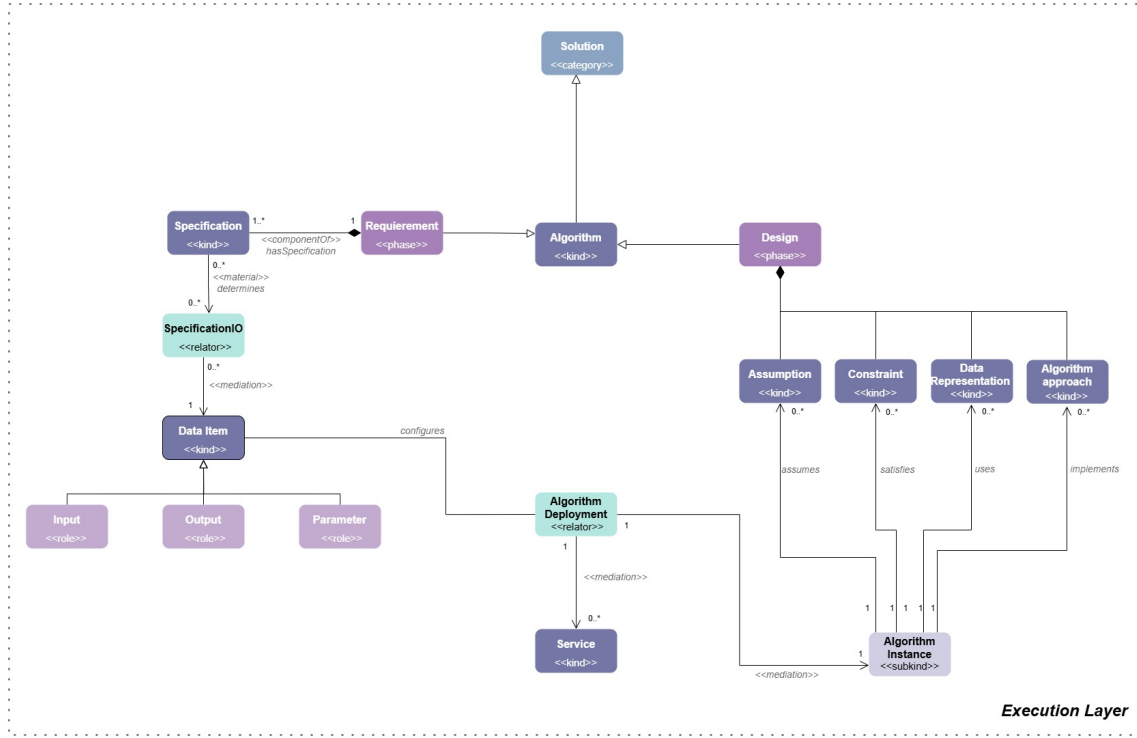


Figure 5.6.: O-DMIoT Execution layer

- i denotes an individual of type *AlgorithmInstance*.
- d denotes an individual of type *Design*.
- s denotes an individual of type *Service*.
- di denotes an individual of type *DataItem*.
- dpl denotes an individual of type *AlgorithmDeployment*.

Binary Predicates:

- $instantiates(i, a)$
- $definedBy(a, d)$
- $deploys(dpl, i, s)$
- $configures(d, i)$
- $executes(s, i)$

Axiom 5.6.16 (Algorithm Instances Instantiate Algorithms). Every algorithm instance instantiates exactly one algorithm.

$$\forall i (AlgorithmInstance(i) \rightarrow \exists! a (Algorithm(a) \wedge instantiates(i, a)))$$

Axiom 5.6.17 (Algorithm Instances Are Defined by a Design). Every algorithm instance is grounded in the design that defines its algorithm.

$$\forall i, a, d ((instantiates(i, a) \wedge definedBy(a, d)) \rightarrow Design(d))$$

Axiom 5.6.18 (Deployment Binds Algorithm Instances to Services). Every algorithm instance is deployed through exactly one algorithm deployment, which binds it to exactly one service.

$$\forall i (AlgorithmInstance(i) \rightarrow \exists! dpl, s (Service(s) \wedge deploys(dpl, i, s)))$$

Axiom 5.6.19 (Services Execute Algorithm Instances). Every service executes one or more algorithm instances.

$$\forall s (Service(s) \rightarrow \exists i (AlgorithmInstance(i) \wedge executes(s, i)))$$

Axiom 5.6.20 (Algorithm Instances Are Configured by Data Items). Every algorithm instance is configured using one or more data items.

$$\forall i (AlgorithmInstance(i) \rightarrow \exists d; (DataItem(d) \wedge configures(d, i)))$$

The axioms of the *Execution layer* formalize the operational semantics of decision-making in O-DMIoT. These axioms ensure that algorithm execution is grounded in explicit deployment and configuration relationships, thereby enabling traceability from requirements and design to runtime behavior. By distinguishing algorithm instances from services and explicitly modeling deployment, O-DMIoT supports dynamic reconfiguration and service substitution, which are essential properties for autonomic IoT systems.

5.6.5. Evaluation and Metrics as an Integrative Cross-Cutting Role

Within the O-DMIoT framework, evaluation functions as a cross-cutting function that spans the *Requirement*, *Design*, and *Execution layers*. As shown in figure 5.7, rather than forming an independent layer in the decision-making algorithm lifecycle, evaluation supplies the semantic mechanisms required for validating requirements, comparing designs, and assessing executed services. This approach acknowledges that evaluation criteria are established at multiple stages and must maintain semantic consistency throughout the lifecycle of a decision-making solution.

Figure 5.5 illustrates the Design layer of O-DMIoT. In this layer, a *Solution* represents an abstract resolution to a decision problem. The solution is instantiated through an Algorithm that encapsulates the decision logic required to fulfill the requirements derived from the problem. The structure of an algorithm is defined during the *Design* phase, which consolidates the assumptions, constraints, and defining characteristics that determine the algorithm's applicability.

Performance denotes the evaluable state of a solution relative to defined expectations. In the ontology, *Performance* is modeled as a mode that characterizes a *Specification*, thereby anchoring evaluation at the requirement level. This modeling approach ensures that evaluation remains grounded in explicitly defined expectations rather than being limited to isolated numerical indicators.

Metrics are modeled as qualities that measure performance. Each metric offers a scale, criterion, or quantitative or qualitative indicator for assessing a specific aspect of performance. By explicitly distinguishing performance from metrics, O-DMIoT separates the phenomenon being evaluated from the mechanisms used to measure it. This distinction allows multiple metrics to be associated with the same performance and supports flexible, extensible evaluation strategies across diverse application domains.

O-DMIoT classifies metrics according to their evaluation perspective. *Quality of Service (QoS)* metrics capture objective, system-level performance metrics, such as latency, throughput, availability, and resource consumption. *Quality of Experience (QoE)* metrics

address human-centered aspects, including user satisfaction, trust, perceived effectiveness, and behavioral impact. Additionally, *Algorithm characteristic* metrics describe intrinsic properties of algorithm designs, such as computational complexity or scalability, enabling systematic comparison of alternative design strategies prior to deployment.

Evaluation is operationalized through explicit relators that contextualize the application of metrics to the entity being assessed. *Design Evaluation* associates metrics with algorithm designs, enabling the assessment of alternative design choices based on design-related characteristics. *Service Evaluation* associates metrics with deployed services, enabling run-time assessment of decision-making behavior using QoS and QoE metrics. In both cases, metrics act as mediating elements that enable performance expectations defined at the requirement level to be assessed at design time and in execution.

Modeling evaluation as a cross-cutting role grounded in performance and mediated by metrics ensures semantic consistency across requirements, design, and execution within O-DMIoT. This approach enables traceability from problem objectives to evaluation outcomes, supports systematic comparison of algorithmic alternatives, and establishes a principled foundation for adaptive reconfiguration and learning-based optimization in IoT systems.

Formal Semantics of the Evaluation and Metrics

To provide semantic rigor, the evaluation aspect of O-DMIoT is partially formalized using typed first-order logic. This formalization makes explicit how metrics relate to requirements, designs, and executions, and ensures that evaluation is consistently grounded across the ontology.

Logical vocabulary

Let the domain of discourse include the following unary predicates:

- *Metric*(x)
- *Performance*(x)
- *Requirement*(x)
- *Design*(x)
- *Service*(x)
- *Algorithm*(x)
- *QualityOfService*(x)
- *QualityOfExperience*(x)
- *AlgorithmCharacteristic*(x)
- *DesignEvaluation*(de)
- *ServiceEvaluation*(se)

Individual Terms:

- m denotes an individual of type *Metric*.
- p denotes an individual of type *Performance*.
- r denotes an individual of type *Requirement*.

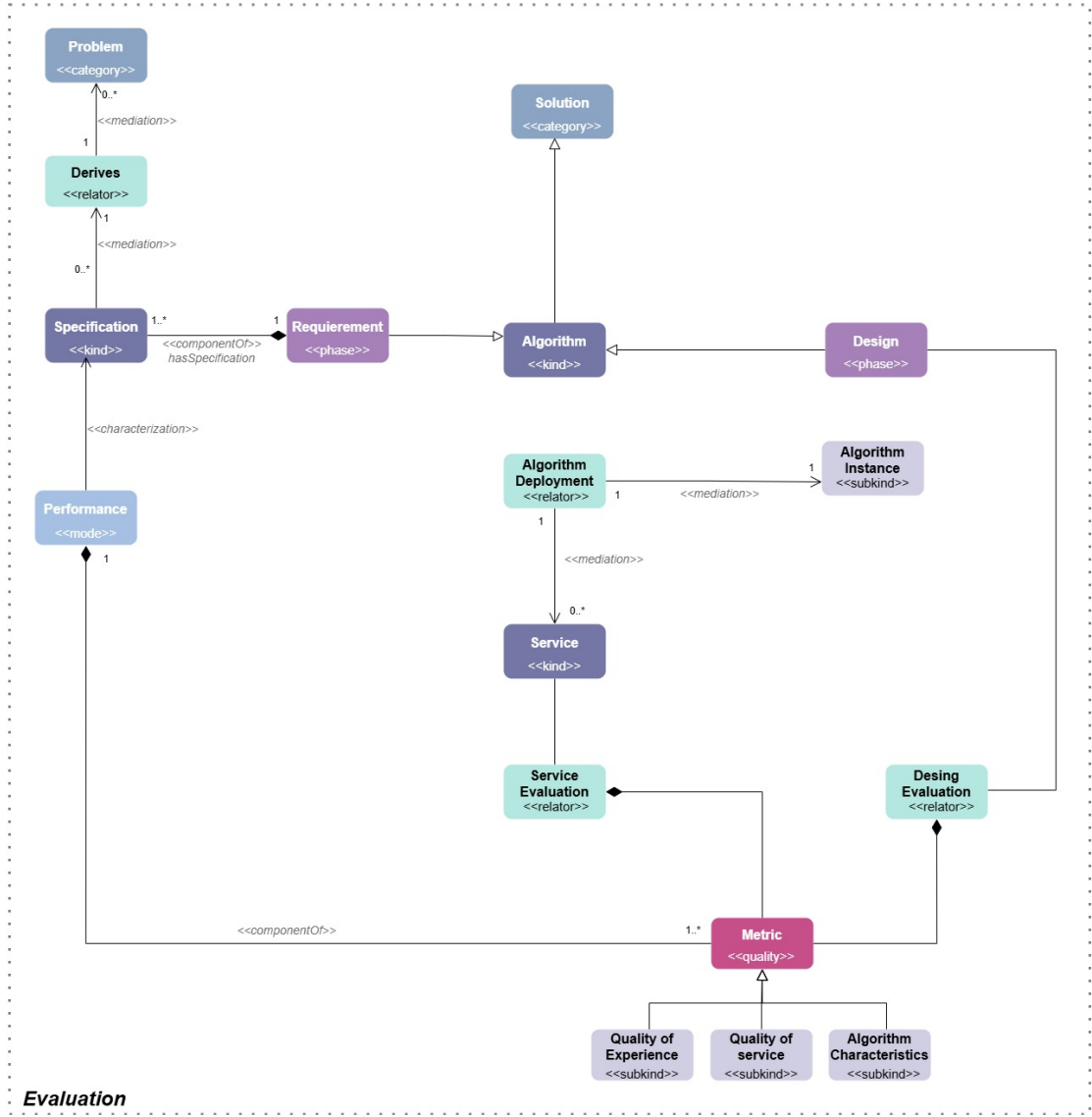


Figure 5.7.: O-DMIoT Evaluation layer

- d denotes an individual of type *Design*.
- s denotes an individual of type *Service*.
- a denotes an individual of type *Algorithm*.
- de denotes an individual of type *DesignEvaluation*.
- se denotes an individual of type *ServiceEvaluation*.

Binary Predicates:

- $characterizes(p, s)$, relating a performance criterion to the specification it characterizes.
- $measuredBy(p, m)$, relating a performance criterion to the metric that measures it.
- $designEvaluates(de, m, d)$, relating a design evaluation to the metric applied to a design.

- $serviceEvaluates(se, m, s)$, relating a service evaluation to the metric applied to a service.

Axiom 5.6.21 (Performance Characterizes Specifications). Every specification is characterized by exactly one performance.

$$\forall s (Specification(s) \rightarrow \exists! p (Performance(p) \wedge characterizes(p, s)))$$

Axiom 5.6.22 (Performance Is Measured by One or More Metrics).

$$\forall p (Performance(p) \rightarrow \exists m (Metric(m) \wedge measuredBy(p, m)))$$

Axiom 5.6.23 (Metrics Are Categorized by Evaluation Scope). Every metric belongs to exactly one evaluation category.

$$\begin{aligned} \forall m (Metric(m) \rightarrow \\ & (QualityOfService(m) \vee \\ & QualityOfExperience(m) \vee \\ & AlgorithmCharacteristic(m))) \end{aligned}$$

Axiom 5.6.24 (Design Evaluation Uses One or More Metrics). Every design evaluation applies at least one metric to a design.

$$\begin{aligned} \forall de (DesignEvaluation(de) \rightarrow \\ & \exists d, m (Design(d) \wedge Metric(m) \wedge \\ & designEvaluates(de, m, d))) \end{aligned}$$

Axiom 5.6.25 (Design Evaluation Metrics Are Proper). Ensures multiple metrics can be used in a design evaluation.

$$\begin{aligned} \forall de, m_1, m_2, d ((designEvaluates(de, m_1, d) \wedge \\ designEvaluates(de, m_2, d)) \rightarrow \\ (Metric(m_1) \wedge Metric(m_2))) \end{aligned}$$

Axiom 5.6.26 (Service Evaluation Uses One or More Metrics). Every service evaluation applies at least one metric to a service.

$$\begin{aligned} \forall se (ServiceEvaluation(se) \rightarrow \\ & \exists s, m (Service(s) \wedge Metric(m) \wedge \\ & serviceEvaluates(se, m, s))) \end{aligned}$$

Axiom 5.6.27 (Service Evaluation Metrics Are Proper). Ensures multiple metrics can be used in a service evaluation.

$$\begin{aligned} \forall se, m_1, m_2, s ((serviceEvaluates(se, m_1, s) \wedge \\ serviceEvaluates(se, m_2, s)) \rightarrow \\ (Metric(m_1) \wedge Metric(m_2))) \end{aligned}$$

Axiom 5.6.28 (Evaluation Metrics Measure Performance). Metrics used in either design or service evaluation must measure performance.

$$\begin{aligned} \forall m, d, s ((designEvaluates(d, m) \vee serviceEvaluates(s, m)) \rightarrow \\ \exists p (Performance(p) \wedge measuredBy(p, m))) \end{aligned}$$

The axioms of the evaluation formalize the consistent use of metrics across the Requirement, Design, and Execution layers. By representing metrics as qualities and evaluation as explicit relators, O-DMIoT ensures that assessment results remain semantically grounded, contextualized, and traceable. This comprehensive evaluation structure supports principled algorithm selection, adaptive reconfiguration, and reinforcement learning-based optimization, as discussed in subsequent chapters.

5.7. Formalization and Implementation

This section presents the formal encoding and implementation of O-DMIoT, demonstrating how the conceptual model and logical constraints introduced in previous sections are translated into a machine-processable ontology. The objective of this section is not to redefine the ontology, but to show that its semantics are explicitly formalized, logically constrained, and suitable for automated reasoning and extension.

O-DMIoT is implemented using the Web Ontology Language (OWL 2 DL), which offers a decidable fragment of first-order logic appropriate for representing domain ontologies with formal semantics. The ontology is serialized in Turtle format and developed with the Protégé software [31]. This implementation adheres to the SABI-O methodology, treating conceptual modeling, formalization, and evaluation as distinct yet interconnected activities.

5.7.1. OWL Encoding of O-DMIoT

The OWL encoding of O-DMIoT directly formalizes the conceptual model described in previous sections. Core ontological constructs, including problems, requirements, designs, algorithms, services, and metrics, are represented as OWL classes. Relationships among these constructs are modeled as object properties with explicitly defined domains and ranges.

Concepts identified as kinds in the conceptual model are encoded as OWL classes. Roles, phases, and relators are also represented as classes, with corresponding object properties that capture their mediation semantics. Cardinality constraints, disjointness axioms, and property restrictions enforce the structural constraints established during conceptual modeling. For example, the association between a specification and performance is encoded using an existential restriction, which ensures that each specification is characterized by at least one performance instance (See Figure 5.8). Similarly, evaluation relations are encoded so that design and service evaluations involve only valid metrics grounded in performance expectations.

Serialization in Turtle format enhances the ontology's readability and facilitates version control. The complete OWL implementation is available in a GitHub repository A.2.4, which serves as the reference artifact for the formalization of O-DMIoT.

5.7.2. Logical Constraints and Reasoning

The OWL implementation of O-DMIoT encodes logical constraints that correspond to the formal axioms introduced in the *Requirement*, *Design*, *Execution*, and *Evaluation* sections. These constraints, including existential and cardinality restrictions, disjointness axioms, and domain and range constraints, maintain the internal consistency of the ontology.

As OWL 2 DL is based on description logic, O-DMIoT supports automated reasoning with standard description logic reasoners. These reasoning services verify ontology consistency, classify concepts, and infer implicit relationships among entities. Reasoning also enables detection of modeling errors, such as incompatible metric assignments, invalid evaluation configurations, or missing specification elements.

Reasoning over O-DMIoT supports ontology-based tasks relevant to this thesis, including:

- Verifying that every evaluation metric is grounded in a performance expectation.
- Ensuring that algorithm instances are correctly associated with designs and services.
- Inferring applicable algorithm candidates based on requirement specifications (See Figure 5.9).


```

:Specification rdf:type owl:NamedIndividual ,
                gufo:Kind .

:Performance rdf:type owl:Class ;
              rdfs:subClassOf gufo:IntrinsicMode .

:Specification rdf:type owl:Class ;
              rdfs:subClassOf gufo:FunctionalComplex ,
                [ rdf:type owl:Restriction ;
                  owl:onProperty [ owl:inverseOf gufo:inheresIn
                                    ] ;
                  owl:someValuesFrom :Performance
                ] ,
                [ rdf:type owl:Restriction ;
                  owl:onProperty [ owl:inverseOf gufo:mediates
                                    ] ;
                  owl:qualifiedCardinality "1"^^xsd:nonNegativeInteger ;
                  owl:onClass :Derives
                ] .

:hasSpecification rdf:type owl:ObjectProperty ;
                  owl:inverseOf :isSpecificationOf ;
                  rdfs:domain :Requirement ;
                  rdfs:range :Specification .

```

Figure 5.8.: **Excerpt of the OWL formalization of O-DMIoT illustrating the encoding of the *Requirement* layer.** The figure presents the formal definition of the class *Specification* as a UFO functional complex, constrained to be characterized by at least one *Performance* instance (mode) and to participate in exactly one *Derives* relator mediating the relationship between a *Problem* and a *Requirement*. The excerpt also illustrates the formalization of object properties and inverse properties used to preserve the ontological semantics defined at the conceptual level.

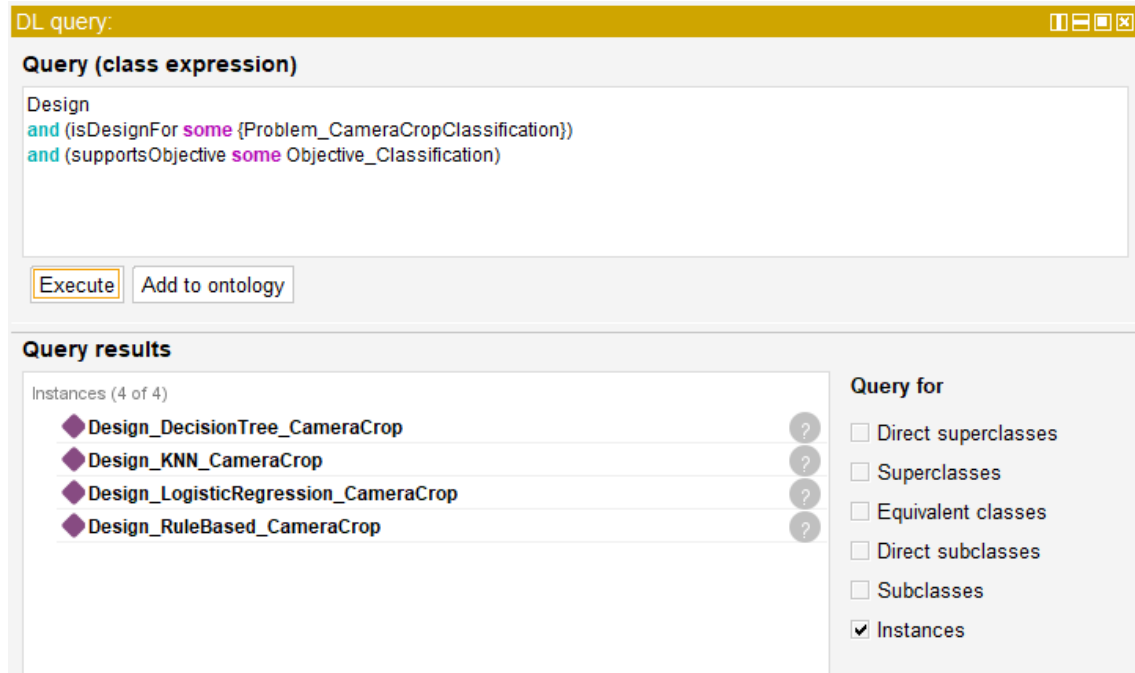


Figure 5.9.: **Example of a Description Logic (DL) query executed in Protégé to retrieve all algorithm designs applicable to a specific decision problem with a classification objective.** The query identifies *Design* instances that are associated with the *Problem_CameraCropClassification* and that support the objective *Objective_Classification*. The inferred results demonstrate how O-DMIoT enables semantic filtering of candidate algorithm designs based on problem characteristics, providing a foundation for ontology-driven algorithm selection.

These reasoning capabilities formally validate the ontology structure and ensure that algorithm selection and reconfiguration processes operate on semantically consistent knowledge.

5.7.3. Consistency and Extensibility

The consistency of O-DMIoT was verified using automated reasoning tools integrated in Protégé, specifically the HermiT v1.4.3.456 reasoner [18]. The ontology was checked for unsatisfiable classes, violated cardinality constraints, and conflicting disjointness axioms. These checks confirmed that the formalization is logically consistent with the conceptual model and the axioms defined in Section 5.6.

Extensibility is a central design objective of O-DMIoT. The ontology is structured to support incremental extension without changes to its core structure. New decision-making algorithms are introduced by instantiating or specializing the *Algorithm* and *Design* concepts, while new evaluation criteria are added by extending the *Metric* hierarchy. New application domains are supported by reusing the Requirement–Design–Execution pattern and introducing domain-specific specializations.

The extensibility of O-DMIoT was demonstrated in case studies presented later in this thesis (See Chapters 6 and 7), where additional metrics and evaluation criteria were incorporated to address domain-specific requirements. These extensions were achieved without

modifying the foundational ontology structure, confirming that O-DMIoT supports controlled evolution and reuse.

The consistency checking and extensibility of the O-DMIoT demonstrate that it is both a theoretically grounded ontology and a practical, reusable semantic model capable of supporting algorithm selection, evaluation, and adaptive reconfiguration in IoT systems.

5.8. Validation

O-DMIoT is validated through a comprehensive approach that integrates conceptual adequacy, ontological soundness, and application-driven assessment. This process follows established ontology engineering best practices and aligns with the SABiO methodology, which emphasizes validation using competency questions, foundational grounding, and application in concrete scenarios.

Validation of O-DMIoT occurs throughout its lifecycle, encompassing conceptualization, formal grounding, and application in realistic IoT decision-making contexts, rather than as a single post-hoc activity. The case studies in Chapters 6 and 7 demonstrate the ontology expressiveness, extensibility, and suitability for algorithm selection and evaluation in socio-technical systems.

5.8.1. Competency Question Coverage

Competency questions (CQs) are the primary means of validating the semantic adequacy of O-DMIoT. The CQs in Section 5.3.3 are based on requirements for describing, selecting, and evaluating decision-making algorithms in IoT systems. These questions guided the identification of core concepts and relationships in the ontology.

Each competency question is linked to one or more ontology elements, as summarized in Table 5.2. The Requirement layer addresses problem-oriented questions (CQ1–CQ3) by structuring decision problems through objectives, uncertainty, interaction, knowledge representation, and processing characteristics. The Design and Execution layers address solution-oriented questions (CQ4–CQ8) by capturing algorithm structure, assumptions, constraints, and service realization. The evaluation model addresses evaluation-oriented questions (CQ9–CQ10) by connecting performance expectations to metrics and evaluation processes.

The answerability of the CQ is demonstrated through semantic queries on the OWL implementation of O-DMIoT. Figure 5.9 shows a DL query that retrieves all algorithm designs for a decision problem with a classification objective, addressing CQ1–CQ5. Figure 5.10 illustrates the derivation of specifications and their associated performance and metrics, addressing CQ6 and CQ9. Addressing all competency questions through semantic queries and reasoning confirms that O-DMIoT fulfills its objectives. The case studies further show that these questions remain relevant even when domain-specific evaluation mechanisms are introduced, confirming the ontology’s appropriate level of abstraction for IoT decision-making.

5.8.2. Foundational Ontology Compliance (UFO)

Grounding O-DMIoT in the Unified Foundational Ontology provides a mechanism for validating its conceptual soundness. Specifically, lifecycle aspects of decision-making solutions are modeled as *phases*, enabling a clear distinction between requirement specification, design, and execution. These *phases* represent alternative conceptual states of a solution rather than temporal events or processes, thereby allowing lifecycle differentiation without embedding assumptions about time.

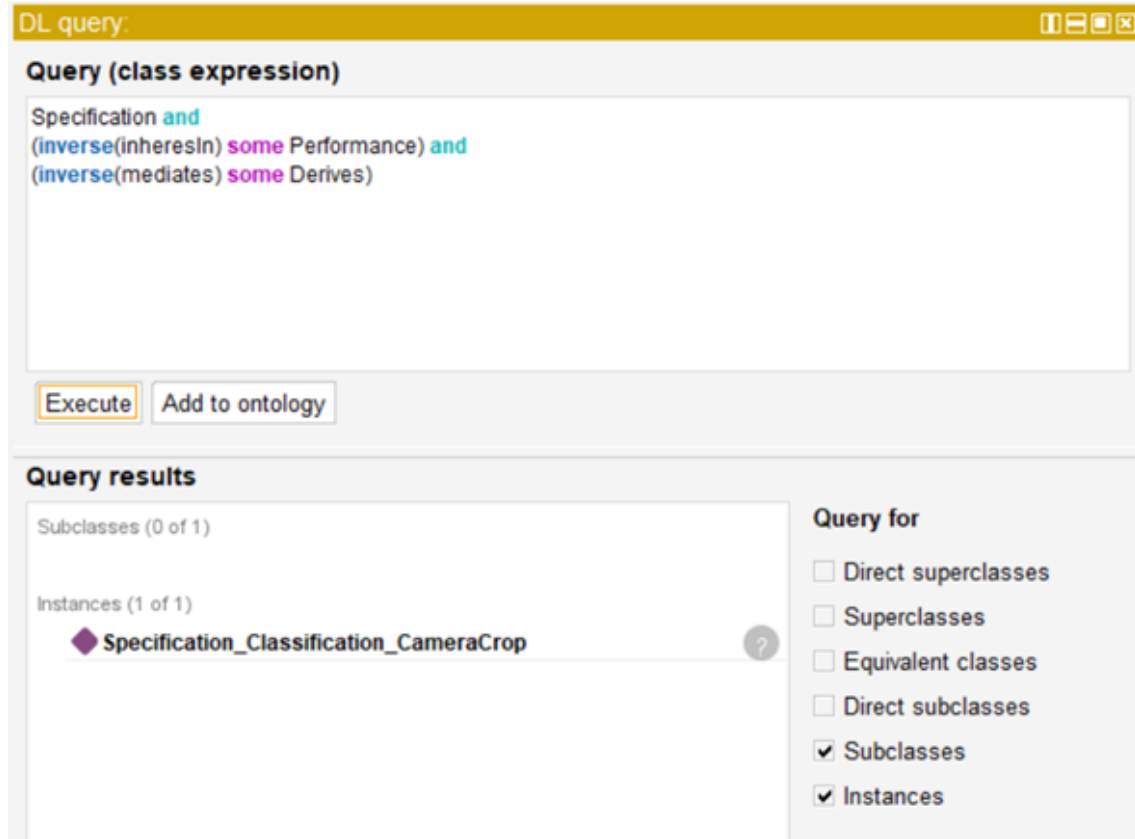


Figure 5.10.: Example of a Description Logic (DL) query executed in Protegé to retrieve requirement specifications derived from a decision problem and characterized by performance. The query selects *Specification* instances that (i) are characterized by a *Performance* mode and (ii) are related to a decision problem through the *Derives* relator. The inferred result, *Specification_Classification_CameraCrop*, illustrates how O-DMIoT supports semantic traceability from problem characterization to requirement specification, enabling ontology-driven reasoning over problem–requirement relationships.

This modeling choice addresses a known source of ontological ambiguity in decision-making models, where lifecycle stages are often represented as activities or time-indexed states. Such representations implicitly introduce assumptions about ordering, duration, or simultaneity that are unnecessary for algorithm selection. By adopting phases, O-DMIoT ensures that lifecycle distinctions are mutually exclusive and semantically well-defined, while remaining independent of explicit temporal commitments.

Validation of this abstraction was conducted through inspection of the OWL implementation and competency-question-driven reasoning. The ontology was examined to confirm that no temporal primitives, time-indexed properties, or event-based constructs are introduced. Furthermore, all defined competency questions were answered using semantic queries over the OWL formalization without requiring temporal parameters, ordering relations, or duration constraints (See Appendix A). These results demonstrate that explicit temporality is not necessary to support the intended reasoning tasks of the ontology.

Time-dependent effects relevant to algorithm selection, such as latency, convergence speed, adaptation behavior, or changes in user perception, are captured in O-DMIoT through *performance* and associated *metrics*, rather than through ontology-level temporal constructs. Performance represents the observable behavior of a solution relative to its requirements, while metrics quantify specific aspects of that behavior. Thus, temporal dynamics influence evaluation outcomes without being explicitly modeled as temporal entities.

This validation confirms that O-DMIoT captures lifecycle distinctions at an appropriate level of abstraction for algorithm selection and evaluation. Temporality is incorporated indirectly through evaluative measures when it affects decision outcomes, while being intentionally excluded from the ontology core. Consequently, O-DMIoT remains reusable across heterogeneous IoT domains with differing temporal assumptions, supporting generality without sacrificing evaluative expressiveness.

5.8.3. Application-driven Validation (Case Studies)

The primary validation of O-DMIoT is application-driven, as demonstrated through the case studies in Chapters 6 and 7. These studies show that O-DMIoT can effectively characterize, compare, and reason about decision-making services under diverse socio-technical conditions.

In both case studies, O-DMIoT described decision problems involving human behavior, uncertainty, and interaction, while remaining agnostic to domain-specific models such as epidemiological or behavioral simulations. External models, specifically agent-based simulations, generated evaluation artifacts, particularly Quality of Experience (QoE) metrics, which were then interpreted within the ontology as performance measures for services.

The results indicate that algorithm selection and evaluation cannot be based solely on technical performance. In the public health and vaccination scenarios, decision-making services with similar technical characteristics produced different human-centered outcomes depending on interaction type, behavioral assumptions, and information exposure. These differences were captured through QoE-based evaluation without changes to the core ontology, thereby validating O-DMIoT's abstraction level and semantic scope.

The case studies resulted in extensions of the algorithmic approach and metrics taxonomy, particularly in the Rule-driven and Quality of Experience categories, without altering the ontology's foundational structure. This evolution validates O-DMIoT's extensibility and confirms its minimal and adaptable design. These extensions demonstrate that O-DMIoT supports controlled growth driven by application needs, as intended.

The observed variation in QoE across scenarios provides empirical support for O-DMIoT as a foundation for adaptive and reconfigurable IoT systems. Although runtime reconfigu-

ration is not implemented in the case studies, the results suggest that performance feedback from semantic evaluation can guide decisions such as enabling, disabling, or reconfiguring decision-making services. This finding validates O-DMIoT's role as the semantic foundation for the learning-based algorithm selection mechanism introduced in later chapters.

5.9. Role of O-DMIoT in Thesis Framework

O-DMIoT plays a central integrative role within the thesis framework by converting the conceptual insights of the Taxonomy for Decision-Making in IoT (TDMIoT) into a formal, machine-processable semantic model. Although TDMIoT offers a structured classification of decision-making problems and algorithmic approaches, it remains primarily descriptive. O-DMIoT formalizes the taxonomy's categories, relationships, and constraints, which enables semantic reasoning, interoperability, and automated support for algorithm selection.

Grounding TDMIoT in a reference ontology, O-DMIoT establishes a shared semantic vocabulary that links problem characterization, algorithm design, and service execution. This semantic layer allows decision-making requirements to be expressed explicitly and consistently across system components, facilitating the comparison and alignment of alternative algorithmic solutions. Thus, O-DMIoT provides the formal knowledge base that supports systematic reasoning about algorithm selection.

Within the thesis, O-DMIoT bridges the gap between conceptual modeling and adaptive system behavior. It supplies the semantic foundation necessary to support reconfiguration mechanisms in IoT systems, enabling the selection, evaluation, and reconfiguration of services that implement decision-making algorithms in response to evolving requirements or contextual changes. Instead of embedding domain-specific logic within individual services, O-DMIoT externalizes decision-relevant knowledge into a reusable and extensible semantic model.

Furthermore, O-DMIoT creates the interface between symbolic knowledge and learning-based adaptation. The ontology's structured representations of problems, designs, execution contexts, and evaluation outcomes allow subsequent chapters to utilize this semantic information as input for learning-based algorithm selection mechanisms. Specifically, O-DMIoT defines the state descriptors, constraints, and evaluative signals that are used by the reinforcement learning approach introduced later in the thesis.

In summary, O-DMIoT represents a foundational contribution that unifies taxonomy and adaptive decision-making within a single framework. It transforms TDMIoT into an operational semantic model, supports the reconfiguration mechanisms, and enables the integration of symbolic reasoning with data-driven learning for algorithm selection in IoT systems.

5.10. Conclusions

This chapter presented O-DMIoT, a semantic ontology designed for decision-making algorithms in IoT systems. Motivated by the limitations identified in existing IoT ontologies and decision-making ontologies, the chapter developed an integrated semantic model that supports problem characterization, algorithm selection, execution, and evaluation across heterogeneous IoT environments.

O-DMIoT operationalizes TDMIoT as a reference ontology based on UFO. This foundation provides conceptual rigor and supports a clear separation among requirement specification, algorithm design, and execution as services, thereby minimizing ontological ambiguities related to lifecycle modeling and temporality. The ontology was developed using the

SABiO methodology, ensuring alignment between its structure, content, and the explicit requirements articulated through competency questions.

The ontology formalizes decision-making problems as categories defined by conceptual dimensions, including objective, uncertainty, interaction, and processing constraints. Algorithmic solutions are described in terms of design-time assumptions, constraints, and algorithmic properties, while execution is represented through service instantiation and deployment relationships. Evaluation is addressed as a cross-cutting concern based on performance and metrics, supporting both system-level (QoS) and human-centered (QoE) assessments without incorporating domain-specific logic into the ontology.

Validation results indicate that O-DMIoT fulfills its intended objectives. The ontology addresses the defined competency questions, maintains ontological consistency through UFO grounding, and enables semantic reasoning and traceability in application-driven scenarios. The case studies in subsequent chapters illustrate the application of O-DMIoT for comparing, evaluating, and reasoning about decision-making services in socio-technical IoT contexts.

In summary, O-DMIoT offers a reusable semantic foundation for algorithm selection and reconfiguration in IoT systems. By externalizing decision-relevant knowledge into a formal ontology, this chapter establishes the knowledge layer necessary for the adaptive, learning-based reconfiguration mechanism developed in the subsequent sections of the thesis.

References

- [1] R. Agarwal et al. “Unified IoT ontology to enable interoperability and federation of testbeds”. In: *2016 IEEE 3rd World Forum on Internet of Things (WF-IoT)*. IEEE, Dec. 2016, pp. 70–75. URL: <http://dx.doi.org/10.1109/wf-iot.2016.7845470>.
- [2] A. Aguilera et al. “IoTO++: An Enhanced Interoperability Based on Semantic for IoT Environments”. In: *2024 L Latin American Computer Conference (CLEI)*. IEEE, Aug. 2024, pp. 1–10. URL: <http://dx.doi.org/10.1109/clei64178.2024.10700398>.
- [3] M. B. Alaya et al. “Toward semantic interoperability in oneM2M architecture”. In: *IEEE Communications Magazine* 53.12 (Dec. 2015). Conference Name: IEEE Communications Magazine, pp. 35–41.
- [4] C. Alexakos and A. Kalogeras. “Internet of Things integration to a Multi Agent System based manufacturing environment”. In: *2015 IEEE 20th Conference on Emerging Technologies & Factory Automation (ETFA)*. ISSN: 1946-0759. Sept. 2015, pp. 1–8.
- [5] C. Alexakos et al. “Adaptive manufacturing utilizing ontology-driven multi-agent systems: Extending Pabadis’ Promise approach”. en. In: *2012 IEEE International Conference on Industrial Technology*. Athens: IEEE, Mar. 2012, pp. 42–47. URL: <http://ieeexplore.ieee.org/document/6209911/> (visited on 02/15/2023).
- [6] M. Arazzi, A. Nocera, and E. Storti. “The SemIoE Ontology: A Semantic Model Solution for an IoE-Based Industry”. In: *IEEE Internet of Things Journal* 11.24 (Dec. 2024), pp. 40376–40387. URL: <http://dx.doi.org/10.1109/jiot.2024.3452945>.

- [7] F. Baader, ed. *The description logic handbook: theory, implementation, and applications*. 4. print. Cambridge: Cambridge Univ. Press, 2005. 555 pp.
- [8] G. Bajaj et al. *A study of existing Ontologies in the IoT-domain*. 2017. URL: <https://arxiv.org/abs/1707.00112>.
- [9] K. Benali and S. A. Rahal. “OntoDTA: Ontology-Guided Decision Tree Assistance”. In: *Journal of Information & Knowledge Management* 16.03 (Sept. 2017). Publisher: World Scientific Publishing Co., p. 1750031. URL: <https://www.worldscientific.com/doi/10.1142/S0219649217500319> (visited on 05/18/2023).
- [10] M. Bermudez-Edo et al. “IoT-Lite: a lightweight semantic model for the internet of things and its use with dynamic semantics”. In: *Personal and Ubiquitous Computing* 21.3 (Feb. 2017), pp. 475–487. URL: <http://dx.doi.org/10.1007/s00779-017-1010-8>.
- [11] D. Bonino and F. Corno. “DogOnt - Ontology Modeling for Intelligent Domestic Environments”. en. In: *The Semantic Web - ISWC 2008*. Ed. by A. Sheth et al. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2008, pp. 790–803.
- [12] M. Brut et al. “WoO Approach General Overview”. en. In: (2014).
- [13] M. Compton et al. “The SSN ontology of the W3C semantic sensor network incubator group”. en. In: *Journal of Web Semantics* 17 (Dec. 2012), pp. 25–32. URL: <https://www.sciencedirect.com/science/article/pii/S1570826812000571> (visited on 02/07/2023).
- [14] M. Cornejo-Lupa, A. Aguilera, and I. Dongo. “Developing an Ontology to Enhance Semantic Interoperability in IoT Environment”. In: *CLEI Electronic Journal* 28.3 (May 2025). URL: <http://dx.doi.org/10.19153/cleiej.28.3.4>.
- [15] B. B. Duarte et al. “Ontological foundations for software requirements with a focus on requirements at runtime”. In: *Applied Ontology* 13.2 (May 2018), pp. 73–105. URL: <https://www.medra.org/servlet/aliasResolver?alias=iospress&doi=10.3233/AO-180197> (visited on 07/16/2024).
- [16] T. Elsaleh et al. “IoT-Stream: A Lightweight Ontology for Internet of Things Data Streams and Its Use with Data Analytics and Event Detection Services”. In: *Sensors* 20.4 (Feb. 2020), p. 953. URL: <http://dx.doi.org/10.3390/s20040953>.
- [17] R. Falbo. “SABiO: Systematic Approach for Building Ontologies”. In: 2014. URL: <https://www.semanticscholar.org/paper/SABiO%3A-Systematic-Approach-for-Building-Ontologies-Falbo/2660676b0d784d46d3c407793c0cee0c10e76c5b> (visited on 07/16/2024).
- [18] B. Glimm et al. “HermiT: An OWL 2 Reasoner”. In: *J. Autom. Reason.* 53.3 (Oct. 2014), pp. 245–269. URL: <https://doi.org/10.1007/s10817-014-9305-1> (visited on 01/20/2026).
- [19] J. Guerson et al. “OntoUML Lightweight Editor: A Model-Based Environment to Build, Evaluate and Implement Reference Ontologies”. In: *2015 IEEE 19th International Enterprise Distributed Object Computing Workshop*. IEEE, Sept. 2015, pp. 144–147. URL: <http://dx.doi.org/10.1109/EDOCW.2015.17>.

- [20] G. Guizzardi. “Ontological foundations for structural conceptual models”. English. PhD Thesis - Research UT, graduation UT. University of Twente, Oct. 2005.
- [21] G. Guizzardi et al. “Ufo: Unified Foundational Ontology”. In: *Applied ontology* 1.17 (2022), pp. 167–210.
- [22] R. Guizzardi et al. “A core ontology on decision making”. English. In: *CEUR workshop proceedings*. Vol. 2728. ISSN: 1613-0073. Rheinisch Westfälische Technische Hochschule, Nov. 2020, pp. 9–21. URL: <https://research.utwente.nl/en/publications/a-core-ontology-on-decision-making> (visited on 07/29/2022).
- [23] M. Hilario et al. “Ontology-Based Meta-Mining of Knowledge Discovery Workflows”. en. In: *Meta-Learning in Computational Intelligence*. Ed. by N. Jankowski, W. Duch, and K. Grabczewski. Studies in Computational Intelligence. Berlin, Heidelberg: Springer, 2011, pp. 273–315. URL: https://doi.org/10.1007/978-3-642-20980-2_9 (visited on 05/02/2023).
- [24] B. G. Humm and A. Zender. “An Ontology-Based Concept for Meta AutoML”. en. In: *Artificial Intelligence Applications and Innovations*. Ed. by I. Maglogiannis, J. Macintyre, and L. Iliadis. Vol. 627. Series Title: IFIP Advances in Information and Communication Technology. Cham: Springer International Publishing, 2021, pp. 117–128. URL: https://link.springer.com/10.1007/978-3-030-79150-6_10 (visited on 03/13/2023).
- [25] M. A. Jarwar, J. Watson, and S. Ali. “Modeling Industrial IoT Security Using Ontologies: A Systematic Review”. In: *IEEE Open Journal of the Communications Society* 6 (2025), pp. 2792–2821. URL: <http://dx.doi.org/10.1109/ojcoms.2025.3532224>.
- [26] X. Jin et al. “A fast and scalable approach for IoT service selection based on a physical service model”. en. In: *Information Systems Frontiers* 19.6 (Dec. 2017), pp. 1357–1372. URL: <https://doi.org/10.1007/s10796-016-9650-1> (visited on 05/03/2023).
- [27] E. Kornyshova and R. Deneckère. “Decision-Making Ontology for Information System Engineering”. en. In: *Conceptual Modeling – ER 2010*. Ed. by J. Parsons et al. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2010, pp. 104–117.
- [28] E. Kornyshova and R. Deneckère. “Decision-Making Ontology for Information System Engineering”. en. In: *Conceptual Modeling – ER 2010*. Ed. by J. Parsons et al. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2010, pp. 104–117.
- [29] M. Mohsin et al. “IoTChecker: A data-driven framework for security analytics of Internet of Things configurations”. en. In: *Computers & Security* 70 (Sept. 2017), pp. 199–223. URL: <https://www.sciencedirect.com/science/article/pii/S0167404817301190> (visited on 02/13/2023).
- [30] B. A. Mozzaquatro, R. Jardim-Goncalves, and C. Agostinho. “Towards a reference ontology for security in the Internet of Things”. In: *2015 IEEE International Workshop on Measurements & Networking (M&N)*. Oct. 2015, pp. 1–6.

- [31] M. A. Musen. “The protégé project: a look back and a look forward”. en. In: *AI Matters* 1.4 (June 2015), pp. 4–12. URL: <https://dl.acm.org/doi/10.1145/2757001.2757003> (visited on 01/20/2026).
- [32] A. Nayak, B. Božić, and L. Longo. “An Ontological Approach for Recommending a Feature Selection Algorithm”. en. In: *Web Engineering*. Ed. by T. Di Noia et al. Lecture Notes in Computer Science. Cham: Springer International Publishing, 2022, pp. 300–314.
- [33] *OWL - Semantic Web Standards*. URL: <https://www.w3.org/2001/sw/wiki/OWL> (visited on 01/30/2026).
- [34] F. Qaswar et al. “Applications of Ontology in the Internet of Things: A Systematic Analysis”. In: *Electronics* 12.1 (Dec. 2022), p. 111. URL: <http://dx.doi.org/10.3390/electronics12010111>.
- [35] H. Rahman and M. I. Hussain. “A comprehensive survey on semantic interoperability for Internet of Things: State-of-the-art and research challenges”. In: *Transactions on Emerging Telecommunications Technologies* 31.12 (Feb. 2020). URL: <http://dx.doi.org/10.1002/ett.3902>.
- [36] R. Ranpara. “A semantic and ontology-based framework for enhancing interoperability and automation in IoT systems”. In: *Discover Internet of Things* 5.1 (Mar. 2025). URL: <http://dx.doi.org/10.1007/s43926-025-00122-8>.
- [37] F. Ruy et al. “SEON: A Software Engineering Ontology Network”. In: Nov. 2016, pp. 527–542.
- [38] M. Vila et al. “Critical infrastructure awareness based on IoT context data”. In: *Internet of Things* 23 (Oct. 2023), p. 100855. URL: <http://dx.doi.org/10.1016/j.iot.2023.100855>.
- [39] J. Wątróbski. “An Ontology Supporting Multiple-Criteria Decision Analysis Method Selection”. In: *Intelligent Decision Technologies 2016*. Ed. by I. Czarnowski et al. Vol. 56. Series Title: Smart Innovation, Systems and Technologies. Cham: Springer International Publishing, 2016, pp. 89–99. URL: http://link.springer.com/10.1007/978-3-319-39630-9_8 (visited on 12/30/2022).
- [40] M. Zappatore et al. “Semantic models for IoT sensing to infer environment–wellness relationships”. In: *Future Generation Computer Systems* 140 (Mar. 2023), pp. 1–17. URL: <http://dx.doi.org/10.1016/j.future.2022.10.005>.

6. Agent-based evaluation of information and communication technology interventions

Abstract. Human-centered Internet of Things (IoT) systems operate at the intersection of algorithmic decision-making, technological infrastructure, and human behavior. Evaluating these systems requires approaches that capture both technical outcomes and socio-technical dynamics. This chapter provides an agent-based evaluation of information and communication technology (ICT)-mediated decision-making within the O-DMIoT framework, illustrating how agent-based modeling (ABM) enables systematic analysis of decision-making services in human-centered environments. A technological non-pharmaceutical intervention (NPI), operationalized as a point-of-interest (POI) recommendation service, is incorporated into an agent-based simulation that models human behavior, epidemiological dynamics, and algorithmic decision processes. The behavioral model includes factors such as anxiety, trust, and mobility choices, facilitating the examination of feedback loops between algorithmic recommendations and human responses. Simulation results are synthesized into a human-centric Quality of Experience (QoE) metric that integrates infection impact, anxiety variation, and participation in recreational activities, allowing for comparative evaluation under standardized conditions. Findings demonstrate that even simple, interpretable algorithms can drive behavioral adaptation and influence system-level outcomes at scale. This chapter positions ABM as a foundational evaluation method within O-DMIoT, connecting algorithm design to human-centered performance and enabling systematic comparison of decision-making services.

6.1. Introduction

The COVID-19 pandemic highlighted the importance of decision-making in managing complex socio-technical systems under uncertainty. Governments and organizations worldwide implemented both pharmaceutical interventions (PI), such as vaccines, and non-pharmaceutical interventions (NPI), including mobility restrictions, social distancing, and mask mandates, to mitigate contagion [11]. Although these measures effectively reduced transmission, they also had unintended consequences, particularly for mental health and social well-being. Prolonged isolation, disrupted routines, and ongoing uncertainty increased anxiety among workers and complicated their reintegration into pre-pandemic activities [2, 19].

During this period, Information and Communication Technologies (ICT) emerged as an additional layer of intervention. Mobile applications, IoT-based sensing, and digital public-health tools enabled functionalities such as contact tracing, symptom monitoring, and crowd management [4, 17]. These technological NPIs aimed to build resilience and

support safe social interaction. Their effectiveness, however, extended beyond epidemiological outcomes. They influenced behavioral responses, trust, and perceived risk, all of which shape the Quality of Experience (QoE) in socio-technical systems [14, 24]. A comprehensive understanding of these human-centric dimensions is essential for designing IoT services that address both technical requirements and societal responses.

This chapter supports Contribution 2: The Human-Centric Evaluation Framework by presenting an agent-based model (ABM) designed to evaluate ICT-mediated interventions in the context of the COVID-19 pandemic. The model integrates epidemiological dynamics, vaccination strategies, and behavioral submodels encompassing anxiety, trust, and technology adoption [13, 15]. Through simulations, this work examines how technological NPIs interact with pharmaceutical strategies to influence well-being, reintegration processes, and perceived safety. The resulting metrics provide qualitative insights that complement traditional epidemiological indicators and the integration of human-centric considerations into O-DMIoT for algorithm selection and adaptive service reconfiguration.

The remainder of this chapter is structured as follows. Section 6.2 examines NPIs and ICT-based strategies within the context of pandemic response. Section 6.3 presents the architectural framework of the ABM and its constituent submodels. Section 6.4 details the modeling contributions, including the technological NPI model, anxiety and adoption dynamics, and their integration with worker routines. Section 6.5 outlines the evaluation methodology and computational experiments. Section 6.6 addresses the derivation of human-centric metrics for O-DMIoT and discusses their implications for semantic-driven decision-making. Finally, Section 6.8 concludes with a summary of key findings.

6.2. Background: Non-Pharmaceutical Interventions and ICT in Pandemic Response

The global response to COVID-19 relied on a diverse set of strategies intended to reduce transmission. These strategies are typically categorized into PI, such as vaccines, and NPI, including measures such as social distancing, mask mandates, and mobility restrictions [11]. NPIs were crucial in the early phases of the pandemic, when vaccines were not yet available. However, their prolonged implementation introduced significant social and psychological consequences, especially for workers navigating remote or hybrid work arrangements [2, 19].

As the pandemic evolved, ICT played a central role in enabling public health resilient responses. ICT-based NPIs capitalized on mobile devices and IoT infrastructure to operationalize interventions such as Bluetooth-enabled contact tracing, geolocation services, and self-assessment platforms. Research into the efficacy of these applications, as conducted in [12, 1], provided important insights. For instance, [1] employs an agent-based model (ABM) with a susceptible-infectious-recovered (SIR) structure to examine the epidemiological impact of contact-tracing applications. The authors concluded that isolated deployment of these tools is insufficient to curb disease transmission. Conversely, [12] evaluates both centralized and decentralized contact tracing approaches using a similar modeling framework. The study similarly emphasizes that the effectiveness of such tools depends on integration with broader mitigation strategies. Both works examine the effectiveness of an ICT, focusing on epidemiological outcomes, such as infection rates and reductions in transmission. However, they overlook key human-centric aspects, including mental health, trust, and behavioral adaptation [18, 24].

This omission is significant because the effectiveness of technological NPIs depends on technical accuracy and user acceptance. Factors such as anxiety, risk perception, and trust critically influence the adoption of recommended behaviors and the utilization of digital

tools [14, 19]. Consequently, evaluating ICT interventions requires a holistic perspective that integrates epidemiological outcomes with social and psychological dimensions. Unlike other modeling strategies, agent-based modeling uniquely captures the interactions among heterogeneous agents and system layers, enabling insights into emergent socio-technical behaviors that shape intervention success [13, 15].

6.3. An agent-based model to evaluate NPI to support Post-Pandemic work activities and the well being of workers

This work introduces an ABM designed to study how a technological NPI can help workers resume routine activities after the COVID-19 outbreak while supporting their mental well-being. The intervention targets behaviors associated with the COVID-19 anxiety syndrome—particularly the tendency to avoid leaving home. To mitigate this effect, the model incorporates a decision-support system (DSS) accessible through a mobile application. This DSS generates personalized recommendations by identifying nearby recreational places with low occupancy; thus, it is a POI recommendation system.

To operate, the NPI relies on lightweight monitoring infrastructure capable of estimating the number of people in each recreational venue. In this model, this is represented by smart thermometers placed at the entrances of indoor spaces that report temperature readings for all incoming visitors. While temperature screening later proved ineffective as a health barrier, it continued to be used in many contexts as an acceptable precautionary measure and as part of broader monitoring strategies. In this scenario, these thermometers are used solely to approximate occupancy in real time. When a worker wishes to engage in leisure activities, the mobile App sends their current location to the POI recommendation system, and it returns a ranked list of recommended places.

The ABM also incorporates the broader pandemic context, including a vaccination process, social interactions shaped by workers' routines, and behavioral changes associated with risk perception and anxiety. The model was implemented using the GAMA platform (version 1.8) [25] and applied to a case study in Mexico. Figure 6.1 depicts the scenario for this simulation.

6.3.1. Model Overview

Figure 6.1 provides an overview of the spatial and technological context in which the NPI operates. The left side of the figure depicts the simulated urban environment, including workplaces, recreational venues, hospitals, and vaccination centers. Worker agents navigate this environment as part of their daily routines, interacting with different spaces and encountering varying levels of crowding.

The right side illustrates the operational logic of the technological intervention. Recreational venues are equipped with crowd-monitoring devices—in this model represented by smart thermometers placed at their entrances—which continuously estimate the number of visitors inside each location. These occupancy readings are transmitted to a cloud service that aggregates the data and feeds it into the POI recommendation algorithm. When a worker decides to engage in a leisure activity, the mobile application sends the worker's location to this service, which computes a ranked list of nearby venues using both distance and current crowd level as inputs. The recommendations are then delivered back to the worker in real time. Fig. 6.1 highlights how the ABM integrates individual movement patterns, IoT-based sensing, and an online decision-support mechanism into a single socio-technical system.

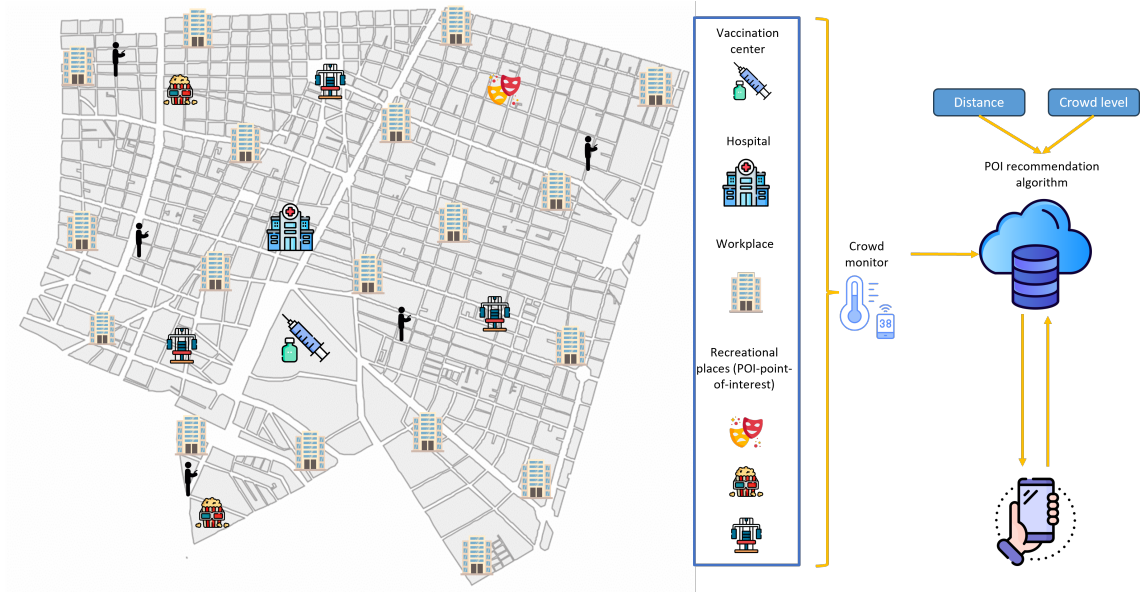


Figure 6.1.: Interaction between urban infrastructure, IoT monitoring, and the POI recommendation algorithm.

Figure 6.2 presents the high-level architecture of the ABM, organized into Behavioral, Epidemiological, and Technological layers. This multi-layered structure captures the socio-technical dynamics that emerged during the pandemic, showing how health processes, psychological responses, and ICT-based interventions interact within a single simulation framework.

While the ABM contains several interacting submodels, the behavioral and technological components—particularly the anxiety model, the NPI adoption mechanism, and the technological intervention—constitute the core methodological contributions of this thesis. These components are introduced briefly here and examined in depth in Section 6.4.

Epidemiological Layer

The epidemiological layer reproduces disease dynamics that shape workers exposure and health states. This layer integrates an extended SEIR model enriched with behavioral states and a vaccination model.

- **Epidemiological submodel**

Each worker transitions through COVID-19 health states according to an SEIR(susceptible - exposed - infected - recovered) epidemiological model [21]. These transitions influence behavioral states:

1. whether the worker continues normal activities,
2. whether they must isolate, or
3. whether hospitalization is required (Hospitals serve as destinations for agents who reach a severe infection state).

- **Vaccination submodel**

The vaccination model implements a priority-based scheduling strategy consistent with public health guidelines, giving precedence to workers in face-to-face schemes,

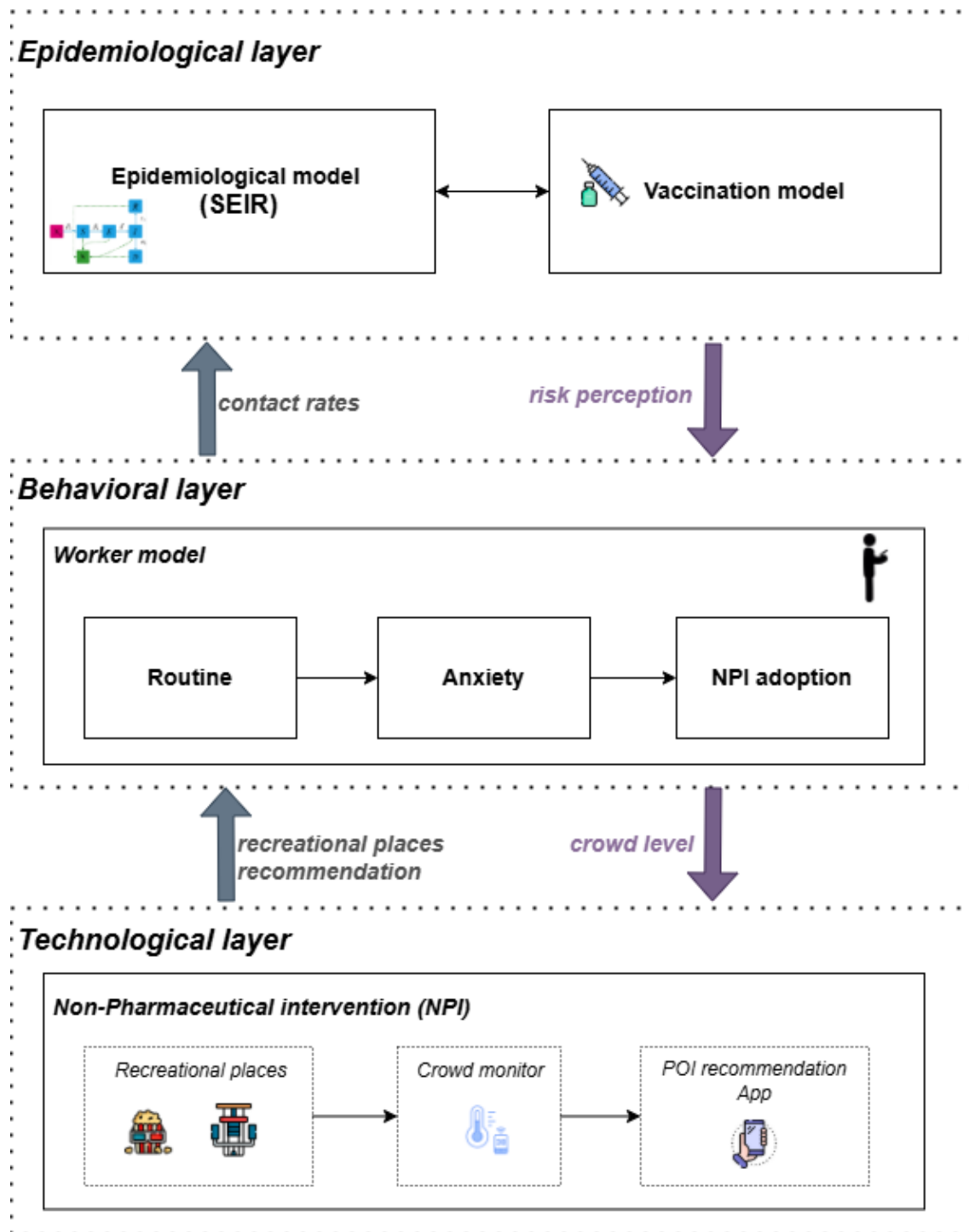


Figure 6.2.: High-level architecture of the ABM

older individuals, and agents with comorbidities. Vaccine efficacy and scheduling influence susceptibility, infection spread, and the epidemiological conditions perceived by workers.

A vaccination schedule is modeled to represent the progressive administration of vaccine doses:

1. A Vaccine Manager agent oversees appointment scheduling and notifies workers when it is time to receive their vaccine.
2. Vaccination centers act as fixed geographic locations where workers receive their shots.

This layer determines the overall infection dynamics and interacts with the behavioral layer through workers' activity choices.

Behavioral Layer

The behavioral layer models how workers adjust their psychological and behavioral responses to changing epidemiological conditions and technological interventions. This layer captures routines, personalities, and decisions of workers as they navigate their daily activities. The worker agent in this layer contains the behavioral submodels that lead their actions:

1. Follow daily schedules.
2. Update their anxiety level based on individual traits and contextual epidemiological states.
3. Choose whether to comply with or ignore App recommendations.
4. Interact with the epidemiological and technological layers.

• Routine submodel

Each worker follows a daily routine that includes work and recreational activities. A worker is under one of three work schemes that reflect labor practices:

- *Home-office*: All work is performed from home.
- *Hybrid*: Workers commute on a subset of days and stay home for the rest.
- *Face-to-face*: Daily travel to the workplace is required.

Outside working hours, workers may engage in leisure activities, either at home or elsewhere. However, their decision to leave home is influenced by:

1. Current anxiety level,
2. Recommendations delivered by the App.
3. Trust-based adoption of the technological NPI.

As a contextual factor influencing routine, vaccination can be added as an additional activity once during the simulation. If a worker is scheduled to receive a vaccine, their routine for that day will include this activity.

• Anxiety submodel

Each worker is characterized by personality traits based on the Big Five model [27]. According to [20], traits as neuroticism, extraversion, and conscientiousness are directly related to the COVID-19 Anxiety Syndrome, characterized by:

- *Avoidance*: Avoiding public spaces out of fear of contracting the disease.
- *Checking*: Constantly looking for symptoms in ourselves.
- *Worrying*: Compulsive research about the virus at the cost of other activities.
- *Thread monitoring*: Paying attention to others for possible symptoms.

The anxiety level of a worker is influenced by their routine (eg, a worker in a face-to-face scheme experiences major levels of anxiety), the contextual epidemiological state, personality traits, and risk factors (eg, comorbidities and immunization).

The proposed technological intervention aims to address the *Avoidance* factor by equipping workers with a tool that enables informed decision-making regarding crowd levels in recreational areas, thereby reducing infection risk. Anxiety levels influence the adoption of the technological intervention, while vaccination and positive experiences with the intervention contribute to anxiety reduction.

- **NPI adoption**

The technological NPI incorporates a recommendation system accessible via a mobile application, which identifies the nearest and least crowded venues for recreational activities, thereby reducing the risk of virus transmission during social engagement. Recognizing that adaptation is required for new technology adoption, a dynamic trust model based on personality traits was implemented [5]. This trust model enables users to develop trust in the application through experience. The NPI adoption model determines whether a worker utilizes the technological intervention.

The behavioral layer integrates feedback from both the epidemiological layer (e.g., infection levels) and the technological layer (e.g., app recommendations), enabling workers to dynamically adjust their routines and, in turn, their anxiety levels.

Technological Layer

This layer contains the technological components of the NPI, from sensing infrastructure to the decision-support logic. Each recreational venue has a monitoring infrastructure capable of estimating the number of people inside. In this model, this is represented by smart thermometers placed at the entrances of indoor spaces that report temperature readings for all incoming visitors. Therefore, as shown in Fig. 6.2, Recreational places include crowd-monitoring infrastructure that feeds a POI recommendation system.

- **Crowd monitoring**

Recreational places are equipped with smart thermometers that measure the temperature of visitors as they enter. In the model, these readings are used solely to infer current occupancy levels, which are sent to the POI recommendation system

- **POI Recommendation System**

This component represents the server-side logic of the NPI:

1. It maintains the state of all recreational venues.
2. It processes occupancy updates.
3. When a worker requests recommendations, it executes the recommendation algorithm and returns a list of suitable places.

- **App**

Workers interact with the POI recommendation system through a mobile App:

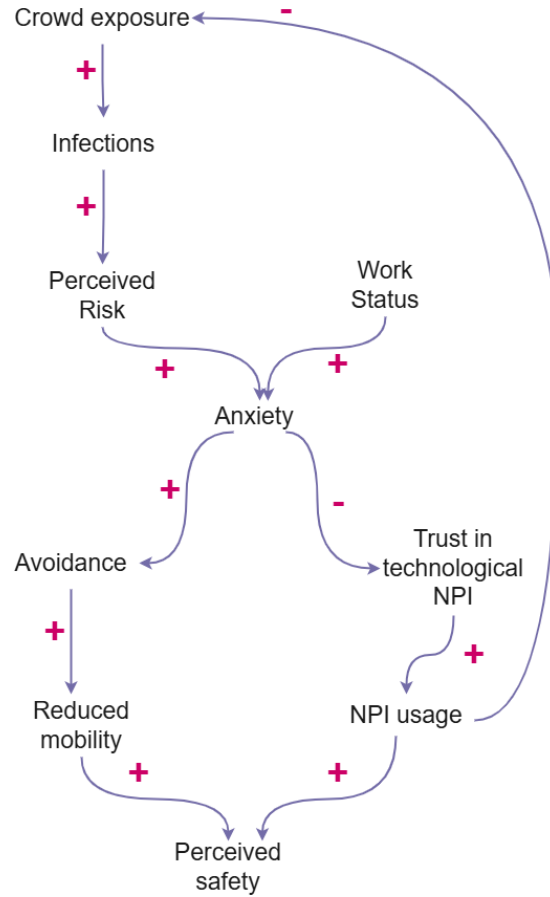


Figure 6.3.: Causal loop diagram of behavioral dynamics and ICT-mediated intervention effects.

1. They send their location when they want to perform a recreational activity.
2. The App displays the recommended venues generated by the system.
3. Workers decide whether to trust and follow these suggestions.

To clarify the dynamic interactions captured within the Behavioral Layer, Figure 6.3 presents a causal loop diagram (CLD) that summarizes the key feedback mechanisms linking epidemiological conditions, psychological responses, and ICT-based interventions. The diagram provides a qualitative representation of how workers' anxiety, mobility decisions, and trust in technological NPIs co-evolve over time.

6.4. Modeling contributions

This section presents the modeling contributions developed in this thesis. While the previous section offered a conceptual overview of the entire agent-based system, the focus now shifts to the specific components that were designed, formalized, and implemented to operationalize the socio-technical problem under study.

The first part introduces the Technological NPI model, which formalizes how the proposed non-pharmaceutical intervention functions within the IoT infrastructure. Beyond describing its internal logic, this section presents the TDMIoT classification, which positions the intervention within the broader taxonomy of IoT decision-making mechanisms.

This classification clarifies the technological assumptions guiding the intervention, makes its characteristics explicit, and establishes its role as a service within the simulation environment.

The second part addresses the Human Behavior Modeling contributions, which operationalize the psychological and social processes that shape workers' responses to the technological intervention. This includes a computational Anxiety Model grounded in psychological theory, an NPI Adoption Model driven by dynamic trust based on experiences, and the integration of these mechanisms within the Worker agent. This integration ensures that psychological states, adoption decisions, and mobility patterns evolve over time and directly interact with the technological NPI. Rather than treating behavior as a static input, the model captures how workers adapt their routines, assess risks, and build trust in the system as the simulated pandemic unfolds.

These contributions provide the analytical foundation for the experimental scenarios and the QoE evaluation framework that complements O-DMIoT.

6.4.1. Technological NPI model

The Technological NPI formalizes the operation of the POI recommendation system that mediates workers' recreational mobility during the post-pandemic transition. In the ABM, this intervention is implemented as an IoT service that provides personalized recommendations for low-occupancy recreational places. Its design reflects two key characteristics of technological NPIs: (i) they do not alter health-related biological processes directly, but instead modify behavioral patterns; and (ii) they rely on ICT infrastructure to sense, model, and influence human decision-making.

6.4.2. IoT monitoring infrastructure

The Technological NPI assumes the availability of a minimal, non-intrusive monitoring infrastructure capable of estimating the instantaneous occupancy of recreational venues (e.g., gyms, galleries, indoor studios). In this model, each venue is equipped with a smart thermometer placed at the entrance. Although temperature screening later proved to have limited epidemiological impact, it was widely deployed and accepted as a pragmatic monitoring tool during the COVID-19 pandemic. In this work, the thermometers are repurposed solely as counting sensors: each temperature reading corresponds to a single entry, enabling an approximate estimation of the current occupancy within the venue.

This monitoring is represented in the ABM as a distributed set of IoT devices that continuously send occupancy updates to the Technological NPI service. These sensing events feed the recommendation algorithm.

POI recommendation algorithm

When a worker decides to engage in recreational activities, the mobile App sends their current location to the POI recommendation system. The system identifies the nearest spatial cluster of venues and ranks venues by their current crowd levels, returning the three least crowded nearby options. This process balances efficiency (by restricting the search to nearby venues) with simplicity (by applying a single ranking criterion).

The operational logic underlying the recommendation mechanism is detailed in Algorithm 1. The definitions of the constituent functions are provided in Algorithms 2, 3, 4 and 5. This modular approach abstracts the core logic of the recommendation process and ensures that it is systematically executed each time a worker requests a recommendation.

Algorithm 1: POI Recommendation Algorithm

Input: User location $user_location$
Output: List of the three closest low-crowd recreational venues
 $cluster \leftarrow \text{SELECTCLOSESTCLUSTER}(user_location);$
 $poi_list \leftarrow \text{GETPOIS}(cluster);$
 $poi_list \leftarrow \text{SORTBYCROWDLEVEL}(poi_list);$
return $TOP(poi_list, 3);$

Algorithm 2: SELECTCLOSESTCLUSTER

Input: User location $user_location$, set of clusters $\mathcal{C}$
Output: Nearest cluster to the user
 $min_dist \leftarrow \infty;$
 $nearest \leftarrow \emptyset;$
foreach $c \in \mathcal{C}$ **do**
 $d \leftarrow \text{GEOGRAPHICALDISTANCE}(user_location, c.centroid);$
 if $d < min_dist$ **then**
 $min_dist \leftarrow d;$
 $nearest \leftarrow c;$
return $nearest;$

POI recommendation algorithm classification under TDMIoT

The POI recommendation algorithm is classified across the TDMIoT dimensions based on its goal, information flow, decision logic, and computational structure, as summarized in Table 6.1. The rationale for each classification is provided below.

- **Objective $\rightarrow$ Optimization**

The algorithm aims to identify the optimal alternative among recreational places by minimizing distance and promoting lower crowd levels. Rather than predicting a value, the approach seeks to determine the best choice using a multi-criteria decision rule. This aligns with the *Optimization* objective, as the algorithm balances competing factors to generate the most suitable recommendation for the user.

- **Processing $\rightarrow$ Centralized**

All computations are executed within the system's backend, which receives the user's location, processes cluster data, ranks places, and returns the recommendation. The decision is made by a single computational component rather than distributed nodes, thus following a *Centralized* processing model.

- **Processing Layer $\rightarrow$ Application Layer**

The POI recommendation works at the *Application* level, where high-level decision logic and user services run. It does not interact directly with network routing or sensing hardware. Instead, it uses processed inputs to deliver tailored service recommendations, a feature of application-layer algorithms.

- **Interaction $\rightarrow$ Human**

The decision process is initiated when a human user explicitly requests or accesses a recommendation. The algorithm's output is designed for direct human use to

Algorithm 3: SORTBYCROWDLEVEL

Input: List of POIs *poi_list*
Output: List sorted by crowd level (descending)
sorted_list $\leftarrow$ SORT(*poi_list*, *key* = *crowd_level*);
return *sorted_list*;

Algorithm 4: TOP

Input: Sorted list *poi_list*, integer *k*
Output: Top *k* elements of *poi_list*
return *first k elements of poi_list*;

support individual decision-making regarding recreational mobility. This categorizes the algorithm as involving *Human* interaction.

- **Uncertainty $\rightarrow$ Parameter**

The algorithm addresses uncertainty arising from assumptions encoded as fixed parameters, rather than from sensor noise or real-time measurement errors. User preferences for proximity and low crowd levels are incorporated into the scoring criteria as design-time assumptions, not as stochastic or noisy data. Consequently, the uncertainty in the decision process is classified as *Parameter* uncertainty.

- **Knowledge Representation $\rightarrow$ Mathematical**

The algorithm employs a mathematical scoring function that integrates distance and crowd level to rank POIs. This function serves as the knowledge base for evaluating alternatives, supporting the classification as a *Mathematical* knowledge representation.

- **Algorithm Approach $\rightarrow$ Criterion-Driven**

The algorithm selects an option based on explicit criteria (distance and crowd density), which directly determine the ranking of recreational places. Because the decision is guided by predefined evaluation criteria rather than learning from data or modeling utility under uncertainty, the approach is classified as *Criterion-Driven*.

- **Algorithm Technique $\rightarrow$ Heuristic**

The ranking and selection mechanism utilizes a straightforward heuristic: POIs are scored according to defined criteria, sorted, and the top candidates are selected. While this method does not ensure a globally optimal distribution of users, it offers a practical, near-optimal solution with low computational cost. Therefore, the technique is classified as *Heuristic* within the criterion-driven approach.

- **Algorithm $\rightarrow$ POI Recommendation Algorithm**

The POI recommendation algorithm operationalizes these decision-making processes by integrating the aforementioned characteristics to deliver personalized, context-aware recommendations for recreational places, see Alg. 1.

It is important to note that the contribution of this work does not lie in proposing a novel recommendation algorithm, but in formalizing and integrating a simple, transparent decision-making mechanism as a Technological NPI within an agent-based simulation framework. The algorithm is deliberately designed to be interpretable, computationally

Algorithm 5: GEOGRAPHICALDISTANCE

Input: Coordinates p_1, p_2
Output: Euclidean distance between p_1 and p_2
 $dx \leftarrow p_1.x - p_2.x;$
 $dy \leftarrow p_1.y - p_2.y;$
return $\sqrt{dx^2 + dy^2};$

TDMIoT dimensions	Values
Objective	Optimization
Processing	Centralized
Processing layer	Application
Interaction	Human
Uncertainty	Parameter
Knowledge	Mathematical
Algorithm approach	Criterion-driven
Algorithm technique	Heuristic
Algorithm	POI recommendation algorithm

Table 6.1.: TDMIoT classification of the POI recommendation algorithm

lightweight, and behaviorally plausible, enabling its repeated execution at scale and its coupling with psychological and epidemiological dynamics. This design choice allows the effects of ICT-mediated recommendations to be studied as a socio-technical intervention.

Role of the Technological NPI within the ABM

Within the agent-based model, the Technological NPI serves as an active mediator between workers' behavioral decisions and the evolving epidemiological context. The POI recommendation system is embedded in the simulation and interacts with both the Behavioral and Epidemiological layers.

From a behavioral perspective, the NPI is invoked conditionally. Workers request recommendations only when planning recreational activities outside the home. The system's recommendations influence activity choices by reducing the perceived risk of crowd exposure, thereby affecting workers' anxiety levels. These changes affect mobility decisions, adherence to recommendations, and continued use of applications.

From the epidemiological perspective, the NPI indirectly shapes contact patterns by redistributing recreational mobility toward less crowded venues. Although the system does not enforce behavior, its recommendations alter the spatial distribution of social interactions, thereby influencing population-level exposure risk.

The Technological NPI is involved in the ABM's intrinsic feedback loops. Positive experiences with recommendations, such as successful recreational outings with reduced infection risk, reinforce trust in the application, increase adoption, and lower anxiety. In contrast, unfavorable outcomes may diminish trust and adoption, limiting the intervention's effectiveness. These feedback mechanisms allow the simulation to capture the adaptive and emergent effects of ICT-based interventions over time.

By mediating between sensing infrastructure, algorithmic decision-making, and human behavior, the Technological NPI plays a central role. It operationalizes the socio-technical dynamics under study. Its integration into the ABM enables the evaluation of technological

interventions not only on recommendation accuracy but also on behavioral adaptation, psychological well-being, and broader system outcomes.

6.4.3. Human Behavior Modeling

Accurate modeling of human behavior is fundamental to this thesis, as behavioral processes play a pivotal role in shaping the effectiveness of technological change within socio-technical systems. Conceptualizing behavior as a dynamic process shaped by environmental conditions and ICT-based interventions is essential for evaluating the performance of decision-making services in human-centered contexts. Accordingly, human behavior modeling advances the thesis objective of integrating agent-based modeling as an evaluation strategy, enabling the assessment of ICT-mediated interventions beyond technical criteria alone.

This section presents the behavioral models that underpin technology adoption within the simulation. The Worker model captures individual risk perception, emotional responses, and mobility patterns. The Anxiety model formalizes the influence of perceived epidemiological risk and recreational activity on emotional states, while the NPI Adoption model describes the emergence of trust in the recommendation system based on user experience. Collectively, these models offer an integrated representation of the psychological and decision-making processes governing interaction with the Technological NPI.

Worker Model

The Worker model specifies how individual behavior and decision-making are operationalized within the agent-based simulation. Each worker is characterized by a work scheme (home-office, hybrid, or face-to-face) and a personality profile, which together influence mobility decisions, psychological responses, and interaction with technological interventions..

The personality of each worker is characterized using the five major personality traits: Openness, Neuroticism, Extroversion, Agreeableness, and Conscientiousness [27]. This characterization is integrated within the BEN (Behavior with Emotions and Norms) architecture [3]. All personality traits are within the interval $[0, 1]$. Personality traits remain constant throughout the simulation, serving as intrinsic parameters that shape emotional dynamics and behavioral tendencies.

Figure 6.4 presents the class diagram of the ABM, focusing on entities and relationships relevant to the modeling contributions of this thesis. The diagram shows how behavioral constructs, including anxiety, personality traits, routines, and trust, are encapsulated within the Worker model and how they interact with the Technological NPI via the mobile application and backend recommendation service. The abstraction of spatial entities through the Place hierarchy clarifies how workers interact with different types of locations and enables role-specific behavior at the agent level. The structural organization of these entities and their relationships, as shown in the class diagram, connects the conceptual modeling discussed in previous sections with the computational mechanisms described later in this chapter. This diagram offers a clear representation of how psychological processes, technological interventions, and decision-making logic are integrated within the agent-based simulation.

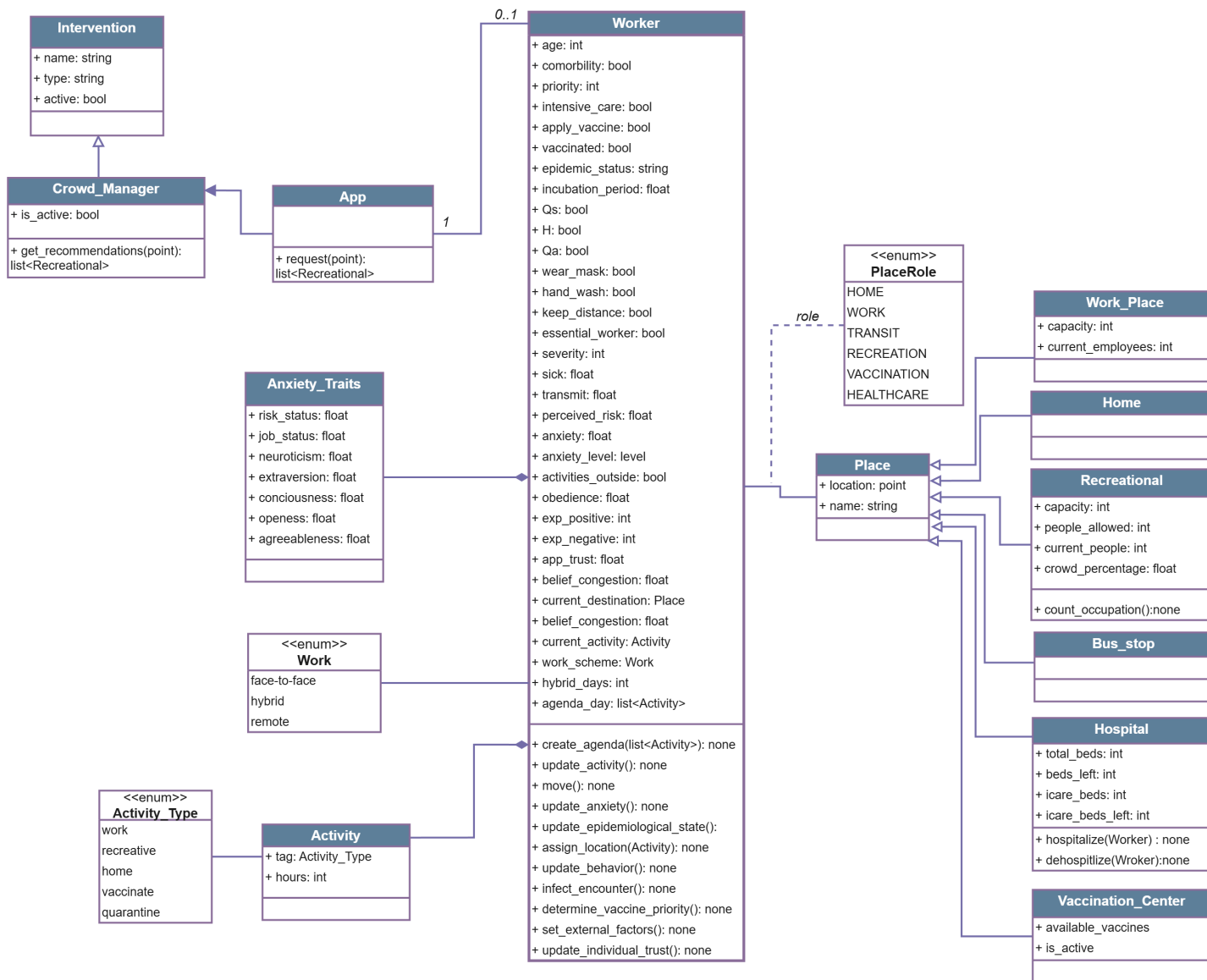


Figure 6.4.: Modeling contributions class diagram.

Anxiety Model

As illustrated in Figure 6.3, anxiety functions as a mediating internal state that connects epidemiological conditions, technological interventions, and recreational mobility decisions. Increased perceived risk elevates anxiety and promotes avoidance behavior, whereas successful interaction with the Technological NPI enhances perceived safety and reduces avoidance. These feedback loops justify the formalization of anxiety dynamics and the mechanisms underlying NPI adoption within the model.

Anxiety is modeled and quantified using the COVID-19 Anxiety Syndrome Scale (C-19ASS) proposed by [19]. The C-19ASS captures anxiety specifically associated with the COVID-19 pandemic and focuses on four behavioral dimensions. The original scale ranges from 0 (no anxiety) to 36 (severe anxiety). For integration into the ABM, this scale is normalized to the interval $[0, 1]$, allowing anxiety to be treated as a continuous internal state variable that evolves over time.

The authors of [19] analyzed the correlation between their scale, the Big Five personality traits [27] and contextual factors (work and perceived risk) and found that: neuroticism, perceived risk status, and employment status are positively correlated with the COVID-19 anxiety, while extroversion and conscientiousness are negatively correlated and represent protective factors against the COVID-19 anxiety. Based on this, to calculate the anxiety, each worker has a data structure to save their personality traits and contextual factors (*anxiety_traits*) (see Eq. 6.15):

Each worker is associated with a set of anxiety-related traits, defined as:

$$\begin{aligned} anxiety_traits = \{ & extroversion, conscientiousness, neuroticism, \\ & openness, agreeableness, \\ & risk_status, job_status \} \end{aligned} \quad (6.1)$$

where personality traits (*extroversion, conscientiousness, neuroticism, openness, agreeableness*) are represented as continuous values in $[0, 1]$, and contextual variables capture employment conditions and perceived epidemiological risk.

Job Status

The *job_status* variable represents the work scheme. Individuals engaged in face-to-face or hybrid work schemes are subject to increased exposure risk, consequently exhibiting elevated levels of anxiety compared to those working exclusively from home. For modeling purposes, *job_status* is operationalized as a binary variable:

$$job_status = \begin{cases} 1 & \text{if } work_scheme \in [face - to - face, hybrid], \\ 0 & \text{if } work_scheme \in [remote] \end{cases} \quad (6.2)$$

Perceived Risk Status

The *risk_status* variable characterizes an individual's perceived personal risk and is operationalized as a composite measure comprising four distinct components: perceived likelihood of infection ($P_{likelihood}$), perceived severity of the disease ($P_{severity}$), concern about contracting the disease (C_{sick}), and concern regarding the potential transmission of the disease to others ($C_{transmit}$). This formulation is adapted from [14], with the exclusion of general perceived gravity, which is deemed less pertinent in a geographically homogeneous context.

The risk status is computed as:

$$risk_status = \frac{P_{likelihood} + P_{severity} + C_{sick} + C_{transmit}}{4} \quad (6.3)$$

where

- **Perceived severity ($P_{severity}$)**

The severity perception component, $P_{severity}$, quantifies an individual's subjective assessment of COVID-19's potential impact. In the proposed model, $P_{severity}$ is defined as a binary variable:

$$P_{severity} = \begin{cases} 1 & \text{if the worker has a comorbidity (e.g., diabetes, hypertension),} \\ 0 & \text{otherwise.} \end{cases} \quad (6.4)$$

- **Concern about contracting the disease (C_{sick})**

The concern of contracting the disease, C_{sick} , is modeled as a continuous random variable in the interval $[0, 1]$:

$$C_{sick} \sim \mathcal{U}(0, 1) \quad (6.5)$$

where $\mathcal{U}(0, 1)$ denotes the uniform distribution.

- **Concern of disease transmission ($C_{transmit}$)**

The concern regarding the transmission of the disease to others, $C_{transmit}$, is also represented as a continuous variable within $[0, 1]$. This component captures the uncertainty associated with engagement in activities outside the home and is dynamically updated according to the degree of NPI adoption (see subsection 6.4.3):

$$C_{transmit} = g(app_trust) \quad (6.6)$$

where app_trust denotes the adoption level of the technological NPI (trust in the app recommendations) and $g(\cdot)$ is a monotonically decreasing function.

- **Perceived Likelihood of Infection ($P_{likelihood}$)**

The perceived likelihood of infection is linked to the Epidemiological Traffic Light System (ETLS) implemented in Mexico [6, 23]. Rather than relying on precise epidemiological statistics, workers assess risk using discrete, color-coded indicators that reflect public communication practices.

The ETLS is derived from two indicators: percentage of positive cases and hospital occupancy. The positive cases indicator is computed as:

$$P_{positive} = \frac{P_{I_s} + P_{I_a}}{Total_Population} \quad (6.7)$$

where P_{I_s} refers to people in the epidemiological status I_s (Infected symptomatic) and P_{I_a} refers to people in the epidemiological status I_a (Infected asymptomatic). The positive cases indicator (I_P) is calculated by:

$$I_P(P_{positive}) = \begin{cases} 0.3 & P_{positive} \leq 0.2 \\ 0.5 & 0.2 < P_{positive} < 0.3 \\ 0.7 & 0.3 < P_{positive} < 0.39 \\ 1 & P_{positive} > 0.4 \end{cases} \quad (6.8)$$

Hospital occupancy is computed based on the number of beds within a hospital that comprises common beds ($total_beds_{common}$) and intensive care unit (ICU) beds

($total_beds_{ICU}$). The hospital occupancy ($H_{occupancy}$) is calculated according to the ETLS in Mexico [23]:

$$H_{ICU} = \frac{beds_occupied_{ICU}}{total_beds_{ICU}} \quad (6.9)$$

$$H_{common} = \frac{beds_occupied_{common}}{total_beds_{common}} \quad (6.10)$$

$$H_{occupancy} = 0.6 \times H_{ICU} + 0.4 \times H_{common} \quad (6.11)$$

The hospital occupancy indicator (I_H) is calculated by:

$$I_H(H_{occupancy}) = \begin{cases} 0.3 & H_{occupancy} \leq 0.3 \\ 0.5 & 0.3 < H_{occupancy} \leq 0.5 \\ 0.7 & 0.5 < H_{occupancy} \leq 0.7 \\ 1 & H_{occupancy} > 0.7 \end{cases} \quad (6.12)$$

A general indicator to assign the ETLS indicator is computed by:

$$I = \max(I_P, I_H) \quad (6.13)$$

Finally, the ETLS indicator is assigned:

$$Traffic_Light = \begin{cases} \text{Green} & I = 0.3 \\ \text{Yellow} & I = 0.5 \\ \text{Orange} & I = 0.7 \\ \text{Red} & I = 1 \end{cases} \quad (6.14)$$

The *Traffic_Light* variable functions as an indicator of the epidemic state, with its value I mapped to the perceived likelihood of infection, $P_{likelihood}$. In this model, $P_{likelihood}$ is assigned default, discrete values to reflect the general population's reliance on categorical signals rather than precise metrics such as case counts or hospital bed occupancy rates. Similar findings in nutrition research demonstrate that individuals frequently associate color cues with nutritional value [26, 16]. Accordingly, in this context, perceived epidemiological risk is more strongly shaped by the traffic light color code than by exact numerical indicators. The model uses parameter values corresponding to the maximum hospital occupancy threshold, but these values can be adjusted to suit the objectives of specific studies.

Anxiety Computation

The computation of the anxiety is formulated as a weighted sum:

$$\mathbf{anxiety_traits} = [extroversion, conscientiousness, neuroticism, risk_status, job_status] \quad (6.15)$$

Correspondingly, the vector of weights is defined as:

$$\mathbf{weight_factors} = [W_{extroversion}, W_{conscientiousness}, W_{neuroticism}, W_{risk}, W_{job}] \quad (6.16)$$

The anxiety score for each agent is then given by:

$$anxiety = \sum_{i=0}^4 weight_factors_i \cdot anxiety_traits_i \quad (6.17)$$

The resulting anxiety value is a continuous variable constrained to $[0, 1]$. For interpretative and analytical purposes, anxiety is stratified into three categories: mild ($0 \leq anxiety < 0.3$), moderate ($0.3 \leq anxiety < 0.6$), and severe ($0.6 \leq anxiety \leq 1$).

Anxiety Dynamic Evolution

The anxiety metric dynamically evolves in response to simulation events, which are formalized as follows:

1. The perceived likelihood of infection, $P_{likelihood}$, depends on the ETLS indicator:

$$P_{likelihood} = \begin{cases} 0.3 & \text{if } Traffic_Light = Green \\ 0.5 & \text{if } Traffic_Light = Yellow \\ 0.7 & \text{if } Traffic_Light = Orange \\ 1.0 & \text{if } Traffic_Light = Red \end{cases}$$

2. If an agent is vaccinated, then:

$$P_{severity} = 0$$

$$C_{sick} = 0$$

3. Upon following app recommendations:

$$C_{transmit} = 1 - app_trust$$

NPI adoption Model

Workers experiencing anxiety may be unable to avoid attending work but often refrain from participating in other activities outside the home, as crowds during the COVID-19 pandemic were perceived as significant sources of contagion. The parameter $C_{transmit}$ (see subsection 6.4.3) quantifies concern about disease transmission and, within this model, is associated with uncertainty about engaging in activities in enclosed spaces because of crowd density. The NPI is a recommendation system, accessible via a mobile application, that identifies nearby, less crowded locations for recreational activities. This system is designed to reduce the risk of contracting the virus during such activities and the likelihood of transmitting it to family members or friends.

The adoption of new technology involves an adaptation process. To represent this process, a dynamic trust model based on personality traits was implemented [5]. This model enables users, specifically workers, to develop trust in the application through accumulated experience. The resulting trust value, denoted as app_trust , is a continuous variable within the interval $[0,1]$. The app_trust value is determined by integrating the agent's own experiences, the experiences of other agents, and the agent's willingness to experiment.

$$app_trust_A(t + \Delta t) = app_trust_A(t) + \gamma_A * (E_A(t) - app_trust_A(t)) * \Delta t \quad (6.18)$$

In this formulation, γ_A represents the openness personality trait, which is associated with an individual's willingness to experiment. Workers with higher openness are more likely to follow the application's recommendations. E_A integrates direct experience (E_A^d , defined as the ratio of positive experiences with the application to total experiences) and indirect experience (E_A^i , defined as the average app_trust of all agents).

$$E_A^d(t) = \frac{\#PositiveExperiences}{\#TotalExperiences} \quad (6.19)$$

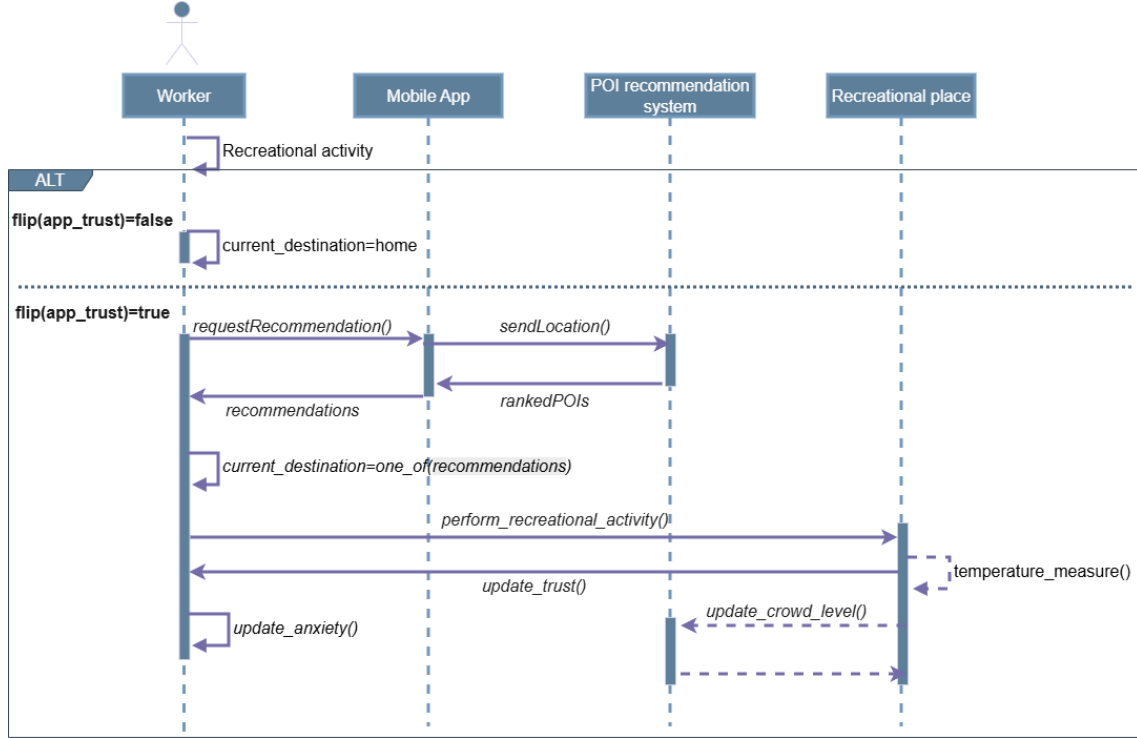


Figure 6.5.: **Sequence diagram of the interaction between the Worker model and the Technological NPI**, illustrating conditional recommendation requests, POI ranking, and feedback effects on trust and anxiety.

$$E_A^i(t) = \frac{\sum_{B=1, B \neq A}^N app_trust_B(t)}{N - 1} \quad (6.20)$$

$$E_A(t) = \alpha_A * E_A^i(t) + (1 - \alpha_A) * E_A^d(t) \quad (6.21)$$

In this context, α_A quantifies the extent to which an agent is influenced by the experiences of others, and this parameter is determined by the personality traits of conscientiousness and agreeableness. Positive experiences with the application increase trust. A positive experience is defined as a situation in which a worker follows a recommendation and finds that the suggested location is not crowded. The likelihood of following the application's recommendations is directly related to the level of *app_trust*. Higher trust in the application increases the probability that a worker will follow recommendations and reduces concerns about transmitting the virus ($C_{transmit}$), as personal experience indicates that recommended locations are safer.

Integration of the Worker model and the Technological NPI model

The integration of the Worker model with the Technological NPI model represents a central contribution of this thesis by providing an explicit representation of socio-technical feedback loops within the agent-based simulation. Instead of treating behavioral processes and technological services as separate components, the proposed model couples these elements during runtime. This approach enables worker behavior, psychological states, and algorithmic recommendations to co-evolve throughout the simulation.

Operationally, this interaction is event-driven and conditional. Figure 6.5 presents the sequence of interactions that occur when a worker plans a recreational activity. The decision to request recommendations from the Technological NPI is not deterministic; it depends on the worker’s current anxiety state, and the level of trust in the mobile application. This decision is modeled probabilistically using a weighted coin flip function ($flip(app_trust)$), which captures variability in human compliance and technology adoption.

If the probabilistic condition is met, the worker initiates a recommendation request through the mobile application. The application transmits the worker’s current location to the backend POI recommendation service, which processes real-time crowd data and returns a ranked list of nearby, low-occupancy recreational places. The worker then selects one of the recommended locations and performs the planned activity.

The outcome of this interaction influences the worker’s internal state. A recommendation that leads to a low-crowd environment constitutes a positive experience, increasing trust in the application and reducing concerns about disease transmission. These updates directly affect the worker’s anxiety dynamics and the likelihood of requesting recommendations in future decision cycles. In contrast, unfavorable outcomes reduce trust, reinforce avoidance behavior, and decrease subsequent engagement with the Technological NPI.

This bidirectional coupling establishes feedback mechanisms between individual behavior and technological intervention. At the individual level, recommendations shape mobility choices, emotional responses, and continued technology use. At the population level, aggregated adoption patterns modify the spatial distribution of recreational mobility, indirectly influencing crowd exposure and epidemiological dynamics.

Integrating behavioral states, trust dynamics, and algorithmic decision-making within a single simulation loop enables the evaluation of technological NPIs as adaptive socio-technical interventions. This integration supports the assessment of functional correctness, recommendation quality, behavioral adaptation, psychological well-being, and emergent system-level effects over time.

6.5. Evaluation and Computational Experiments

This section presents the experimental design used to evaluate the proposed agent-based model and the integrated NPI. The key objective is to assess the interactions between ICT-mediated decision-making services and human behavior, and their influence on socio-technical outcomes. The evaluation focuses on quantitative comparisons across scenarios, supporting analysis of how algorithmic services embedded within socio-technical systems affect population-level dynamics and individual behaviors. Key evaluation metrics include infection rates, mortality rates, anxiety levels, and changes in mobility patterns, providing a comprehensive overview of intervention effectiveness.

6.5.1. Scenario Setup and Model Assumptions

Scenario Description

The simulation scenario represents an urban area surrounding the Agua Azul Park in Guadalajara, Mexico, which served as a vaccination center during the COVID-19 pandemic. The modeled population consists of workers aged 20 to 60 residing within the area. Demographic data were obtained from the Mexican census [7], resulting in a simulated population of 19,071 workers.

Workplaces and recreational venues were selected using the National Statistical Directory of Economic Units (DENUE) [10]. Workplaces correspond to economic categories associated with office-based and service activities, while recreational venues are limited

to cultural and sports entertainment services. A single hospital is modeled to reflect the healthcare infrastructure available in the selected area. Public transportation is represented through a network of bus stops, enabling workers to commute within and outside the simulated zone.

This geographically grounded scenario establishes a realistic and controlled environment for evaluating the interactions between behavioral responses and technological interventions over time.

Model Assumptions

To ensure reproducibility and interpretability of the results, the simulation is conducted under the following assumptions:

- The population is homogeneous and is made up of workers. The total population is 19,071 workers [7].
- A total of 297 workplaces, 57 recreational venues, 258 bus stops, one hospital, and one vaccination center are modeled.
- DENEUE [10] provides an estimated number of workers for every workplace, according to this a population of 9,480 workers has a workplace in the area, the rest of the population works outside. Workers who have a workplace outside the area are transported by bus, in the model, this is represented by sending the worker to a bus stop, and when they have finished their workday they return to carry out a recreational activity in the area or he goes home.
- There is one hospital in the zone and contains common beds ($total_beds_{common} = 40$) and intensive care unit (ICU) beds ($total_beds_{ICU} = 10$).
- The personality traits are initialized following a uniform distribution within the interval $[0,1]$.
- Weights for anxiety calculation are: $W_{risk} = 0.45$, $W_{job} = 0.05$, $W_{neuroticism} = 0.45$, $W_{extraversion} = -0.1$, $W_{consciousness} = -0.05$. These values can be changed to match future studies.
- Anxiety is classified in three categories: mild anxiety (anxiety value $[0,0.29]$), moderate anxiety (anxiety value $[0.3,0.59]$), severe anxiety (anxiety value $[0.6,1]$).
- Diabetes is the comorbidity modeled and workers could have it with a probability of 10% [8].
- Trust in the technological NPI is initialized to $app_{trust} = 0.5$, assuming the application's institutional credibility.
- The vaccine provides immediate immunity and only one dose is required.
- A total of 51 vaccines is applied each day. This value has been obtained from the total of vaccinated people in Mexico ($P_{vaccinated} = 81\,300\,000$ [22]), the Mexican population ($P_{total} = 126\,014\,024$ [9]), the days from the start of the vaccination process ($12/24/2020 - 08/22/2021 = 241 = days$), and the simulation population ($P_{simulation} = 19\,071$):

$$estimated_vaccines_by_day = \frac{P_{vaccinated}}{days} = 337334.4$$

$$vaccine_ratio = \frac{estimated_vaccines_by_day}{P_{total}} = 0.0027$$

$$vaccines_by_day = vaccine_ratio \times P_{simulation} = 51$$

- The vaccination process, in the model, only considers the population that lives within the studied area (19,071 workers). *vaccines_by_day* refers to the number of vaccines applied daily to this population throughout the simulation.
- The epidemiological status of a worker changes from *S* (Susceptible) to *E* (Exposed) when the social distance is trespassed. The social distance is 1.5 meters.
- The simulation lasts for 60 days, where each simulation step is equivalent to 1 hour.

6.5.2. Experimental results

Three experimental scenarios are defined to isolate and compare the effects of interventions:

- (i) **Baseline scenario (no interventions):** epidemic evolution without vaccination or technological NPI.
- (ii) **Pharmaceutical intervention scenario:** vaccination deployed according to priority guidelines.
- (iii) **Combined intervention scenario:** vaccination and technological NPI deployed jointly.

To ensure comparability, all simulations are executed under the identical structural assumptions outlined above (See section 6.5.1). The principal epidemiological, behavioral, and intervention-related parameters employed in each simulation are detailed in Table 6.2. The resulting quantitative outcomes are synthesized in Table 6.3, which serves as the primary reference for the evaluation.

Table 6.3 reports epidemiological, behavioral, and socio-technical metrics extracted from the simulation runs. These metrics include peak reproduction numbers, infection and mortality rates, anxiety distributions, recreational mobility indicators, and timing of epidemiological risk escalation.

Scenario (i): Baseline scenario (no interventions)

In the absence of pharmaceutical and technological interventions, the epidemic unfolds without mitigation beyond imposed work schemes. This scenario exhibits the highest peak basic reproduction number ($R_0^{max} = 36$), reflecting unrestricted contact patterns and the absence of adaptive behavioral responses. Nearly the entire population becomes infected, with only 0.005% remaining uninfected by the end of the simulation (See Fig. 6.6).

Behaviorally, this scenario is characterized by sustained high anxiety levels, particularly among face-to-face workers, and minimal engagement in recreational activities outside the home. These results establish a baseline against which subsequent interventions are evaluated.

Scenario (ii): Pharmaceutical intervention scenario

Introducing vaccination substantially alters epidemic dynamics. The peak reproduction number is reduced to $R_0^{max} = 2.35$, and mortality decreases by 0.66% relative to Scenario (i) (See Fig. 6.6). Anxiety levels decline across all work schemes, particularly among face-to-face workers prioritized for vaccination (See Fig. 6.7).

Parameter	Model	Sce. (i)	Sce. (ii)	Sce. (iii)
<i>Vaccinations per day</i>	Vaccination	-	51	51
<i>Vaccine efficacy</i>	Vaccination	-	91%	91%
<i>Face-to-Face workers</i>	Population	45%	45%	45%
<i>Hybrid workers</i>	Population	45%	45%	45%
<i>Home office</i>	Population	10%	10%	10%
<i>Worker initially exposed</i>	Epidemiological	10	10	10
β	Epidemiological	0.01	0.01	0.01
ρ	Epidemiological	0.86834	0.86834	0.86834
δ_s	Epidemiological	0.5	0.5	0.5
δ_a	Epidemiological	0.3	0.3	0.3
γ	Epidemiological	0.1	0.1	0.1
λ	Epidemiological	0.9	0.9	0.9

Table 6.2.: Model parameters used across simulation scenarios. Parameters are grouped conceptually into epidemiological dynamics, vaccination process, and population structure models.

However, behavioral patterns remain largely unchanged. Although recreational activity increases modestly among individuals with mild and moderate anxiety, no mechanism actively redistributes mobility or crowd exposure. Consequently, hybrid workers remain disproportionately affected, and population-level adaptation is limited to biological protection rather than socio-technical mediation.

Scenario (iii): Combined intervention scenario (PI and technological NPI)

The combined deployment of vaccination and the technological NPI yields the most favorable overall outcomes. The peak reproduction number is further reduced to $R_0^{max} = 2.0$, and the proportion of the population that remains uninfected increases to 4.03%, the highest across all scenarios. Mortality is reduced by 1.80% relative to Scenario (i) (See Fig. 6.6).

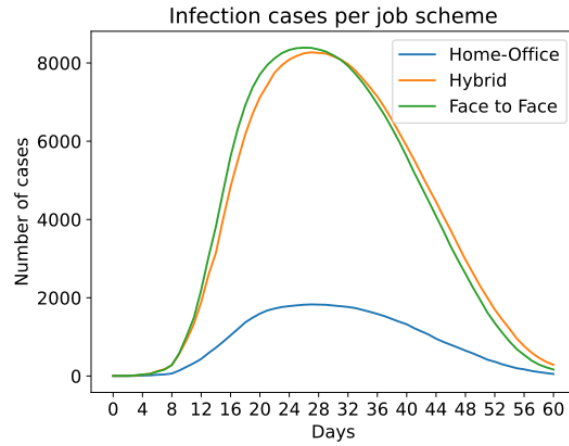
In addition to epidemiological improvements, this scenario displays distinct behavioral effects. Recreational activities increase by 13.2% among individuals with mild anxiety and decrease by 87.74% among those with severe anxiety, indicating selective reintegration rather than indiscriminate mobility (See Fig. 6.8). Furthermore, the onset of the highest epidemiological risk phase is delayed by 3 days, demonstrating the capacity of the technological NPI to slow epidemic escalation by redistributing mobility rather than through direct enforcement.

Although final anxiety levels remain elevated due to delayed ETLS recovery, mean anxiety values across work schemes are lower than in other scenarios (See Fig. 6.7). This outcome highlights the decoupling between infection risk and perceived safety enabled by the technological intervention.

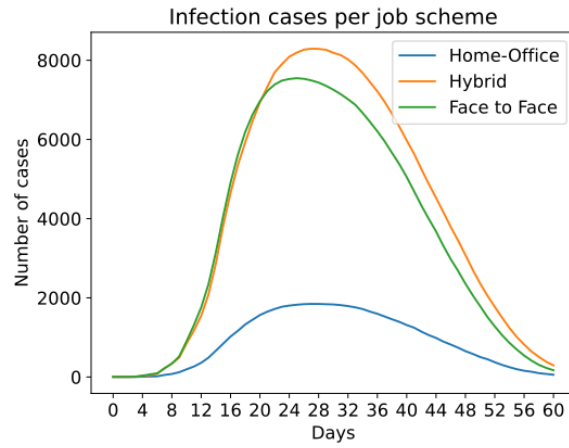
Metric	No Intervention (i)	PI Only (ii)	PI + Technological NPI (iii)
Peak basic reproduction number R_0^{max}	36.0	2.35	2.0
Total infected population (%)	99.995	99.854	95.973
Deaths (% of population)	Highest (baseline)	−0.66% vs (i)	−1.80% vs (i)
Population never infected (%)	0.005	0.146	4.027
Peak anxiety among face-to-face workers	Highest (0.491)	Reduced (0.487)	Lowest (0.480)
Severe anxiety population (% , final state)	High (24%)	Moderate (12%)	Moderate–High (22%)*
Recreational activities — mild anxiety group (%)	Baseline	+10.53%	+13.2%
Recreational activities — moderate anxiety group (%)	Baseline	+0.59%	+1.81%
Recreational activities — severe anxiety group (%)	Baseline	−7.47%	−87.74%
ETLS red phase onset (day)	Day 13	Day 13	Day 16
Mobility redistribution effect	None	None	Present
Behavioral–technological feedback strength	None	None	Strong

Table 6.3.: Comparative evaluation of simulation scenarios across epidemiological, behavioral, and socio-technical dimensions.

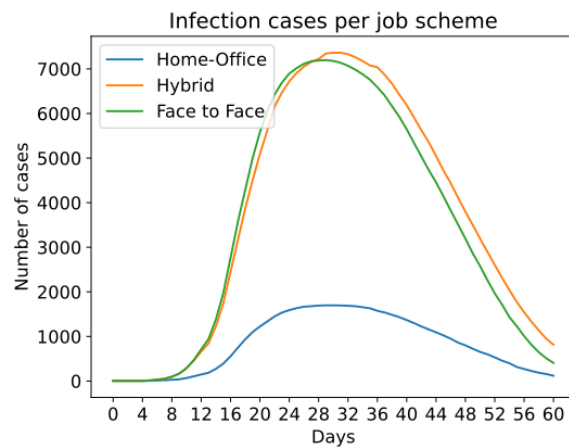
*High final anxiety in Scenario (iii) is explained by delayed ETLS improvement rather than higher infection rates.



(a) Scenario (i)



(b) Scenario (ii)



(c) Scenario (iii)

Figure 6.6.: **Infection dynamics by work scheme across simulation scenarios.** The figure depicts the temporal evolution of infection cases among home-office, hybrid, and face-to-face workers.

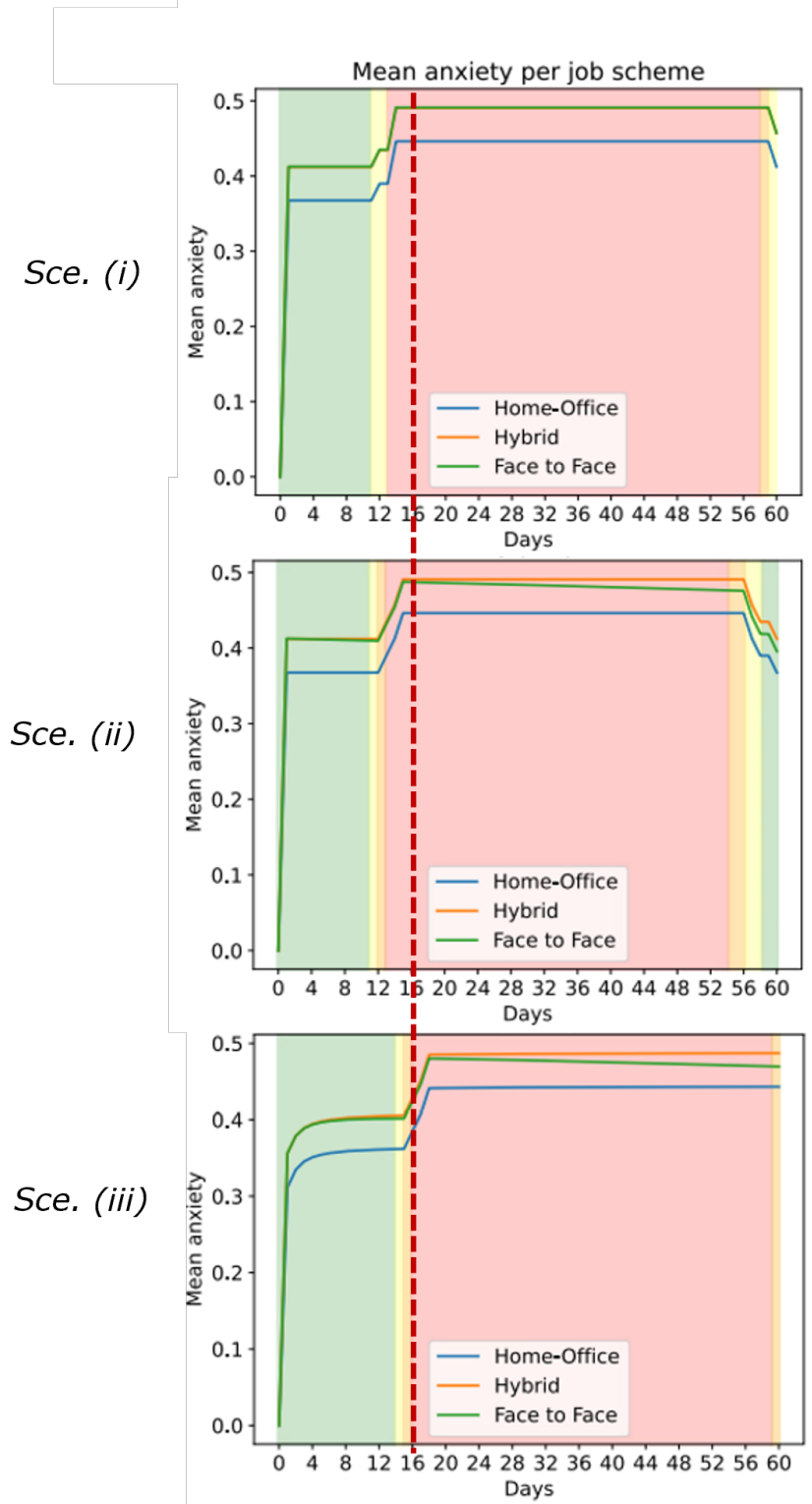
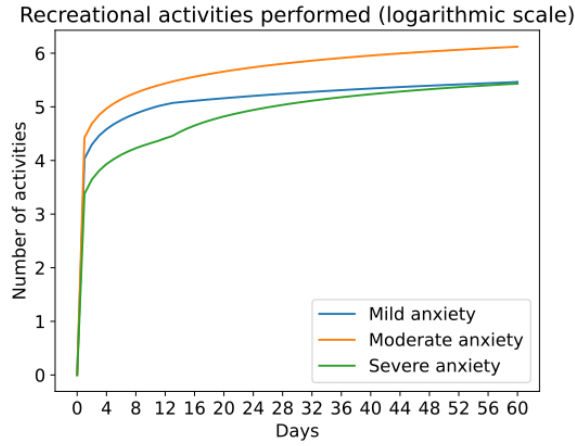
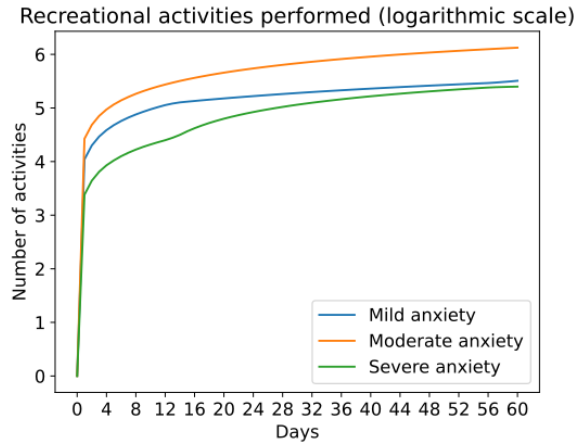


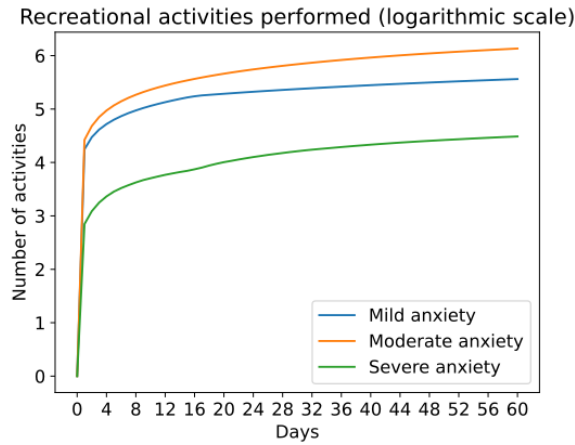
Figure 6.7.: **Mean anxiety levels by work scheme across simulation scenarios.** The figure illustrates the temporal progression of mean anxiety among workers in home-office, hybrid, and face-to-face work schemes across three simulation scenarios: (i) no intervention, (ii) pharmaceutical intervention (vaccination), and (iii) combined pharmaceutical and technological NPI. Shaded regions indicate the ETLs, which reflect changes in perceived risk over time.



(a) Scenario (i)



(b) Scenario (ii)



(c) Scenario (iii)

Figure 6.8.: **Recreational activities performed by anxiety level across simulation scenarios (logarithmic scale).** The figure shows the cumulative number of recreational activities performed by workers classified with mild, moderate, and severe anxiety.

6.5.3. Robustness and Sensitivity Considerations

The evaluation relies on fixed scenario configurations. Nevertheless, the agent-based model incorporates stochastic elements at the individual level, including personality initialization, activity selection, and NPI adoption decisions. As a result, the reported outcomes reflect aggregate tendencies that emerge from heterogeneous, probabilistic agent behavior rather than deterministic individual trajectories.

Robustness is assessed by emphasizing relative differences across scenarios rather than absolute outcome values. Key findings (such as reductions in the peak basic reproduction number, delays in epidemiological risk escalation, and differentiated reintegration of recreational activities) are consistent in direction across simulation executions observed during model development and calibration. These patterns arise at the population level despite stochastic variation at the agent level.

The analysis adopts a comparative robustness perspective. Effects are considered meaningful when they produce consistent directional changes across scenarios and manifest simultaneously across multiple independent dimensions, including epidemiological, behavioral, and socio-technical metrics. For example, the technological NPI contributes to lowering R_0 , delaying ETLs escalation, increasing recreational mobility among individuals with mild anxiety, and reducing transitions into severe anxiety states. The convergence of these effects across distinct indicators strengthens confidence in the observed trends.

Sensitivity is addressed by structurally isolating intervention mechanisms while holding all other parameters constant. By comparing scenarios that differ exclusively in the activation of vaccination and technological recommendation services, observed differences can be attributed directly to these interventions rather than to parameter variation. Additionally, parameters governing anxiety weighting, trust dynamics, and vaccination rates are explicitly documented, enabling future studies to explore alternative calibrations or extended sensitivity analyses ¹.

6.6. Deriving Human-Centric Metrics for O-DMIoT

Human-centric evaluation metrics are designed to assess decision-making algorithms within the O-DMIoT framework. Agent-based simulations produce multidimensional outputs that represent epidemiological progression, psychological states, and behavioral patterns. For effective algorithm evaluation and potential reconfiguration, these outputs are transformed into structured metrics that quantify the achievement of human-centered objectives. In O-DMIoT, this transformation is accomplished through Quality of Experience (QoE) metrics.

In the epidemiological application domain addressed in this chapter, the primary human-centered objectives are: (i) minimizing infection risk, (ii) reducing anxiety, and (iii) facilitating safe reintegration into recreational activities. These objectives are operationalized in the simulation through aggregate population-level indicators. Consequently, three QoE components are established:

¹A formal sensitivity analysis of key epidemiological and behavioral parameters was conducted and reported in the original publication associated with this work. That analysis examined the effects of parameter variation on infection dynamics, anxiety levels, and intervention outcomes. In this thesis, the sensitivity study is not reproduced in full, as the primary focus lies on the conceptual modeling contributions, the integration of technological NPIs within an agent-based evaluation framework, and the comparative assessment of intervention mechanisms rather than on parameter calibration. Nevertheless, all simulation parameters are explicitly documented to ensure transparency and reproducibility, and the reported results should be interpreted in conjunction with the sensitivity findings presented in the published study.

- **Infection impact (I):** defined as the percentage of the population infected during the simulation, capturing the health-related consequences of algorithm-mediated mobility decisions:

$$I = \text{percentage of population infected} \quad (6.22)$$

- **Anxiety change (A):** computed as the relative variation between final and initial average anxiety levels, representing the psychological effect of technological interventions:

$$A = \frac{\text{avg_anxiety}_{end} - \text{avg_anxiety}_{start}}{\text{avg_anxiety}_{start}} \quad (6.23)$$

- **Activity participation (P):** defined as the ratio between performed recreational activities and the estimated maximum, reflecting behavioral and social reintegration:

$$P = \frac{\text{total performed activities}}{\text{total estimated activities}} \quad (6.24)$$

The overall QoE is computed as a weighted aggregation of these components:

$$QoE = -w_1 \cdot I - w_2 \cdot A + w_3 \cdot P \quad (6.25)$$

The negative signs for I and A indicate their interpretation as costs from a human-centered perspective, while P is considered a benefit associated with safe social participation. The weights w_1 , w_2 , and w_3 represent the relative importance of health protection, psychological well-being, and behavioral reintegration, respectively. This formulation explicitly captures the trade-offs inherent in socio-technical interventions, where risk reduction must be balanced with the maintenance of social activity and mental health.

QoE is not designed to serve as an absolute or normalized performance score. Rather, it functions as a relative utility signal that facilitates comparative evaluation of decision-making services under consistent contextual conditions. Therefore, meaningful conclusions are drawn from relative differences in QoE values, not from their absolute magnitude. This approach aligns with the role of QoE in O-DMIoT, where metrics are intended to support comparison, ranking, and informed reconfiguration rather than standalone interpretation.

In the proposed framework, QoE functions as a decision-support metric that links simulation-based evaluation to algorithmic design choices. Distinct decision-making algorithms, or alternative parameterizations of a single algorithm, produce different QoE outcomes when applied within the same socio-technical environment. Consequently, QoE enables: (i) comparative evaluation of decision-making services, (ii) selection of algorithms that best fulfill human-centered objectives, and (iii) informed reconfiguration in response to evolving system requirements or contextual conditions.

6.6.1. QoE Application Across Simulation Scenarios

This subsection presents the operational application of the human-centric metrics defined in Eq. 6.25 to enable comparative evaluation within O-DMIoT. The section illustrates how simulation outcomes from various intervention configurations can be interpreted through the QoE construct to enhance the assessment of algorithms.

The QoE formulation is applied to the results of the three simulation scenarios described in Section 6.5. In each scenario, simulation outputs related to epidemiological evolution, psychological dynamics, and recreational mobility are aggregated into a unified evaluative signal (See Tab. 6.4). This approach demonstrates that domain-specific indicators can be synthesized into a comparable metric without modifying the underlying simulation models.

Scenario	I	A	P	QoE
(i) No intervention	0.9693	0.108	1.099	-0.0904
(ii) PI	0.9249	-0.015	1.109	-0.0325
(iii) PI + NPI	0.8515	0.366	1.810	0.0926

Table 6.4.: **Application of the proposed QoE metric across the three simulation scenarios.** I represents infection impact, A captures anxiety variation, and P denotes recreational activity participation. QoE is computed using the formulation $QoE = -w_1I - w_2A + w_3P$, with weights $w_1 = 0.4$, $w_2 = 0.3$, and $w_3 = 0.3$, reflecting the relative importance of health protection, psychological well-being, and social reintegration. Higher QoE values indicate better fulfillment of human-centered objectives. The estimation of total recreational activity participation assumes an average of 1.5 recreational activities performed per individual per day throughout the simulation.

Illustrative weights are selected to reflect plausible prioritization in pandemic-response contexts, assigning primary importance to infection mitigation, followed by psychological well-being and support for safe social participation. These weights are not intended as optimal or universal. Instead, they exemplify how stakeholder preferences or policy priorities can be encoded within the O-DMIoT evaluation process, allowing interpretation of simulation results under different value systems.

The application of the QoE metric produces a consistent ranking across scenarios, as shown in Tab. 6.4. The scenario without interventions results in the least favorable outcome, characterized by high infection prevalence, increased anxiety, and restricted recreational participation. The introduction of vaccination improves overall performance by reducing epidemiological and psychological burdens, though it leads to limited behavioral adaptation. The combined use of pharmaceutical and technological NPIs produces the most favorable outcome by reducing infection risk and enabling selective reintegration of recreational activities.

This application demonstrates how QoE aggregates heterogeneous simulation outputs into a single evaluative signal suitable for comparison in O-DMIoT. QoE facilitates evaluation of algorithmic alternatives or configurations based on their emergent human-centered effects, thereby complementing traditional technical metrics. QoE could inform selection or reconfiguration decisions in adaptive IoT systems by grounding them in simulation-based evidence of human impact. By enabling consistent ranking of alternative intervention strategies under identical contextual conditions, the QoE formulation provides an operational foundation for linking human experience to algorithm assessment. These results illustrate that QoE can distinguish intervention strategies based on their emergent human-centered effects under identical contextual conditions.

While this section establishes how QoE can be derived, computed, and applied to compare algorithmic interventions, it does not prescribe how these results should influence ontology-driven design or reconfiguration decisions. The following section examines the implications of QoE-based evaluation for O-DMIoT, discussing how such metrics inform algorithm selection, semantic reasoning, and adaptive decision-making mechanisms.

6.6.2. Mapping QoE to O-DMIoT Layers

Problem Layer. At the Problem layer, O-DMIoT characterizes decision-making problems by objectives, uncertainty, and human involvement. In this case study, the competing objectives of minimizing infection risk, reducing anxiety, and enabling safe participation are represented by the QoE components I , A , and P . Simulation results, as summarized in Table 6.3, demonstrate that varying intervention configurations produce distinct epidemiological and behavioral outcomes. Aggregating these outcomes into QoE converts abstract human-centered objectives into concrete, measurable indicators grounded in simulation dynamics.

Design Layer. At the Design layer, O-DMIoT specifies decision-making algorithms by their knowledge representation, algorithmic approach, and interaction mechanisms. The POI recommendation algorithm examined in this chapter is classified as a criterion-driven heuristic at the application layer with direct human interaction. Comparative results reveal that enabling or disabling this algorithm across intervention configurations results in different QoE values. Notably, the technological NPI scenario demonstrates stronger behavioral feedback and delayed escalation of epidemiological risk, and increased participation among individuals with low to moderate anxiety, all contributing to higher QoE (see Table 6.3). These findings indicate that algorithmic design choices can be evaluated not only for functional correctness but also for their broader human-centered impact.

Metrics Layer. The Metrics layer functions as the formal interface for comparing, selecting, and potentially reconfiguring decision-making services. The QoE formulation operationalizes this layer by integrating epidemiological, psychological, and behavioral indicators into a unified evaluative signal (see Eq. 6.25). This approach enables consistent comparison of alternative algorithms or configurations under identical contextual conditions.

6.7. Implications for O-DMIoT

The findings of this chapter have direct implications for the O-DMIoT framework in evaluating, comparing, and potentially reconfiguring decision-making algorithms in human-centered IoT systems. Instead of introducing a new class of algorithms, this work demonstrates that O-DMIoT enables principled reasoning about the appropriateness of specific decision-making services under defined socio-technical conditions.

6.7.1. Implications for Problem Characterization

At the Problem layer, this case study underscores the necessity of explicitly representing human-centered objectives and sources of uncertainty when characterizing decision problems in IoT. The epidemiological application domain analyzed here involves inherently competing objectives: minimizing infection risk, preserving psychological well-being, and enabling social participation. These objectives cannot be reduced to a single technical performance criterion.

Operationalizing these objectives through simulation-derived QoE components demonstrates how problem characterizations in O-DMIoT can be anchored in measurable human experience. The results indicate that problem definitions excluding behavioral dimensions may favor technically efficient solutions that underperform in real-world contexts.

6.7.2. Implications for Algorithm Design and Selection

At the Design layer, the analysis demonstrates that algorithmic simplicity and transparency offer advantages in socio-technical contexts. The POI recommendation algorithm evaluated in this chapter is a criterion-driven heuristic that employs mathematical knowledge representation and facilitates direct human interaction. Despite its simplicity, the algorithm yields measurable improvements in human-centered outcomes when integrated into a broader socio-technical system.

The comparative evaluation across scenarios reveals that the presence or absence of a decision-making service, rather than its internal sophistication alone, can influence system-level behavior. This finding shows that algorithm selection should be based on contextual fitness rather than solely on technical optimality.

From the O-DMIoT perspective, these findings validate the value of classifying algorithms according to objective, interaction type, uncertainty, and knowledge representation, as these characteristics directly shape human experience and system outcomes.

6.7.3. Implications for Metrics and Evaluation

At the Metrics layer, the QoE formulation demonstrates how heterogeneous simulation outputs can be consolidated into a unified evaluative signal suitable for algorithm comparison. Notably, QoE functions as a relative utility measure that facilitates ranking and trade-off analysis under consistent contextual conditions, rather than serving as an absolute indicator of success.

This approach aligns with the role of metrics in O-DMIoT as tools for comparison and decision support, rather than as definitive indicators. The capacity to encode key stakeholder objectives through weighting further indicates that O-DMIoT can embrace multiple value systems, policy objectives, or domain-specific constraints without modifying the underlying algorithmic models.

The application of QoE further demonstrates that agent-based modeling can function as an external evaluation mechanism within O-DMIoT, enabling the assessment of decision-making services prior to deployment or reconfiguration.

6.7.4. Implications for Adaptation and Reconfiguration Mechanisms

Although runtime reconfiguration is not implemented in this case study, the results offer clear insights into how such mechanisms could be supported within O-DMIoT. Variations in QoE across scenarios indicate that alternative configurations produce different human-centered outcomes under identical environmental conditions. This finding suggests that, within an adaptive IoT system, QoE could serve as a feedback signal to guide reconfiguration decisions, such as enabling, disabling, or retuning decision-making services, when contextual conditions or system objectives change. Integrating QoE with the semantic descriptions of algorithms proposed in earlier chapters establishes a foundation for adaptive behavior without embedding domain-specific logic.

6.8. Conclusions

This chapter demonstrated that agent-based modeling can be systematically employed as an evaluation strategy for decision-making algorithms in human-centered IoT systems. By embedding a Technological NPI into an agent-based simulation, the analysis revealed how

ICT-mediated decision services interact with human behavior, psychological states, and epidemiological dynamics in a socio-technical context.

The proposed evaluation integrates three interdependent components: (i) a Technological NPI that provides algorithmic recommendations, (ii) a human behavior model that captures anxiety, trust, and mobility decisions, and (iii) an epidemiological layer representing population-level health dynamics. This integration enables explicit representation of feedback loops between algorithmic decisions and human responses, demonstrating that even simple, transparent decision-making algorithms can generate complex emergent effects when deployed at scale.

Comparative simulation scenarios demonstrated that technological interventions cannot be adequately assessed using only technical or performance-based criteria. Their effectiveness depends on how algorithmic decisions affect human experience, influence behavior, and interact with broader system dynamics. The introduction of a human-centric QoE metric established a structured mechanism for translating multidimensional simulation outcomes into evaluative signals aligned with human-centered objectives.

By linking problem characterization, algorithm design choices, and evaluation metrics, this chapter operationalized the O-DMIoT framework within a specific application domain. The results indicate that O-DMIoT facilitates a transition from isolated algorithm evaluation to evidence-based, human-centered assessment grounded in simulation. In this context, agent-based modeling serves as a bridge between algorithm design and real-world socio-technical impact.

Although the case study addressed an epidemiological scenario, the modeling and evaluation approach presented here is not limited to a single domain. Rather, it demonstrates that O-DMIoT can serve as a general framework for designing, evaluating, and comparing decision-making services in IoT systems operating in dynamic, human-centered environments. This chapter establishes the empirical and methodological foundation for the subsequent discussion of broader implications for O-DMIoT design and adaptation.

References

- [1] J. Almagor and S. Picascia. “Exploring the effectiveness of a COVID-19 contact tracing app using an agent-based model”. In: *Scientific Reports* 10.1 (Dec. 2020). URL: <https://doi.org/10.1038/s41598-020-79000-y>.
- [2] J. J. V. Bavel et al. “Using social and behavioural science to support COVID-19 pandemic response”. en. In: *Nature Human Behaviour* 4.5 (May 2020), pp. 460–471. URL: <http://www.nature.com/articles/s41562-020-0884-z> (visited on 02/26/2021).
- [3] M. Bourgais, P. Taillandier, and L. Vercouter. “BEN: An Architecture for the Behavior of Social Agents”. In: *Journal of Artificial Societies and Social Simulation* 23.4 (2020). URL: <https://doi.org/10.18564/jasss.4437>.
- [4] V. Chamola et al. “A Comprehensive Review of the COVID-19 Pandemic and the Role of IoT, Drones, AI, Blockchain, and 5G in Managing its Impact”. In: *IEEE Access* 8 (2020), pp. 90225–90265. URL: <https://ieeexplore.ieee.org/document/9086010/> (visited on 02/26/2021).
- [5] L. Durán-Polanco and M. Siller. “Crowd management COVID-19”. In: *Annual Reviews in Control* (Apr. 2021). URL: <https://doi.org/10.1016/j.arcontrol.2021.04.006>.

- [6] *Embajada de México en Estados Unidos*. URL: <https://embamex.sre.gob.mx/eua/index.php/en/2016-04-09-20-40-51/tourism/1760-mexico-s-covid-19-monitoring-system>.
- [7] I. N. de Estadística y Geografía (INEGI). URL: <https://www.inegi.org.mx/programas/ccpv/2020/default.html>.
- [8] I. N. de Estadística y Geografía (INEGI). URL: <https://www.inegi.org.mx/programas/ensanut/2018/>.
- [9] I. N. de Estadística y Geografía (INEGI). URL: <https://www.inegi.org.mx/programas/ccpv/2020/default.html>.
- [10] I. N. de Estadística y Geografía (INEGI). *Directorio Estadístico Nacional de Unidades Económicas. DENUÉ*. URL: <https://www.inegi.org.mx/app/mapa/denue/default.aspx>.
- [11] European Centre for Disease Prevention and Control. *Guidelines for non-pharmaceutical interventions to reduce the impact of COVID-19 in the EU/EEA and the UK*. Sept. 2020. URL: <https://www.ecdc.europa.eu/en/publications-data/covid-19-guidelines-non-pharmaceutical-interventions>.
- [12] E. Hernández-Orallo et al. “Evaluating the Effectiveness of COVID-19 Bluetooth-Based Smartphone Contact Tracing Applications”. en. In: *Applied Sciences* 10.20 (Oct. 2020), p. 7113. URL: <https://www.mdpi.com/2076-3417/10/20/7113> (visited on 05/20/2021).
- [13] I. Holmdahl and C. Buckee. “Wrong but Useful — What Covid-19 Epidemiologic Models Can and Cannot Tell Us”. In: *New England Journal of Medicine* 383.4 (July 2020). Publisher: Massachusetts Medical Society, pp. 303–305. URL: <https://www.nejm.org/doi/10.1056/NEJMp2016822> (visited on 07/12/2022).
- [14] L. C. Karlsson et al. “Fearing the disease or the vaccine: The case of COVID-19”. In: *Personality and Individual Differences* 172 (2021), p. 110590. URL: <https://www.sciencedirect.com/science/article/pii/S0191886920307819>.
- [15] C. C. Kerr et al. “Covasim: An agent-based model of COVID-19 dynamics and interventions”. en. In: *PLoS Comput. Biol.* 17.7 (July 2021), e1009149.
- [16] J. Koenigstorfer, A. Groeppel-Klein, and F. Kamm. “Healthful Food Decision Making in Response to Traffic Light Color-Coded Nutrition Labeling”. In: *Journal of Public Policy & Marketing* 33.1 (2014), pp. 65–77. URL: <http://www.jstor.org/stable/43305367>.
- [17] E. Mbunge. “Integrating emerging technologies into COVID-19 contact tracing: Opportunities, challenges and pitfalls”. In: *Diabetes & Metabolic Syndrome* 14.6 (2020), pp. 1631–1636. URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7833487/> (visited on 07/13/2022).
- [18] L. McMullan et al. *The psychological toll of coronavirus in Britain – a visual guide*. Publication Title: The Guardian. Guardian News and Media, July 2020. URL: <https://www.theguardian.com/world/ng-interactive/2020/jul/22/psychological-toll-coronavirus-britain-visual-guide-anxiety-mental-strain>.

- [19] A. V. Nikčević and M. M. Spada. “The COVID-19 anxiety syndrome scale: Development and psychometric properties”. In: *Psychiatry Research* 292 (Oct. 2020), p. 113322. URL: <https://doi.org/10.1016/j.psychres.2020.113322>.
- [20] A. V. Nikčević et al. “Modelling the contribution of the Big Five personality traits, health anxiety, and COVID-19 psychological distress to generalised anxiety and depressive symptoms during the COVID-19 pandemic”. en. In: *Journal of Affective Disorders* 279 (Jan. 2021), pp. 578–584. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0165032720328974> (visited on 02/26/2021).
- [21] G. A. Palomo-Briones, M. Siller, and A. Grignard. “An agent-based model of the dual causality between individual and collective behaviors in an epidemic”. en. In: *Computers in Biology and Medicine* 141 (Feb. 2022), p. 104995. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0010482521007897> (visited on 04/23/2024).
- [22] H. Ritchie et al. *Coronavirus (COVID-19) Vaccinations - Statistics and Research*. Mar. 2020. URL: https://ourworldindata.org/covid-vaccinations?country=OWID_WRL.
- [23] S. de Salud. *Lineamientos para la estimación de riesgos de semáforo por regiones COVID-19*. 2021. URL: https://coronavirus.gob.mx/wp-content/uploads/2020/10/SemaforoCOVID_Metodo.pdf.
- [24] J. Santabárbara et al. “Prevalence of anxiety in the COVID-19 pandemic: An updated meta-analysis of community-based studies”. In: *Progress in Neuro-Psychopharmacology and Biological Psychiatry* 109 (2021), p. 110207. URL: <https://www.sciencedirect.com/science/article/pii/S0278584620305236>.
- [25] P. Taillandier et al. “Building, composing and experimenting complex spatial models with the GAMA platform”. In: *GeoInformatica* 23.2 (Dec. 2018), pp. 299–322. URL: <https://doi.org/10.1007/s10707-018-00339-6>.
- [26] R. Trudel et al. “The impact of traffic light color-coding on food health perceptions and choice.” In: *Journal of Experimental Psychology: Applied* 21.3 (2015), pp. 255–275. URL: <https://doi.org/10.1037/xap0000049>.
- [27] J. S. Wiggins. *The Five-factor Model of Personality*. Mar. 1996.

7. Agent-based analysis of trust and adoption dynamics accross populations

Abstract. Vaccination campaigns can be understood as information-based decision-making services within socio-technical systems, requiring evaluation methods that extend beyond traditional epidemiological indicators. This chapter employs an agent-based model that incorporates epidemic dynamics, information dissemination, trust development, and deliberative individual decision-making to investigate how identical vaccination interventions yield different outcomes among populations with distinct cultural orientations. The analysis focused on operationalizing Quality of Experience (QoE) as an evaluative construct that reflects how populations perceive and respond to information-mediated interventions. QoE is represented through composite metrics that aggregate vaccine acceptance, trust dynamics, and infection reduction, facilitating systematic comparisons across intervention scenarios. Experimental findings indicate that information-based interventions primarily affect the timing and characteristics of vaccination behavior, instead of only influencing final coverage, with trust serving as a pivotal mediator between information exposure and adoption. By integrating behavioral and epidemiological outcomes into a unified, population-sensitive metric, QoE emerges as a crucial mechanism for evaluating, comparing, and adapting decision-making services in adaptive Internet of Things (IoT) systems.

7.1. Introduction

The COVID-19 pandemic revealed significant challenges in the design and evaluation of interventions within socio-technical systems. These systems are characterized by uncertainty, population heterogeneity, and dynamic information flows. While pharmaceutical interventions such as vaccines have been central to pandemic control, their effectiveness relies on human decision-making processes, including trust formation, risk perception, and behavioral adoption. However, these processes vary across populations and directly affect the outcomes of public health interventions [65, 44, 67, 27].

As COVID-19 becomes endemic, vaccination remains a primary strategy for preventing infections and limiting the emergence of new variants [15]. However, vaccine uptake is shaped by anti-vaccine attitudes, misinformation, and reduced perceived risk [38, 67, 25]. These factors indicate that vaccination constitutes not only a medical intervention but also an information-driven, socio-technical process mediated by communication channels such as mass media, social networks, and institutional messaging [32, 48, 60, 64].

In adaptive IoT systems, vaccination campaigns can be conceptualized as information-based decision-making services delivered via digital communication infrastructures that influence individual beliefs and behaviors under epidemiological risk. In contrast to the

prescriptive nature of algorithmic recommendation systems, these services do not dictate specific actions; they influence trust and intention through the dissemination of information. Prior research has established trust as a critical determinant of vaccine acceptance, although the effectiveness of information-based interventions remains debated [36, 48, 29, 31, 28]. Yet, the impact of these interventions cannot be fully understood without considering the social and cultural context in which they are implemented.

Cultural orientation shapes how individuals interpret information, trust institutions, and respond to collective health interventions, making it a critical factor in evaluating population-level outcomes. Therefore, this chapter investigates how identical information-based interventions can produce divergent outcomes across populations with distinct cultural orientations. An agent-based model is employed to analyze trust dynamics, vaccine acceptance, and epidemiological evolution under varying cultural profiles. Building on the human-centric evaluation framework introduced in Chapter 6, this chapter develops Quality of Experience (QoE) metrics to enable comparative evaluation of intervention outcomes across populations.

Through this approach, the chapter demonstrates that population-sensitive evaluation is essential for adaptive IoT systems. Furthermore, it establishes that human-centric metrics provide the necessary abstraction to assess interventions whose effectiveness cannot be determined solely by technical or epidemiological indicators.

7.2. Background: Epidemiology, Information Interventions, and Trust Dynamics

Epidemic dynamics have traditionally been modeled using compartmental approaches, such as the SIR and SEIR models, to analyze disease transmission at the population level. During the COVID-19 pandemic, these models were extended to incorporate pharmaceutical interventions (PI), non-pharmaceutical interventions (NPIs), and behavioral responses. Agent-based models (ABMs) have played a key role in this development, as they enable explicit representation of heterogeneous individuals, social interactions, and decision-making processes [37].

Recent ABM-based studies have examined a wide range of mechanisms related to the pandemic, including vaccination strategies, mobility restrictions, contact tracing, and behavioral compliance. As shown in Fig. 7.1, existing models encompass various combinations of epidemiological dynamics, vaccination processes, NPIs, and human behavior [51, 14, 30, 7]. However, only a limited subset of these studies explicitly integrate all four dimensions.

Within this body of literature, vaccination has primarily been modeled in two ways: as an epidemiological variable affecting disease susceptibility and transmission, and as a strategy involving allocation, scheduling, promotion, and uptake [10, 32]. Models that address vaccine uptake increasingly recognize that acceptance is shaped by psychological and social factors, such as trust, perceived risk, and exposure to information [6, 36].

Several studies have integrated opinion dynamics, trust, or decision-making processes into agent-based models to analyze vaccine-related behaviors. These studies show that information exchange through social networks and media can influence vaccine adoption and epidemic outcomes [39, 68, 7]. However, most existing models assess interventions within a single population or under fixed behavioral assumptions.

A major limitation of current research is the lack of systematic evaluation of how identical interventions perform across populations with distinct cultural or social characteristics. Although cultural orientation and social influence have been studied individually [54, 45], their combined impact on the effectiveness of information-based interventions remains insufficiently explored.

Furthermore, although many studies report epidemiological indicators such as infection rates or hospitalizations, fewer provide integrated human-centric metrics that capture behavioral adoption, trust evolution, and health outcomes within an integrated evaluative framework [24, 5]. This gap is particularly relevant for adaptive IoT systems, where interventions must be evaluated and adjusted based on population-specific responses rather than on assumed universal effectiveness.

In this context, agent-based modeling provides a robust framework for evaluating interventions, and QoE metrics establish a unified basis for comparing outcomes across populations. This chapter advances these concepts by presenting an ABM that explicitly links epidemiological dynamics, information dissemination, trust formation, and individual decision-making. In addition, QoE metrics designed to enable population-sensitive evaluation within the O-DMIoT framework are introduced.

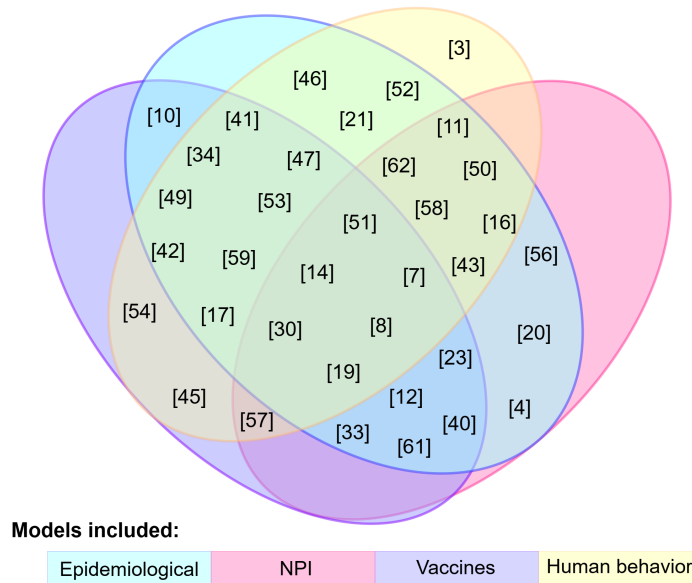


Figure 7.1.: **State of the art in agent-based modeling of COVID-19, classified by inclusion of epidemiological dynamics, vaccination processes, non-pharmaceutical interventions (NPIs), and human behavior.** While many models address subsets of these dimensions, few integrate all four. This limited intersection underscores the need for comprehensive frameworks to evaluate population-specific, information-based vaccination interventions and their effects on human behavior.

7.3. An agent-based model for Epidemics and Vaccination Response Dynamics

7.3.1. Model overview

An ABM was employed to examine the effects of information-based interventions on trust, vaccine adoption, and epidemic evolution in different populations with distinct cultural

orientations. The model integrates epidemiological dynamics, information dissemination processes, and individual decision-making within a unified socio-technical simulation framework.

Figure 7.2 illustrates the principal components of the model, demonstrating their collective role in addressing how information-based interventions influence vaccine uptake within a population. The model consists of (i) an extended SEIR-based epidemiological submodel, (ii) an information flow submodel representing communication through mass media and interpersonal networks, and (iii) a decision-making submodel that addresses vaccine acceptance and uptake at the individual level.

The epidemiological submodel simulates disease transmission based on agent mobility and contact patterns, incorporating the effects of quarantine, hospitalization, immunity, and vaccination. Changes in epidemic dynamics influence individual risk perception through institutional indicators such as hospital occupancy, which are communicated to the population as epidemic-related information.

Information dissemination is modeled through centralized and decentralized channels. Media agents represent mass communication sources (eg, television, radio, and online platforms) and broadcast information about the epidemic state and vaccination campaigns. Simultaneously, interpersonal communication within family groups and online social networks enables agents to exchange experiences related to infection and vaccination outcomes.

People agents represent individuals who perform daily routines, consume information, and make vaccination decisions. Their behavior is influenced by cultural orientation, accumulated information, and evolving trust. Vaccine uptake emerges from an internal decision-making process driven by perceived risk, trust in vaccines, and vaccine availability.

The interactions among these components generate population-level outcomes, including vaccination coverage, trust dynamics, and epidemiological trajectories. This integrated structure provides a foundation for analyzing how identical information-based interventions produce different effects across populations, a topic addressed in subsequent sections.

7.3.2. Conceptual Architecture

The proposed agent-based model conceptualizes a socio-technical system in which epidemiological dynamics, information dissemination, and individual decision-making are interconnected through feedback mechanisms. Rather than treating vaccination as a prescriptive control process, the architecture frames vaccination campaigns as information-based decision-making services that shape trust and intention within the context of epidemiological risk.

The architecture encompasses three interacting submodels that operate concurrently:

- (i) An *epidemiological submodel* capturing population-level disease dynamics and health outcomes.
- (ii) An *information flow submodel* representing both centralized and decentralized dissemination of epidemic and vaccine related information.
- (iii) A *behavioral decision-making submodel* governing the formation of trust, development of intention, and decisions regarding vaccination acceptance.

Each submodel contributes to emergent system behavior through structured interactions. Figure 7.3 provides a conceptual overview of these interactions and the resulting feedback loops. Epidemiological conditions generate institutional indicators, such as hospital occupancy, which shape perceived risk and influence individual intention. Concurrently, information disseminated through media and social networks shapes trust formation. Trust

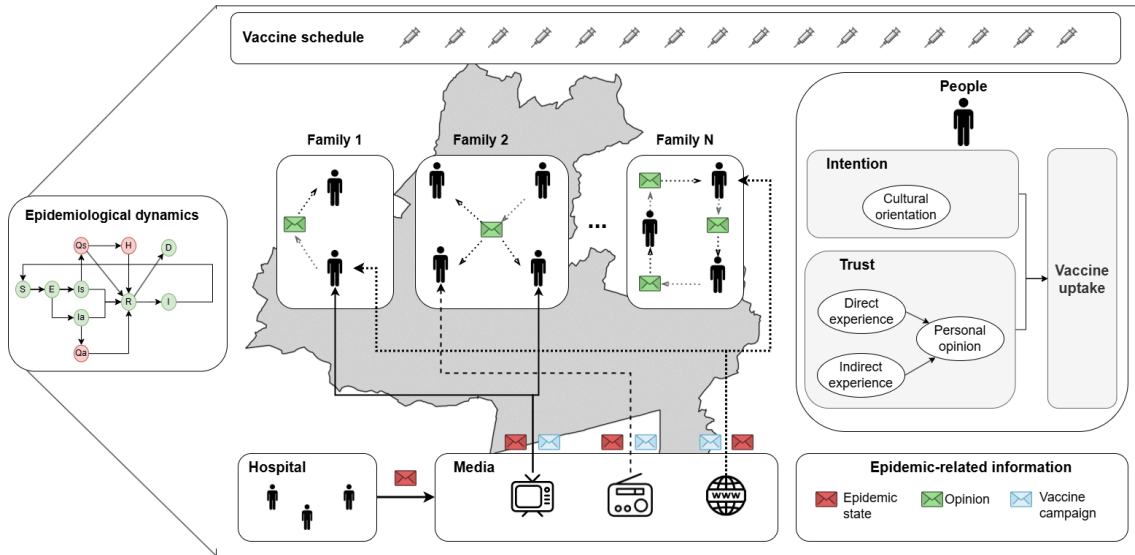


Figure 7.2.: **ABM overview** The figure presents an overview of the proposed model, demonstrating the coordination among epidemiological dynamics, information exposure, trust evolution, and vaccine adoption decisions. The figure emphasizes feedback relationships and offers a comprehensive perspective on the evolution of individual cognition and population dynamics during the simulation.

and intention jointly inform a deliberative decision-making mechanism that determines vaccine acceptance. Vaccine uptake, constrained by availability, modifies the epidemiological state of the population, thereby completing the feedback loop.

This architecture explicitly captures the non-linear and temporally extended nature of vaccination decision-making. Information does not directly determine behavior; instead, it alters cognitive and perceptual states that accumulate over time through deliberation. Consequently, identical interventions may yield different outcomes depending on population characteristics, information exposure, and timing of execution. These properties are not evident in isolated submodels and emerge only through their interaction.

By structuring the model around interacting feedback loops, the architecture enables population-sensitive analysis of information-based interventions. This approach is consistent with the objectives of human-centric evaluation in adaptive IoT systems and establishes the framework for assessing how decision-making services influence human experience and system-level outcomes without embedding domain-specific logic into the O-DMIoT framework.

7.3.3. Agents and System Components

The model incorporates heterogeneous agents representing both decision-making individuals and information generating entities within the socio-technical environment. *People* agents function as the primary decision-makers, performing daily activities, processing information, building trust, and deciding whether to accept vaccination based on their internal states and external stimuli. Each individual is characterized by attributes such as cultural orientation, trust, intention, and epidemiological status.

Information is introduced and disseminated through media agents and interpersonal networks. *Media* agents represent mass communication channels, including television, radio, and online platforms, and are responsible for broadcasting epidemic-related information

and vaccine promotion messages. Interpersonal communication within family groups and online social networks enables the exchange of experiences related to infections, vaccinations, and side effects.

Institutional components, including *hospitals* and *vaccine* entities, provide structural context. Hospital occupancy serves as an institutional indicator of epidemiological risk, while vaccine availability constrains the timing of individual decisions. Collectively, these agents and components create a heterogeneous environment in which information, trust, and behavior evolve.

7.3.4. Information Flow and Decision Cycle

Information flow serves as the primary mechanism by which interventions influence individual behavior in the model. Epidemic-related information is disseminated through centralized channels, such as media broadcasts, and decentralized channels, including family communication and social networks. Information is abstracted as messages encoding experiences, opinions, and institutional indicators, such as hospital occupancy, enabling agents to update their internal beliefs through direct and indirect experience [6, 26].

Each People agent integrates incoming information into an internal decision-making process that handles vaccination acceptance. This process combines perceived epidemiological risk, trust in vaccines, and individual predisposition determined by cultural orientation. Vaccination is not imposed but instead emerges from repeated deliberation under changing informational and contextual conditions, consistent with behavioral decision theories [1].

The resulting decision cycle forms a closed feedback loop: information influences trust and intention; decisions affect vaccination uptake; vaccination alters epidemiological trajectories; and updated epidemic states generate new information. This cycle enables analysis of how identical information-based interventions can have different outcomes across populations, providing a foundation for population-sensitive evaluation and the derivation of human-centric metrics in subsequent sections.

7.4. Behavioral Foundations and Supporting Submodels

This section presents the behavioral foundations and supporting submodels that underpin the agent-based analysis in this chapter. These elements establish the conceptual and computational framework necessary for modeling trust dynamics, information-driven behavior, and vaccine adoption. They are included to ensure methodological completeness and to clarify the assumptions and mechanisms underlying the proposed decision-making framework.

Specifically, this section formalizes how cultural orientation, epidemiological states, and institutional signals are translated into internal agent states that influence behavior. These foundations are essential for integrating the trust formation and deliberative decision-making mechanisms introduced later in the chapter. Without these elements, the interpretation of adoption dynamics and the effects of information-based interventions would remain under specified.

The submodels presented are primarily adapted from established behavioral and epidemiological literature. Cultural orientation is operationalized as an intention construct derived from social psychology, while epidemiological progression and vaccination availability are modeled using validated compartmental and scheduling approaches. This integration enables subsequent modeling contributions to be evaluated within a controlled and theoretically grounded context.

By distinguishing foundational components from original modeling contributions, this chapter emphasizes that its novelty resides in the integration of information exchange, trust evolution, and deliberative decision-making within a unified agent-based framework.

7.4.1. Epidemiological Submodel

Epidemiological dynamics are modeled using an extended SEIR framework adapted from [50]. The model integrates both epidemiological and behavioral states, enabling the representation of disease progression alongside individual response mechanisms.

The compartmental structure includes:

- S - Susceptible
- E - Exposed
- I_s - Infected symptomatic
- I_a - Infected asymptomatic
- Q_s - Quarantined with symptoms
- Q_a - Quarantined asymptomatic
- H - Hospitalized
- R - Recovered
- D - Death
- I - Immune

The dynamics of the model follow these rules:

1. A people agent begins in a Susceptible (S) state.
2. When an agent is in the Susceptible (S) state and has contact with an agent in the Infected (I_a, I_s) state, then it changes its state to Exposed (E) according to probability β .
3. An agent in the Exposed (E) state spends t_1 (incubation time).
4. When t_1 ends, based on a probability ρ , the agent will change to the Infected symptomatic state (I_s). Consequently, it will change to an Infected asymptomatic (I_a) state based on probability $(1 - \rho)$.
5. An agent will spend time t_2 in the Infected symptomatic (I_s) state. The agent can change to the Quarantine symptomatic (Q_s) state during this period according to the probability δ_s or be hospitalized (H) according to the probability γ .
6. An agent will spend time t_3 in the Infected asymptomatic (I_a) state. The agent can change to the Quarantine asymptomatic (Q_a) state during this period according to the probability δ_a .
7. When t_2 ends, the agent can enter the Immune state (I) according to probability λ or a terminal state called death (D) with probability $1 - \lambda$.
8. The agent can change to the Immune (I) state with probability λ after it enters the Recovery state.

9. An agent that has reached the Immune (I) state remains there for a time t_4 .
10. When t_4 ends, the agent returns to the Susceptible (S) state.
11. A vaccinated agent enters the Immune(I) state.
12. Agents in the Susceptible (S) and Exposed (E) states can be vaccinated. In this case, it is assumed that the agent might not know that he/she has been exposed and hence decides to get vaccinated.
13. An agent in the Asymptomatic Infected state (Ia) who is not in the Quarantined Asymptomatic behavioral state (Qa) can be vaccinated. Once again, it is assumed that the agent may not know that he/she has been infected and therefore decides to get vaccinated.

A modelling contribution in this submodel is the explicit integration of vaccination within the epidemiological state space. Vaccinated agents transition directly to the *Immune* state. Additionally, agents in the *Susceptible*, *Exposed*, and *Infected Asymptomatic* states may be vaccinated, reflecting realistic uncertainty regarding infection status. This design supports the representation of preventive vaccination, delayed behavioral responses, and imperfect situational awareness, which are essential for evaluating information-based interventions.

7.4.2. Vaccine Schedule Submodel

The vaccine schedule is modeled as an external mechanism reflecting real-world vaccination campaigns. Availability periods are defined exogenously based on the Mexican vaccination rollout [22].

Vaccines are available during discrete five-day windows, separated by intervals of non-availability. Availability does not guarantee uptake. During availability periods, agents deliberate and decide whether to be vaccinated based on their internal cognitive states.

This submodel introduces a temporal separation between vaccine availability and adoption, enabling analysis of scenarios in which vaccines exist but remain underutilized due to trust deficits, exposure to misinformation, or cultural factors. This separation is critical for evaluating interventions that rely on persuasion rather than enforcement.

7.4.3. Intention

In the social and behavioral sciences, cultural orientation has been shown to influence behavioral norms, attitudes toward collective risk, and responses to public health interventions [63]. Prior research has demonstrated that individuals in collectivist contexts tend to be more sensitive to shared risk and institutional signals, whereas individualistic orientations emphasize personal autonomy and risk assessment. These distinctions are particularly relevant in vaccination contexts, where decisions involve trade-offs between individual preferences and collective outcomes.

In this work, cultural orientation is operationalized as an internal intention variable that captures an agent's predisposition toward vaccination in response to epidemiological risk. The formulation adopted here is based on the model introduced in [50], where individual behavior during the COVID-19 pandemic is influenced by both cultural orientation and institutional indicators of risk. Intention is computed as:

$$\beta^t = \frac{\delta\beta^{t-1}}{\delta\beta^{t-1} + (1-\delta)(1-\beta^{t-1})}\epsilon^t + \frac{(1-\delta)\beta^{t-1}}{(1-\delta)\beta^{t-1} + \delta(1-\beta^{t-1})}(1-\epsilon^t) \quad (7.1)$$

where $\delta \in [0, 1]$ represents the cultural orientation coefficient, with values close to 0 indicating collectivist tendencies and values close to 1 indicating individualistic tendencies. The variable $\epsilon^t \in [0, 1]$ denotes institutional risk perception, operationalized as hospital occupancy at time t . The resulting value β^t reflects the agent's evolving behavioral inclination in response to perceived epidemic severity.

Unlike previous applications in which β directly influenced compliance behavior, this chapter interprets β as intention, consistent with the Theory of Planned Behavior, which posits that intentions precede and guide actions [2]. Intention, therefore, functions as a mediating construct between external risk signals and individual decision-making regarding vaccine uptake.

In the proposed framework, intention does not directly determine vaccine uptake. Rather, it is one of two cognitive inputs, along with trust, that inform the deliberative decision-making process described in Section 7.5.3. This separation permits intention to evolve independently of trust and enables the analysis of scenarios where high perceived risk does not necessarily result in adoption due to low trust, or conversely.

Treating intention as a behavioral foundation rather than a decision rule, the model preserves analytical flexibility and ensures theoretical grounding. This design supports the central objective of examining how information-based interventions interact with trust dynamics and cultural context to influence vaccination outcomes across populations.

7.5. Modelling contributions

The primary contribution is the integration of epidemiological dynamics with information exchange, trust formation, and individual decision-making within a unified agent-based framework. This approach enables systematic analysis of how information-based interventions propagate through heterogeneous populations and influence adoption behavior under epidemiological risk.

Specifically, this work presents a novel operationalization of Decision Field Theory (DFT) as a mechanism for vaccine adoption within an agent-based epidemiological model. DFT has not been applied to model vaccination decisions in the context of information-driven interventions and epidemic dynamics. Integrating DFT into the agent-based model allows explicit representation of deliberation, hesitation, and reversible preferences.

7.5.1. Information Exchange Submodel

This subsection describes the information exchange model, which characterizes the dissemination of epidemic- and vaccine-related information within the population and its influence on individual beliefs, trust formation, and vaccination decisions. The approach explicitly models information content, communication channels, and cognitive mediation mechanisms.

The information exchange model consists of two tightly coupled components: a structured representation of information units, and a set of dissemination mechanisms governing how information is generated, transmitted, consumed, and re-transmitted among agents.

Information Unit

The objective is to analyze vaccine acceptance as a socio-technical phenomenon shaped by attitudes, beliefs, and information exposure. To capture these dimensions, information is abstracted using the Vaccination Attitude Examination (VAX) scale [38], a validated instrument widely employed in studies of vaccination hesitancy and information-seeking behavior.

The VAX scale distinguishes four attitudinal dimensions that influence vaccination behavior:

1. Mistrust of vaccine benefits
2. Preference for natural immunity
3. Concerns about side effects
4. Concerns about commercial profiteering

In the proposed model, epidemic-related information is represented as a multidimensional vector:

$$E_A = \begin{bmatrix} vaccine_brand_acceptance \\ vaccine_efficiency_belief \\ natural_immunity_preference \\ side_effects_belief \end{bmatrix}$$

Each component assumes values within the interval $[-1, 1]$, enabling the encoding of supportive and adverse attitudes toward vaccination. This representation allows partial, conflicting, or ambivalent information to coexist within a single message.

Specifically:

- *vaccine_brand_acceptance* represents attitudes related to commercial trust and brand differentiation, where -1 indicates rejection of a vaccine brand, and 1 indicates strong acceptance.
- *vaccine_efficiency_belief* captures mistrust of vaccine benefits; negative values indicate disbelief in vaccine effectiveness.
- *natural_immunity_preference* models the preference for acquiring immunity through infection versus vaccination.
- *side_effects_belief* represents perceived safety concerns, where negative values indicate a strong fear of adverse effects.

This multidimensional abstraction represents a key modeling contribution, treating information as a structured cognitive stimulus rather than a uniform signal. This approach enables selective influence on trust, intention, and decision-making processes.

Information Exchange Mechanisms

The dissemination of information is governed by three complementary exchange mechanisms, each corresponding to a distinct socio-technical communication channel:

(i) *Family exchange (People agent ↔ People agent)*

Family exchange models interpersonal communication driven by lived experience. Families are represented as small social groups with cardinalities ranging from two to five individuals, randomly assigned at the beginning of the simulation.

Information exchange within families is event-driven and occurs only when salient health-related events take place. Specifically, agents share information with family members under the following conditions:

- Experiencing side effects after vaccination
- Becoming infected despite having been vaccinated

- Recovering from an infection

This design ensures that family communication reflects experiential credibility. Family exchange functions as a high-trust, low-frequency information channel that directly influences indirect experience within the trust formation model.

(ii) **Mass media exchange (*Radio/TV agent* $\rightarrow$ *People agents*)**

Mass media exchange represents centralized, institutional communication through television and radio channels. Media agents broadcast epidemic-related and vaccine-promoting messages at a frequency determined by the parameter *posting_rate*, which is initialized using empirical media consumption statistics [18].

Messages are disseminated in bulk but are not received uniformly by the population. Instead, each broadcast targets a randomly sampled subset of agents drawn from pre-defined audience profiles, reflecting heterogeneous media consumption habits. This stochastic sampling enables the model to represent temporal variation in exposure, scheduling effects, and partial campaign reach.

Mass media messages are unidirectional and cannot be re-transmitted by recipients. Their influence is mediated exclusively through updates to trust and intention, rather than through network diffusion.

(iii) **Online social network exchange (*People agent* $\leftrightarrow$ *People agent*)**

Online social network exchange models decentralized digital communication using a scale-free network topology, where nodes represent people, agents and edges represent social ties. Information propagation follows a selective flooding mechanism combined with a Susceptible–Infected–Recovered (SIR) diffusion process [66, 55, 13].

At initialization, a health authority is modeled as an external source that injects messages into the network at intervals determined by *posting_rate*. A subset of highly connected nodes (influencers) is selected as the initial target audience. Unlike mass media, online messages may be reposted by agents.

Message consumption is governed by a confirmation bias mechanism [38]. Upon receiving a message, an agent evaluates it against its current trust level. Only if the message aligns sufficiently with existing beliefs does the agent transition from the Susceptible to the Infected state, indicating that the information has been cognitively accepted. Accepted messages are then re-posted to neighboring nodes, excluding the original sender. Once an agent disseminates the message, it enters the Recovered state, preventing immediate re-posting.

This mechanism enables belief-dependent diffusion, producing the emergence of misinformation resistance and echo-chamber effects as natural outcomes of trust dynamics.

Collectively, these mechanisms represent interpersonal communication, institutional broadcasting, and decentralized digital diffusion. Mass media and online social network exchanges are structured to reflect the public vaccine-promotion campaign “It’s up to you” [itsuptoyou_covid-19_nodate], which serves as the NPI analyzed in this chapter.

The information exchange model does not directly induce behavioral changes. Instead, information alters agents’ experiences, which subsequently update trust and intention values. These internal states influence vaccine adoption through the decision-making sub-model. Therefore, information acts as an indirect, mediating factor rather than a deterministic driver.

7.5.2. Trust Submodel

To incorporate the individual and interpersonal factors influencing vaccine uptake, as well as the impact of vaccination campaigns on perceptions and trust, this work adopts the model of trust with indirect experience proposed by [26]. This model conceptualizes trust as evolving through accumulated experience. It distinguishes between direct experience (personal experience) and indirect experience (observed experiences of others). Within this framework, experiences are exchanged, and trust T at time t is subsequently computed.

The proposed Trust submodel offers a novel perspective by conceptualizing experience as a form of information. Accordingly, experience is abstracted as a unit of information, as described in Sec. 7.5.1. People agents acquire experience in three distinct forms:

- By vaccine application.
- By exchanging information with other individuals.
- Through the dissemination of information by the media.

To implement the indirect trust model developed by [26], each people agent contains a direct experience vector (E_A^d , see Sec. 7.5.1) whose values are randomly established at the beginning of the simulation and change according to the following rules:

- When having side effects after being vaccinated, the value of the variable *side_effects_belief* is set to -1.
- When an agent is recovered, the value of variable *natural_immunity_preference* is set to -1.
- When an agent gets sick even though it has been vaccinated, variables *vaccine_brand_acceptance* and *vaccine_efficiency_belief* are set to -1, indicating that the vaccine is ineffective.

The indirect experience vector E_A^i is computed as an average of trust T at time t , excluding the agent (A) who is currently performing the calculation:

$$E_A^i(t) = \frac{\sum_{B=1, B \neq A}^N T_B(t)}{N - 1} \quad (7.2)$$

The overall experience $E_A(t)$ merges both experiences as a weighted sum where the weights are determined by α , which represents an influence factor:

$$E_A(t) = \alpha_A * E_A^i(t) + (1 - \alpha_A) * E_A^d(t) \quad (7.3)$$

Since the overall experience is a vector, an average is calculated with its elements, and this is compounded with the trust ($T_A(t)$) in the trust equation 7.4.

Trust $T_A(t)$ is computed by:

$$T_A(t + \Delta t) = T_A(t) + \gamma_A * (E_A(t) - T_A(t)) * \Delta t \quad (7.4)$$

where T_A represents the current trust of the people agent, and γ is an individual flexibility feature that entails the intrinsic desire to experience. The factors α and γ_A are set randomly at the start of the simulation and remain constant for each people agent.

The calculated trust value T_A is used as a factor in making decisions regarding vaccine acceptance (see Sec. 7.5.3); as well as a cognitive bias [38] for the acceptance of messages in the information exchange model (see Sec. 7.5.1). In the simulation, experience and trust are computed every 24 cycles.

7.5.3. Decision-making Submodel

The decision-making submodel handles vaccine adoption at the individual level using a deliberation approach. This submodel integrates trust and intention into a dynamic preference evolution process based on Decision Field Theory (DFT) [9, 35]. While DFT itself is an established cognitive decision framework, its operationalization in this work constitutes a key modeling contribution. This integration enables explicit representation of hesitation, delayed commitment, and reversible preferences, which are challenging to capture using static thresholds or adoption rules commonly employed in epidemic models.

Dynamic Preference Representation

At each time step t , a people agent maintains a preference vector:

$$P(t) = \begin{bmatrix} be_vaccinated \\ not_be_vaccinated \end{bmatrix} \quad (7.5)$$

where each component represents the degree of desirability assigned to the corresponding alternative. Preference values are continuous and non-negative, allowing partial inclination.

Initial preferences are assigned according to the agent's initial vaccine attitude:

- Refusal: $P(0) = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix}$
- Acceptance: $P(0) = \begin{bmatrix} 0.5 \\ 0 \end{bmatrix}$
- Hesitation: $P(0) = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

This initialization enables agents to begin in a neutral or undecided state, avoiding premature commitment.

Preference Evolution via DFT

Preferences evolve according to the DFT update equation:

$$P(t+h) = SP(t) + CM(t+h)W(t+h) \quad (7.6)$$

This formulation decomposes decision-making into four interacting components:

1. Personal evaluation Matrix (M)

The matrix M encodes how internal cognitive factors contribute to preference formation:

$$M(t) = \begin{bmatrix} T_A & \beta^t \\ 1 - T_A & 1 - \beta^t \end{bmatrix} \quad (7.7)$$

where:

T_A represents the individual trust in the vaccine, derived endogenously from the trust formation submodel (See section 7.5.2); β^t represents intention to vaccinate, derived from cultural orientation and perceived epidemiological risk (See section 7.4.3). The personal evaluation matrix changes at every time step.

This formulation represents a central contribution by treating trust and intention as distinct yet complementary dynamic cognitive drivers.

2. Attention Weight Vector (W)

The vector W captures fluctuating attention to decision relevant factors:

$$W(t) = \begin{bmatrix} T_{avg} \\ H_t \end{bmatrix} \quad (7.8)$$

This structure enables epidemiological signals, such as hospital occupancy (H_t) and average trust (T_{avg}) representing the population's overall perception of vaccination, to influence decision salience indirectly. This approach maintains the autonomy of individual deliberation by not directly enforcing behavior.

3. Contrast Matrix (C)

The contrast matrix:

$$C = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \quad (7.9)$$

models competitive evaluation between alternatives, ensuring that increased preference for vaccination simultaneously reduces preference for non-vaccination.

4. Feedback Matrix (S)

The feedback matrix:

$$S = \begin{bmatrix} 0.9 & -0.01 \\ -0.01 & 0.9 \end{bmatrix} \quad (7.10)$$

provides a memory effect; it contains the self-connections and the interconnections between the choices. The self-connections represent the memory of the previous time step; when set to 1, it represents perfect memory, and 0 represents no memory. In this work, it is established at 0.9, which indicates slow deterioration in memory [35].

Fig. 7.4 provides a visual representation of this process.

Decision Threshold and Reversibility

A vaccination decision is triggered when the preference for vaccination exceeds an internal threshold:

$$p_{\text{vaccinate}}(t) \geq 1.5$$

This threshold operates on accumulated preference, which allows:

- Delayed adoption despite favorable conditions.
- Hesitation under conflicting information.
- Reversibility of inclination prior to threshold crossing.

As trust and intention evolve over time, preference trajectories remain non-deterministic and are sensitive to both information exposure and the epidemic context.

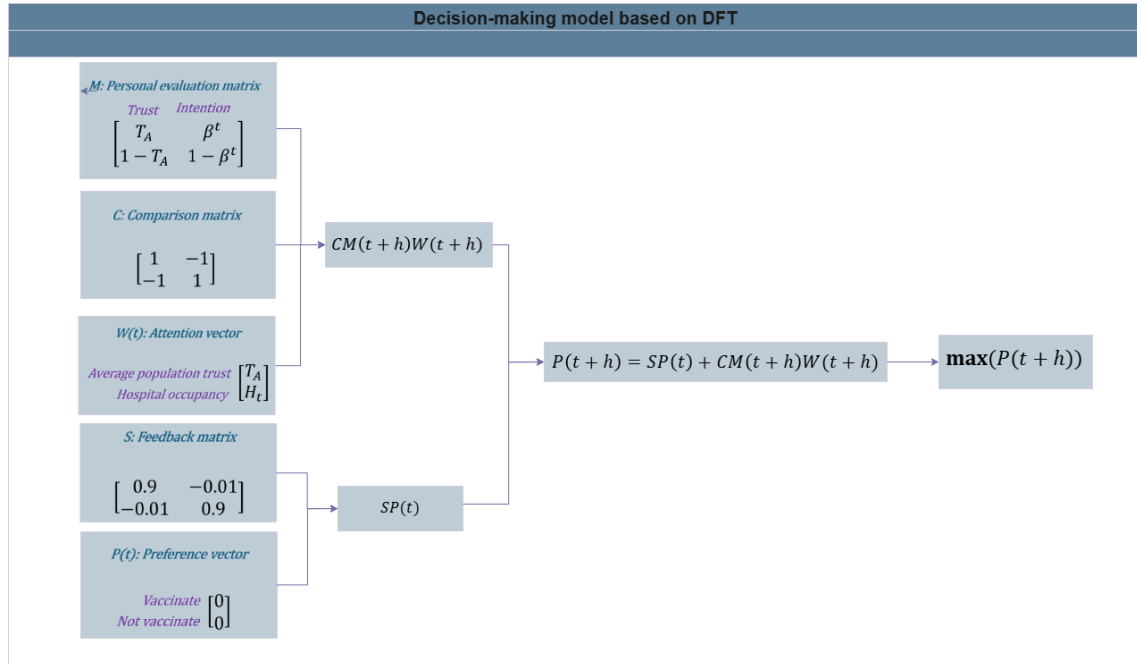


Figure 7.4.: **Operationalization of deliberative vaccination decisions using Decision Field Theory.** The diagram illustrates the integration of trust and intention with contextual signals to drive dynamic preference accumulation over time. Feedback and attention mechanisms support non-deterministic and reversible decision trajectories, enabling vaccine acceptance to result from deliberative processes. This structure forms the basis for the timing effects and population-sensitive outcomes examined in Chapter 7.

7.5.4. Modeling Contribution Summary

This decision-making submodel contributes three novel capabilities to agent-based epidemic modeling:

1. **Deliberative vaccine uptake**

Vaccination is not imposed probabilistically but emerges from a cognitive process that unfolds over time.

2. **Reversible and non-monotonic preferences**

Agents may oscillate between acceptance and hesitation in response to evolving trust and information.

3. **Explicit coupling of information, trust, and epidemiology**

Epidemiological indicators influence attention, information shapes trust, and trust modulates decision outcomes.

Embedding DFT within an ABM provides a behavioral foundation for analyzing information-based interventions whose effects depend on timing, perception, and population heterogeneity. This capability is essential for evaluating adaptive IoT decision-making services within the O-DMIoT framework.

7.6. Experimental Setup and Population Scenarios

This section describes the experimental design used to evaluate the proposed agent-based model and to analyze the effects of information-based vaccination interventions across populations with different cultural orientations. The objective of the experimental setup is twofold:

- (i) Examine how information exposure, trust dynamics, and decision-making processes influence vaccine adoption.
- (ii) Assess how these behavioral dynamics translate into epidemiological outcomes under different intervention configurations.

7.6.1. Scenario Design

The experimental design consists of eight simulation scenarios constructed by combining three binary dimensions:

1. Cultural orientation of the population
 - Collectivist.
 - Individualist.
2. Information-based intervention (Non-Pharmaceutical Intervention, NPI)
 - Active: vaccine-promoting information disseminated through media and social networks.
 - Inactive: no centralized information campaign.
3. Vaccine availability (Pharmaceutical Intervention, PI)
 - Available: vaccines provided according to a predefined schedule

- Unavailable: no vaccine access

These dimensions yield a total of eight scenarios, enabling systematic comparison of how identical interventions perform under different social and informational conditions.

For scenarios with vaccine availability, vaccination follows the schedule described in Section 7.4.2. Vaccines are offered during discrete five-day availability windows, beginning on day 14 of the simulation. Vaccine availability does not imply uptake: individuals must deliberate and decide whether to be vaccinated based on trust, intention, and information exposure. To isolate the behavioral effects under study, all scenarios share:

- Identical epidemiological parameters.
- Identical mobility routines.
- The same initial random seed.

This design ensures that observed differences result from information flow, trust evolution, and decision-making, rather than from stochastic variation in disease spread.

7.6.2. Observed Variables

Simulation outputs are observed at both the individual and population levels. The following indicators are monitored throughout the simulation:

- **Epidemiological variables:** Infected, Quarantined, Immune, Hospitalized, and Deaths.
- **Behavioral and cognitive variables:** Trust, Intention, Vaccine Acceptance, and Vaccine Uptake

These variables enable analysis of feedback loops between epidemic dynamics, information exposure, and individual decision-making. In particular, they support evaluation of how information-based interventions influence preventive vaccination and the timing of adoption, rather than only final vaccination coverage.

Simulation results are analyzed across multiple infection waves and at the end of the simulation. Table 7.1 summarizes the evolution of key variables at representative time points. For clarity, vaccine acceptance refers to the internal preference to be vaccinated, while vaccine uptake denotes the realized action of vaccination.

	Population	Scenario		ID	Day	Infected	Quarantine	Immune	Hospitalization	Deaths	Trust	Intention	Vaccinated	Vaccine acceptance
		Intervention	Vaccine											
Peak 1	Colectivist	Inactive	Unavailable	(i)	4	59.30%	30.31%	1.03%	72.00%	0.25%	0.029	0.5	0.00%	0.00%
			Available	(iii)	4	59.30%	30.47%	2.14%	64.00%	0.25%	0.029	0.5	0.00%	0.00%
		Active	Unavailable	(v)	4	59.13%	30.56%	2.39%	56.00%	0.29%	0.103	0.5	0.00%	0.00%
			Available	(vii)	4	59.13%	30.76%	2.39%	46.00%	0.29%	0.103	0.5	0.00%	0.00%
	Individualist	Inactive	Unavailable	(ii)	4	59.30%	30.31%	2.14%	72.00%	0.25%	0.029	0.5	0.00%	0.00%
			Available	(iv)	4	59.30%	30.47%	2.14%	64.00%	0.25%	0.029	0.5	0.00%	0.00%
		Active	Unavailable	(vi)	4	59.13%	30.56%	2.39%	56.00%	0.29%	0.103	0.5	0.00%	0.00%
			Available	(viii)	4	59.13%	30.76%	2.39%	46.00%	0.29%	0.103	0.5	0.00%	0.00%
Peak 2	Colectivist	Inactive	Unavailable	(i)	25	20.95%	10.56%	26.10%	52.00%	7.46%	0.0744	0.065	0.00%	4.41%
			Available	(iii)	25	20.70%	10.31%	26.97%	32.00%	7.34%	0.0744	0.0396	0.00%	2.18%
		Active	Unavailable	(v)	25	20.78%	21.65%	25.86%	24.00%	7.63%	0.2183	0.031	0.00%	27.62%
			Available	(vii)	26	20.49%	10.56%	24.49%	18.00%	9.90%	0.2098	0.0311	17.81%	38.30%
	Individualist	Inactive	Unavailable	(ii)	25	21.07%	10.47%	26.10%	50.00%	7.46%	0.0744	0.029969	0.00%	3.13%
			Available	(iv)	25	20.70%	10.31%	26.80%	32.00%	7.34%	0.0744	0.029965	0.00%	1.94%
		Active	Unavailable	(vi)	26	20.62%	10.31%	26.27%	30.00%	7.75%	0.2241	0.030586	0.00%	0.08%
			Available	(viii)	26	20.41%	10.43%	25.15%	16.00%	9.07%	0.2255	0.030865	12.41%	25.77%
Peak 3	Colectivist	Inactive	Unavailable	(i)	50	13.20%	6.85%	27.51%	12.00%	13.57%	0.080324	0.0661	0.00%	98.56%
			Available	(iii)	47	12.99%	6.89%	27.05%	18.00%	14.23%	0.093431	0.032635	2.31%	5.73%
		Active	Unavailable	(v)	47	7.59%	7.30%	29.65%	14.00%	11.67%	0.228733	0.030779	0.00%	29.56%
			Available	(vii)	47	12.58%	6.47%	27.42%	12.00%	14.89%	0.196091	0.031134	18.88%	42.30%
	Individualist	Inactive	Unavailable	(ii)	47	13.24%	6.93%	27.75%	20.00%	13.36%	0.088329	0.029985	0.00%	5.90%
			Available	(iv)	48	13.32%	7.09%	27.01%	12.00%	14.31%	0.084427	0.02994	1.48%	3.13%
		Active	Unavailable	(vi)	48	13.20%	6.56%	27.75%	14.00%	13.98%	0.229382	0.030583	0.00%	27.05%
			Available	(viii)	47	12.62%	6.56%	27.18%	14.00%	14.93%	0.212177	0.030858	13.73%	31.17%
Simulation end	Colectivist	Inactive	Unavailable	(i)	62	9.28%	5.20%	28.08%	8.00%	16.78%	0.077644	0.065497	0.00%	9.93%
			Available	(iii)	62	9.98%	5.07%	27.09%	14.00%	17.32%	0.082218	0.032633	2.31%	5.73%
		Active	Unavailable	(v)	62	10.89%	5.48%	28.54%	12.00%	14.97%	0.227827	0.030764	0.00%	30.35%
			Available	(vii)	62	9.57%	5.36%	26.31%	4.00%	18.89%	0.197276	0.031134	19.34%	43.51%
	Individualist	Inactive	Unavailable	(ii)	62	9.61%	4.99%	28.04%	18.00%	16.70%	0.095365	0.029985	0.00%	6.39%
			Available	(iv)	62	9.40%	4.62%	27.67%	12.00%	17.24%	0.089848	0.02994	1.48%	3.13%
		Active	Unavailable	(vi)	62	9.48%	4.82%	28.04%	14.00%	16.82%	0.229032	0.030583	0.00%	31.63%
			Available	(viii)	62	9.69%	4.82%	27.18%	16.00%	17.86%	0.201143	0.030858	15.09%	34.10%

Table 7.1.: Analysis by wave of infections. NPI - Information flow intervention regarding COVID-19 vaccination / PI - COVID-19 vaccine

7.6.3. Experimental Results and Analysis

Epidemic Dynamics and Vaccination Outcomes

Across all scenarios, the overall structure of epidemic progression remains with three major infection waves observed. This similarity is expected due to the shared epidemiological parameters and daily routines. However, as shown in Fig. 7.5, despite similar infection trajectories, significant differences emerge in hospital occupancy, immunity levels, and vaccination-related variables (vaccine acceptance and uptake).

Scenarios with the NPI active exhibit lower hospital occupancy after the first wave, particularly in collectivist populations. This reduction influences subsequent behavioral dynamics by lowering perceived risk, which directly affects intention. These results indicate that epidemiological indicators function as inputs to behavioral processes rather than as direct determinants of vaccine adoption.

Impact of Cultural Orientation on Decision Dynamics

Cultural orientation plays a decisive role in shaping vaccination behavior (See Fig. 7.6). Collectivist populations consistently exhibit higher intention peaks in response to high hospital occupancy, reflecting stronger sensitivity to collective risk. In contrast, individualist populations show more muted intention responses, particularly in later waves.

These differences persist even under identical intervention configurations, indicating that population characteristics mediate the effectiveness of information-based interventions. This finding supports the argument that adaptive IoT systems must account for human and social context when evaluating intervention strategies.

Trust, Information Exposure, and Vaccine Acceptance

Trust evolves differently depending on the presence of information-based interventions (See Fig. 7.6). In scenarios without an active NPI, trust remains low and is shaped primarily by interpersonal experiences, which may include negative outcomes such as side effects or breakthrough infections.

When an NPI is active, trust increases earlier and reaches higher levels overall. Although trust may fluctuate or decline later due to negative personal experiences, early exposure to consistent, positive information leads to earlier vaccine acceptance and higher preventive vaccination rates.

In scenarios with active NPIs, higher vaccine acceptance is observed among Susceptible agents, indicating a shift toward preventive rather than reactive vaccination. This contrasts with scenarios without NPIs, where acceptance is concentrated among agents who have already experienced infection (See Fig. 7.7).

Timing Effects and Deliberative Adoption

An important result enabled by the proposed decision-making model is the observation of timing effects. In scenarios without NPIs, vaccine acceptance tends to increase during later infection waves. In contrast, scenarios with NPIs show earlier adoption, often immediately following vaccine availability.

This behavior emerges from the deliberative nature of the decision-making model: agents accumulate preference over time based on evolving trust and intention, rather than reacting instantaneously to vaccine availability. These results illustrate the value of modeling decision-making as a dynamic process rather than as a static threshold.

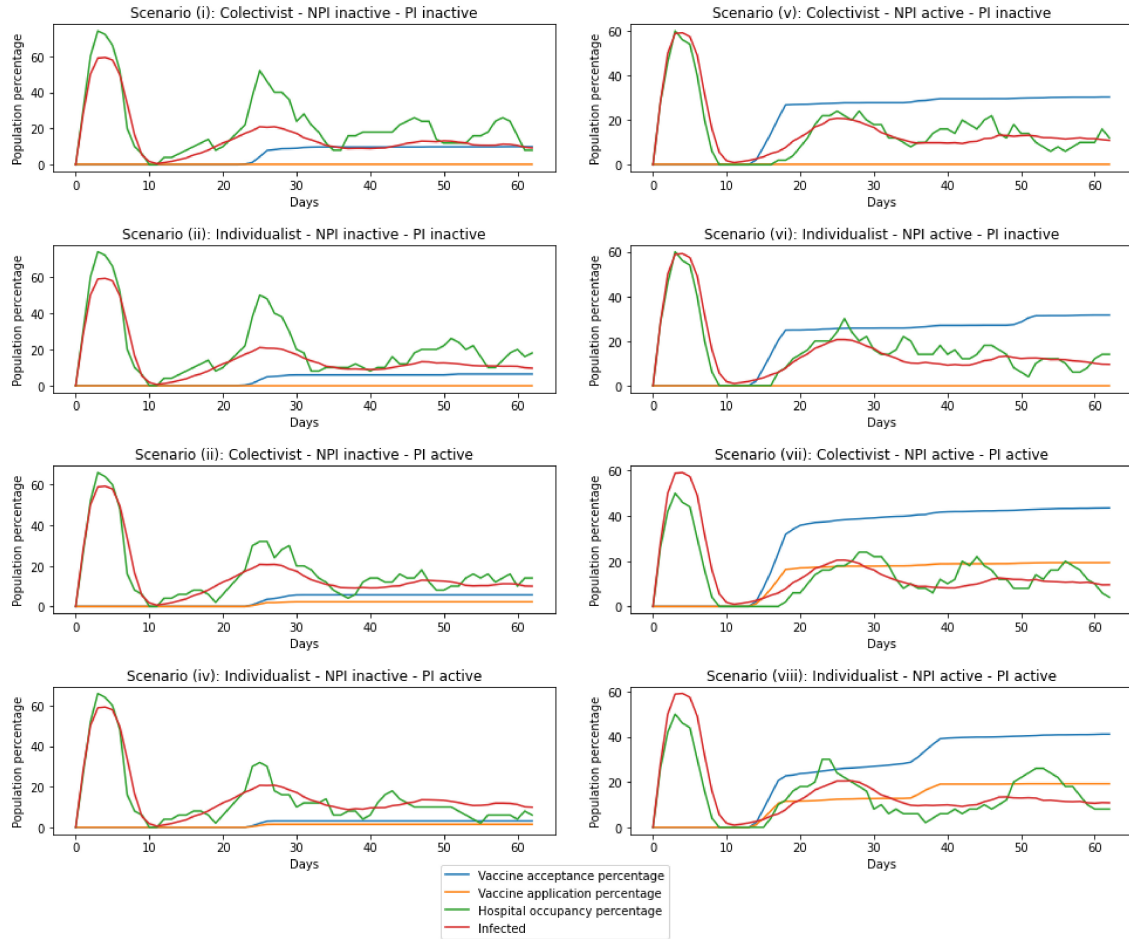


Figure 7.5.: **Comparative dynamics of epidemiological outcomes and vaccination behavior across experimental scenarios.** Results are organized by cultural orientation and intervention configuration. Although epidemiological trajectories remain comparable due to identical initial conditions, substantial differences are observed in vaccine acceptance and uptake, especially under active information-based interventions. These findings highlight the impact of trust dynamics and deliberative decision-making on vaccination behavior.

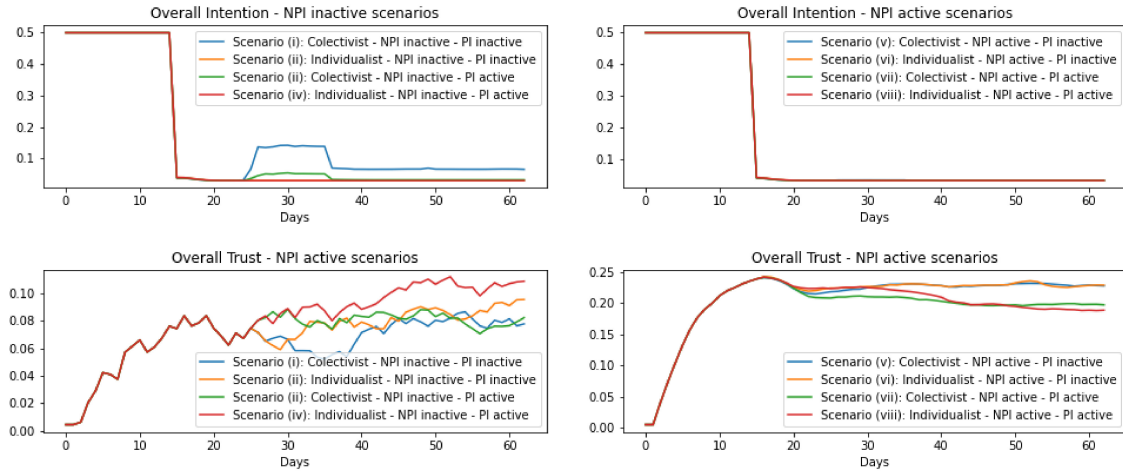


Figure 7.6.: **Evolution of population-level intention and trust across experimental scenarios, grouped by the presence or absence of information-based interventions.** The figure reveals that intention declines rapidly following the initial epidemic wave in all scenarios. However, trust increases sustainably when information-based interventions are active. These dynamics demonstrate that information exposure and cultural orientation influence deliberative vaccination behavior beyond the effects of epidemiological pressure.

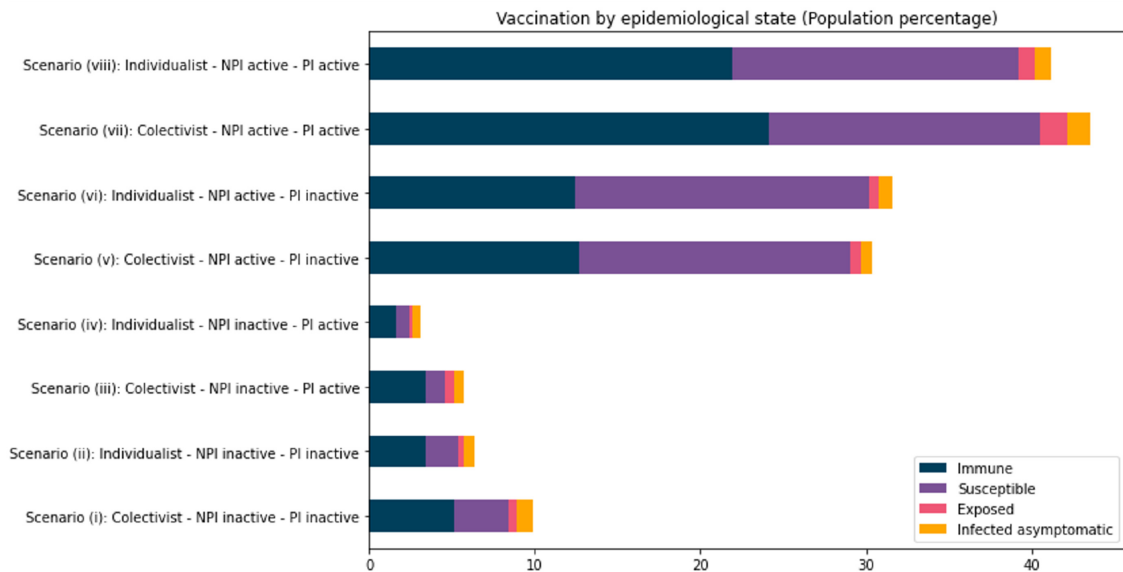


Figure 7.7.: **Distribution of vaccinated individuals by epidemiological state at the end of the simulation across all experimental scenarios.** Results are organized according to cultural orientation and the presence or absence of information-based interventions (representing NPI) and vaccine availability (representing PI). Scenarios with active information-based interventions exhibit higher vaccination among susceptible agents, reflecting preventive adoption. In contrast, scenarios lacking NPIs show vaccination concentrated among agents who have already experienced infection or exposure, reflecting a reactive adoption.

7.6.4. Summary of findings

The experimental results demonstrate that:

- Identical interventions can produce substantially different outcomes across populations.
- Information-based interventions primarily influence when vaccination occurs, not only how much vaccination occurs.
- Trust serves as a critical mediating variable between information exposure and vaccine adoption.
- Cultural orientation shapes both responsiveness to risk and sensitivity to information.

Collectively, these findings validate the modeling contributions introduced in this chapter and provide empirical support for employing human-centric evaluation metrics within the O-DMIoT framework.

7.7. Derived Human-Centric Metrics

For the vaccination case study, human-centric metrics are derived by instantiating the general QoE framework introduced in Chapter 6. Observable behavioral and epidemiological indicators generated by the agent-based simulation serve as the basis for these metrics, which are then computed across the experimental scenarios defined in Section 7.6.

The proposed QoE formulation captures the human-centered impact of information-based interventions by jointly considering vaccine acceptance, trust dynamics, and infection reduction. As a result, QoE functions as an integrative metric that reflects how populations experience and respond to ICT-mediated interventions over time.

7.7.1. QoE Instantiation

QoE is instantiated as a composite metric that aggregates multiple dimensions of human experience relevant to vaccination campaigns. These dimensions are selected to reflect the behavioral and perceptual factors emphasized throughout the study, specifically:

- Vaccine acceptance captures the population's willingness to be vaccinated.
- Trust dynamics reflect how confidence in vaccination evolves over time.
- Infection reduction represents the epidemiological consequences of behavioral decisions.

Each dimension is derived directly from simulation outputs, ensuring that QoE remains grounded in observable agent behavior rather than in abstract assumptions. This approach is consistent with the principle introduced in Chapter 6, which asserts that QoE should emerge from the interaction among human behavior, algorithmic mediation, and system-level outcomes.

7.7.2. Metric definition

Three quantitative components are used to construct the QoE metric.

- **Vaccine Acceptance**

Vaccine acceptance is defined as the proportion of the population that has accepted vaccination, regardless of whether vaccination has already been administered:

$$V_{\text{acceptance}} = \frac{\text{population that accepted the vaccine}}{\text{total population}} \quad (7.11)$$

This metric captures the translation of intention into commitment and serves as a direct indicator of behavioral alignment with the intervention objectives.

- **Trust Growth Rate**

To account for the dynamic nature of trust, a normalized trust growth metric is defined as:

$$T = \frac{\text{trust}_{\text{end}} - \text{trust}_{\text{initial}}}{\text{trust}_{\text{max}} - \text{trust}_{\text{initial}}} \quad (7.12)$$

This formulation emphasizes changes in trust rather than absolute trust levels, thereby enabling comparisons across scenarios with differing initial conditions.

- **Infection Reduction**

Infection reduction is measured as the relative change in infection levels over the course of the simulation:

$$I = \frac{\text{infections}_{\text{end}} - \text{infections}_{\text{initial}}}{\text{infections}_{\text{initial}}} \quad (7.13)$$

Since higher infection levels correspond to adverse outcomes, this component is incorporated as a penalization term in the QoE computation.

To ensure comparability, all components are normalized to the interval $[0, 1]$. The overall QoE score is subsequently computed as a weighted aggregation:

$$QoE = w_1 \cdot V_{\text{acceptance}} + w_2 \cdot T - w_3 \cdot I \quad (7.14)$$

In this formulation, w_1 , w_2 , and w_3 denote weighting factors that represent the relative importance of behavioral adoption, trust evolution, and epidemiological impact, respectively.

7.7.3. Scenario-Based Computation

The QoE metric is calculated for each of the eight experimental scenarios described in Section 7.6, which encompass combinations of cultural orientation, information-based intervention (NPI), and vaccine availability (PI). This scenario-based computation enables direct comparison of intervention performance under varying social and information exposure conditions.

QoE for every scenario is shown in table 7.2. Differences in QoE across scenarios can be attributed to variations in trust formation, timing of vaccine acceptance, and responsiveness to information exposure, rather than to random fluctuations in disease spread.

QoE does not replace traditional epidemiological metrics; instead, it complements them by providing a human-centric perspective for assessing intervention effectiveness. This feature is particularly relevant for adaptive IoT systems, where decisions must account for both human response and technical performance, and where evaluation metrics must support comparison, selection, and potential reconfiguration.

Scenario	Population	PI	NPI	$V_{acceptance}$	T	I	QoE
(i)	Colectivist	NA	NA	0.09938	0.85526	-0.66814	0.49677
(ii)	Individualist	NA	NA	0.06391	1.00005	-0.65634	0.52248
(iii)	Colectivist	A	NA	0.05896	1.41834	-0.64896	0.64378
(iv)	Individualist	A	NA	0.031340	0.96929	-0.64601	0.49712
(v)	Colectivist	NA	A	0.303505	0.94404	-0.61061	0.58780
(vi)	Individualist	NA	A	0.316288	0.94367	-0.66076	0.60784
(vii)	Colectivist	A	A	0.518350	0.75467	-0.63569	0.62445
(viii)	Individualist	A	A	0.411546	0.77468	-0.61356	0.58109

Table 7.2.: **Application of the proposed QoE metric across the eight simulation scenarios.** I denotes infection, T represents trust, and $V_{acceptance}$ indicates vaccine acceptance. NA signifies “Not Available” and A denotes “Available,” indicating whether interventions are active. QoE is calculated as $QoE = w_1V_{acceptance} + w_2T - w_3I$, where $w_1 = 0.4$, $w_2 = 0.3$, and $w_3 = 0.3$, reflecting the relative importance of vaccine acceptance, trust, and infection. Higher QoE values correspond to greater fulfillment of human-centered objectives.

7.8. Human-Centric Evaluation as QoE in O-DMIoT

Drawing on the instantiation and computation of QoE for the vaccination case study (Section 7.7), this section elucidates how QoE serves as a human-centric evaluation mechanism for information-based decision-making in adaptive IoT systems within the O-DMIoT framework.

The intervention examined in this chapter differs from prescriptive or optimization-driven algorithmic services. Rather than directly determining actions, it influences trust, intention, and perception through information dissemination. Consequently, its effectiveness cannot be evaluated solely using technical performance metrics or epidemiological indicators. QoE offers an abstraction that connects observed behavioral responses to system-level evaluation, facilitating population-sensitive comparisons of interventions whose impact arises from human decision-making rather than direct control.

7.8.1. Interpreting QoE Across O-DMIoT Layers

Within O-DMIoT, decision-making services are described across conceptual layers spanning problem characterization, design, execution, and evaluation. QoE provides a unifying evaluative signal that can be interpreted consistently across these layers without introducing domain-specific representations into the ontology.

Problem Layer

At the Problem Layer, O-DMIoT does not encode domain-specific models of vaccination, trust, or epidemiology. Instead, it provides abstract descriptors for decision-making problems, such as objectives, interaction types, and sources of uncertainty.

In this chapter, the epidemiological and behavioral problem context is modeled externally through an agent-based simulation. QoE functions as an evaluation artifact that projects domain-specific outcomes onto the abstract problem dimensions supported by the ontology.

From this perspective, differences in QoE across scenarios demonstrate that problem instances that differ in human interaction patterns and uncertainty profiles (when evaluated externally) lead to measurably different human-centered outcomes. QoE thus enables empirical assessment of how problem characterizations perform with respect to human behavior.

Design Layer

At the Design Layer, QoE enables comparison of alternative decision-making service configurations. The deliberative adoption mechanism utilized in this chapter interacts with trust dynamics and information exposure, resulting in distinct behavioral trajectories. QoE synthesizes the emergent effects of these interactions, demonstrating how design choices, such as enabling information-based interventions or incorporating trust-sensitive decision processes, lead to differences in experienced outcomes across populations. Thus, QoE serves as an indirect indicator of design suitability within a given socio-technical context, while remaining independent of algorithm specification.

Execution Layer

At the Execution Layer, QoE captures the cumulative effects of runtime dynamics, such as the timing of information dissemination, vaccine availability, and evolving epidemiological conditions. Although QoE is computed post-simulation in this study, it encapsulates execution-level phenomena including early versus delayed adoption and preventive versus reactive vaccination behavior.

This demonstrates the suitability of QoE as an execution-aware evaluation construct that reflects temporal and contextual variability, even when execution policies are not explicitly modeled within the ontology.

Metrics Layer

At the Metrics Layer, QoE formalizes evaluation by integrating diverse behavioral and epidemiological indicators into a unified comparative signal. QoE complements domain-specific metrics by supporting structured comparisons, rankings, and trade-off analyses under consistent contextual conditions.

This layered interpretability reinforces the role of QoE as a human-centric evaluation construct within O-DMIoT, supporting comparative reasoning without compromising the framework's domain independence.

7.8.2. Linking Observed Outcomes to Design and Execution Decisions

QoE offers a mechanism for linking observed simulation outcomes to design and execution decisions in adaptive IoT systems. The scenario-based results presented in Section 7.7 indicate that identical epidemiological conditions can produce substantially different QoE values depending on social behavior.

From a design perspective, QoE underscores the significance of integrating human-centric mechanisms into decision-making services. Scenarios featuring active information-based interventions consistently achieve higher QoE values, attributable not only to reduced

infection impact but also to earlier vaccine acceptance and sustained trust. These effects are not discernible from epidemiological indicators or technical criteria alone, highlighting the need to evaluate information-based services through experiential outcomes.

From an execution perspective, QoE reveals timing effects that static performance metrics fail to capture. Variations in vaccination timing, rather than solely in coverage levels, generate meaningful differences in QoE. Early trust formation and deliberative adoption foster preventive vaccination behavior, resulting in higher QoE even when final vaccination coverage is similar.

QoE enables these assessments without incorporating domain-specific logic into O-DMIoT. Simulation-derived QoE values can be linked to semantic descriptions of decision-making services, enabling O-DMIoT to support informed comparison and selection. Although this chapter does not implement adaptive reconfiguration, the results establish the evaluative foundation required for such mechanisms.

QoE serves in this chapter as more than a numerical outcome. It operates as a human-centric evaluation mechanism that integrates external problem modeling, algorithmic design choices, and execution dynamics through a unified evaluative signal. This interpretation reinforces the central claim of this work: effective evaluation of IoT decision-making services requires metrics that capture system actions and user experience.

7.9. Implications for O-DMIoT

The findings of this chapter have direct implications for the O-DMIoT framework in terms of problem characterization, algorithm design, evaluation metrics, and future adaptation mechanisms. Instead of proposing new decision-making algorithms, this case study demonstrates how O-DMIoT can be employed to assess the human-centered impact of existing decision-making services within socio-technical systems.

By evaluating deliberative decision-making, information exposure, and trust dynamics through external simulation and QoE-based metrics, this chapter demonstrates that algorithm assessment in IoT systems must consider human perception and response to interventions. The following implications clarify how O-DMIoT supports principled evaluation and comparison without incorporating domain-specific models.

7.9.1. Implications for Problem Characterization

This case study provides empirical evidence that decision-making problems involving human behavior cannot be adequately assessed using only technical or epidemiological objectives. In the vaccination context, uptake is determined by perception, trust, and deliberation, rather than by resource availability or scheduling constraints.

The observed differences in QoE across scenarios indicate that interventions appearing equivalent under epidemiological indicators can result in substantially different human-centered outcomes when population characteristics vary. These results do not suggest that O-DMIoT represents such problem details internally. Rather, they demonstrate the necessity of external problem modeling combined with evaluation artifacts that can project outcomes onto abstract problem dimensions.

This supports the role of O-DMIoT as a framework for reasoning about problem characteristics, such as interaction type and uncertainty, while remaining agnostic to domain-specific representations.

7.9.2. Implications for Algorithm Design and Selection

At the Design Layer, the results suggest that the effectiveness of a decision-making service depends not only on its computational structure but also on its interaction with human cognition and social context. The deliberative decision model introduced in this chapter is not presented as a novel algorithmic contribution; rather, it serves as an analytical instrument to reveal behavioral and temporal effects. The evaluation based on QoE shows that enabling information-mediated decision-making processes leads to earlier, more preventive adoption under certain population configurations, even when epidemiological conditions remain unchanged. This finding highlights the importance of selecting decision-making services based on interaction characteristics and behavioral assumptions, rather than solely on optimization efficiency or predictive accuracy.

These results support the proposed design taxonomy TDMIoT and demonstrate that distinctions based on interaction type yield tangible consequences for human-centered outcomes.

7.9.3. Implications for Metrics and Evaluation

At the Metrics Layer, the chapter demonstrates that the previously introduced QoE formulation gains interpretive significance when applied to behavioral and epidemiological dynamics. In this context, QoE consolidates diverse simulation outputs into a unified evaluative signal that reflects emergent human-centered effects resulting from the interplay of information exposure, trust formation, and decision-making.

Instead of proposing a new evaluative principle, this approach demonstrates how QoE operationalizes relative utility through observable behavioral outcomes. Comparisons among algorithmic configurations are thus based not only on technical performance but also on their distinct effects on population-level behavioral trajectories. The weighting of behavioral and epidemiological components allows the evaluation process to accommodate varying stakeholder priorities and policy perspectives within a unified framework.

7.9.4. Implications for Adaptation and Reconfiguration Mechanisms

Although runtime adaptation is not implemented in this chapter, the observed variation in QoE across scenarios provides insight into the type of feedback that could inform reconfiguration mechanisms within O-DMIoT. Variations in QoE demonstrate that different configurations of the same decision-making service can produce measurably distinct human-centered outcomes, even when environmental conditions remain constant.

In this context, QoE does not introduce new adaptation capabilities but instead characterizes the evaluative signal that would inform such mechanisms. When anchored in behavioral and epidemiological dynamics, QoE captures both performance differences and the broader societal implications of configuration choices. This feedback may be utilized by external reasoning mechanisms to determine when adjustments to service activation or deployment strategies are necessary, thereby avoiding the need to embed domain-specific adaptation logic within the services themselves.

7.10. Conclusions

This chapter extends the human-centric evaluation framework introduced in Chapter 6 by applying it to an information-based vaccination intervention conceptualized as a socio-technical decision-making service. Using an agent-based simulation that integrates epi-

demiological dynamics, information exposure, trust formation, and deliberative adoption behavior, the analysis shows how population-level outcomes arise from individual decision-making processes determined by trust, perceptions, and cultural context.

The main contribution of this chapter is the integration of existing models, rather than the introduction of new epidemiological or behavioral theories. Epidemiological dynamics are adapted from established compartmental models, and intention and trust formulations are based on prior research. The novelty resides in their coordinated operationalization within a unified agent-based framework, where information explicitly drives trust, trust influences deliberative decision-making, and individual decisions subsequently affect epidemiological evolution. This integration promotes the study of interventions mediated by human cognition and social interaction, rather than by direct algorithmic control.

Additionally, this chapter presents a behaviorally grounded operationalization of Decision Field Theory within the context of epidemic simulation. By embedding deliberative preference accumulation into the vaccination decision process, the model captures hesitation, delayed adoption, and reversible preferences, which are not represented in static or threshold-based adoption models. This approach enables analysis of timing effects and preventive behaviors, which are critical for understanding the impact of information-based interventions.

Building on these modeling elements, the chapter introduces QoE as a human-centric evaluation metric specifically designed for vaccination interventions. In contrast to traditional epidemiological metrics, QoE aggregates behavioral adoption, trust dynamics, and infection impact into a single evaluative signal. This approach shows how heterogeneous simulation outputs can be transformed into comparative metrics that reflect population experiences of interventions over time, rather than only their final epidemiological outcomes.

Consistent with Chapter 6, QoE is interpreted as a relative, scenario-dependent evaluation signal rather than an absolute measure of performance. Applying QoE across populations with different cultural orientations reveals that identical interventions can produce substantially different outcomes, underscoring the necessity for population-sensitive evaluation in adaptive IoT systems. Notably, this evaluation is accomplished without incorporating domain-specific representations into O-DMIoT, thereby preserving its ontology-level generality.

This chapter demonstrates how human-centric evaluation can be operationalized within O-DMIoT without compromising domain independence. By integrating agent-based simulation, deliberative decision-making, and QoE-based evaluation, the study provides concrete evidence that human experience must be treated as an explicit evaluation target in IoT systems, particularly for interventions whose effectiveness emerges through trust, perception, and deliberation.

These conclusions reinforce the broader contribution of this thesis, which is to enable systematic, human-centered reasoning about decision-making services in adaptive IoT systems through ontology-driven evaluation rather than domain-specific modeling. Alongside Chapter 6, this chapter establishes a coherent methodological foundation for assessing and comparing IoT decision-making services based on their emergent human impact, thereby supporting informed design, selection, and future adaptation.

References

- [1] I. Ajzen. “From Intentions to Actions: A Theory of Planned Behavior”. In: *Action Control: From Cognition to Behavior*. Berlin, Heidelberg: Springer Berlin Heidelberg, 1985, pp. 11–39. URL: https://doi.org/10.1007/978-3-642-69746-3_5C_2.

- [2] I. Ajzen. “The theory of planned behavior”. en. In: *Organizational Behavior and Human Decision Processes*. Theories of Cognitive Self-Regulation 50.2 (Dec. 1991), pp. 179–211. URL: <https://www.sciencedirect.com/science/article/pii/074959789190020T> (visited on 02/13/2023).
- [3] M. Al Atiqi, S. Chang, and H. Deguchi. “Simulating the influence of Facebook fan pages on individual attitudes toward vaccination using agent-based modelling”. en. In: *Systems Research and Behavioral Science* 40.3 (May 2023), pp. 595–610. URL: <https://onlinelibrary.wiley.com/doi/10.1002/sres.2889> (visited on 04/23/2024).
- [4] A. Aleta et al. “Modelling the impact of testing, contact tracing and household quarantine on second waves of COVID-19”. en. In: *Nature Human Behaviour* 4.9 (Aug. 2020), pp. 964–971. URL: <https://www.nature.com/articles/s41562-020-0931-9> (visited on 04/23/2024).
- [5] J. Almagor and S. Picascia. “Exploring the effectiveness of a COVID-19 contact tracing app using an agent-based model”. In: *Scientific Reports* 10.1 (Dec. 2020). URL: <https://doi.org/10.1038/s41598-020-79000-y>.
- [6] A. M. L. Amit et al. “COVID-19 vaccine brand hesitancy and other challenges to vaccination in the Philippines”. In: *PLOS Global Public Health* 2.1 (Jan. 2022). Ed. by D. Benjumea-Bedoya, e0000165. URL: <https://doi.org/10.1371/journal.pgph.0000165>.
- [7] J. T. Beerman, G. G. Beaumont, and P. J. Giabbanelli. “On the Necessity of Human Decision-Making Errors to Explain Vaccination Rates for Covid-19: an Agent-Based Modeling Study”. In: *2023 Annual Modeling and Simulation Conference (ANNSIM)*. May 2023, pp. 413–424. URL: <https://ieeexplore.ieee.org/document/10155377> (visited on 02/27/2024).
- [8] P. Bhattacharya et al. “AI-Driven Agent-Based Models to Study the Role of Vaccine Acceptance in Controlling COVID-19 Spread in the US”. In: *2021 IEEE International Conference on Big Data (Big Data)*. IEEE, Dec. 2021. URL: <https://doi.org/10.1109/bigdata52589.2021.9671811>.
- [9] J. R. Busemeyer and J. T. Townsend. “Decision field theory: A dynamic-cognitive approach to decision making in an uncertain environment.” In: *Psychological Review* 100.3 (1993), pp. 432–459. URL: <https://doi.org/10.1037/0033-295x.100.3.432>.
- [10] E. L. Campos et al. “Multi-generational SIR modeling: Determination of parameters, epidemiological forecasting and age-dependent vaccination policies”. en. In: *Infectious Disease Modelling* 6 (2021), pp. 751–765. URL: <https://linkinghub.elsevier.com/retrieve/pii/S2468042721000373> (visited on 04/23/2024).
- [11] B. M. Castro et al. “Multi-agent simulation model for the evaluation of COVID-19 transmission”. en. In: *Computers in Biology and Medicine* 136 (Sept. 2021), p. 104645. URL: <https://linkinghub.elsevier.com/retrieve/pii/S001048252100439X> (visited on 04/23/2024).

- [12] A. Cattaneo et al. “An agent-based model to assess large-scale COVID-19 vaccination campaigns for the Italian territory: The case study of Lombardy region”. en. In: *Computer Methods and Programs in Biomedicine* 224 (Sept. 2022), p. 107029. URL: <https://www.sciencedirect.com/science/article/pii/S0169260722004114> (visited on 04/27/2023).
- [13] Y. Chai, Y. Wang, and L. Zhu. “A Stochastic Information Diffusion Model in Complex Social Networks”. In: *IEEE Access* 7 (2019), pp. 175897–175906. URL: <https://ieeexplore.ieee.org/document/8924669/> (visited on 10/26/2022).
- [14] P. Ciunkiewicz et al. “Agent-based epidemiological modeling of COVID-19 in localized environments”. en. In: *Computers in Biology and Medicine* 144 (May 2022), p. 105396. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0010482522001883> (visited on 04/23/2024).
- [15] *Covid World Vaccination Tracker*. The New York Times. 2021. URL: <https://www.nytimes.com/interactive/2021/world/covid-vaccinations-tracker.html> (visited on 03/02/2022).
- [16] E. Cuevas. “An agent-based model to evaluate the COVID-19 transmission risks in facilities”. en. In: *Computers in Biology and Medicine* 121 (June 2020), p. 103827. URL: <https://linkinghub.elsevier.com/retrieve/pii/S001048252030192X> (visited on 04/23/2024).
- [17] R. P. Curiel and H. G. Ramírez. “Vaccination strategies against COVID-19 and the diffusion of anti-vaccination views”. In: *Scientific Reports* 11.1 (Mar. 2021). URL: <https://doi.org/10.1038/s41598-021-85555-1>.
- [18] *Encuesta Nacional sobre Disponibilidad y Uso de Tecnologías de la Información en los Hogares (ENDUTIH) 2021. (Comunicado de prensa) 4 de julio / Instituto Federal de Telecomunicaciones - IFT*. URL: <https://www.ift.org.mx/comunicacion-y-medios/comunicados-ift/es/encuesta-nacional-sobre-disponibilidad-y-uso-de-tecnologias-de-la-informacion-en-los-hogares-endutih> (visited on 10/27/2022).
- [19] B. Faucher et al. “Agent-based modelling of reactive vaccination of workplaces and schools against COVID-19”. In: *Nature Communications* 13.1 (Mar. 2022). URL: <https://doi.org/10.1038/s41467-022-29015-y>.
- [20] E. A. Ferguson et al. “Modelling how face masks and symptoms-based quarantine synergistically and cost-effectively reduce SARS-CoV-2 transmission in Bangladesh”. en. In: *Epidemics* 40 (Sept. 2022), p. 100592. URL: <https://linkinghub.elsevier.com/retrieve/pii/S1755436522000391> (visited on 04/23/2024).
- [21] B. K. Fosdick et al. “Model-based evaluation of policy impacts and the continued COVID-19 risk at long term care facilities”. en. In: *Infectious Disease Modelling* 7.3 (Sept. 2022), pp. 463–472. URL: <https://linkinghub.elsevier.com/retrieve/pii/S2468042722000549> (visited on 04/23/2024).
- [22] Gobierno de Jalisco. *Ruta de la vacunación – Gobierno de Jalisco*. 2022. URL: <https://coronavirus.jalisco.gob.mx/ruta-de-la-vacunacion/> (visited on 10/27/2022).

- [23] N. Groves-Kirkby et al. “Large-scale calibration and simulation of COVID-19 epidemiologic scenarios to support healthcare planning”. en. In: *Epidemics* 42 (Mar. 2023), p. 100662. URL: <https://linkinghub.elsevier.com/retrieve/pii/S1755436522001025> (visited on 04/23/2024).
- [24] E. Hernández-Orallo et al. “Evaluating the Effectiveness of COVID-19 Bluetooth-Based Smartphone Contact Tracing Applications”. en. In: *Applied Sciences* 10.20 (Oct. 2020), p. 7113. URL: <https://www.mdpi.com/2076-3417/10/20/7113> (visited on 05/20/2021).
- [25] *Infodemic*. en. <https://www.who.int/westernpacific/health-topics/infodemic>. [Online; accessed 05-March-2022]. URL: <https://www.who.int/westernpacific/health-topics/infodemic> (visited on 03/05/2022).
- [26] S. W. Jaffry and J. Treur. “Agent-Based and Population-Based Modeling of Trust Dynamics”. In: *Transactions on Computational Collective Intelligence IX*. Springer Berlin Heidelberg, 2013, pp. 124–151. URL: https://doi.org/10.1007/978-3-642-36815-8_5C_6.
- [27] W. Jennings et al. “Lack of Trust, Conspiracy Beliefs, and Social Media Use Predict COVID-19 Vaccine Hesitancy”. en. In: *Vaccines* 9.6 (June 2021), p. 593. URL: <https://www.mdpi.com/2076-393X/9/6/593> (visited on 03/02/2022).
- [28] R. Kachurka, M. Krawczyk, and J. Rachubik. “Persuasive Messages Will Not Increase COVID-19 Vaccine Acceptance: Evidence from a Nationwide Online Experiment”. en. In: *Vaccines* 9.10 (Sept. 2021), p. 1113. URL: <https://www.mdpi.com/2076-393X/9/10/1113> (visited on 10/26/2022).
- [29] P. D. M. C. Katoto et al. “Predictors of COVID-19 Vaccine Hesitancy in South African Local Communities: The VaxScenes Study”. In: *Vaccines* 10.3 (Feb. 2022), p. 353. URL: <https://doi.org/10.3390/vaccines10030353>.
- [30] C. C. Kerr et al. “Covasim: An agent-based model of COVID-19 dynamics and interventions”. en. In: *PLOS Computational Biology* 17.7 (July 2021). Ed. by M. Marz, e1009149. URL: <https://dx.plos.org/10.1371/journal.pcbi.1009149> (visited on 04/28/2023).
- [31] J. R. Kerr et al. “Effect of Information about COVID-19 Vaccine Effectiveness and Side Effects on Behavioural Intentions: Two Online Experiments”. eng. In: *Vaccines* 9.4 (Apr. 2021), p. 379.
- [32] C. A. Latkin et al. “Trust in a COVID-19 vaccine in the U.S.: A social-ecological perspective”. In: *Social Science & Medicine* 270 (Feb. 2021), p. 113684. URL: <https://doi.org/10.1016/j.socscimed.2021.113684>.
- [33] R. Latkowski and B. Dunin-Kępicz. “An Agent-Based Covid-19 Simulator: Extending Covasim to the Polish Context”. en. In: *Procedia Computer Science* 192 (2021), pp. 3607–3616. URL: <https://linkinghub.elsevier.com/retrieve/pii/S1877050921018731> (visited on 04/23/2024).
- [34] S. Lee et al. *COVID-19 Endemic Plan: Impact of Vaccination and Non-pharmaceutical Interventions with Viral Variants and Waning Immunity Using an Agent-Based Simulation*. en. June 2022. URL: <http://medrxiv.org/lookup/doi/10.1101/2022.06.03.22275834> (visited on 04/23/2024).

- [35] S. Lee, Y.-J. Son, and J. Jin. “Decision field theory extensions for behavior modeling in dynamic environment using Bayesian belief network”. In: *Information Sciences* 178.10 (May 2008), pp. 2297–2314. URL: <https://doi.org/10.1016/j.ins.2008.01.009>.
- [36] A. A. Malik et al. “Determinants of COVID-19 vaccine acceptance in the US”. In: *EClinicalMedicine* 26 (Sept. 2020), p. 100495. URL: <https://doi.org/10.1016/j.eclinm.2020.100495>.
- [37] S. Manson, S. Sun, and D. Bonsal. “Agent-Based Modeling and Complexity”. In: *Agent-based Models of Geographical Systems*. Journal Abbreviation: Agent-based Models of Geographical Systems. Oct. 2012, pp. 125–139.
- [38] L. R. Martin and K. J. Petrie. “Understanding the Dimensions of Anti-Vaccination Attitudes: the Vaccination Attitudes Examination (VAX) Scale”. en. In: *Annals of Behavioral Medicine* 51.5 (Oct. 2017), pp. 652–660. URL: <https://academic.oup.com/abm/article/51/5/652-660/4648576> (visited on 03/02/2022).
- [39] A. D. Medus and C. O. Dorso. “Vaccination and public trust: a model for the dissemination of vaccination behavior with external intervention”. In: *arXiv preprint arXiv:1412.0583* (2014).
- [40] A. N. Micuda et al. “Exploring a targeted approach for public health capacity restrictions during COVID-19 using a new computational model”. en. In: *Infectious Disease Modelling* 9.1 (Mar. 2024), pp. 234–244. URL: <https://linkinghub.elsevier.com/retrieve/pii/S2468042724000010> (visited on 04/23/2024).
- [41] S. M. Moghadas et al. “The Impact of Vaccination on Coronavirus Disease 2019 (COVID-19) Outbreaks in the United States”. en. In: *Clinical Infectious Diseases* 73.12 (Dec. 2021), pp. 2257–2264. URL: <https://academic.oup.com/cid/article/73/12/2257/6124429> (visited on 04/23/2024).
- [42] S. Moore et al. “Projecting the future impact of emerging SARS-CoV-2 variants under uncertainty: Modeling the initial Omicron outbreak”. en. In: *Epidemics* 47 (June 2024), p. 100759. URL: <https://linkinghub.elsevier.com/retrieve/pii/S1755436524000203> (visited on 04/23/2024).
- [43] H. S. Mortveit et al. “BESSIE: A Behavior and Epidemic Simulator for Use With Synthetic Populations”. In: (2022). Publisher: [object Object] Version Number: 1. URL: <https://arxiv.org/abs/2203.11414> (visited on 04/23/2024).
- [44] G. Muric, Y. Wu, and E. Ferrara. “COVID-19 Vaccine Hesitancy on Social Media: Building a Public Twitter Data Set of Antivaccine Content, Vaccine Misinformation, and Conspiracies”. In: *JMIR Public Health and Surveillance* 7.11 (Nov. 2021), e30642. URL: <https://doi.org/10.2196/30642>.
- [45] P. T. Naderi et al. *COVID-19 Vaccine Hesitancy and Information Diffusion: An Agent-based Modeling Approach*. arXiv:2109.01182 [physics]. Sept. 2021. URL: <http://arxiv.org/abs/2109.01182> (visited on 04/23/2024).
- [46] H. Nunner et al. “Health behavior homophily can mitigate the spread of infectious diseases in small-world networks”. en. In: *Social Science & Medicine* 312 (Nov. 2022), p. 115350. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0277953622006566> (visited on 04/23/2024).

- [47] K. Okita et al. “Stochastic resonance effect observed in a vaccination game with effectiveness framework obeying the SIR process on a scale-free network”. en. In: *Chaos, Solitons & Fractals* 167 (Feb. 2023), p. 113029. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0960077922012085> (visited on 04/23/2024).
- [48] S. B. Omer et al. “Promoting COVID-19 vaccine acceptance: recommendations from the Lancet Commission on Vaccine Refusal, Acceptance, and Demand in the USA”. In: *The Lancet* 398.10317 (Dec. 2021), pp. 2186–2192. URL: [https://doi.org/10.1016/s0140-6736\(21\)02507-1](https://doi.org/10.1016/s0140-6736(21)02507-1).
- [49] C. M. Pais et al. “City-scale model for COVID-19 epidemiology with mobility and social activities represented by a set of hidden Markov models”. en. In: *Computers in Biology and Medicine* 160 (June 2023), p. 106942. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0010482523004079> (visited on 04/23/2024).
- [50] G. A. Palomo-Briones, M. Siller, and A. Grignard. “An agent-based model of the dual causality between individual and collective behaviors in an epidemic”. en. In: *Comput. Biol. Med.* 141.104995 (Feb. 2022), p. 104995.
- [51] M. Patón, J. M. Acuña, and J. Rodríguez. “Evaluation of vaccine rollout strategies for emerging infectious diseases: A model-based approach including protection attitudes”. en. In: *Infectious Disease Modelling* 8.4 (Dec. 2023), pp. 1032–1049. URL: <https://linkinghub.elsevier.com/retrieve/pii/S2468042723000702> (visited on 04/23/2024).
- [52] R. G. Qiu, E. Wang, and I. Gong. “Data-Driven Modeling to Facilitate Policymaking in Fighting to Contain the COVID-19 Pandemic”. en. In: *Procedia Computer Science* 185 (2021), pp. 320–329. URL: <https://linkinghub.elsevier.com/retrieve/pii/S1877050921011169> (visited on 04/23/2024).
- [53] A. Rodríguez et al. “An agent-based transmission model of COVID-19 for re-opening policy design”. en. In: *Computers in Biology and Medicine* 148 (Sept. 2022), p. 105847. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0010482522006035> (visited on 04/23/2024).
- [54] A. A. Romanyukha et al. “The trade-off between COVID-19 and mental diseases burden during a lockdown: Mathematical modeling of control measures”. en. In: *Infectious Disease Modelling* 8.2 (June 2023), pp. 403–414. URL: <https://linkinghub.elsevier.com/retrieve/pii/S2468042723000283> (visited on 04/23/2024).
- [55] E. Serrano, C. Á. Iglesias, and M. Garijo. “A Novel Agent-Based Rumor Spreading Model in Twitter”. In: *Proceedings of the 24th International Conference on World Wide Web. WWW ’15 Companion*. New York, NY, USA: Association for Computing Machinery, May 2015, pp. 811–814. URL: <https://doi.org/10.1145/2740908.2742466> (visited on 10/27/2022).
- [56] P. C. Silva et al. “COVID-ABS: An agent-based model of COVID-19 epidemic to simulate health and economic effects of social distancing interventions”. en. In: *Chaos, Solitons & Fractals* 139 (Oct. 2020), p. 110088. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0960077920304859> (visited on 04/23/2024).

- [57] P. Sobkowicz and A. Sobkowicz. “Agent Based Model of Anti-Vaccination Movements: Simulations and Comparison with Empirical Data”. en. In: *Vaccines* 9.8 (July 2021), p. 809. URL: <https://www.mdpi.com/2076-393X/9/8/809> (visited on 03/02/2022).
- [58] F. Spooner et al. “A dynamic microsimulation model for epidemics”. en. In: *Social Science & Medicine* 291 (Dec. 2021), p. 114461. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0277953621007930> (visited on 04/23/2024).
- [59] A. Srivastava. “The variations of SIkJalpha model for COVID-19 forecasting and scenario projections”. en. In: *Epidemics* 45 (Dec. 2023), p. 100729. URL: <https://linkinghub.elsevier.com/retrieve/pii/S1755436523000658> (visited on 04/23/2024).
- [60] P. G. Szilagyi et al. “The role of trust in the likelihood of receiving a COVID-19 vaccine: Results from a national survey”. In: *Preventive Medicine* 153 (Dec. 2021), p. 106727. URL: <https://doi.org/10.1016/j.ypmed.2021.106727>.
- [61] F. Trad and S. E. Falou. “Testing Different COVID-19 Vaccination Strategies Using an Agent-Based Modeling Approach”. In: *SN Computer Science* 3.4 (May 2022). URL: <https://doi.org/10.1007/s42979-022-01199-6>.
- [62] S. Utsumi et al. “How and to what extent does the anti-social behavior of violating self-quarantine measures increase the spread of disease?” en. In: *Chaos, Solitons & Fractals* 159 (June 2022), p. 112178. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0960077922003885> (visited on 04/23/2024).
- [63] J. A. Vandello and D. Cohen. “Patterns of individualism and collectivism across the United States.” In: *Journal of personality and social psychology* 77.2 (1999), p. 279.
- [64] L. D. Williamson and A. Tarfa. “Examining the relationships between trust in providers and information, mistrust, and COVID-19 vaccine concerns, necessity, and intentions”. In: *BMC Public Health* 22.1 (Nov. 2022). URL: <http://dx.doi.org/10.1186/s12889-022-14399-9>.
- [65] L. P. Wong et al. “COVID-19 Anti-Vaccine Sentiments: Analyses of Comments from Social Media”. In: *Healthcare* 9.11 (Nov. 2021), p. 1530. URL: <https://doi.org/10.3390/healthcare9111530>.
- [66] J. Woo, J. Son, and H. Chen. “An SIR model for violent topic diffusion in social media”. In: *Proceedings of 2011 IEEE International Conference on Intelligence and Security Informatics*. July 2011, pp. 15–19.
- [67] O. Yaqub et al. “Attitudes to vaccination: a critical review”. In: *Social science & medicine* 112 (2014), pp. 1–11.
- [68] X. Yuan and A. Crooks. “From cyber space opinion leaders and the diffusion of anti-vaccine extremism to physical space disease outbreaks”. In: *International conference on social computing, behavioral-cultural modeling and prediction and behavior representation in modeling and simulation*. Springer. 2017, pp. 114–119.

8. Algorithm Selection Based on a Semantic Model for Decision-Making in the IoT

Abstract. Algorithm selection represents a fundamental challenge in decision-making systems and is traditionally approached as a one-shot mapping from problem instances to algorithms based on performance criteria. However, in Internet of Things (IoT) environments, this approach is increasingly insufficient due to heterogeneous data sources, dynamic execution contexts, evolving objectives, and multiple sources of uncertainty. This chapter introduces a semantic-driven formulation of algorithm selection as a planning problem. Both decision problems and algorithmic solutions are explicitly defined using the O-DMIoT ontology, which enables algorithm selection to be guided by semantic compatibility rather than solely by empirical performance. The selection process is formalized as a Markov Decision Process and implemented using a Monte Carlo Tree Search algorithm guided by Thompson Sampling. Semantic reasoning is integrated throughout all phases of the search process, constraining the action space and guiding evaluation. A proof of concept demonstrates the feasibility of the proposed modeling and integration strategy within an IoT decision-making scenario, with an emphasis on explainability and adaptability rather than competitive performance.

8.1. Introduction

Algorithm selection addresses the problem of identifying an appropriate algorithmic solution for a given decision problem. Traditionally, this problem has been studied as a mapping from problem instances to algorithms that optimize a performance criterion [9]. Subsequent work has extended this formulation through learning-based performance models, problem and algorithm characterizations, and statistical decision mechanisms. Despite these advances, algorithm selection is still predominantly treated as a one-shot recommendation problem, in which a single algorithm is selected based on offline features or prior performance estimates.

As established in Chapter 1, IoT systems function as cyber-physical infrastructures that influence human, social, and environmental processes through software-driven decision-making. Within this context, algorithms are deployed as services that must operate with heterogeneous data sources, shifting objectives, and dynamic execution environments. Therefore, algorithm selection in IoT systems should not be regarded as a static, design-time decision. Rather, it constitutes an adaptive decision problem that requires consideration of problem semantics, algorithmic assumptions, execution constraints, and various sources of uncertainty.

This chapter argues that algorithm selection in IoT systems should be conceptualized as a *planning problem*. In this framework, selection occurs through a sequence of decisions that incrementally refine the set of admissible solutions. Each decision imposes semantic constraints, thereby reducing uncertainty and narrowing the range of compatible algorithm designs. This approach diverges from traditional recommendation-based methods and instead aligns algorithm selection with the principles of sequential decision-making under uncertainty.

To support this formulation, the chapter introduces a semantic model for algorithm selection grounded in ***O-DMIoT***, an ontology developed to characterize decision problems and algorithmic solutions within IoT contexts. This ontology offers explicit representations of problem objectives, interaction modalities, processing characteristics, knowledge structures, and types of uncertainty, along with complementary descriptions of algorithmic approaches, techniques, and instantiations. By articulating these dimensions, algorithm selection can be guided by semantic compatibility and system requirements, rather than relying exclusively on empirical performance.

Building on this semantic foundation, algorithm selection is formalized as a Markov Decision Process (MDP) and implemented using a semantic-driven Monte Carlo Tree Search (MCTS) algorithm guided by Thompson Sampling. Semantic reasoning is integrated throughout the search process, including the definition of the initial state, the constraint of the action space, the guidance of exploration, and the computation of rewards. The chapter concludes with a proof of concept that demonstrates the feasibility of this modeling and integration strategy within an IoT decision-making scenario. The primary objective is not to demonstrate performance superiority, but to validate algorithm selection as a semantic-driven planning process that enables adaptive and explainable decision-making in IoT systems [2, 5].

8.2. Algorithm Selection: Background and Related Work

Algorithm selection involves identifying the most suitable algorithm for a specific problem instance, dataset, or application context. This need is driven by the established observation that no single algorithm consistently outperforms all others across diverse instances, particularly in heterogeneous and dynamic environments. The complexity of this challenge increases in IoT systems, where decision-making algorithms must operate under varying data characteristics, objectives, and system constraints.

The algorithm selection problem was formally introduced by Rice as a mapping from problem instances to algorithms that optimize a given performance metric [9]. Subsequent research has expanded this formulation to encompass instance-level selection, optimization, and learning-based approaches. Within the machine learning discipline, algorithm selection is commonly framed as a learning problem that relates dataset characteristics, algorithmic spaces, and observed performance outcomes. In this context, solution paradigms such as meta-learning and surrogate modeling have emerged as dominant approaches for predicting algorithm performance and guiding selection decisions [10].

Building on this learning-based perspective, a comprehensive survey of automated algorithm selection methods for optimization and decision problems highlights the importance of informative, interpretable, and computationally efficient instance features for effective performance prediction [7]. The survey frames algorithm selection primarily as a feature-driven modeling task, adopting a domain-independent approach that emphasizes feature design, performance estimation, and evaluation methodologies. Consequently, algorithm selection is predominantly addressed as an offline learning or prediction problem, with

semantic knowledge representation, contextual reasoning, and runtime decision-making in highly dynamic environments such as IoT systems explicitly excluded from the survey scope.

8.2.1. Theoretical Foundations of Algorithm Selection

Early work on algorithm selection emphasized theoretical analyses and empirical performance comparisons. A formal learning treatment is presented by Gupta and Roughgarden, who model the algorithm selection problem within the Probably Approximately Correct (PAC) framework, establishing conditions under which an algorithm can be learned to perform near-optimally for a given application domain, with guarantees on sample complexity and generalization [3]. While this formulation provides a strong theoretical foundation, it assumes well-defined instance distributions and frames algorithm selection as a static learning problem. Consequently, it does not explicitly account for application semantics, runtime adaptation, or uncertainty, nor does it address the evolving objectives characteristic of highly dynamic environments such as IoT systems.

8.2.2. Semantic and Knowledge-Based Characterization Approaches

To address the limitations of statistical or feature-based approaches, several studies have explored semantic and knowledge-driven representations of algorithms and problems. One of the earliest examples is the ontology-based meta-mining framework, which centers on the Data Mining Optimization Ontology (DMOP) [4]. DMOP extends Rice's framework by offering explicit semantic characterizations of data mining tasks, algorithms, models, and workflows. This enables reasoning not only about which algorithm performs well, but also about the underlying factors contributing to its effectiveness in specific problem classes. DMOP illustrates the potential of ontologies to increase transparency in algorithm behavior and to support meta-learning and workflow analysis. However, its primary focus is on offline data mining processes. Consequently, mechanisms for runtime adaptation are not addressed.

A similar focus on semantic modeling appears in service-oriented IoT research, with the Physical Service Model (PSM), which describes IoT services using device characteristics, spatial and temporal constraints, and Quality of Service (QoS) attributes [6]. Their results indicate that ontological descriptions can improve service selection by enabling filtering and ranking based on user preferences and execution constraints. However, this approach considers algorithmic behavior only implicitly and does not conceptualize algorithm selection as an independent decision-making process.

8.2.3. Algorithm Selection in IoT and Dynamic Environments

Within IoT systems, algorithm selection becomes significantly more complex due to heterogeneous data sources, evolving data streams, and changing application goals. Consequently, several studies address algorithm selection in IoT by focusing on specific tasks such as data mining, anomaly detection, and stream processing.

Poormohammady et al. introduce a dynamic algorithm selection mechanism for IoT stream processing systems, selecting logic operators according to task characteristics and system conditions [8]. This approach supports runtime adaptation to improve performance metrics such as latency and throughput. Nevertheless, algorithm selection is mainly driven by low-level execution metrics and remains tightly coupled with the underlying stream processing framework, limiting its generalizability across diverse application domains.

Similarly, Singh et al. seek to enhance QoS in practical IoT applications through adaptive algorithm mechanisms [12]. Although their work highlights the importance of adaptation in IoT environments, algorithm selection is considered a component of system tuning rather than as a structured decision-making problem grounded in explicit problem and algorithm characterization.

Other IoT-focused approaches address specific application domains. Shafiq et al. propose a framework for selecting machine learning algorithms for IoT network traffic anomaly detection using a bijective soft set approach [11]. Algorithms are evaluated against a predefined set of categorical metrics, and selection is performed through a mathematical decision-making process. Although structured, this approach assumes fixed evaluation criteria and does not account for scenarios in which multiple algorithms satisfy the same requirements or when system conditions change dynamically.

Similarly, Yang et al. develop an algorithm selection service for IoT stream data that employs feature extraction with Tsfresh and supervised learning [13]. In this framework, algorithm selection is cast as a classification problem, with historical data streams labeled by the best-performing algorithm. While effective within its defined scope, the selection process is statistical, trained offline, and tailored to specific tasks, which restricts its applicability to broader and more adaptive IoT decision-making contexts.

8.2.4. Learning-Based and Decision-Theoretic Perspectives

Beyond IoT-specific research, algorithm selection has also been conceptualized as a sequential decision-making problem. For instance, algorithm selection has been modeled as a multi-armed bandit problem, where each algorithm corresponds to an arm and execution times are interpreted as losses [2, 1]. This approach enables online learning and adaptation from partial feedback, but it does not incorporate semantic knowledge about problem structure or algorithm properties.

Recent studies have demonstrated the effectiveness of MCTS for navigating large combinatorial decision spaces in automatic feature engineering [5]. In this line of research, feature transformation sequences are modeled as a Markov decision process, and MCTS is combined with Thompson sampling to balance exploration and exploitation. Although this approach does not directly address algorithm selection, it illustrates the capacity of MCTS to integrate learning, uncertainty, and sequential decision-making in complex optimization contexts.

8.2.5. Identified Research Gap

Across the literature, several limitations consistently emerge. First, many algorithm selection approaches rely on statistical or heuristic feature representations, lacking explicit semantic models that capture problem objectives, algorithmic assumptions, and contextual constraints. Second, algorithm selection is often treated as a one-shot recommendation problem rather than as a sequential decision-making process under uncertainty. Third, surveys of the field explicitly acknowledge that existing approaches predominantly operate offline and provide limited support for runtime adaptation or semantic reasoning in dynamic environments such as IoT systems [10]. These limitations prevent algorithm selection mechanisms from being integrated as adaptive services within autonomic IoT architectures.

8.3. Semantic Model for Algorithm Selection

Algorithm selection in IoT systems requires a structured representation of both the decision problem and the available algorithmic solutions. Due to the heterogeneity of IoT applications, relying solely on data-driven or performance-based descriptions does not adequately support adaptive and explainable selection mechanisms. To address this challenge, this work adopts *O-DMIoT*, a semantic model developed to characterize decision-making problems and their corresponding algorithmic solutions within the IoT domain.

O-DMIoT provides a categorical abstraction that enables reasoning about problem specifications and algorithm properties at different levels of granularity. Instead of emphasizing low-level implementation details, the model prioritizes conceptual characteristics that remain consistent across diverse application domains and execution contexts. This abstraction enables algorithm selection to be subsequently formulated as a planning and decision-making problem

8.3.1. Problem characterization in O-DMIoT

In *O-DMIoT*, a decision problem is defined by a finite set of semantic dimensions, each represented by a set of categorical values. These dimensions capture essential aspects of the problem that influence the suitability of different algorithmic approaches:

- The *objective* (*O*) dimension describes the primary purpose of the decision-making task, such as estimation, classification, forecasting, optimization, or planning. This dimension constrains the space of admissible algorithms by excluding approaches that are inherently incompatible with the problem goal.
- The *processing* (*Pr*) and *processing layer* (*L*) dimensions define how and where computation occurs, distinguishing between centralized execution and deployment across cloud, edge, or fog layers. These characteristics are especially significant in IoT environments, where computational resources, latency requirements, and data locality can differ substantially.
- The *interaction* (*I*) dimension describes the relationship between the algorithm and its environment, encompassing sensor-driven, data-driven, sequential, behavioral, or active forms of interaction. This dimension indicates whether the decision-making process is reactive, deliberative, or continuous, and whether it relies on human behavior or solely on system-generated inputs.
- The *knowledge representation* (*K*) dimension specifies the format in which information is processed, differentiating among data-based, symbolic, mathematical, or hybrid representations. This distinction is essential for aligning problem requirements with algorithmic paradigms that depend on particular representational assumptions.
- The *uncertainty* (*U*) dimension identifies the primary sources of uncertainty influencing the problem, including measurement, parameter, or model uncertainty. Explicit representation of uncertainty supports the selection of algorithms that are robust to the specific types of variability present in the environment.

Collectively, these dimensions provide a semantic description of the decision problem while preserving the information required to guide algorithm selection.

8.3.2. Algorithm Characterization in O-DMIoT

O-DMIoT characterizes algorithmic solutions using a complementary semantic structure that mirrors the abstraction applied to problems. Algorithms are characterized at three levels:

- The *algorithm approach* dimension captures the fundamental principle that drives the decision-making process. In *O-DMIoT*, this includes *criterion-driven*, *data-driven*, *utility-driven*, *rule-driven*, and *probability-driven* approaches. This level of abstraction enables reasoning about classes of algorithms without committing to specific techniques.
- The *algorithm technique* dimension further refines this characterization by specifying concrete methodological families, including *heuristic* methods, *dynamic programming*, *supervised* and *unsupervised learning*, *multicriteria decision-making*, *game-theoretic* approaches, *stochastic* methods, *Bayesian* inference, and *evidence theory*. This dimension connects conceptual approaches to executable algorithms.
- An *algorithm instantiation* denotes a specific implementation that can be executed within a given context. Instantiations are linked to evaluation metrics derived from prior executions in similar contexts or from on-demand execution when such data are unavailable (Algorithms are executed only when evaluation data is unavailable, enabling incremental enrichment of the semantic knowledge base).

Algorithm evaluation in *O-DMIoT* comprises three categories of metrics. *Algorithm characteristics* capture intrinsic properties such as temporal and spatial complexity. *QoE* metrics assess how well the algorithm fulfills user-oriented requirements, including accuracy, utility, trust, success rate or human-centric metrics. *QoS* metrics evaluate execution-related factors such as latency and throughput. This multi-dimensional evaluation framework enables selection decisions that balance performance, resource constraints, and user-centered objectives.

8.4. Algorithm Selection as a Planning Problem

The algorithm selection problem was originally formalized by Rice as the task of identifying the most suitable algorithm for a given problem instance [9]. In its classical formulation, the problem is defined by the triple

$$\langle X, A, P \rangle \quad (8.1)$$

where X denotes the set of problem instances, A the set of candidate algorithms, and

$$P : A \times X \rightarrow \mathbb{R} \quad (8.2)$$

is a performance function. The objective is to determine a mapping $\alpha : X \rightarrow A$ that associates each problem with an algorithm that maximizes performance.

This formulation conceptualizes algorithm selection as a direct, single-step mapping from problems to algorithms. Although foundational, it lacks explicit representation of the internal structures of problems and algorithms, as well as the decision process involved in identifying an appropriate algorithm.

To address this limitation, extended formulations of the algorithm selection problem introduce explicit characterizations of problems and algorithms using feature spaces [4]. In this context, problems are represented by feature vectors $f(x) \in F$, and algorithms by feature vectors $g(a) \in G$. The selection process is then guided by a function that relates

these characterizations to expected performance. This extension clarifies that algorithm selection operates over abstract representations rather than raw problem instances.

However, even in its extended form, algorithm selection remains conceptualized as a one-shot decision. Given a problem characterization, an algorithm is selected directly, without explicit modeling of the intermediate decisions involved in combining, constraining, or refining problem–solution compatibility.

In this work, algorithm selection is reconceptualized as a planning problem, with the objective of constructing a sequence of decisions that progressively refines the solution space. The semantic characterization provided by ***O-DMIoT*** defines the initial planning state, and each planning action involves selecting a semantic category from the algorithm characterization dimensions. Each action incrementally constrains the set of admissible algorithms by enforcing semantic compatibility between problem and solution characteristics. Consequently, the selection process is decomposed into a sequence of state transitions, with each state representing a partial specification of the final algorithm selection.

From this perspective, the goal of the planning process is not solely to select an algorithm, but to reach a state in which the accumulated semantic constraints identify one or more suitable algorithm instantiations. This interpretation aligns algorithm selection with sequential decision-making under uncertainty and establishes the conceptual basis for its formalization as a Markov Decision Process in the subsequent section.

8.5. Algorithm Selection as a Markov Decision Process

Recasting algorithm selection as a planning problem allows it to be represented as a Markov Decision Process (MDP). This formalization facilitates rigorous and structured reasoning about sequences of decisions, inherent uncertainty, and the long-term implications of algorithmic choices.

8.5.1. MDP definition

The algorithm selection MDP is defined by the tuple $\langle S, A, T, R \rangle$. The state space (S) comprises semantic representations derived from ***O-DMIoT***. The action space (A) involves selecting categories from the *algorithm approach* and *algorithm technique* dimensions. The transition function (T) specifies how actions expand the current state by adding solution characteristics. The reward function (R) assesses the quality of the selected algorithm using predefined metrics.

The Markov property holds because each state encapsulates all relevant information needed to evaluate future decisions, independent of the sequence of actions that led to it.

8.5.2. Ontology-based state representation

States in the MDP are represented as vectors of categorical values defined by ***O-DMIoT***. The initial state encodes problem characteristics such as objective, processing, processing layer, interaction, uncertainty, and knowledge representation. Subsequent states are generated by accumulating semantic constraints over solution dimensions.

This augmented state representation enables the identification of statistical relationships between problem descriptions and algorithmic solutions while preserving semantic interpretability. Each state represents a partial specification of an algorithm selection decision.

8.5.3. Action semantics

Actions in the MDP involve selecting categories from the algorithm characterization dimensions. At each decision step, an action restricts the set of candidate algorithms by ensuring semantic compatibility with the current state. In practice, selecting a category imposes a semantic constraint that limits the admissible set of algorithm designs. Rather than directly selecting an algorithm instantiation, actions progressively refine the solution space.

Interpreting actions as semantic constraints distinguishes the proposed approach from traditional algorithm recommendation systems, which typically perform direct selection without explicitly modeling intermediate decisions.

8.5.4. Reward

The reward is assigned at the conclusion of an episode, after an algorithm instantiation has been selected and evaluated. The reward function aggregates metrics related to *algorithm characteristics*, *QoE*, and *QoS* to determine whether the selected algorithm satisfies the problem requirements more effectively than alternatives.

Deferring reward assignment until the end of the decision sequence allows the model to capture the cumulative impact of semantic decisions and supports exploration strategies that balance short-term uncertainty with long-term performance.

In this work, the MDP formulation serves as an abstract decision model, and its solution is obtained through Monte Carlo Tree Search guided by semantic constraints derived from *O-DMIoT*.

8.6. Semantic-driven Monte Carlo Tree Search with Thompson Sampling

The algorithm selection problem in IoT systems is formulated as a semantic-driven sequential decision-making process, in which a Monte Carlo Tree Search (MCTS) algorithm operates over a knowledge base structured by the *O-DMIoT* ontology. The ontology offers an explicit and structured representation of decision problems and algorithm designs, which supports principled definitions of the search space, state transitions, and reward signals. This section presents the specifications that enable the operationalization of the semantic knowledge base, culminating in algorithm 6, which outlines the complete MCTS procedure used for semantic-driven algorithm selection in *O-DMIoT*.

8.6.1. Assumptions

The proposed semantic-driven MCTS approach relies on the following assumptions:

1. **Semantic problem completeness:** Each decision problem is fully characterized using the mandatory dimensions defined in *O-DMIoT*. Exactly one value per dimension is assumed, ensuring an unambiguous initial state.
2. **Finite semantic design space:** All candidate algorithms are represented as *Design* individuals within the ontology. Algorithm selection is limited to this explicitly modeled design space.
3. **Ontology-based compatibility:** Semantic compatibility between problems and algorithm designs is expressed through Description Logic constraints, which restrict the action space during the search process.

4. **Episodic decision process:** Algorithm selection is modeled as an episodic process. Each MCTS iteration is treated as an episode that samples a semantic decision path and evaluates the resulting candidate designs.
5. **Metric availability:** Evaluation metrics (algorithm characteristics, QoE, and QoS) are assumed to be retrievable, either from prior executions or through on-demand execution.
6. **Comparable evaluation metrics:** All metrics are assumed to be normalizable to a common scale, which enables their aggregation into a single reward signal.

These assumptions define the scope of applicability for the approach and allow algorithm selection to be formulated as a semantically constrained sequential decision-making problem.

8.6.2. Tree Structure

The Monte Carlo Tree Search explores a semantic decision tree whose structure is determined by the *O-DMIoT* ontology. The root node represents the initial state, defined by the problem characterization and an empty set of solution constraints.

Each node corresponds to a semantic state

$$s = \langle P, C_s \rangle \quad (8.3)$$

where P is the fixed problem instance and C_s is the set of semantic constraints accumulated along the path from the root. These constraints progressively refine the space of admissible algorithm designs.

Edges in the tree represent actions that introduce new semantic constraints, such as the selection of an *Algorithm_Approach* or an *Algorithm_Technique*. Each action restricts the solution space by enforcing compatibility conditions specified in the ontology.

The depth of the tree is limited by the number of *O-DMIoT* solution dimensions, while the branching factor is dynamically determined through ontology queries. At each node, only semantically valid actions are considered, resulting in a problem-dependent and adaptive search space.

Terminal nodes correspond to states in which the accumulated constraints identify one or more fully specified algorithm designs. These nodes initiate rollout and evaluation phases, allowing the MCTS to balance exploration and exploitation while progressively narrowing the search space through semantic reasoning.

8.6.3. State and Action Space Formalization

Let P be a decision problem instance represented in *O-DMIoT*. Each problem is fully characterized by exactly one individual of each mandatory problem dimension:

$$P = \langle O, I, K, Pr, L, U \rangle \quad (8.4)$$

where O is the objective, I the interaction mode, K the knowledge representation, Pr the processing type, L the processing layer, and U the uncertainty type.

An MCTS node corresponds to a semantic state s , defined as:

$$s = \langle P, C_s \rangle \quad (8.5)$$

where C_s is the set of semantic constraints over *Design* individuals accumulated along the path from the root to node s .

Actions correspond to the selection of semantic categories that further constrain the solution space. Formally, an action $a \in A$ adds a new constraint c_a to C_s , producing a successor state:

$$s' = \langle P, C_s \cup \{c_a\} \rangle \quad (8.6)$$

In practice, these constraints correspond to **O-DMIoT** solution dimensions such as *Algorithm_Approach* or *Algorithm_Technique*.

8.6.4. Selection Policy (Thompson Sampling)

During the selection phase, the algorithm traverses the tree by selecting actions according to a Thompson Sampling policy. Each action a available at a node is associated with a Beta distribution:

$$\theta_a \sim \text{Beta}(\alpha_a, \beta_a) \quad (8.7)$$

At each node, a sample $\tilde{\theta}_a$ is drawn for every available action, and the selected action a^* is given by:

$$a^* = \arg \max_{a \in A_s} \tilde{\theta}_a \quad (8.8)$$

The set of available actions A_s is constrained by the ontology through Description Logic queries. For a problem instance P , the compatible designs at a given node are retrieved using queries of the form:

```
Design and isDesignForProblem value P
and hasAlgorithmApproach value Data_Driven
```

These queries ensure that only semantically valid actions are considered during selection.

8.6.5. Expansion Phase

If the selected node is not fully expanded, the expansion phase introduces a new child node by adding an additional semantic constraint. In this work, the expansion policy π_e is identical to the selection policy π_s , favoring exploration due to the lack of prior training data.

An example of an expanded query after adding an algorithm technique constraint is:

```
Design and isDesignForProblem value P
and hasAlgorithmApproach value Data_Driven
and hasAlgorithmTechnique value Supervised_Learning
```

This incremental restriction reduces the search space and systematically guides the MCTS toward fully specified algorithm designs.

8.6.6. Rollout and Semantic Evaluation

Once a node reaches a sufficient level of specification, a rollout is performed. During this phase, candidate *Design* individuals compatible with the accumulated constraints are retrieved from the ontology. Each retrieved design d represents a terminal state:

$$s_{terminal} = \langle P, d \rangle \quad (8.9)$$

For each design, evaluation metrics are obtained through the associated *Design_Evaluation* individuals. These metrics include algorithm characteristics, QoE, and QoS, which are retrieved using queries such as:

Metric and isMetricOfDesignEvaluation some
(Design_Evaluation and evaluatesDesign value d)

If no evaluation data are available, the corresponding algorithm is executed, and new metric individuals are instantiated and associated with the design.

8.6.7. Reward Function

The reward function is defined as a weighted aggregation of normalized evaluation metrics:

$$R(d) = w_{QoE} \cdot f(QoE_d) + w_{QoS} \cdot f(QoS_d) - w_C \cdot f(Complexity_d) \quad (8.10)$$

where $f(\cdot)$ denotes a normalization function mapping metric values to the interval $[0, 1]$, and w_{QoE} , w_{QoS} , and w_C are application-dependent weights. For a given rollout, the design with the highest continuous reward is selected as:

$$d^* = \arg \max_{d \in D_s} R(d) \quad (8.11)$$

where D_s denotes the set of candidate designs compatible with the semantic constraints of the reached state.

The continuous reward $R(d)$ is used exclusively for ranking candidate designs within a rollout and is not directly propagated through the search tree. In order to update the Beta-distributed beliefs associated with semantic actions in the Thompson Sampling policy, the outcome of each rollout is discretized into a binary reward signal.

Formally, a unit reward is assigned to the semantic decision path π that leads to the selection of d^* :

$$r(\pi) = 1 \quad (8.12)$$

No explicit negative reward is assigned to alternative designs or decision paths. Instead, paths that do not lead to the best-ranked design in a rollout remain unreinforced, which implicitly decreases their probability of being selected in future iterations through the Bayesian update of the Beta distributions.

8.6.8. Backpropagation

After completing an episode, the reward r is propagated backward along the selected path in the tree. For each action a in the path, the corresponding Beta distribution parameters are updated as follows:

$$\alpha_a \leftarrow \alpha_a + r \quad (8.13)$$

$$\beta_a \leftarrow \beta_a + (1 - r) \quad (8.14)$$

This update mechanism enables the MCTS to learn probabilistic preferences over semantic decisions, thereby capturing statistical regularities between problem characterizations and successful algorithm designs.

Algorithm 6: Semantic-driven MCTS for Algorithm Selection using Thompson Sampling

Input: Problem instance P , ontology $\mathcal{O}$, number of iterations N
Output: Selected algorithm design d^*
Initialize root state $s_0 \leftarrow \langle P, \emptyset \rangle$;
Initialize root node n_0 with state s_0 ;
Initialize MCTS tree $T \leftarrow \{n_0\}$;
for $i \leftarrow 1$ **to** N **do**
 $n \leftarrow n_0$;
 Initialize path $\pi \leftarrow []$;
 // Selection (Semantic Traversal)
 while n has children and n is not terminal **do**
 Retrieve available semantic actions A_n using ontology queries;
 foreach $a \in A_n$ **do**
 Sample $\tilde{\theta}_a \sim \text{Beta}(\alpha_a, \beta_a)$;
 Select $a^* \leftarrow \arg \max_{a \in A_n} \tilde{\theta}_a$;
 Append a^* to path π ;
 $n \leftarrow \text{Child}(n, a^*)$;
 // Expansion
 if n is not terminal **then**
 Retrieve available semantic actions A_n ;
 Select expansion action a_e using Thompson Sampling;
 Create new state $s' \leftarrow \text{Apply}(n.\text{state}, a_e)$;
 Create new node n' with state s' ;
 Add edge (n, a_e, n') to T ;
 Append a_e to path π ;
 $n \leftarrow n'$;
 // Rollout (Design Evaluation)
 Retrieve candidate designs D_n using DL queries over $\mathcal{O}$;
 foreach $d \in D_n$ **do**
 Retrieve evaluation metrics (QoE, QoS, complexity);
 Compute continuous reward $R(d)$;
 Select best design $d^+ \leftarrow \arg \max_{d \in D_n} R(d)$;
 // Binary reward assignment
 Assign unit reward $r(\pi) \leftarrow 1$ to the semantic path π leading to d^+ ;
 // Backpropagation
 foreach $a \in \pi$ **do**
 $\alpha_a \leftarrow \alpha_a + r$;
 $\beta_a \leftarrow \beta_a + (1 - r)$;
 Select best child n^* of root using posterior beliefs;
return $d^* \in n^*.\text{candidates}$;

8.6.9. Integration of Semantic Reasoning within Monte Carlo Tree Search

The proposed MCTS formulation integrates semantic reasoning into the algorithm selection process as a core component. Ontological knowledge encoded in *O-DMIoT* establishes the initial state through problem characterization, restricts the admissible action space

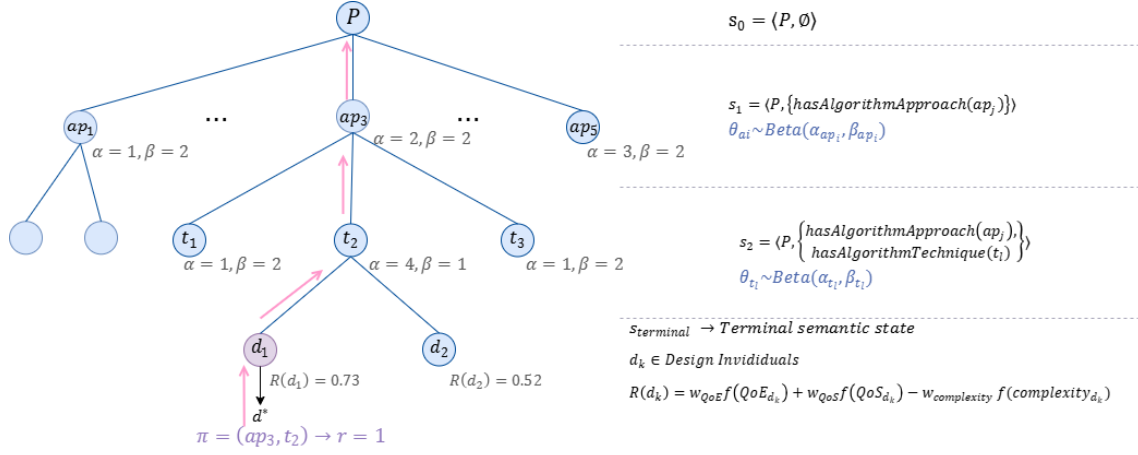


Figure 8.1.: **Semantic-driven Monte Carlo Tree Search with Thompson Sampling.** Each node denotes a semantic state $s = \langle P, C_s \rangle$, where P represents a fixed problem instance and C_s denotes the set of accumulated semantic constraints. Edges indicate semantic actions, such as algorithm approach ap_i and algorithm technique t_i , each linked to a Beta distribution for Thompson Sampling. A sampled trajectory, shown as a highlighted path, terminates in a semantic state where candidate algorithm designs are evaluated using a continuous reward function based on QoE, QoS, and complexity. A unit reward $r = 1$ is assigned to the semantic decision path that yields the highest ranked design, and this reward is propagated backward to update the posterior beliefs of the selected actions.

using semantic compatibility rules, and directs the expansion and rollout phases through Description Logic queries over algorithm designs. Evaluation metrics obtained during rollout are interpreted within this semantic framework and aggregated to compute the reward signal, which is then propagated through the search tree. As a result, semantic reasoning is embedded within the search and learning dynamics of MCTS. This integration enables algorithm selection to be both explainable and adaptive in dynamic IoT environments.

Figure 8.1 depicts the semantic-driven Monte Carlo Tree Search process for algorithm selection and summarizes the operationalization of the proposed methodology. The root node represents the initial state $s_0 = \langle P, \emptyset \rangle$, in which the decision problem P is fully characterized, but no solution constraints have been applied. From this point, the search proceeds by selecting semantic actions that impose constraints on algorithm characterization dimensions, such as *algorithm approach* and *algorithm technique*. Each action incrementally narrows the solution space, resulting in terminal semantic states that correspond to fully specified algorithm designs. These designs are subsequently evaluated and used to update the selection policy.

8.7. Proof of Concept: Algorithm Selection Scenario

This section presents a proof of concept (PoC) designed to demonstrate the feasibility of the proposed ontology-driven algorithm selection approach. The PoC examines a specific IoT decision-making scenario and illustrates how *O-DMIoT* functions as a knowledge

base to filter, evaluate, and select decision-making algorithms through a semantic-driven MCTS process.

8.7.1. Scenario Description

Within O-DMIoT, the problem is instantiated as an individual of type *Problem* and is fully characterized by a predefined set of problem dimensions. In this PoC, the problem is defined by the following characteristics:

- **Objective:** Classification
- **Interaction:** System-driven
- **Knowledge representation:** Data-driven
- **Processing:** Centralized
- **Processing layer:** Application
- **Uncertainty:** Model uncertainty

These characteristics are represented in the ontology as individuals linked to the problem instance through inheritance relations, ensuring semantic completeness and suitability for automated reasoning. The resulting problem characterization serves as the initial state for the MCTS process.

8.7.2. Candidate Algorithms and Metrics

Candidate algorithms are represented in *O-DMIoT* as individuals of the *Design* type. Each design encapsulates a specific algorithmic solution and is characterized by its algorithmic approach, data representation, and intrinsic complexity. Table 8.1 summarizes the individuals instantiated in the A-Box of *O-DMIoT* for this proof of concept.

In this proof of concept, the ontology incorporates a deliberately limited set of algorithm designs that are applicable to the defined classification problem. The candidate algorithms include Logistic Regression, k-Nearest Neighbors (kNN), decision tree, and rule-based classifiers. This selection is not exhaustive but is intended to represent distinct algorithmic paradigms frequently encountered in IoT decision-making contexts. These algorithms encompass data-driven parametric models, instance-based learning, symbolic rule induction, and explicit rule-based reasoning. This diversity provides sufficient semantic contrast to demonstrate how the proposed ontology-driven formulation distinguishes differences in knowledge representation, algorithmic assumptions, and execution characteristics. As a result, this set facilitates the illustration of algorithm selection as a semantic-driven planning process rather than a purely performance-oriented recommendation task. Compatibility between a problem instance and an algorithm design is established through ontology-based semantic queries that filter candidate designs based on the specified problem characteristics.

Regarding the evaluation, each candidate design is linked to one or more evaluation metrics through instances of *Design_Evaluation*. The metrics considered in this PoC are organized into three categories:

- **Algorithm characteristics**, which describe intrinsic properties such as temporal and spatial complexity.
- **Quality of Experience (QoE)**, which assesses the accomplishment of functional requirements from a user or application perspective.

- **Quality of Service (QoS)**, which captures execution-related aspects such as latency or throughput.

These metrics form the basis for computing the reward function during the rollout phase of the MCTS algorithm. If metric values are not present in the knowledge base, the corresponding algorithm is executed and the resulting measurements are incorporated into the ontology for future reference. Table 8.2 presents a summary with the proof of concept configuration grounded in a specific domain.

Category	Description	Count
Problem	Decision problem instances	1
Design	Algorithm design instances	4
Objective	Objective categories (e.g., Classification)	1
Interaction	Interaction types (e.g., System-driven)	1
Knowledge	Knowledge representation types	1
Processing	Processing modes	1
Processing Layer	Execution layers	1
Uncertainty	Uncertainty types	1
QoE Metrics	Quality of Experience metrics	12
QoS Metrics	Quality of Service metrics	10
Complexity Metrics	Algorithm characteristics	6
Design Evaluation	Evaluation instances linking designs and metrics	12

Table 8.1.: Summary of A-Box individuals instantiated in O-DMIoT for the proof of concept

8.7.3. Selection Outcome

The algorithm selection process utilizes the semantic-driven MCTS formulation introduced in this chapter (see Sec. 8.6). Beginning with the initial state defined by the problem characterization, MCTS incrementally explores the space of admissible algorithm designs by applying semantic constraints derived from the *O-DMIoT* ontology.

At each decision step, the set of available actions is limited to semantically compatible refinements of the solution space. This restriction ensures that only algorithmic approaches and techniques consistent with the problem specification are considered. Consequently, the search process incrementally transforms an abstract problem description into a fully specified algorithm design through a sequence of semantic decisions.

Upon reaching a terminal state, which corresponds to a complete algorithm design, the design is evaluated using a multi-criteria reward function. This function aggregates algorithm characteristics, QoE, and QoS. The resulting reward is discretized and propagated backward through the search tree, updating the selection policy using Thompson Sampling.

The proof of concept results in the identification of an algorithm design that is semantically compatible with the problem and consistent with the specified evaluation criteria. The selected algorithm is not obtained through direct recommendation or exhaustive comparison. Instead, it emerges from a planning process in which semantic reasoning explicitly guides state transitions, action selection, and evaluation.

This proof of concept demonstrates that the proposed approach enables algorithm selection to be formulated and executed as a semantic-driven planning problem, in which ontological knowledge is tightly integrated with sequential decision-making. The proof of

PoC Scenario	Vision-Based Crop Classification
Problem Description	The system shall classify crop health status based on image-derived features captured from vision-based sensing components in an IoT-enabled agricultural environment.
Objective	Classification
Interaction	System-driven
Processing	Centralized
Processing Layer	Application (Cloud)
Knowledge Representation	Data-driven
Uncertainty	Model uncertainty
Input Data Assumptions	Image-derived feature vectors obtained from vision-based sensors; identical feature representation and pre-processing applied across all candidate algorithms.
Learning Paradigm	Supervised learning
Dataset Variability	Fixed dataset; no resampling, augmentation, or stochastic variation applied.
Candidate Algorithms	Logistic Regression; Decision Tree; k-Nearest Neighbors; Rule-Based Classifier
Evaluation Dimensions	Algorithm characteristics (temporal and spatial complexity); QoS; QoE

Table 8.2.: Problem instantiation and experimental configuration for the algorithm selection proof of concept

concept therefore validates the feasibility of the proposed modeling and integration strategy, rather than the performance of any specific algorithm.

The main outcomes of the semantic-driven MCTS execution are summarized in Table 8.3. The table provides a compact view of the planning process, including the size of the semantic action space, the evolution of posterior beliefs, and the final algorithm design selected through semantic-guided exploration.

8.8. Discussion and Limitations

This chapter introduced a semantic-driven formulation of algorithm selection as a planning problem, implemented using Monte Carlo Tree Search and based on the O-DMIoT ontology. The primary contribution is the modeling and integration of semantic reasoning with sequential decision-making, rather than the development or comparison of advanced machine learning algorithms. By framing algorithm selection as a sequence of semantically constrained decisions, the proposed approach facilitates a structured, interpretable, and extensible selection process appropriate for dynamic IoT environments.

The proof of concept demonstrates the feasibility of integrating semantic reasoning with a planning-based algorithm selection mechanism. Consequently, the evaluation scope is

Aspect	Outcome
Problem instance	<i>Problem_CameraCropClassification</i>
Total MCTS iterations	50
Initial candidate designs	4
Semantic action space	7 admissible semantic actions
Terminal design selected	<i>Design_LogisticRegression_CameraCrop</i>
Final posterior belief	Beta(41, 1)
Reward type	Discretized Bernoulli reward
Episodes with valid rollouts	Majority of iterations
Episodes with empty candidate sets	Observed (reward $r = 0$)
Learning behavior	Progressive reinforcement of semantically valid paths

Table 8.3.: Summary of semantic-driven MCTS execution for the algorithm selection proof of concept

intentionally limited. The results support the validity of the proposed formulation and its implementation through MCTS, but do not seek to establish performance superiority over alternative algorithm selection methods. Quantitative comparisons with baseline selection mechanisms are therefore excluded from this chapter.

Within this scope, several limitations are recognized. First, the set of candidate algorithms is intentionally limited and chosen to ensure semantic diversity rather than comprehensive coverage. Second, the evaluation is confined to a single decision-making scenario, with the ontology populated by a limited number of individuals. Third, algorithm execution occurs exclusively for evaluation and is not integrated into a live IoT deployment. Comprehensive quantitative evaluation would require standardized benchmarks, large-scale experimentation, and domain-specific tuning, which are reserved for future research.

Despite these limitations, the proof of concept demonstrates that the proposed semantic-driven planning formulation establishes a coherent foundation for adaptive algorithm selection in IoT systems. This formulation is generalizable and can be extended to additional application domains, algorithm families, and evaluation criteria, thereby supporting future research on autonomous and human-centered decision-making services.

In the subsequent chapter, this selection mechanism is integrated into a service reconfiguration pipeline, where selected designs directly influence runtime system behavior.

References

- [1] M. Gagliolo and J. Schmidhuber. “Algorithm Selection as a Bandit Problem with Unbounded Losses”. en. In: *Learning and Intelligent Optimization*. Ed. by C. Blum and R. Battiti. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2010, pp. 82–96.
- [2] M. Gagliolo and J. Schmidhuber. “Learning dynamic algorithm portfolios”. en. In: *Annals of Mathematics and Artificial Intelligence* 47.3-4 (Feb. 2007),

- pp. 295–328. URL: <http://link.springer.com/10.1007/s10472-006-9036-z> (visited on 07/14/2023).
- [3] R. Gupta and T. Roughgarden. “A PAC Approach to Application-Specific Algorithm Selection”. en. In: *SIAM Journal on Computing* 46.3 (Jan. 2017), pp. 992–1017. URL: <https://epubs.siam.org/doi/10.1137/15M1050276> (visited on 01/21/2026).
- [4] M. Hilario et al. “Ontology-Based Meta-Mining of Knowledge Discovery Workflows”. en. In: *Meta-Learning in Computational Intelligence*. Ed. by N. Jankowski, W. Duch, and K. Grabczewski. Studies in Computational Intelligence. Berlin, Heidelberg: Springer, 2011, pp. 273–315. URL: https://doi.org/10.1007/978-3-642-20980-2_9 (visited on 05/02/2023).
- [5] Y. Huang et al. “Automatic Feature Engineering Through Monte Carlo Tree Search”. en. In: *Machine Learning and Knowledge Discovery in Databases*. Ed. by M.-R. Amini et al. Cham: Springer Nature Switzerland, 2023, pp. 581–598.
- [6] X. Jin et al. “A fast and scalable approach for IoT service selection based on a physical service model”. en. In: *Information Systems Frontiers* 19.6 (Dec. 2017), pp. 1357–1372. URL: <https://doi.org/10.1007/s10796-016-9650-1> (visited on 05/03/2023).
- [7] P. Kerschke et al. “Automated Algorithm Selection: Survey and Perspectives”. In: *Evolutionary Computation* 27.1 (Mar. 2019), pp. 3–45. URL: https://doi.org/10.1162/evco_a_00242 (visited on 01/09/2023).
- [8] E. Poormohammady et al. “Dynamic algorithm selection for the logic of tasks in IoT stream processing systems”. In: *2017 13th International Conference on Network and Service Management (CNSM)*. ISSN: 2165-963X. Nov. 2017, pp. 1–5. URL: <https://ieeexplore.ieee.org/document/8256009> (visited on 01/21/2026).
- [9] J. R. Rice. “The Algorithm Selection Problem”. en. In: *Advances in Computers*. Vol. 15. Elsevier, 1976, pp. 65–118. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0065245808605203> (visited on 01/09/2023).
- [10] M. Salisu et al. “An Overview of the Algorithm Selection Problem”. In: *International Journal of Computer (IJC)* (Jan. 2017).
- [11] M. Shafiq et al. “Selection of effective machine learning algorithm and Bot-IoT attacks traffic identification for internet of things in smart city”. en. In: *Future Generation Computer Systems* 107 (June 2020), pp. 433–442. URL: <https://www.sciencedirect.com/science/article/pii/S0167739X19334880> (visited on 08/08/2023).
- [12] S. P. Singh et al. “Improving quality of service for Internet of Things(IoT) in real life application: A novel adaptation based Hybrid Evolutionary Algorithm”. In: *Internet of Things* 27 (Oct. 2024), p. 101323. URL: <https://www.sciencedirect.com/science/article/pii/S2542660524002646> (visited on 01/21/2026).

- [13] Z. Yang et al. “An Anomaly Detection Algorithm Selection Service for IoT Stream Data Based on Tsfresh Tool and Genetic Algorithm”. en. In: *Security and Communication Networks* 2021 (Feb. 2021). Publisher: Hindawi, e6677027. URL: <https://www.hindawi.com/journals/scn/2021/6677027/> (visited on 08/07/2023).

9. Adaptive Reconfiguration of IoT services: a proof of concept in the field of protected and controlled agriculture

Abstract. This chapter presents an integrated proof of concept that implements the semantic-driven adaptive reconfiguration framework proposed in this thesis within a dynamic Internet of Things (IoT) system. The aim is to illustrate how system evolution, formal problem characterization, algorithm selection, and service-level reconfiguration can be achieved as a unified, end-to-end execution process. The case study focuses on protected and controlled agriculture, combining agent-based modeling with a mechanistic crop growth model to simulate realistic environmental dynamics. System evolution is modeled by activating modular cyber-physical components, each linked to a predefined semantic characterization using the O-DMIoT ontology. These semantic characterizations function as declarative specifications for decision-making requirements and initiate algorithm selection via an external synthesizer component. The chosen algorithm is then incorporated into the system through service activation and configuration-level binding. This proof of concept demonstrates that semantic problem characterization provides a sufficient, traceable foundation for adaptive reconfiguration, thereby validating the feasibility, internal coherence, and architectural integrity of the proposed framework in a simulated IoT environment.

9.1. Introduction

This chapter introduces an integrated proof of concept that synthesizes the semantic, methodological, and computational contributions developed in this thesis. The objective is to demonstrate the instantiation and execution of semantic models for decision-making, algorithm selection, and adaptive reconfiguration as an executable, closed-loop system within a specific IoT scenario.

The case study is situated within the domain of protected and controlled agriculture, which is characterized by dynamic environmental conditions, heterogeneous sensing infrastructure, and continuously adaptive decision processes. This context illustrates how evolving physical conditions can be abstracted into semantic problem representations and subsequently used to drive adaptive behavior in IoT systems. The focus is on the structural and semantic mechanisms that enable principled and systematic runtime reconfiguration, rather than on domain-specific optimization.

A central component of this chapter is the implementation of an end-to-end reconfiguration loop that integrates semantic modeling of decision problems and algorithms, a

planning-based algorithm selection mechanism, and the service-level reconfiguration strategy proposed in this thesis. Simulated environmental changes are reflected in the evolving characterization of decision problems, which are processed using the ontology-driven algorithm selection framework described in Chapter 5. This process results in the selection and activation of decision-making algorithms as services.

Agent-based modeling is employed for evaluation and experimentation, enabling observation of interactions among the IoT system, its environment, and its services over time. The inclusion of a mechanistic crop growth model introduces realistic and complex environmental dynamics, ensuring that reconfiguration events result from changes in system state rather than from scripted scenarios. This combination provides a controlled, yet expressive environment for exercising and analyzing the proposed reconfiguration mechanisms.

In summary, this chapter consolidates the thesis contributions by demonstrating the practical implementation and internal coherence of semantic-driven decision-making and adaptive reconfiguration in IoT systems. The proof of concept confirms the feasibility of the proposed framework and provides a foundation for future research on quantitative evaluation, optimization, and large-scale deployment.

9.2. Case Study Overview

The case study presented in this chapter is designed to operationalize the proposed semantic-driven adaptive reconfiguration framework within an evolving IoT system. The primary objective is to demonstrate how an IoT system can evolve through the addition of new cyber-physical components, how this evolution can be semantically characterized, and how such characterizations can inform the selection and deployment of decision-making algorithms as services.

Protected and controlled agriculture is selected as the application domain for the case study, providing a concrete and well-structured setting where environmental dynamics, sensing infrastructure, and decision-making processes are tightly coupled. In the proposed scenario, the physical environment is represented by a tomato crop whose growth evolves over time in response to environmental conditions such as temperature, radiation, and CO₂ concentration. These dynamics introduce significant changes in system state that are directly relevant to monitoring and decision-making tasks.

The IoT system is modeled as an evolving cyber-physical system composed of sensing, actuation, and monitoring modules. An initial configuration provides basic monitoring of environmental variables, while additional evolution modules can be incrementally incorporated to extend system capabilities. Each evolution module introduces new functionality, such as climate control, illumination management, ventilation, or vision-based crop monitoring, and is associated with a semantic characterization expressed using the O-DMIoT ontology. This characterization captures key aspects of the decision problem, including processing model, system configuration, and data characteristics.

Adaptive behavior emerges from the interaction between the IoT system and an external synthesizer component. The synthesizer incorporates mechanisms for algorithm selection and service determination, converting semantic problem characterizations into executable decision-making services. Given a specific problem characterization and a directory of candidate algorithms, the synthesizer identifies and selects an appropriate decision-making algorithm and its corresponding service. This selected capability is subsequently activated within the IoT system configuration, thereby completing the reconfiguration cycle. In the present case study, problem characterizations are predefined and associated with evolution modules; the IoT system activates the relevant characterization rather than inferring it dynamically.

The case study is structured to emphasize the end-to-end flow from environmental evolution to system adaptation. Instead of exploring all possible configurations or maximizing domain-specific performance, the scenario focuses on a limited but expressive set of evolution events that make the reconfiguration process explicit and traceable. This design ensures that the interactions among environment, semantic models, algorithm selection, and system deployment can be clearly observed and analyzed.

Overall, the case study provides a concrete instantiation of the thesis framework, illustrating how semantic characterization supports algorithm selection and adaptive reconfiguration in practice. It serves as a unifying scenario that connects the conceptual models, methodological choices, and computational mechanisms introduced in earlier chapters.

9.3. Simulation Environment

The proof of concept presented in this chapter is implemented within a simulation environment designed to enable observation of system evolution and adaptive reconfiguration over time. This environment provides a controlled setting for instantiating and analyzing environmental dynamics, IoT system behavior, and reconfiguration mechanisms as an integrated process. Instead of replicating a deployment-scale system, the simulation environment supports systematic exploration of interactions among the physical environment, semantic reasoning, and algorithm-driven adaptation.

The simulation environment integrates agent-based modeling with domain-specific environmental processes. This integration enables representation of heterogeneous entities, explicit interactions, and temporal evolution, while maintaining flexibility to incorporate semantic characterization and external synthesizer component responsible for algorithm selection and service resolution mechanisms defined within the proposed framework.

Figure 9.1 presents a deployment view of the proof-of-concept environment. The diagram illustrates the separation between the simulation environment, which hosts the physical environment, crop model, IoT system, and its evolution modules, and the external synthesizer component. Within the simulation environment, the IoT system interacts with the evolving environment and activates a semantic characterization of decision problems defined using O-DMIoT. This characterization is communicated to an external synthesizer component, which selects an appropriate decision-making algorithm from a directory of candidates. The selected algorithm is then deployed as part of the IoT system configuration. This deployment view establishes the architectural context for subsequent sections detailing simulation dynamics, system modeling, and the adaptive reconfiguration process. Figure 9.2 provides an integrated architectural view of the simulation environment and serves as a reference for the modeling, reconfiguration, and execution processes described in the remainder of this chapter.

9.3.1. Agent-based Modeling Approach

Agent-based modeling is the primary modeling paradigm for the simulation environment. ABM provides an effective framework for representing systems composed of interacting entities whose behavior evolves over time, making it particularly suitable for modeling IoT systems embedded in dynamic physical environments.

In this study, ABM is used as a tool for evaluation and experimentation rather than as an end in itself. Agents represent key system elements, including the IoT infrastructure and its interactions with the environment. The agent-based abstraction enables explicit modeling of system evolution, such as the incremental addition of new cyber-physical components, and facilitates observation of how these changes influence decision-making requirements and reconfiguration behavior.

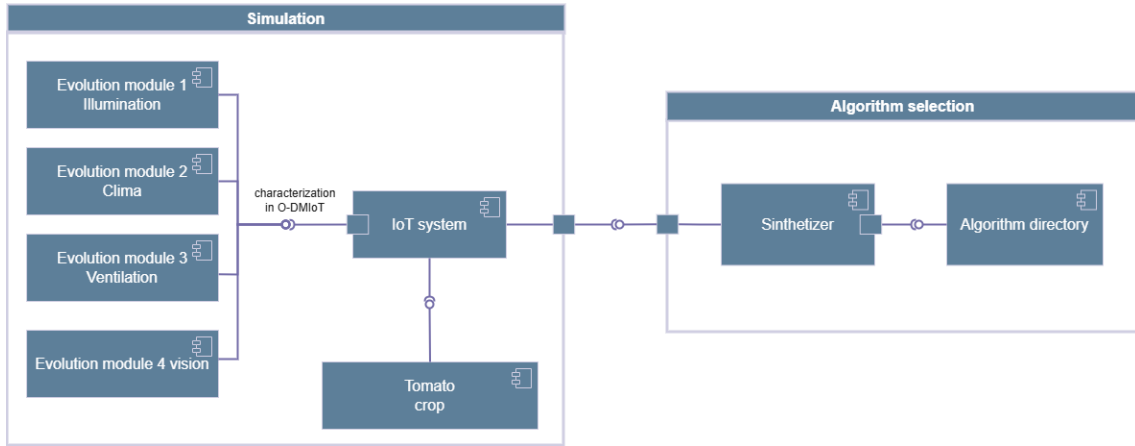


Figure 9.1.: **Deployment view of the proof-of-concept environment**, illustrating the separation between the simulation environment and the external synthesizer component.

The use of ABM supports temporal coupling between environmental evolution and system adaptation. Environmental processes evolve continuously during the simulation, producing observable changes that influence the characterization of decision problems. These characterizations trigger semantic reasoning and algorithm selection processes, which may result in system reconfiguration. By explicitly representing these interactions, the agent-based approach enables traceability of cause-and-effect relationships throughout the reconfiguration loop.

The agent-based model does not encode decision logic for algorithm selection or reconfiguration policies. These decisions are delegated to semantic models and selection mechanisms defined outside the simulation. This separation of concerns ensures that the simulation environment functions as a neutral execution and observation layer, supporting evaluation of the proposed framework without conflating modeling assumptions with decision-making logic.

9.3.2. Simulation Platform

The simulation environment is implemented using the Gama simulation platform, which supports discrete-time execution, interaction among heterogeneous agents, and integration with external computational services [3]. The platform provides the necessary infrastructure to model environmental dynamics, manage agent interactions, and capture system evolution over time.

The platform hosts simulations of the physical environment and the IoT system, including sensing and actuation components. Evolution modules can be incorporated dynamically during simulation execution, enabling explicit representation of system evolution scenarios. The platform also facilitates information exchange between the simulated IoT system and external services, such as the algorithm selection service operating outside the simulation environment.

A primary advantage of the selected platform is its support for modular model design. Environmental processes, system components, and monitoring mechanisms are implemented as distinct model elements, allowing for incremental composition and extension. This modularity supports the objectives of the proof of concept by enabling clear separation between environmental modeling, system behavior, and adaptive reconfiguration mechanisms.

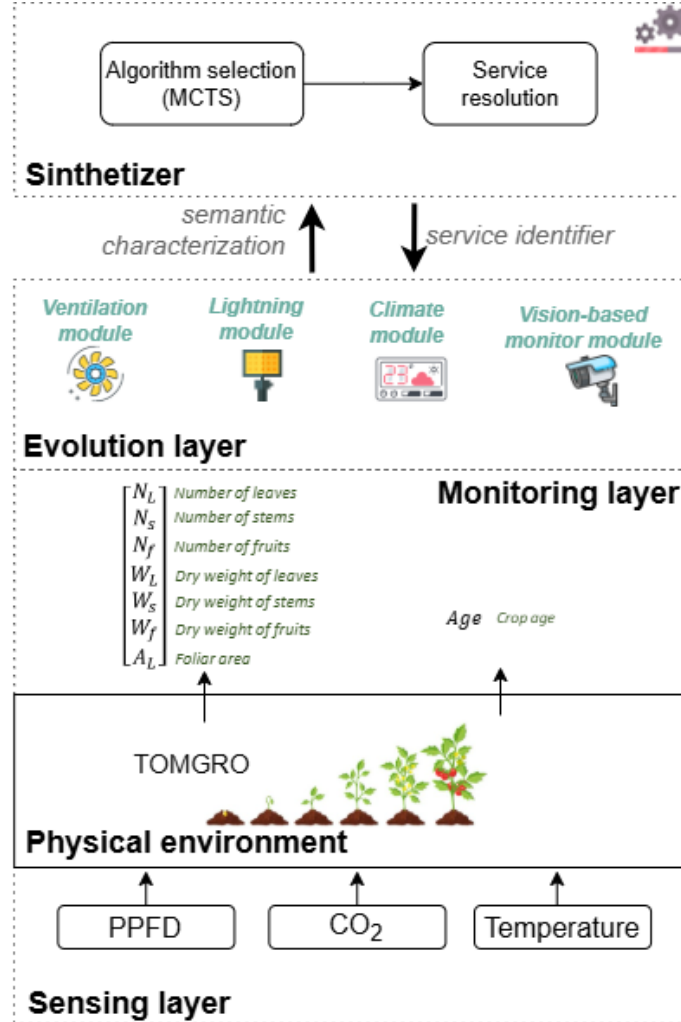


Figure 9.2.: **Overview of the simulation environment, the TOMGRO crop growth model, the evolution of the IoT system, and the semantic-driven reconfiguration process.** The diagram illustrates how environmental inputs, provided as exogenous data and abstracted as a sensing layer, parameterize the TOMGRO crop growth model; how the resulting crop state variables are monitored to support system evolution through modular extensions, and how semantic problem characterizations guide the synthesizer in selecting and activating decision-making services.

In summary, the simulation platform provides a flexible and extensible execution environment for instantiating and evaluating the proposed semantic-driven reconfiguration framework. In this chapter, it supports controlled observation of system behavior and adaptation, establishing the foundation for the introduction of the crop growth environmental model in the following section.

9.4. Crop Growth Environmental Model

The simulation environment integrates a crop growth environmental model to represent the physical dynamics monitored by the IoT system. Rather than optimizing agricultural production, this model provides a realistic, time-evolving environment whose state influences monitoring tasks, decision-making requirements, and system reconfiguration. A mechanistic crop growth model is employed to ensure that environmental evolution is grounded in established domain knowledge and generates complex system dynamics.

9.4.1. TOMGRO Crop Growth Model

Crop growth is modeled using TOMGRO, a dynamic tomato growth and production model originally developed to analyze the response of greenhouse-grown tomatoes to environmental conditions [1]. TOMGRO expresses crop growth as a function of key environmental variables, including temperature, carbon dioxide concentration, and photosynthetic photon flux density (PPFD). The model captures physiological processes, including photosynthesis, maintenance respiration, and biomass allocation, thereby providing a mechanistic description of plant development over time.

TOMGRO is classified as a medium-granularity crop model, abstracting the plant into major structural components such as leaves, stem segments, and fruits [2]. Growth is described through state variables representing both structural properties (e.g., the number of leaves and fruits) and biomass accumulation (e.g., the dry weights of plant components). This abstraction balances biological realism with computational tractability, making the model suitable for integration within a simulation-based evaluation framework.

Within this study, TOMGRO functions as the environmental process linking physical conditions to observable crop states. These states are monitored by the IoT system and provide the basis for defining decision problems related to crop monitoring and management. Thus, TOMGRO serves as a source of structured environmental dynamics rather than as an optimization target.

9.4.2. TOMGRO Adaptation for Agent-Based Simulation

The original TOMGRO formulation was not intended for operation within an agent-based simulation environment or for direct interaction with IoT system models. Therefore, significant adaptation was necessary to integrate the model into the simulation framework employed in this study. This adaptation addresses differences in temporal resolution, execution flow, and model interaction.

TOMGRO operates with a dual-time-scale structure. Environmental inputs are processed at an hourly resolution to compute photosynthesis and respiration rates, while plant growth and state variable updates occur daily. To support this behavior within an agent-based simulation, the model was restructured into two coordinated execution cycles: a fast cycle that accumulates physiological processes based on hourly environmental conditions and a slow cycle that updates plant state variables at the end of each simulated day.

In the adapted model, the crop is represented as an environmental process whose state evolves independently of the IoT system but remains observable through sensing and mon-

itoring agents. Model execution is synchronized with the simulation clock to ensure consistent interaction with other simulated components. This design enables environmental dynamics to emerge from the interaction between inputs and physiological processes, rather than being scripted or externally imposed.

The adaptation also required explicit handling of age-dependent plant structures. TOMGRO represents aging through vectors of age classes for leaves, stem segments, and fruits. These structures were preserved in the adapted model, enabling the simulation to capture developmental stages that influence observable variables such as foliar area and fruit count. Maintaining these internal structures was essential for preserving the model's mechanistic behavior within the broader simulation context.

Overall, the adaptation of TOMGRO enables its use as a dynamic environmental driver within an agent-based simulation. This supports continuous interaction with the IoT system and provides a realistic foundation for evaluating monitoring, decision-making, and reconfiguration processes.

9.4.3. Model Assumptions

The crop growth environmental model is implemented with explicit assumptions that define its scope and ensure consistency within the simulation environment. These assumptions reflect both the original TOMGRO formulation and the requirements of integration into an agent-based framework.

The primary assumptions adopted in this study are as follows:

- Only the main vegetative stem of the tomato plant is considered. Secondary stems and branching effects are not modeled.
- Environmental inputs, including temperature, CO₂ concentration, and PPFD, are assumed to be spatially uniform across the crop.
- Physiological processes such as photosynthesis and maintenance respiration are computed at an hourly resolution, while plant growth and biomass allocation are updated on a daily basis.
- Plant aging is represented using discrete age classes for leaves, stem segments, and fruits, following the original TOMGRO formulation.
- The crop growth model operates independently of decision-making and control logic; interactions with the IoT system are limited to observation and monitoring of model outputs.

These assumptions are intentionally selected to preserve the mechanistic behavior of the crop growth model while ensuring compatibility with the simulation environment and the objectives of the proof of concept.

9.4.4. Inputs and Outputs

The adapted TOMGRO model requires environmental inputs at an hourly resolution. The following inputs are provided to the model during simulation execution:

- Air temperature (°C)
- Carbon dioxide concentration
- Photosynthetic photon flux density (PPFD)

Based on these inputs, the model computes a set of state variables that abstract crop growth and development. The primary outputs used in this study are:

- N_L : number of leaves
- N_S : number of stem segments
- N_F : number of fruits
- W_L : dry weight of leaves
- W_S : dry weight of stem segments
- W_F : dry weight of fruits
- A_L : foliar area

These outputs provide a structured representation of crop state that can be observed by the IoT system through sensing and monitoring components. In this chapter, selected outputs are used to define monitoring tasks and to characterize decision problems that may trigger adaptive reconfiguration. The crop growth model, therefore, serves as a bridge between physical environmental conditions and the semantic characterization of decision-making requirements.

9.5. IoT System Model

The IoT system model constitutes the cyber-physical infrastructure responsible for monitoring the crop growth environment and delivering decision-making services, with a configuration that may change over time. In this chapter, the IoT system is conceptualized as an evolving architecture, where capabilities are incrementally extended through the integration of new modules. This ongoing evolution directly shapes the characterization of decision problems and acts as a primary catalyst for adaptive reconfiguration.

Within the proof of concept, the IoT system fulfills two primary roles. It provides continuous observation of the physical environment, encompassing both environmental variables and crop state as determined by the growth model. Additionally, it serves as the semantic interface between the simulated environment and the external algorithm selection service, translating system capabilities and monitoring objectives into formalized problem characterizations.

9.5.1. IoT System Architecture

The IoT system architecture is structured around sensing, actuation, and monitoring components that interact with the crop growth environment. These components are represented within the simulation environment and are tasked with acquiring environmental data, observing crop state variables, and supporting decision-making processes related to crop monitoring and management.

At the sensing level, the IoT system incorporates sensors for key environmental variables, including temperature, PPFD, and carbon dioxide concentration. These sensors supply the environmental inputs necessary for the crop growth model and allow the IoT system to monitor changing conditions affecting crop development. Actuation components corresponding to these variables are modeled to represent the system's capacity to influence environmental conditions; however, the development of control strategies is beyond the scope of this chapter.

Beyond environmental sensing, the IoT system integrates monitoring components that observe specific outputs of the crop growth model, such as foliar area, fruit count, and other structural indicators of plant development. These monitored variables define the observable crop state and form the foundation for establishing monitoring objectives and decision-making requirements.

From an architectural perspective, the IoT system functions as an intermediary between the physical environment and the decision-making layer. It aggregates data from sensors and monitoring components, maintaining an internal representation of system state relevant to monitoring capabilities and decision-making requirements that supports the activation and management of semantic problem characterizations associated with evolution modules. These characterizations are then communicated to an external service responsible for algorithm selection, as depicted in the deployment view discussed earlier in this chapter (See Fig. 9.1).

The interaction between the IoT system and the algorithm selection service is realized through a service-oriented interface, enabling the exchange of semantic problem descriptions and selection results.

9.5.2. Evolution Modules

System evolution is represented by a set of evolution modules that may be incrementally integrated into the IoT system during simulation execution. Each evolution module constitutes a cyber-physical component that extends the system's monitoring or interaction capabilities and introduces additional requirements for decision-making.

The evolution modules examined in this case study include, but are not limited to, modules for illumination management, climate monitoring, ventilation, and vision-based crop monitoring. Each module encapsulates distinct sensing or actuation capabilities and specifies the type of data provided to the IoT system. For instance, integrating a vision module enables the observation of crop attributes such as fruit count and foliar area, which were previously inaccessible to the system.

Each evolution module is linked to a semantic characterization defined using the O-DMIoT ontology. This characterization describes how the module's integration alters the decision problem, encompassing factors such as application type, processing model, system configuration, and data characteristics. Upon incorporation of a new module, its characterization is activated within the current system context to define an updated problem description.

This mechanism explicitly connects system evolution to changes in problem characterization, rather than treating it solely as a structural modification. Consequently, the addition of evolution modules serves as a trigger for adaptive reconfiguration, potentially necessitating the selection of alternative decision-making algorithms to process new data types or address revised monitoring objectives.

9.5.3. Model Assumptions

The IoT system model is developed based on a set of assumptions that delineate its scope and align with the objectives of the proof of concept. These assumptions are deliberately selected to emphasize system evolution and adaptive reconfiguration, rather than low-level communication or control mechanisms.

The main assumptions adopted in this work are as follows:

- The protected and controlled agriculture environment is equipped with sensors for temperature, PPFD, and CO₂, and the evolution modules refer to the corresponding actuation capabilities.

- The IoT system initially provides basic monitoring functionality and can be extended by adding evolution modules during simulation execution.
- Evolution modules represent cyber-physical components that either monitor or influence the physical environment and expose new data to the system.
- Decision-making algorithms are not executed within the simulation environment; instead, algorithm selection is requested as an external service based on semantic problem characterization.
- The IoT system does not embed algorithm selection logic; its role is limited to problem characterization, invoking external services, and activating selected algorithmic services.

These assumptions establish a clear separation among environmental dynamics, system capabilities, and decision-making mechanisms, thereby enabling the analysis of system evolution and semantic-driven reconfiguration in a controlled and interpretable context.

9.6. Adaptive Reconfiguration Mechanism

The adaptive reconfiguration mechanism defines how changes in the IoT system and its operational context are translated into modifications of decision-making behavior. In this study, reconfiguration is conceptualized as a decision process driven by semantic characterization of evolving problem conditions, rather than as a reactive control action. This mechanism establishes the operational connection between system evolution, formal problem representation, and algorithm selection, thereby integrating the contributions presented in Chapters 5 and 8.

9.6.1. Reconfiguration Triggers

Reconfiguration is initiated by changes that alter the nature of the decision problem addressed by the IoT system. Instead of responding to individual sensor values or low-level events, the proposed mechanism responds to structural changes in system capabilities and information availability that influence the formulation and solution of decision-making tasks.

Within this framework, reconfiguration triggers are linked to the activation of evolution modules that introduce new decision-making requirements. Each evolution module is accompanied by a predefined semantic characterization of the decision problem it entails. When such a module is incorporated into the IoT system, its associated characterization becomes active and serves as the basis for triggering adaptive reconfiguration.

This approach explicitly ties reconfiguration triggers to changes in the decision context, establishing a clear and interpretable link between system evolution and adaptive behavior.

9.6.2. Problem Characterization

In the proposed reconfiguration mechanism, problem characterization is explicitly associated with evolution modules rather than inferred dynamically by the IoT system. Each evolution module is defined alongside a semantic characterization, expressed using the O-DMIoT ontology, which describes the type of decision problem introduced by the module's functionality.

These characterizations capture the essential properties of the decision problem, including the application objective, interaction mode, processing model, knowledge representation, and sources of uncertainty. Associating this information directly with evolution

modules ensures that the semantic meaning of newly introduced system capabilities is defined explicitly and consistently.

When an evolution module is added to the IoT system, its associated problem characterization is activated and integrated with the current system context. This characterization does not encode algorithmic decisions or parameter values; instead, it defines the problem space within which appropriate decision-making algorithms can be selected. Thus, the semantic description serves as a declarative specification of decision requirements rather than a procedural control rule.

This design choice simplifies the reconfiguration process by decoupling semantic modeling from runtime inference. It also enhances traceability, as each reconfiguration event can be directly linked to the evolution module that introduced the corresponding decision problem. Consequently, O-DMIoT serves as a stable semantic interface between system evolution and algorithm selection.

9.6.3. Invocation of the Synthesizer

Following problem characterization, the IoT system initiates the synthesizer utilizing the semantic representation produced by O-DMIoT. In the synthesizer, algorithm selection operates as an external service, remaining independent from both the simulation environment and the IoT system.

Within the synthesizer, the algorithm selection mechanism operates on the formal problem description to identify a suitable decision-making algorithm from a predefined directory of candidates. As described in Chapter 8 the algorithm selection mechanism within the synthesizer is modeled as a planning problem and solved using a Monte Carlo Tree Search (MCTS) approach. The semantic characterization defines the initial state of the search, and successive selection steps progressively narrow the solution space.

The output of the algorithm selection service is a concrete algorithm configuration that satisfies the requirements expressed in the problem characterization. This result is returned to the IoT system, which incorporates the selected algorithm into its service configuration. Deploying the selected algorithm completes the reconfiguration cycle. In the proposal, deployment refers to the logical integration of the selected algorithm as a service within the IoT system configuration. However, in the proof of concept, this process is achieved through service activation, rather than through dynamic code deployment.

Delegating algorithm selection to an external service, decoupled from the IoT system and the simulation environment, maintains a clear separation between semantic reasoning, and algorithmic decision-making. This separation enables flexible adaptation to evolving requirements while preserving the interpretability and modularity of the overall framework. Figure 9.3 summarizes the interactions described previously.

9.7. Proof of Concept Execution

This section presents the proof of concept as an end-to-end reconfiguration trace. It demonstrates how system evolution leads to problem characterization, algorithm selection, and reconfiguration within the proposed framework. The process is illustrated using a single evolution event to ensure clarity and traceability. The approach is designed to generalize to multiple or more complex evolution events, supporting the framework scalability.

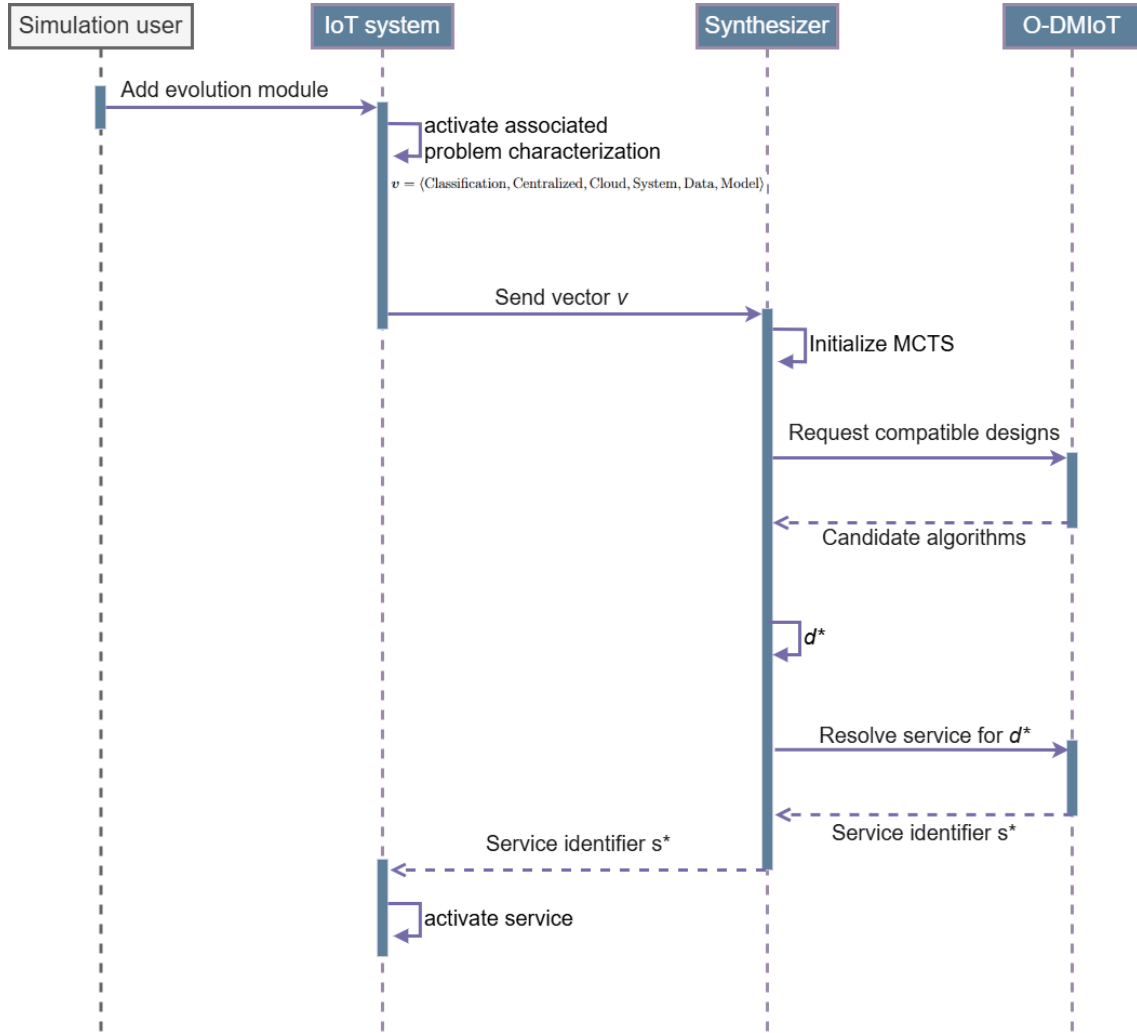


Figure 9.3.: **Semantic-driven reconfiguration decision trace.** The diagram illustrates how activating an evolution module leads to semantic problem characterization, planning-based algorithm selection via MCTS, ontology-driven service resolution, and configuration-level service activation within the IoT system.

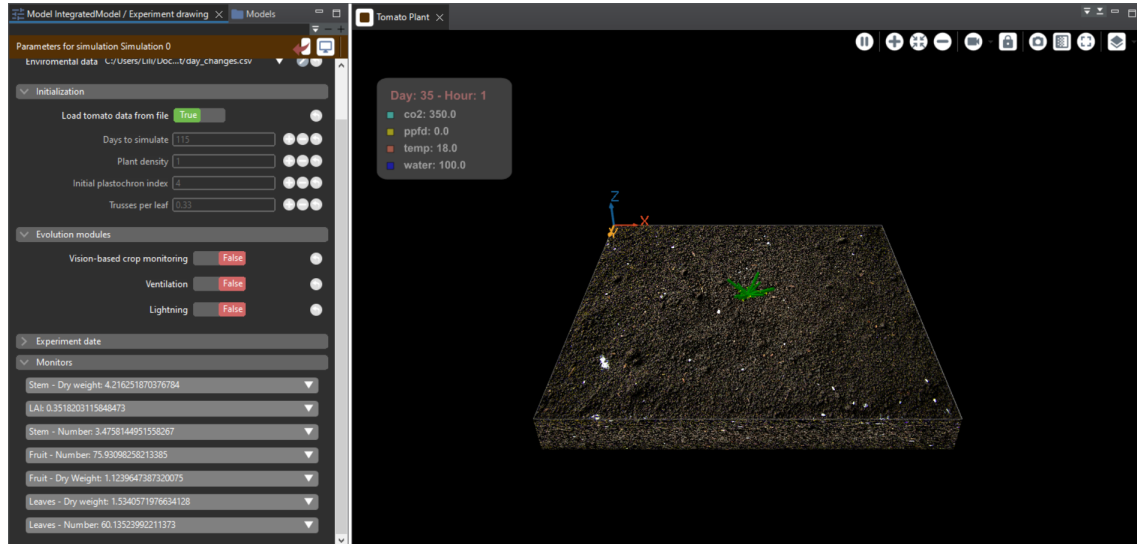


Figure 9.4.: **Initial configuration of the simulation environment** prior to the activation of the vision-based crop monitoring evolution module.

9.7.1. End-to-End Reconfiguration Flow

Initial configuration

At the beginning of the simulation, the IoT system is deployed with an initial configuration that enables basic monitoring of environmental variables. This configuration includes sensors for temperature, PPFD, and carbon dioxide concentration, facilitating observation of the physical conditions that influence crop growth. Advanced monitoring and decision-making modules are not active at this stage.

With this configuration, the IoT system monitors the environment and crop state as determined by the growth model; however, reconfiguration is not required. The system employs a fixed set of monitoring capabilities and does not invoke the algorithm selection service because decision-making requirements remain unchanged.

Figure 9.4 illustrates the initial configuration of the simulation environment, showing the active sensing modules and the absence of evolution modules prior to reconfiguration.

Initial State	
Active Modules	Environmental sensing
Evolution Modules	None
Problem Characterization	None active
Algorithm Selection	Not invoked

Table 9.1.: Initial system state prior to evolution

Evolution event

During the simulation, an evolution event occurs when a new module is integrated into the IoT system, modifying its functional structure and decision-making capabilities. In this scenario, a vision-based crop monitoring module is activated, extending the system's sensing infrastructure with visual observations of crop appearance and age-related features.

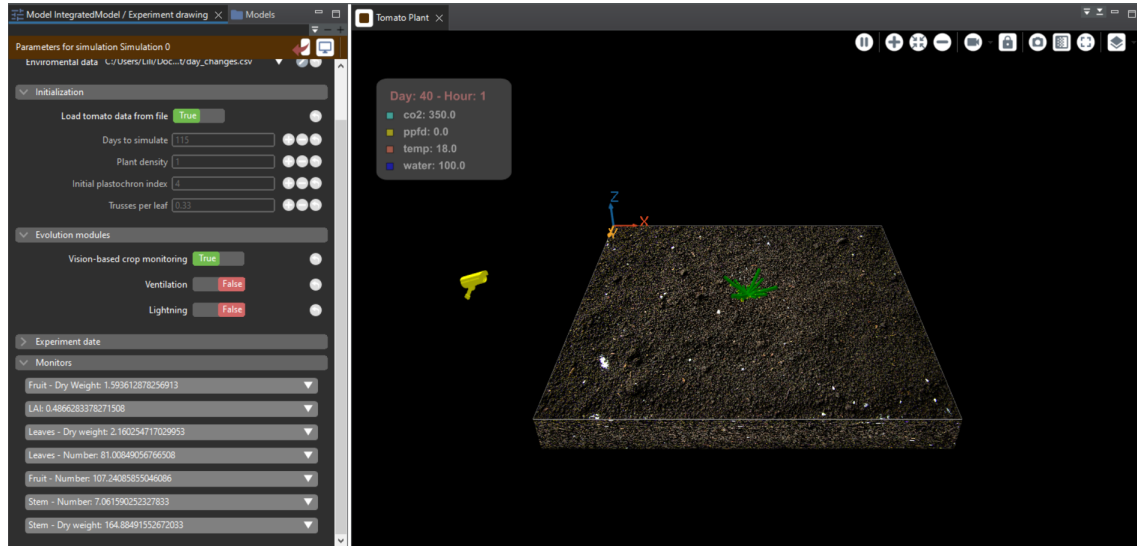


Figure 9.5.: **Simulation state after activation of the vision-based crop monitoring evolution module**, introducing a new decision-making requirement and triggering adaptive reconfiguration.

This evolutionary event constitutes a structural modification of the system. The introduction of vision-based data enables a new class of observations and simultaneously imposes additional decision-making requirements. Consequently, the original problem addressed by the system is extended to include the interpretation and classification of crop health based on visual evidence.

Table 9.2 formalizes the new requirement introduced by the evolution event. The table provides a semantic characterization of the vision-based crop monitoring requirement using the O-DMIoT ontology, specifying the objective, interaction type, processing characteristics, knowledge representation, and uncertainty. This characterization is summarized as a semantic vector that compactly captures the problem dimensions activated by the evolution.

Activation of this requirement triggers the algorithm selection and reconfiguration mechanisms described in the previous sections. Specifically, the semantic vector derived from the requirement serves as the input state for the ontology-driven decision-making process, guiding the selection of compatible algorithm designs.

Figure 9.5 illustrates the simulation state immediately following activation of the vision-based crop monitoring module. The figure highlights the transition from a system with purely environmental sensing to one in which a new decision-making requirement is active, thereby initiating adaptive reconfiguration within the system. Table 9.3 summarizes the system state after the evolution event has been activated, highlighting the transition from passive monitoring to an active decision-making configuration.

Reconfiguration Setup

Following activation of the vision-based crop monitoring module, the system enters an *in-evolution* state characterized by an active new problem characterization and the need for algorithm selection. This state, summarized in Table 9.3, triggers the reconfiguration process by invoking the algorithm selection mechanism defined in Chapter 8.

The reconfiguration setup begins by instantiating the initial state of the Monte Carlo Tree Search (MCTS) algorithm. The semantic characterization of the vision-based crop

Requirement description	de-	Crop health classification: The system shall be able to classify the health status of the crop based on visual observations of crop appearance and age-related features obtained through vision-based sensing components.
Input		Crop appearance, crop age
Output		Categorical crop health state
Semantic mapping using O-DMIoT		Objective: Classification. The decision process aims to assign a categorical health label to the crop based on observed visual features. Interaction: System. Inputs to the decision process originate exclusively from non-human sources, including vision sensors and image-derived data streams. Processing: Centralized. The decision-making process is executed by a single computational entity with access to the required data and models. Processing Layer: Application (Cloud). The computational demands of vision-based classification exceed the capabilities of local devices, requiring execution at the application layer. Knowledge Representation: Data. The decision process relies on datasets of visual observations and extracted features without explicit symbolic or mathematical modeling of crop health. Uncertainty: Model. Crop health is inferred from visual data in the absence of a precise underlying model, and observable features are subject to noise, partial observability, and latent factors.
Problem characterization		$v = \langle \text{Classification, Centralized, Cloud, System, Data, Model} \rangle$

Table 9.2.: Semantic characterization of the vision-based crop monitoring requirement

In-Evolution State	
Active Modules	Environmental sensing, Vision-based crop monitoring
Evolution Modules	Vision-based crop monitoring (active)
Problem Characterization	Crop health classification (active)
Algorithm Selection	Triggered

Table 9.3.: System state after evolution activation

monitoring requirement is encoded as a semantic vector:

$$\mathbf{v} = \langle \text{Classification, Centralized, Cloud, System, Data, Model} \rangle,$$

This vector defines the initial semantic state of the MCTS search tree. It represents the fixed problem instance for the decision-making process and determines the set of admissible algorithm design actions explored during the search.

The MCTS-based algorithm selection procedure, implemented in Python as described in Chapter 8, is provided as a service through a RESTful endpoint. When reconfiguration is required, the simulation environment implemented in GAMA sends an HTTP request to this endpoint via a Flask interface, transmitting the semantic vector and associated configuration parameters as input to the selection service.

Upon receiving the request, the MCTS algorithm explores the space of semantically compatible algorithm designs by querying the O-DMIoT ontology. The outcome is the selection of an algorithm design that satisfies the constraints encoded in the semantic vector. However, the selected design remains an abstract specification and does not yet correspond to an executable system component.

To address this gap, an additional service-selection step is performed. Using the selected algorithm design as input, the system queries the ontology to retrieve a concrete service that provides the required algorithm functionality. This query resolves the abstract design to a deployable service endpoint, enabling integration into the operational IoT system.

As a result, the reconfiguration setup phase establishes a concrete execution pipeline that links system evolution, semantic problem characterization, algorithm selection, and service resolution. In the proposed proof of concept, service resolution yields an abstract service identifier corresponding to an ontology-described service capable of providing the selected algorithm functionality. This identifier is communicated back to the simulation environment, where it is used to bind the selected service to a pre-implemented decision-making capability. In practice, receiving the service identifier in the GAMA simulation activates the corresponding functional module through a control flag, enabling execution of the selected algorithm without dynamic code deployment. This approach allows the proof of concept to focus on semantic-driven selection and reconfiguration logic while maintaining a realistic service-oriented interaction model.

Reconfiguration Outcome

The reconfiguration outcome involves activating a specific decision-making capability following the execution of the algorithm selection service. After the MCTS-based algorithm selection process described in Chapter 8 concludes, the full algorithm selection service provides a service identifier corresponding to an ontology-described service that delivers the selected algorithm's functionality.

In this proof of concept, the output does not necessitate deploying new executable code. Instead, the service identifier is transmitted to the simulation environment as a configuration directive. Upon receiving the identifier, the GAMA simulation activates a pre-implemented decision-making module using a control flag. This control flag triggers the module, transitioning it from a dormant to an active state and enabling the selected algorithm to operate within the system. The control flag ensures that the appropriate algorithm is executed effectively within the simulation.

Consequently, the IoT system transitions from passive monitoring to active decision-making by incorporating vision-based crop health classification. The system processes visual data from the activated monitoring module and generates categorical crop health assessments using the algorithm selected through semantic reasoning.

This outcome completes the reconfiguration cycle initiated by the evolution event. The execution trace demonstrates that a structural system change results in semantic problem

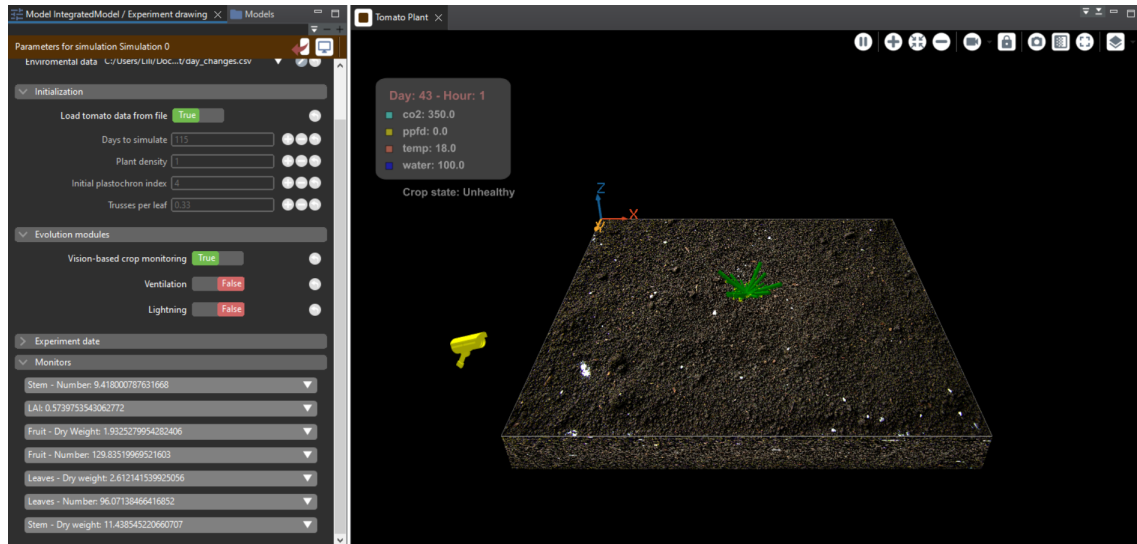


Figure 9.6.: Simulation state after reconfiguration, showing the execution of the vision-based crop health classification service resulting from algorithm selection and service binding.

characterization, algorithm selection, and runtime capability activation without manual intervention. By distinctly separating semantic selection from implementation-level activation, the proof of concept provides a realistic and transparent model for service-oriented adaptive reconfiguration.

Figure 9.6 illustrates the reconfiguration outcome by depicting the simulation state following service resolution and binding, where the vision-based crop monitoring module generates a categorical crop health assessment. The displayed crop health state verifies that the decision-making service selected through semantic reasoning is integrated into the operational system. Table 9.4 presents the system state after reconfiguration is complete.

Post-Reconfiguration State	
Active Modules	Environmental sensing, Vision-based crop monitoring
Evolution Modules	Vision-based crop monitoring (active)
Problem Characterization	Crop health classification (active)
Algorithm Selection	Completed

Table 9.4.: System state after completion of adaptive reconfiguration

Evolution Module	Module	Vision-Based Crop Monitoring
Requirement Description		The system shall be able to classify the health status of the crop based on visual observations of crop appearance and age-related features obtained through vision-based sensing components.
Objective		Classification
Interaction		System
Processing		Centralized
Processing Layer		Application (Cloud)
Knowledge Representation	Rep-	Data
Uncertainty		Model
Candidate Algorithms	Algo-	Logistic Regression; Decision Tree; k-Nearest Neighbors
Selected Algorithm	Algo-	Decision Tree
Resulting Service		DT Crop Health Classification Service

Table 9.5.: Proof-of-concept configuration for the vision-based crop monitoring evolution module

9.7.2. Observations

The proof of concept implementation produces observable outcomes directly attributable to the proposed adaptive reconfiguration mechanism. These outcomes are identified through analysis of system state transitions, configuration tables, and simulation snapshots described in previous sections.

First, activating the vision-based crop monitoring evolution module initiates a transition from passive monitoring to active decision-making. This shift is demonstrated by the system state change from an initial configuration without problem characterization to a post-reconfiguration state where a crop health classification problem is explicitly defined (Tables 9.1, 9.3, and 9.4).

The semantic characterization associated with the evolution module is used to trigger algorithm selection (See Tab. 9.5). The semantic vector derived from the O-DMIoT characterization serves as the fixed input to the algorithm selection service, and its activation during the observed execution completes the selection process. These results demonstrate that, within the scope of the proposed framework, the semantic abstraction defined by the taxonomy (See Chapter 4) is operationally adequate to support algorithm selection, without requiring additional runtime contextual inference.

The selected algorithmic capability is integrated into the operational system through service resolution and activation. The simulation environment demonstrates this integration by enabling a previously inactive decision-making capability, which becomes operational after reconfiguration. The appearance of a categorical crop health assessment in the simulation state (Figure 9.6) provides direct evidence that the decision-making service is active

and generating outputs.

The execution trace shows that the reconfiguration cycle, comprising system evolution, problem characterization, algorithm selection, and runtime activation, is completed without manual intervention. The system maintains operational status throughout, and reconfiguration occurs at the configuration level rather than through dynamic code deployment.

Collectively, these observations indicate that the proposed framework enables coherent, traceable adaptive reconfiguration guided by semantic problem characterization within a simulated IoT environment.

9.8. Discussion and Limitations

This chapter presents an integrated proof of concept that implements the semantic-driven adaptive reconfiguration framework proposed in this thesis. The findings indicate that semantic problem characterization, algorithm selection, and service-level reconfiguration can be integrated into a closed-loop process within an evolving IoT system. Instead of focusing on performance optimization within a specific application domain, the proof of concept highlights structural feasibility, traceability, and internal consistency of the proposed approach.

A primary contribution of the proof of concept is the explicit connection between system evolution and decision-making adaptation through semantic abstraction. By linking evolution modules with predefined semantic problem characterizations, the framework offers a systematic mechanism for translating changes in system capabilities into formal decision requirements. This design eliminates dependence on ad hoc rules or implicit inference and ensures that reconfiguration decisions are grounded in declarative knowledge representations expressed through O-DMIoT.

The end-to-end execution trace demonstrates that adaptive behavior emerges without manual intervention. Structural changes in the IoT system trigger corresponding semantic characterizations, which are processed by the synthesizer to select and activate appropriate decision-making services. This result confirms that the semantic abstractions defined by the ontology are both descriptive and operational, serving as effective inputs for algorithm selection and service resolution.

A further strength of the framework is its architectural modularization. Environmental dynamics, semantic modeling, algorithm selection, and execution are implemented as functionally isolated components that interact through well-defined interfaces. This partitioning enhances interpretability and extensibility by enabling individual components to evolve independently without impacting overall system behavior. Assigning algorithm selection and service determination to an external synthesizer also preserves flexibility in deploying decision-making capabilities while maintaining a stable semantic interface.

The integration of agent-based simulation with a mechanistic crop growth model strengthens the validity of the proof of concept by ensuring that reconfiguration events result from substantive changes in system state rather than artificial triggers. This modeling approach supports the assertion that the proposed framework can function in complex, time-evolving environments typical of real-world IoT systems.

Despite these strengths, several limitations should be acknowledged. First, the evaluation is qualitative and illustrative rather than quantitative. The proof of concept is designed to demonstrate feasibility and traceability, not to assess algorithmic performance, scalability, or optimality. Quantitative evaluation of selection accuracy, computational overhead, and adaptation latency remains a significant direction for future research.

Second, semantic problem characterizations are predefined and explicitly linked to evolution modules. Although this design choice improves clarity and traceability, it restricts the system's capacity to infer problem characterizations dynamically from raw data or un-

expected changes. Expanding the framework to support hybrid approaches that integrate predefined semantic models with runtime inference constitutes a promising direction for future research.

Third, algorithm deployment in the proof of concept is achieved through service activation and binding rather than dynamic code deployment. While this method is sufficient to validate semantic-driven selection and reconfiguration logic, it does not address challenges related to runtime deployment, container orchestration, or service migration across heterogeneous infrastructures. These issues are beyond the scope of the current work.

Finally, the case study addresses a single application domain and a limited set of evolution events. This constraint enhances clarity and interpretability but does not fully demonstrate the framework's applicability across diverse IoT domains or concurrent evolution scenarios. Additionally, the framework assumes the availability of accurate semantic models and curated algorithm directories, the construction and maintenance of which may present significant practical challenges.

Overall, the limitations discussed reflect intentional design choices to prioritize conceptual clarity and internal coherence. The proof of concept provides a robust foundation for future research on quantitative evaluation, dynamic semantic characterization, and large-scale deployment, while clearly defining the boundaries of the current contribution.

References

- [1] J. W. Jones et al. "A DYNAMIC TOMATO GROWTH AND YIELD MODEL (TOMGRO)". en. In: *Transactions of the ASAE* 34.2 (1991), pp. 0663–0672. URL: <http://elibrary.asabe.org/abstract.asp??JID=3&AID=31715&CID=t1991&v=34&i=2&T=1> (visited on 05/30/2024).
- [2] W. J. Kuijpers et al. "Model selection with a common structure: Tomato crop growth models". In: *Biosystems Engineering* 187 (Nov. 2019), pp. 247–257. URL: <http://dx.doi.org/10.1016/j.biosystemseng.2019.09.010>.
- [3] P. Taillandier et al. "Building, composing and experimenting complex spatial models with the GAMA platform". In: *GeoInformatica* 23.2 (Dec. 2018), pp. 299–322. URL: <https://doi.org/10.1007/s10707-018-00339-6>.

10. Conclusions

10.1. Summary of contributions

The primary objective of this thesis was to develop an adaptive reconfiguration mechanism for Internet of Things (IoT) systems that adjusts decision-making services in response to evolving functional requirements across application domains. To achieve this objective, the thesis proposed and validated a semantic-driven framework that integrates decision problem characterization, algorithm selection, evaluation, and service-level reconfiguration within a unified lifecycle.

The first contribution addresses the need for a systematic characterization of decision-making in IoT systems. The Taxonomy for Decision-Making in IoT (TDMIoT) organizes decision problems according to conceptual dimensions such as objective, uncertainty, interaction, and processing constraints, and links these dimensions to families of algorithmic approaches. This taxonomy provides a foundation for consistent reasoning about the relationship between decision problems and algorithmic solutions, a relationship that existing IoT and decision-analysis taxonomies do not explicitly capture.

Building on this foundation, the thesis introduces O-DMIoT, an ontology for decision-making in IoT systems that operationalizes TDMIoT as a formally grounded, machine-processable semantic model. Developed using the SABiO methodology and based on the Unified Foundational Ontology (UFO), O-DMIoT distinguishes requirement, design, and execution layers, and models evaluation as a cross-cutting concern. This structure enables semantic traceability throughout the lifecycle of decision-making algorithms and supports automated reasoning for algorithm selection, comparison, and reconfiguration in IoT environments.

Another contribution is the integration of human-centered evaluation into adaptive decision-making. Using agent-based modeling, this work shows how behavioral, psychological, and trust-related dynamics can be evaluated within decision-making services without embedding domain-specific assumptions into the ontology. The development of Quality of Experience (QoE) metrics offers a structured way to transform multidimensional simulation outcomes into comparative evaluation signals, allowing algorithm assessment based on human impact as well as technical performance.

The thesis achieves its objective by designing and implementing a semantic-driven adaptive reconfiguration mechanism. The integrated proof of concept demonstrates that changes in system structure and environmental conditions can be translated into formal semantic problem characterizations that drive algorithm selection and runtime service-level reconfiguration. The results confirm that adaptive behavior emerges coherently and without manual intervention, validating that the semantic abstractions defined by O-DMIoT are both descriptive and operational.

10.2. Limitations and Future Directions

The scope of this thesis reflects deliberate design decisions that prioritize conceptual clarity and internal coherence. Consequently, several limitations delineate clear directions for future research:

- **Quantitative Evaluation of Semantic-Driven Reconfiguration.**

The evaluation conducted in this work is qualitative and illustrative. While it demonstrates feasibility, traceability, and internal consistency, it does not provide quantitative assessment of properties such as algorithm selection accuracy, computational overhead, scalability, or adaptation latency. Future research should incorporate quantitative evaluation through controlled experiments and benchmarking to assess the performance implications of semantic-driven reconfiguration.

- **Inference of Semantic Problem characterizations.**

Semantic problem characterizations in the proof of concept are predefined and explicitly associated with system evolution modules. While this approach enhances transparency and interpretability, it limits the framework's ability to infer decision requirements dynamically from raw data or unanticipated changes. Extending the framework to support hybrid approaches that combine symbolic semantic models with data-driven or learning-based inference represents a significant direction for future research.

- **Runtime Deployment and Infrastructure-Aware Reconfiguration**

Algorithm deployment in the current framework is accomplished through service activation and binding rather than dynamic code deployment. Addressing challenges related to runtime deployment, container orchestration, service migration, and edge-cloud distribution is necessary to evaluate the framework in deployment-scale IoT infrastructures.

- **Multi-Domain Applicability**

The proof of concept is limited to a single application domain and a narrow set of evolution scenarios. While this constraint supports clarity and interpretability, future research should examine the applicability of the framework across multiple IoT domains and under concurrent or conflicting evolution events.

- **Practical implications.**

The construction and long-term maintenance of curated algorithm repositories and semantic models remain practical challenges that require further investigation.

11. Publications Derived from This Thesis

Parts of this dissertation have been published or submitted for publication in peer-reviewed venues. The contents of these chapters have been adapted and integrated to ensure coherence within the overall structure of the thesis.

- **Chapter 4** is based on the following publication:
L. Durán-Polanco and M. Siller, «A taxonomy for decision making in IoT systems», Internet of Things, vol. 24, p. 100904, dic. 2023, doi: 10.1016/j.iot.2023.100904.
- **Chapter 6** is based on the following publication:
M. Siller, G. A. Palomo-Briones, L. Durán-Polanco, and F. Aleman, «9 - An agent-based model for COVID-19 and its interventions and impact in different social phenomena», en Mathematical Modelling, Simulations, and AI for Emergent Pandemic Diseases, E. A. Hernandez-Vargas y J. X. Velasco-Hernández, Eds., Academic Press, 2023, pp. 145-174. doi: 10.1016/B978-0-323-95064-0.00009-9.
- **Chapter 7** corresponds to a manuscript that was previously submitted for publication and is currently under revision for resubmission.

A. Formal Specification of the O-DMIoT Ontology

This appendix provides the consolidated formal specification of the Ontology for Decision-Making in the Internet of Things (O-DMIoT). While Chapter 5 presents the conceptual foundations, design rationale, and illustrative axioms of the ontology, this appendix serves as a reference artifact intended to support reproducibility, inspection, and reuse.

A.1. Ontology Overview

O-DMIoT is implemented using the Web Ontology Language (OWL 2 DL) and serialized in Turtle format. The ontology is grounded in the Unified Foundational Ontology (UFO) and developed following the SABiO methodology.

- **Name:** Ontology for Decision-Making in the IoT (O-DMIoT)
- **Ontology language:** OWL 2 DL
- **Serialization:** Turtle (.ttl)
- **Development tool:** Protégé
- **Foundational ontology:** Unified Foundational Ontology (UFO)

A.1.1. Visual Ontology Model

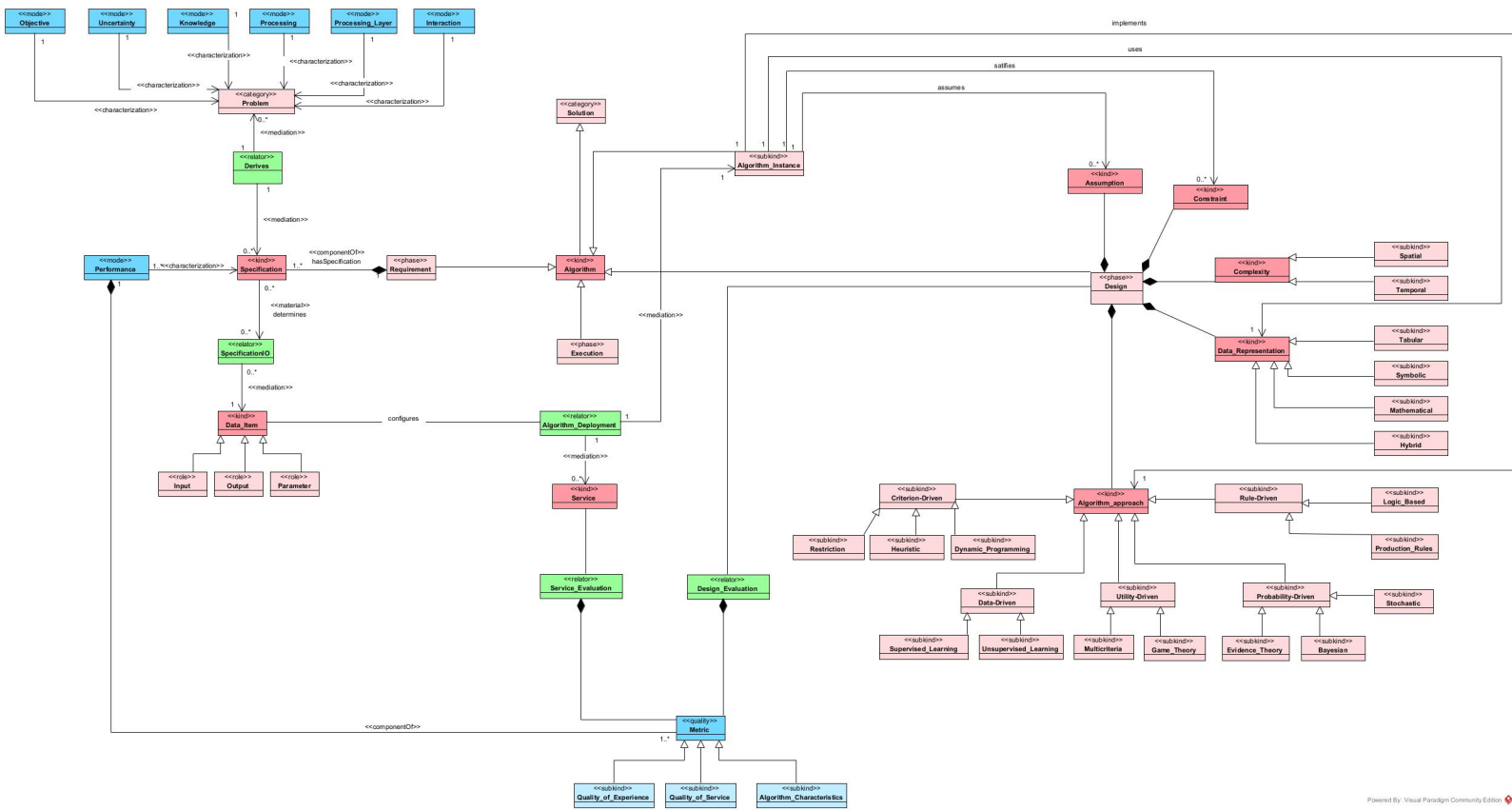


Figure A.1.: O-DMIoT full diagram

A.2. Core Ontological Elements

Table A.1 lists the principal classes of O-DMIoT, grouped according to the conceptual layers of the ontology.

Table A.1.: Core classes of the O-DMIoT ontology

Layer	Class
Problem Characterization	Problem
	Objective
	Uncertainty
	Interaction
	Processing
	ProcessingLayer
Requirement Specification	Requirement
	Specification
	Performance
	DataItem
Design	Algorithm
	Design
	AlgorithmicApproach
	Constraint
	Assumption
Execution	Service
	Deployment
Evaluation	Metric
	DesignEvaluation
	ServiceEvaluation

A.2.1. Core Data Properties

O-DMIoT makes limited use of data properties, primarily to annotate quantitative or categorical values associated with metrics and algorithm characteristics.

A.2.2. Logical Constraints and Axioms

Table A.2 summarizes representative logical constraints of O-DMIoT using Description Logic (DL) notation. These axioms correspond to the OWL 2 DL formalization implemented in the Turtle ontology file and reflect the constraints discussed in Chapter 5.

A.2.3. Query listing

This section presents representative ontology queries used to validate competency questions and to support ontology-driven algorithm selection in O-DMIoT. The queries are expressed using Description Logic (DL) query syntax as supported by Protégé, see Tab. A.3.

The queries listed in this appendix are representative examples intended to illustrate how O-DMIoT supports semantic reasoning and ontology-driven algorithm selection. Additional queries can be defined by combining the same ontological elements and constraints encoded in the OWL implementation.

Table A.2.: Representative Description Logic axioms of O-DMIoT

Concern	DL Axiom	Intent
Problem characteriza- tion	$Problem \sqsubseteq$ $1 \text{ characterizedBy.Objective} \sqcap$ $1 \text{ characterizedBy.Uncertainty} \sqcap$ $(= 1 \text{ characterizedBy.Interaction}) \sqcap$ $(= 1 \text{ characterizedBy.Processing}) \sqcap$ $(= 1 \text{ characterizedBy.ProcessingLayer})$	$(=$ Complete prob- $(=$ lem definition $\sqcap$ $\sqcap$ $\sqcap$ $\sqcap$
Requirement ground- ing	$Problem \sqsubseteq \exists \text{ derives.Specification}$	Specifications derived from problems
Requirement aggrega- tion	$Requirement$ $\exists \text{ hasSpecification.Specification}$	$\sqsubseteq$ Requirements aggregate speci- fications
Specification evalua- bility	$Specification$ $\exists \text{ characterizedBy.Performance}$	$\sqsubseteq$ Specifications are evaluable
Performance measure- ment	$Performance \sqsubseteq \exists \text{ measuredBy.Metric}$	Performance is measurable
Algorithm design typ- ing	$Algorithm \sqsubseteq \exists \text{ hasDesign.Design}$	Algorithms have explicit designs
Algorithmic approach consistency	$CriterionDriven \sqcap DataDriven \sqsubseteq \perp$	Mutually exclu- sive strategies
Evaluation applica- tion	$DesignEvaluation$ $\exists \text{ designEvaluates.Design}$ $\exists \text{ designEvaluates.Metric}$	$\sqsubseteq$ Design evalua- $\sqcap$ tion uses metrics
Metric categorization	$Metric \sqsubseteq QoS \sqcup QoE \sqcup$ $AlgorithmCharacteristic$	Evaluation per- spective

A.2.4. OWL Implementation and Availability

<https://github.com/LiliDupDev/ODMIoT.git>

CQ	Competency Question	Example DL Query (Protégé)
CQ1	What constitutes a decision problem in the IoT?	Problem
CQ2	What are the characteristics of a problem?	Problem and (distinguishedBy some (Objective or Interaction or KnowledgeRepresentation or Processing or ProcessingLayer or Uncertainty))
CQ3	How are problem characteristics related to each other?	Problem and (distinguishedBy some Objective) and (distinguishedBy some ProcessingLayer)
CQ4	What constitutes a solution to a problem?	Solution and (realizedBy some Algorithm)
CQ5	What are the characteristics of a problem solution?	Algorithm and (hasDesign some Design)
CQ6	What are the elements of a requirement specification?	Specification and (characterizedBy some Performance)
CQ7	What are the elements of a design specification?	Design and (aggregates some (Assumption or Constraint))
CQ8	How is execution linked to a problem solution?	Service and (deploys some Algorithm)
CQ9	What constitutes the evaluation of a solution?	(DesignEvaluation or ServiceEvaluation) and (evaluates some Performance)
CQ10	Which metrics can be used to assess whether a solution meets requirements?	Metric and (measures some Performance)

Table A.3.: Mapping of competency questions to example Description Logic (DL) queries executable in Protégé

B. Computational Artifacts and Reproducibility

This appendix details the computational artifacts developed and utilized to support evaluation, simulation, and validation in this thesis. Given the complexity and scale of the implemented models, complete implementations are made available via public repositories instead of being reproduced in full within this document.

B.1. Agent-Based Models

The agent-based models (ABMs) described in Chapters 6 and 7 were implemented to evaluate human-centered aspects of decision-making and system adaptation. These models represent individual behavior, interaction dynamics, and contextual factors relevant to the application domains under study.

The full implementation of the ABMs, including agent definitions, behavioral rules, simulation parameters, and execution scripts, is available in the following GitHub repositories:

- **Case Study 1 (Chapter 6):** <https://github.com/LiliDupDev/Back2RoutineModel.git>
- **Case Study 2 (Chapter 7):** <https://gitfront.io/r/LiliDupDev/5HVGgcZfeZmR/ABM-VaccinationResponse/>

These repositories provide all source code and configuration files necessary to reproduce the simulation experiments presented in the thesis, along with analyzed results, processing scripts, and figures.

B.2. Algorithm Selection and Reconfiguration Mechanisms

The algorithm selection and reconfiguration mechanisms discussed in Chapters 8 and 9 were implemented as modular services operating independently of the simulation environment. These implementations include the ontology-driven synthesizer, the planning model, and the Monte Carlo Tree Search (MCTS) components.

The complete source code and configuration files for these components are accessible in the following repository:

- **Algorithm Selection and Synthesizer:** https://github.com/LiliDupDev/Semantic_MCTS.git
- **Agricultural environment simulation:** https://github.com/LiliDupDev/IoT_Evolution_Simulation.git